QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Qualcomm AI Residency (QARE) — Complete 5-Pillar Theory Textbook

All 5 Pillars in One Volume: Math Foundations • Classical ML & DL • Computer Vision • NLP & Sequence Models • Transformers, LLMs & Efficient Edge AI

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

82 Sections Indexed

Pillar 1 (Days 1–3): Mathematics for AI — Intuitive Step-by-Step Study Guide p. 4

Day 1: Linear Algebra (Eigenvalues, Subspaces, SVD/PCA, Projections) p. 4

1.1 Eigenvalues, Eigenvectors & The 15-Second Shortcut Arsenal p. 4

1.2 Rank & The Four Fundamental Subspaces (Image Space $\mathrm{Im}(A)$ & Kernel $\mathrm{Ker}(A)$) p. 7

1.3 Symmetric, Orthogonal, and Positive Semi-Definite (PSD) Matrices p. 9

1.4 Singular Value Decomposition (SVD: $A = U\Sigma V^T$), Pseudo-Inverse & PCA p. 12

1.5 Orthogonal Projections & Least Squares Geometry p. 15

Day 2: Multivariable Matrix Calculus & Optimization p. 18

2.1 Matrix Calculus Without Memorization p. 18

2.2 Canonical Deep Learning Backprop Derivations (Softmax + CE & Linear Layers) p. 20

2.3 Convexity, Gradient Descent Step-Size Limits & Newton's Method p. 22

2.4 Constrained Optimization: Lagrange Multipliers & KKT Conditions p. 24

Day 3: Probability, Statistics & Information Theory p. 28

3.1 Conditional Probability, Bayes' Theorem & The Odds-Ratio Speed Hack p. 28

3.2 Core Probability Distributions & Their ML Roles p. 30

3.3 Expectation, Variance, Correlated Ensembles & Probability Inequalities p. 31

3.4 Maximum Likelihood (MLE) vs. MAP & Estimator Bias p. 33

3.5 Information Theory: Entropy, Cross-Entropy, KL Divergence & Mutual Information p. 35

Pillar 2 (Days 4–5): Classical ML & Deep Learning Core — Step-by-Step Study Guide p. 40

Day 4: Classical Machine Learning & Evaluation Metrics p. 40

1.1 Linear, Ridge (L_2), Lasso (L_1) & ElasticNet Regression (ML01) p. 40

1.2 Logistic Regression, Probability Calibration & Support Vector Machines (ML02 , ML03) p. 44

1.3 Decision Trees, Random Forests & Gradient Boosting (XGBoost Math) (ML04 , ML05) p. 48

1.4 Unsupervised Learning: K-Means, GMM (EM) & Dimensionality Reduction (ML06) p. 52

1.5 Bias-Variance Decomposition, Cross-Validation & Imbalanced Metrics (ML07 – ML10) p. 55

Day 5: Deep Learning Core Mechanics (Activations, Norms, Init, Optimizers) p. 60

2.1 Activation Functions & Their Failure Modes (ML11) p. 60

2.2 Normalization Layers & Inverted Dropout: Shapes, Buffers & Traps (ML12 , ML13) p. 62

2.3 Weight Initialization Derivations & Symmetry Breaking (ML14 , ML15) p. 65

2.4 Modern Optimizers, Decoupled Weight Decay & Training Schedules (ML16 – ML20) p. 68

Pillar 3 (Days 6–8): Computer Vision (CV) — Step-by-Step Intuitive Study Textbook p. 72

Day 6: Convolution Arithmetic, FLOPs & Efficient Edge CNNs p. 72

1.1 The 3 Master Spatial Dimension Formulas (Standard, Dilated & Transposed Convolutions — CV01 , CV03) p. 72

1.2 Exact Parameter & MACs/FLOPs Formulas Across Convolution Variants (CV02 , CV04) p. 75

1.3 Landmark CNN & Edge Architectures: VGG, ResNet, MobileNetV2, GAP & SENet (CV05 , CV06 , CV07 , CV18 , CV19) p. 77

Day 7: Receptive Field Recursion, Object Detection & Segmentation p. 82

2.1 Iterative Receptive Field Calculation (CV01 , CV08) p. 82

2.2 Object Detection Part 1: IoU, GIoU, NMS & Feature Pyramid Networks (CV09 , CV10) p. 84

2.3 Object Detection Part 2: Two-Stage vs. One-Stage, RoI Align (2D Bilinear Interpolation), Focal Loss, DETR & mAP (CV11 , CV12) p. 87

2.4 Semantic, Instance & Panoptic Segmentation (CV13 , CV14) p. 90

Day 8: Vision Transformers (ViT, Swin), Multimodal (CLIP) & Diffusion p. 92

3.1 Vision Transformer (ViT) & Swin Transformer (CV15 , CV16) p. 92

3.2 CLIP (Contrastive Language-Image Pre-training) & Modern Augmentation/SSL (CV17 , CV20) p. 95

3.3 Diffusion Models & Generative Vision: DDPM, DDIM, Latent Diffusion & CFG p. 97

Pillar 4 (Days 9–10): NLP & Sequence Models — Intuitive Step-by-Step Study Textbook p. 102

Day 9: Tokenization, Statistical NLP & Word Embeddings p. 102

1.1 Subword Tokenization: BPE vs. Byte-Level BPE vs. WordPiece vs. Unigram LM p. 103

1.2 N-Gram Language Models, Smoothing Hierarchy & Perplexity p. 106

1.3 TF-IDF, BM25 & Levenshtein Edit Distance p. 109

1.4 Static vs. Contextual Word Embeddings & Vector Space Geometry p. 112

Day 10: Sequence Models (RNN/LSTM/GRU), Sequence Labeling & NLP Metrics p. 116

2.1 Vanilla RNN, LSTM, GRU & BiLSTM: Equations, Parameter Counts & BPTT Gradients p. 118

2.2 Sequence Labeling: BIO Tagging, Linear-Chain CRF & CTC Loss p. 121

2.3 Seq2Seq, Attention Variants, Decoding & NLP Evaluation Metrics p. 124

Rapid-Fire 60-Second Exam Checklist (All 20 NLP01 – NLP20 Targets) p. 129

Pillar 5: Transformers, LLMs & Efficient On-Device AI — Intuitive Study Textbook p. 131

1. The Transformer Architecture from First Principles p. 131

1.1 Why Self-Attention Replaced Recurrence p. 131

1.2 Scaled Dot-Product Attention & The $\sqrt{d_k}$ Variance Proof (LLM01) p. 132

1.3 Multi-Head Attention (MHA): Shapes, Parameters & The $N = 2D$ Crossover (LLM02 , LLM03) p. 134

1.4 Causal Masking, Pre-LN vs Post-LN & Encoder vs Decoder Paradigms (LLM04) p. 136

2. Modern LLM Architectural Upgrades (LLaMA / Qwen / Mistral DNA) p. 137

2.1 RMSNorm vs LayerNorm & SwiGLU FFN Parameter Count Math (LLM05) p. 138

2.2 MHA vs MQA vs GQA Parameter & KV-Cache Math (LLM05 , LLM06) p. 140

2.3 RoPE (Rotary Position Embedding), ALiBi & Context Extension (LLM07) p. 141

2.4 Sparse Mixture-of-Experts (MoE — Mixtral 8x7B / DeepSeek-MoE) p. 143

2.5 Chinchilla Compute-Optimal Scaling Laws vs. Edge Inference Over-Training p. 144

3. LLM Systems, Memory Arithmetic & Inference Optimization p. 145

3.1 Prefill vs Decode Phases & The Hardware Roofline Model (LLM10) p. 145

3.2 Exact KV-Cache Memory Calculation (LLM08) p. 147

3.3 Mixed-Precision Training Memory (16Ψ Rule) & Activation Checkpointing (LLM09) p. 148

3.4 FlashAttention (IO-Aware Tiling) & PagedAttention (vLLM) (LLM10) p. 150

3.5 Decoding & Sampling Strategies (LLM11) p. 152

4. Fine-Tuning (PEFT / LoRA) & Alignment (RLHF vs DPO vs GRPO) p. 153

4.1 LoRA & QLoRA Math & Exam Traps (LLM12 , LLM13) p. 153

4.2 Post-Training Alignment: SFT $\rightarrow$ RLHF (PPO) vs DPO vs GRPO (LLM14) p. 155

5. Edge AI, Quantization, Distillation & Fast Inference (Qualcomm Focus) p. 158

5.1 Floating-Point Formats: FP32 vs FP16 vs BF16 (LLM15) p. 158

5.2 Exact Quantization Math: Asymmetric (Affine) vs Symmetric (LLM16 , LLM17) p. 160

5.3 PTQ vs QAT & Modern LLM Quantization (GPTQ, AWQ, SmoothQuant) (LLM18) p. 162

5.4 Knowledge Distillation, Pruning, Speculative Decoding & RAG (LLM19 , LLM20) p. 163

6. Master Quick-Reference Checklist (LLM01 – LLM20) p. 166

Pillar 1 (Days 1–3): Mathematics for AI — Intuitive Step-by-Step Study Guide

↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **How This Study Guide Is Structured:** Instead of throwing dense walls of equations at you, every topic in this guide is broken down into **4 human-friendly steps**:
>
> 1. 💡 **Intuition (Plain English First):** *What problem does this solve, why does it exist, and what is the physical/visual mental model?*
> 2. 📐 **The Math & Every Symbol Explained:** Clean equations where **every single letter, subscript, and matrix dimension ($n \times m$ = n rows, m columns)** is explicitly defined.
> 3. 🔢 **Step-by-Step Toy Example:** A concrete calculation with small numbers worked out line by line so you can see how the math actually works in practice.
> 4. 🎯 **Exam Takeaway & Fast Shortcuts:** The exact 60-second trick or conceptual trap tested on the Qualcomm HackerRank MCQ assessment.

Day 1: Linear Algebra (Eigenvalues, Subspaces, SVD/PCA, Projections)

↑ Mục Lục

1.1 Eigenvalues, Eigenvectors & The 15-Second Shortcut Arsenal

💡 Intuition (Plain English First)

When you multiply a square matrix $A \in \mathbb{R}^{n \times n}$ by an arbitrary vector $x \in \mathbb{R}^n$, the output vector Ax usually gets **both rotated and stretched**—it points in a completely new direction.

However, almost every square matrix has a few special "privileged axes" in space. If a vector v lies along one of these special axes, multiplying it by A **does not rotate it at all**—it simply stretches, shrinks, or flips v along that exact same line:

$$Av = \lambda v$$

- v **(Eigenvector):** The special direction/axis that refuses to rotate when multiplied by A.
- λ **(Eigenvalue):** The scalar stretch factor along that axis (e.g., $\lambda = 3$ stretches the vector $3\times$ longer; $\lambda = -1$ flips it backward; $\lambda = 0$ squashes that entire direction into the origin 0).

Why do we care in AI? Because if you apply a matrix A repeatedly ($A^k x$, like in a Recurrent Neural Network or multi-step optimization), any direction with $|\lambda| > 1$ explodes ($1.5^{100} \to \infty$), and any direction with $|\lambda| < 1$ vanishes ($0.5^{100} \to 0$). Furthermore, finding the eigenvalues of a 3×3 matrix on a timed MCQ by expanding $\det(A - \lambda I) = 0$ takes 4 minutes and is prone to sign errors—instead, exam designers expect you to use **Trace, Determinant, and Rank-1 shortcuts** to spot the eigenvalues in 15 seconds.

📐 The Math & Every Symbol Explained

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix (n rows, n columns) with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$:

1. Trace & Determinant Invariants (Always Check These First!)

$$\text{Tr}(A) = \sum_{i=1}^{n} A_{ii} = \sum_{i=1}^{n} \lambda_i, \qquad \det(A) = \prod_{i=1}^{n} \lambda_i$$

- $\text{Tr}(A)$ **(Trace):** The sum of the diagonal entries $A_{11} + A_{22} + \cdots + A_{nn}$. It **always equals the sum of all n eigenvalues**.
- $\det(A)$ **(Determinant):** Geometrically, how much A scales n-dimensional volume. It **always equals the product of all n eigenvalues**. (If even one eigenvalue is $\lambda_i = 0$, then $\det(A) = 0$, meaning A squashes space into a lower dimension and is **not invertible**).
- **For any 2×2 matrix**, the characteristic equation is simply:

$$\lambda^2 - \text{Tr}(A)\lambda + \det(A) = 0 \implies \lambda = \frac{\text{Tr}(A) \pm \sqrt{\text{Tr}(A)^2 - 4\det(A)}}{2}$$

2. Structural Shortcuts for 3×3 (and $n \times n$) Exam Matrices

- **Constant Row Sum Shortcut:** If every row of A adds up to the same number c, then multiplying A by the all-ones vector $\mathbf{1} = [1, 1, \ldots, 1]^T$ just adds each row up, giving $A\mathbf{1} = c\mathbf{1}$. Therefore, $\lambda = c$ **is guaranteed to be an eigenvalue** with right eigenvector $\mathbf{1}$! *(Even if every **column** adds up to c, $\lambda = c$ is still guaranteed to be an eigenvalue of A because A and A^T share the exact same eigenvalues).*
- **Rank-1 Matrix ($A = uv^T$):** Where $u, v \in \mathbb{R}^n$ are column vectors. Because every column of A is just a scalar multiple of u, A squashes all of n-dimensional space onto the single 1D line spanned by u.
  - Therefore, $n - 1$ directions get squashed to zero ($n - 1$ **eigenvalues equal** 0).
  - The single surviving direction is u, and its eigenvalue equals the trace: $\lambda_1 = \text{Tr}(uv^T) = u^T v$.
- **Shifted Rank-1 Matrix ($A = cI + uv^T$):** Adding cI simply adds $+c$ to every eigenvalue of uv^T. Thus the eigenvalues of $cI + uv^T$ are c (repeated $n - 1$ times) and $c + u^T v$ (once).
- **Block-Triangular & Rank-1 Determinant Identities:**

$$\det \begin{bmatrix} A & B \\ 0 & D \end{bmatrix} = \det(A)\det(D), \qquad \det(I + uv^T) = 1 + u^T v$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Sherman-Morrison Rank-1 Inverse Formula**

If you already know A^{-1} and someone modifies A by adding a rank-1 matrix uv^T, you do **not** need to re-invert the matrix from scratch in $\mathcal{O}(n^3)$ time. The **Sherman-Morrison formula** updates the inverse in $\mathcal{O}(n^2)$ time:

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1} u}$$

(Used in Quasi-Newton methods like BFGS and Recursive Least Squares, though too algebra-heavy to compute by hand in a 90-second MCQ.)

3. How Operations Change Eigenvalues vs. Eigenvectors

Suppose $Av_i = \lambda_i v_i$. What happens when we transform A?

New Matrix	New Eigenvalues	New Eigenvectors	Why It Works Intuitively
$A + cI$ (Shift diagonal by c)	$\lambda_i + c$	**Unchanged (v_i)**	$(A + cI)v_i = Av_i + cv_i = (\lambda_i + c)v_i$
A^k (Multiply A k times)	λ_i^k	**Unchanged (v_i)**	Each multiplication by A scales v_i by another factor of λ_i. Also $\text{Tr}(A^k) = \sum \lambda_i^k$
A^{-1} (Inverse, if $\lambda_i \neq 0$)	$\frac{1}{\lambda_i}$	**Unchanged (v_i)**	If A stretches v_i by λ_i, undoing A shrinks v_i by $1/\lambda_i$
$P^{-1}AP$ (Similarity transform)	**Unchanged (λ_i)**	**Rotated to $P^{-1}v_i$**	Same linear transformation viewed in a new coordinate basis P

🔢 Step-by-Step Toy Example

Problem: Find the three eigenvalues of:

$$A = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Step 1 — Spot the structure (do NOT expand the 3×3 determinant!): Notice that every off-diagonal number is 1, and every diagonal number is $3 = 2 + 1$. We can split A into a diagonal matrix $2I$ plus an all-ones matrix $\mathbf{1}\mathbf{1}^T$:

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = 2I + \mathbf{1}\mathbf{1}^T$$

Step 2 — Find the eigenvalues of the rank-1 all-ones matrix $\mathbf{1}\mathbf{1}^T$:

- Since all 3 rows of $\mathbf{1}\mathbf{1}^T$ are identical, it has rank 1.
- A 3×3 rank-1 matrix always has two eigenvalues equal to 0, and one eigenvalue equal to its trace:

  $$\text{Tr}(\mathbf{1}\mathbf{1}^T) = 1 + 1 + 1 = 3$$

- So the eigenvalues of $\mathbf{1}\mathbf{1}^T$ are $\{3, 0, 0\}$.

Step 3 — Shift by $+2I$:

- Adding $2I$ adds $+2$ to every eigenvalue:

  $$\{3 + 2,\ 0 + 2,\ 0 + 2\} = \{\mathbf{5}, \mathbf{2}, \mathbf{2}\}$$

Step 4 — 3-Second Sanity Check:

- Sum of our eigenvalues: $5 + 2 + 2 = 9$.
- Trace of A: $3 + 3 + 3 = 9$. Matches! Total time: **10 seconds.**

🎯 Exam Takeaway & Fast Shortcuts

- Whenever an MCQ asks for eigenvalues of a 3×3 matrix, **eliminate wrong options first** by checking if the options sum to $\text{Tr}(A)$! Often, only 1 of the 4 multiple-choice options will even add up to the trace.
- Watch out for the **Similarity Transform Trap:** $B = P^{-1}AP$ has the **exact same eigenvalues, trace, determinant, and rank** as A, but it does **not** have the same eigenvectors ($v_i \mapsto P^{-1}v_i$).

1.2 Rank & The Four Fundamental Subspaces (Image Space $\text{Im}(A)$ & Kernel $\text{Ker}(A)$)

💡 Intuition (Plain English First)

Let's standardize our matrix dimensions: throughout this guide, a rectangular matrix $A \in \mathbb{R}^{n \times m}$ always has n **rows** and m **columns**:

$$A = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \Bigg\} \, n \text{ rows} \quad \text{with } m \text{ columns}$$

When A multiplies an input vector x to produce an output vector $y = Ax$:

- **Input x must have m entries ($x \in \mathbb{R}^m$):** Because each of the m entries of x weights one of the m columns of A.
- **Output $y = Ax$ has n entries ($y \in \mathbb{R}^n$):** Because each of the n rows of A produces one dot-product entry in y.

So A is a machine that maps **Domain $\mathbb{R}^m$ (m columns)** $\longrightarrow$ **Codomain $\mathbb{R}^n$ (n rows)**.

What is the **rank** $r = \text{rank}(A)$? It is the number of **truly independent directions** (columns or rows) that survive when A transforms space. Because you can't have more independent directions than the number of rows or columns, $r \leq \min(n, m)$.

This splits the input space $\mathbb{R}^m$ and output space $\mathbb{R}^n$ into **Four Fundamental Subspaces**:

1. **Image Space (Column Space) $\text{Im}(A) = \mathcal{C}(A) \subseteq \mathbb{R}^n$:** The set of all possible outputs $y = Ax$ that the matrix can actually reach. Since every output Ax is a linear combination of the columns of A, the Image Space is spanned by the columns of A and has dimension r.
2. **Kernel (Nullspace) $\text{Ker}(A) = \mathcal{N}(A) \subseteq \mathbb{R}^m$:** The set of all input vectors $x \in \mathbb{R}^m$ that get **completely crushed to zero** ($Ax = 0$). Since you start with m input dimensions and r of them survive into the Image Space, the remaining $m - r$ input dimensions must get crushed to zero!
3. **Row Space $\text{Im}(A^T) = \mathcal{C}(A^T) \subseteq \mathbb{R}^m$:** The r-dimensional subspace of the input domain $\mathbb{R}^m$ that *survives* multiplication by A without being crushed to zero. It is strictly perpendicular (orthogonal) to the Kernel $\text{Ker}(A)$.
4. **Left Nullspace (Cokernel) $\text{Ker}(A^T) = \mathcal{N}(A^T) \subseteq \mathbb{R}^n$:** The $(n - r)$-dimensional subspace of the output space $\mathbb{R}^n$ that is **unreachable** by Ax and sits strictly perpendicular to the Image Space $\text{Im}(A)$.

📐 The Math & Every Symbol Explained

For any matrix $A \in \mathbb{R}^{n \times m}$ (n rows, m columns) with $\text{rank}(A) = r$:

$$\underbrace{\dim(\text{Im}(A))}_{\text{Rank } r} + \underbrace{\dim(\text{Ker}(A))}_{\text{Nullity } m-r} = \mathbf{m} \text{ (Number of Columns!)} \qquad \text{(Rank-Nullity Theorem)}$$

Subspace Name	School / Math Notation	Lives In	Dimension	Geometric Meaning
Image Space (Column Space)	$\text{Im}(A) = \mathcal{C}(A) = \{Ax : x \in \mathbb{R}^m\}$	Output $\mathbb{R}^n$	r	All reachable outputs $y = Ax$
Kernel (Nullspace)	$\text{Ker}(A) = \mathcal{N}(A) = \{x \in \mathbb{R}^m : Ax = 0\}$	Input $\mathbb{R}^m$	$m - r$	All inputs crushed to 0; $\text{Ker}(A) \perp \text{Im}(A^T)$
Row Space	$\text{Im}(A^T) = \mathcal{C}(A^T)$	Input $\mathbb{R}^m$	r	Input directions mapped 1-to-1 onto $\text{Im}(A)$
Left Nullspace (Cokernel)	$\text{Ker}(A^T) = \mathcal{N}(A^T) = \{y \in \mathbb{R}^n : A^T y = 0\}$	Output $\mathbb{R}^n$	$n - r$	Unreachable output directions; $\text{Ker}(A^T) \perp \text{Im}(A)$

High-Frequency Rank Identities

- **Gram Matrix Rank Invariance:** For *any* real matrix $A \in \mathbb{R}^{n \times m}$:

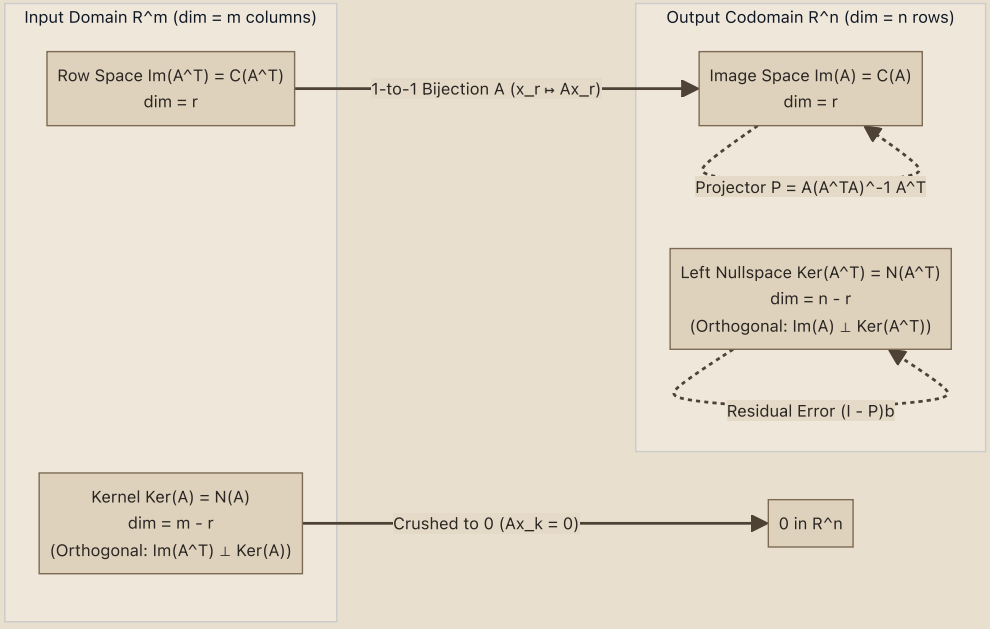
  $$\text{rank}(A) = \text{rank}(A^T) = \text{rank}(A^T A) = \text{rank}(A A^T) = r$$

  (Why? Because $A^T A x = 0 \implies x^T A^T A x = \|Ax\|_2^2 = 0 \implies Ax = 0$, so A and $A^T A$ have the exact same Kernel $\text{Ker}(A^T A) = \text{Ker}(A)$!)

- **Rank of a Sum:** $|\text{rank}(A) - \text{rank}(B)| \leq \text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B)$.

- **Sylvester's Rank Inequality (Rank of a Product):** For $A \in \mathbb{R}^{n \times k}$ and $B \in \mathbb{R}^{k \times m}$:

  $$\text{rank}(A) + \text{rank}(B) - k \leq \text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$$



🔢 Step-by-Step Toy Example

Problem: Suppose A is a 3×4 matrix ($n = 3$ rows, $m = 4$ columns):

$$A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Find $\text{rank}(A)$ and the dimension of all four subspaces.

Step 1 — Find the Rank r:

- Look at the rows: Row 1 and Row 2 are independent (pivots in Column 1 and Column 2), while Row 3 is all zeros.
- Therefore, $\text{rank}(A) = r = 2$.

Step 2 — Find the Image Space $\text{Im}(A)$ and Kernel $\text{Ker}(A)$:

- **Image Space $\text{Im}(A)$:** Lives in $\mathbb{R}^3$ ($n = 3$ rows). Its dimension is $r = \mathbf{2}$ (specifically, the xy-plane where the 3rd coordinate is 0).
- **Kernel $\text{Ker}(A)$:** Lives in $\mathbb{R}^4$ ($m = 4$ columns). By Rank-Nullity, $\dim(\text{Ker}(A)) = m - r = 4 - 2 = \mathbf{2}$. (There are 2 free variables, x_3 and x_4, that you can vary while keeping $Ax = 0$).

Step 3 — Find the Row Space $\text{Im}(A^T)$ and Left Nullspace $\text{Ker}(A^T)$:

- **Row Space $\text{Im}(A^T)$:** Lives in $\mathbb{R}^4$ ($m = 4$ columns), with dimension $r = \mathbf{2}$.
- **Left Nullspace $\text{Ker}(A^T)$:** Lives in $\mathbb{R}^3$ ($n = 3$ rows), with dimension $n - r = 3 - 2 = \mathbf{1}$ (specifically, the z-axis $[0, 0, c]^T$, which is perpendicular to $\text{Im}(A)$).

🎯 Exam Takeaway & Fast Shortcuts

- **The #1 Rank-Nullity Trap:** When given $A \in \mathbb{R}^{n \times m}$ (n rows, m columns), remember that $\dim(\text{Im}(A)) + \dim(\text{Ker}(A)) = \mathbf{m}$ (**number of columns / inputs**), NEVER n!
- If a linear system $Ax = b$ has at least one particular solution x_p, then the set of **all** solutions is the affine subspace $x = x_p + x_k$ for any $x_k \in \text{Ker}(A)$. Thus, a solution is **unique if and only if** $\text{Ker}(A) = \{0\}$ (i.e., $r = m$, full column rank).

1.3 Symmetric, Orthogonal, and Positive Semi-Definite (PSD) Matrices

💡 Intuition (Plain English First)

General square matrices can behave badly: they can have complex eigenvalues, non-perpendicular eigenvectors, or even be missing eigenvectors altogether ("defective" matrices like $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$).

Fortunately, in Machine Learning, three families of matrices behave *beautifully*:

1. **Symmetric Matrices ($A = A^T$):** Examples include Covariance matrices, Hessian matrices ($\nabla^2 f$), and Kernel/Gram matrices ($X^T X$). A symmetric matrix **never rotates vectors into complex space**: its

eigenvalues are guaranteed to be **100% real numbers**, and its eigenvectors are guaranteed to form a **perpendicular (orthogonal) coordinate system**!

2. **Orthogonal Matrices ($Q^TQ = QQ^T = I$):** These are pure **rigid rotations or mirror reflections**. Multiplying a vector x by Q rotates x without changing its length ($\|Qx\|_2 = \|x\|_2$) and without changing the angle between any two vectors.
3. **Positive Definite ($A \succ 0$) & Positive Semi-Definite ($A \succeq 0$) Matrices:** These are the matrix equivalent of "strictly positive numbers ($a > 0$)" and "non-negative numbers ($a \geq 0$)." Geometrically, if $A \succ 0$, the quadratic surface $f(x) = x^TAx$ is an **upward-facing bowl** with a single global minimum at $x = 0$.

📐 The Math & Every Symbol Explained

1. Real Symmetric Matrix ($A = A^T \in \mathbb{R}^{n \times n}$)

- **Spectral Theorem:** Every real symmetric matrix can be factored as:

$$A = Q\Lambda Q^T = \sum_{i=1}^{n} \lambda_i q_i q_i^T$$

  - $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$**:** Diagonal matrix of real eigenvalues ($\lambda_i \in \mathbb{R}$).
  - $Q = [q_1, \ldots, q_n] \in \mathbb{R}^{n \times n}$**:** Orthogonal matrix whose columns are orthonormal eigenvectors ($q_i^T q_j = \delta_{ij}$, so $Q^{-1} = Q^T$).

- **Rayleigh Quotient Bounds:** If you plug any unit vector x ($\|x\|_2 = 1$) into x^TAx, the output is bounded between the smallest and largest eigenvalue:

$$\lambda_{\min}(A) \leq \frac{x^TAx}{x^Tx} \leq \lambda_{\max}(A)$$

  (The maximum $\lambda_{\max}$ is achieved when $x = q_{\max}$, and the minimum $\lambda_{\min}$ when $x = q_{\min}$.)

2. Orthogonal Matrix ($Q^TQ = QQ^T = I_n$)

- **Length & Angle Preservation:** $\|Qx\|_2^2 = (Qx)^T(Qx) = x^TQ^TQx = x^TIx = \|x\|_2^2$.
- **Determinant & Eigenvalues:** $\det(Q) = +1$ (pure rotation) or -1 (rotation + reflection). Every eigenvalue has magnitude $|\lambda_i| = 1$ (can be complex $e^{\pm i\theta}$ for a 2D rotation by angle θ).

3. Positive Definite ($A \succ 0$) vs. Positive Semi-Definite ($A \succeq 0$)

For a **symmetric** matrix $A = A^T \in \mathbb{R}^{n \times n}$:

Property	Positive Definite ($A \succ 0$, PD)	Positive Semi-Definite ($A \succeq 0$, PSD)
Quadratic Form	$x^TAx > 0$ for **all** $x \neq 0$	$x^TAx \geq 0$ for **all** x
Eigenvalues	All $\lambda_i > 0$ (strictly positive)	All $\lambda_i \geq 0$ (non-negative)
Invertibility	Always invertible ($\det(A) > 0$)	May be singular if any $\lambda_i = 0$

Property	Positive Definite ($A \succ 0$, PD)	Positive Semi-Definite ($A \succeq 0$, PSD)
Leading Principal Minors	All $k \times k$ top-left sub-determinants > 0 (**Sylvester's Criterion**)	All principal minors ≥ 0 *(see warning below)*
Factorization	Unique Cholesky $A = LL^T$ ($L_{ii} > 0$)	$A = X^T X$ for some $X \in \mathbb{R}^{n \times m}$

> ⚠️ **WARNING**
>
> **Two Famous PSD/PD Exam Traps:**
>
> 1. **Symmetry is Required for Eigenvalue/Minor Tests:** A non-symmetric matrix like $M = \begin{bmatrix} 1 & 4 \\ 0 & 1 \end{bmatrix}$ has positive eigenvalues $\{1, 1\}$ and positive leading minors, yet for $x = [1, -1]^T$, $x^T M x = 1 - 4 + 1 = -2 < 0$! Always check $x^T M x = x^T \left(\frac{M + M^T}{2} \right) x$.
> 2. **Gram Matrix Guarantee:** For *any* rectangular matrix $X \in \mathbb{R}^{n \times m}$, the matrix $G = X^T X \in \mathbb{R}^{m \times m}$ is **always symmetric PSD** because $x^T (X^T X) x = (Xx)^T (Xx) = \|Xx\|_2^2 \geq 0$. Adding ridge penalty λI_m ($\lambda > 0$) shifts every eigenvalue from ≥ 0 to $\geq \lambda > 0$, making $X^T X + \lambda I_m$ **strictly Positive Definite and guaranteed invertible**!

<details>
<summary>📌 Optional Deep-Dive (Skip for Timed MCQ): Why Sylvester's Leading Minors Fail for PSD</summary>

For strict Positive Definiteness ($A \succ 0$), checking only the n **leading** (top-left) principal minors > 0 is necessary and sufficient. However, for **Positive Semi-Definiteness ($A \succeq 0$)**, checking only top-left minors ≥ 0 is **not enough**! For example, $A = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$ has top-left 1×1 minor $= 0$ and 2×2 determinant $= 0$, yet its eigenvalues are $\{0, -1\}$ (not PSD!). To test PSD via minors, all $2^n - 1$ principal minors must be ≥ 0. On timed exams, simply check $\lambda_i \geq 0$ or $x^T A x \geq 0$.
</details>

🔢 Step-by-Step Toy Example

Problem: For what values of c is the symmetric matrix $A = \begin{bmatrix} 4 & c \\ c & 9 \end{bmatrix}$ strictly Positive Definite ($A \succ 0$)?

Step 1 — Check Symmetry:

- $A = A^T$ is symmetric.

Step 2 — Apply Sylvester's Criterion (Top-Left Leading Principal Minors > 0):

- **1st leading minor (1×1 top-left entry):** $A_{11} = 4 > 0$ ✔
- **2nd leading minor (2×2 determinant):**

$$\det(A) = (4)(9) - c^2 = 36 - c^2 > 0$$

Step 3 — Solve for c:

- $c^2 < 36 \implies \mathbf{-6 < c < 6}$.
- *(If $c = \pm 6$, then $\det(A) = 0$ and $\text{Tr}(A) = 13$, so $\lambda = \{13, 0\}$, making A Positive Semi-Definite $A \succeq 0$ instead of strictly PD.)*

🎯 Exam Takeaway & Fast Shortcuts

- To check if a 2×2 symmetric matrix A is strictly Positive Definite ($A \succ 0$), you only need two instant checks: $A_{11} > 0$ **AND** $\det(A) > 0$ (which is equivalent to $\text{Tr}(A) > 0$ and $\det(A) > 0$).

1.4 Singular Value Decomposition (SVD: $A = U\Sigma V^T$), Pseudo-Inverse & PCA

💡 Intuition (Plain English First)

Let's connect **PCA** and **SVD** from the ground up, because once you see how they fit together, 5 different linear algebra formulas become one single mental picture.

Part A: What is PCA doing? (The Camera Angle Analogy)

Imagine you have a dataset of $N = 1,000$ houses with $d = 2$ features: $x_1 =$ Square Footage and $x_2 =$ Number of Rooms. If you plot these $1,000$ houses on a 2D scatter plot, they don't form a round circle—they form a **tilted cigar-shaped cloud** because bigger houses have more rooms.

- **The Problem:** The two features are redundant. If you need to compress those 2 features down to **1 single number** per house while losing as little information as possible, what should you do?
- **PCA's Solution:** Instead of using the original horizontal/vertical axes (x_1, x_2), **rotate your coordinate axes** so that:
  1. **Principal Component 1 (v_1):** Points directly along the **longest axis of the cigar** (the direction of **maximum variance / spread**).
  2. **Principal Component 2 (v_2):** Points perpendicular to v_1 across the narrow width of the cigar (mostly noise). By projecting every data point onto v_1, you keep the maximum possible variance (which is mathematically identical to minimizing the squared perpendicular reconstruction error!).

Part B: What is SVD doing? (Rotate $\rightarrow$ Stretch $\rightarrow$ Rotate)

Eigendecomposition ($A = Q\Lambda Q^T$) only works on **square symmetric** matrices. What if your matrix $A \in \mathbb{R}^{n \times m}$ (n rows, m columns) is **rectangular**? **Singular Value Decomposition (SVD)** says that **every rectangular matrix** $A \in \mathbb{R}^{n \times m}$ can be broken down into three simple geometric steps:

$$A = U\Sigma V^T$$

When you multiply a vector $x \in \mathbb{R}^m$ by A ($y = U\Sigma V^T x$):

1. $V^T x$ **(Rotate in Input Space $\mathbb{R}^m$):** V^T is an $m \times m$ orthogonal rotation matrix that aligns x with the input principal axes $v_1, \dots, v_m$.
2. $\Sigma(V^T x)$ **(Stretch along the axes):** Σ is an $n \times m$ diagonal matrix of non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ that stretches the i-th axis by σ_i (and pads/drops zeros to change dimension from m to n).
3. $U(\Sigma V^T x)$ **(Rotate in Output Space $\mathbb{R}^n$):** U is an $n \times n$ orthogonal rotation matrix that places the stretched vector onto its final orientation $u_1, \dots, u_n$ in the output space $\mathbb{R}^n$.

Part C: How SVD and PCA Are the Exact Same Thing!

Suppose $X \in \mathbb{R}^{N \times d}$ is your mean-centered data matrix (N rows = data points, d columns = features). In PCA, we form the sample covariance matrix $S = \frac{1}{N-1} X^T X \in \mathbb{R}^{d \times d}$ and find its eigenvectors. Now plug the SVD of $X = U\Sigma V^T$ into $X^T X$:

$$X^T X = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T \underbrace{U^T U}_{I} \Sigma V^T = V\Sigma^2 V^T$$

Look at $X^T X = V\Sigma^2 V^T$:

1. The **right singular vectors** V of X are **literally the PCA principal component directions (eigenvectors of $X^T X$)**!
2. The **eigenvalues of** $X^T X$ are $\lambda_i = \sigma_i^2$ (the squared singular values of X), so the covariance eigenvalues are $\lambda_i(S) = \frac{\sigma_i^2}{N-1}$.

📐 The Math & Every Symbol Explained

1. Full vs. Dyadic SVD ($A \in \mathbb{R}^{n \times m}$, n rows, m columns, rank r)

$$A = U\Sigma V^T = \sum_{i=1}^{r} \sigma_i u_i v_i^T$$

- $U \in \mathbb{R}^{n \times n}$ **(Left Singular Vectors):** Orthogonal matrix ($U^T U = I_n$) whose columns $u_i \in \mathbb{R}^n$ are eigenvectors of $AA^T \in \mathbb{R}^{n \times n}$. First r columns span $\text{Im}(A)$; last $n - r$ columns span $\text{Ker}(A^T)$.
- $\Sigma \in \mathbb{R}^{n \times m}$ **(Singular Values):** Rectangular diagonal matrix with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ on the diagonal, where $\sigma_i = \sqrt{\lambda_i(A^T A)} = \sqrt{\lambda_i(AA^T)}$.
- $V \in \mathbb{R}^{m \times m}$ **(Right Singular Vectors):** Orthogonal matrix ($V^T V = I_m$) whose columns $v_i \in \mathbb{R}^m$ are eigenvectors of $A^T A \in \mathbb{R}^{m \times m}$. First r columns span $\text{Im}(A^T)$; last $m - r$ columns span $\text{Ker}(A)$.

2. Moore-Penrose Pseudo-Inverse ($A^+ \in \mathbb{R}^{m \times n}$)

How do you "invert" a rectangular or rank-deficient matrix A? You invert the 3 SVD steps in reverse order—rotating back by V, inverting only the non-zero singular values $\sigma_i \mapsto \frac{1}{\sigma_i}$ (leaving $0 \mapsto 0$), and rotating by U^T:

$$A^+ = V\Sigma^+ U^T = \sum_{i=1}^{r} \frac{1}{\sigma_i} v_i u_i^T \in \mathbb{R}^{m \times n}$$

- If A has **full column rank** ($r = m$): $A^+ = (A^T A)^{-1} A^T$ (satisfies $A^+ A = I_m$).
- If A has **full row rank** ($r = n$): $A^+ = A^T (AA^T)^{-1}$ (satisfies $AA^+ = I_n$).
- For **any** linear system $Ax \approx b$ (even when underdetermined with infinitely many solutions!), $x^* = A^+ b$ gives the unique solution that minimizes $\|Ax - b\|_2^2$ and has the **smallest possible L_2 norm** $\|x^*\|_2$.

3. Matrix Norms, Condition Number & Eckart-Young-Mirsky Low-Rank Approximation

- **Spectral Norm (L_2 Operator Norm):** Maximum stretch factor of A on any unit vector:

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sigma_1 \quad \text{(Largest singular value!)}$$

- **Frobenius Norm:** Square root of the sum of all squared matrix entries:

$$\|A\|_F = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{m} A_{ij}^2} = \sqrt{\text{Tr}(A^T A)} = \sqrt{\sum_{i=1}^{r} \sigma_i^2}$$

- **Condition Number:** $\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$ (measures how elongated the matrix is; if $\kappa_2(A) \gg 1$, solving $Ax = b$ is numerically unstable and Gradient Descent converges slowly).
- **Eckart-Young-Mirsky Theorem (Best Rank-k Compression):** If you approximate A by keeping only the top $k < r$ singular values ($A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$), the exact approximation error is:

$$\|A - A_k\|_2 = \sigma_{k+1}, \qquad \|A - A_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}$$

- **PCA Proportion of Variance Explained (PVE) by Top k Components:**

$$\text{PVE}(k) = \frac{\sum_{i=1}^{k} \lambda_i(S)}{\sum_{j=1}^{d} \lambda_j(S)} = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{j=1}^{d} \sigma_j^2}$$

🔢 Step-by-Step Toy Example

Problem: A centered data matrix $X \in \mathbb{R}^{100\times 4}$ ($N = 100$ samples, $d = 4$ features) has singular values $\sigma_1 = 6,\ \sigma_2 = 3,\ \sigma_3 = 2,\ \sigma_4 = 1$.

1. What are the spectral norm $\|X\|_2$ and Frobenius norm $\|X\|_F$?
2. If we compress X to a rank-2 matrix X_2 using SVD, what are the errors $\|X - X_2\|_2$ and $\|X - X_2\|_F$?
3. What percentage of total variance is explained by the first $k = 2$ Principal Components?

Step 1 — Norms of X:

- Spectral norm $\|X\|_2 = \sigma_1 = \mathbf{6}$.
- Frobenius norm $\|X\|_F = \sqrt{\sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \sigma_4^2} = \sqrt{36 + 9 + 4 + 1} = \sqrt{\mathbf{50}}$.

Step 2 — Rank-2 Truncation Errors ($k = 2$):

- X_2 keeps $\sigma_1 = 6$ and $\sigma_2 = 3$, and drops $\sigma_3 = 2$ and $\sigma_4 = 1$.
- Spectral error $\|X - X_2\|_2 = \sigma_{k+1} = \sigma_3 = \mathbf{2}$.
- Frobenius error $\|X - X_2\|_F = \sqrt{\sigma_3^2 + \sigma_4^2} = \sqrt{2^2 + 1^2} = \sqrt{\mathbf{5}}$.

Step 3 — PCA Proportion of Variance Explained (PVE for $k = 2$):

- **Crucial Check:** Variance is proportional to σ_i^2 (squared singular values!), NOT σ_i:

$$\text{PVE}(2) = \frac{\sigma_1^2 + \sigma_2^2}{\sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \sigma_4^2} = \frac{36 + 9}{36 + 9 + 4 + 1} = \frac{45}{50} = \mathbf{90\%}$$

🎯 Exam Takeaway & Fast Shortcuts

- **The #1 PCA/SVD Trap:** Singular values of X are σ_i, whereas eigenvalues of $X^T X$ (and variance along each principal component) are proportional to σ_i^2! When an exam gives you singular values $\sigma = \{6, 3, 2, 1\}$ and asks for Variance Explained, **always square them first** $(36, 9, 4, 1)$ before dividing!

1.5 Orthogonal Projections & Least Squares Geometry

💡 Intuition (Plain English First)

Suppose you have an overdetermined linear system $Ax = b$ where $A \in \mathbb{R}^{n \times m}$ is a tall matrix ($n > m$, e.g., 100 equations/rows and only 2 unknowns/columns).

- All possible outputs Ax live inside the m-dimensional **Image Space** $\text{Im}(A)$ (a flat 2D plane sitting inside 100-dimensional space $\mathbb{R}^{100}$).
- Your target vector $b \in \mathbb{R}^n$ almost certainly does **not** lie on that flat plane—so $Ax = b$ has **no exact solution**.

How do we find the best approximate solution $\hat{x}$ that minimizes the squared error $\|Ax - b\|_2^2$? Think of shining a flashlight straight down (perpendicularly) onto the plane $\text{Im}(A)$:

1. The **closest point** on the plane $\text{Im}(A)$ to b is the **orthogonal shadow (projection)** $\hat{b} = Pb = A\hat{x}$.
2. Because $\hat{b}$ is directly underneath b, the **residual error vector** $r = b - A\hat{x}$ **must stick straight up, 90° perpendicular to the plane** $\text{Im}(A)$!
3. Saying "$b - A\hat{x}$ is perpendicular to every column of A" in math is simply:

> $$A^T(b - A\hat{x}) = 0 \implies A^T A\hat{x} = A^T b \quad \text{(The Normal Equations!)}$$

📐 The Math & Every Symbol Explained

1. Projection onto a Single 1D Vector $u \in \mathbb{R}^n$

> $$P_u = \frac{uu^T}{u^T u} \in \mathbb{R}^{n \times n}, \qquad \text{proj}_u(y) = P_u y = \left(\frac{u^T y}{\|u\|_2^2}\right) u$$

- uu^T **($n \times n$ outer product matrix):** A rank-1 matrix.
- $u^T u = \|u\|_2^2$ **(scalar inner product):** Normalizes for the length of u.

2. Projection onto Image Space $\text{Im}(A)$ (for full-column-rank $A \in \mathbb{R}^{n \times m}$, $r = m$)

Solving $A^T A\hat{x} = A^T b$ gives $\hat{x} = (A^T A)^{-1} A^T b$. Multiplying by A gives the projected vector $\hat{b} = A\hat{x} = Pb$, where the **Orthogonal Projection Matrix** $P \in \mathbb{R}^{n \times n}$ is:

> $$P = A(A^T A)^{-1} A^T \in \mathbb{R}^{n \times n}$$

3. The 5 Algebraic Laws of Any Orthogonal Projection Matrix P

If a question asks "*Which of the following is true about $P = A(A^TA)^{-1}A^T$?*", test these 5 geometric properties:

1. **Symmetric ($P^T = P$):** Because it is an *orthogonal* (perpendicular) projection.
2. **Idempotent ($P^2 = P$):** Projecting a shadow that is *already on the floor* onto the floor a second time doesn't move it ($P(Py) = Py$).
3. **Eigenvalues are strictly 0 or 1:** Since $P^2v = Pv \implies \lambda^2 = \lambda \implies \lambda \in \{0, 1\}$.
   - Any vector already inside $\text{Im}(A)$ stays unchanged ($\lambda = 1$, multiplicity r).
   - Any vector perpendicular to $\text{Im}(A)$ (in $\text{Ker}(A^T)$) gets squashed to zero ($\lambda = 0$, multiplicity $n - r$).
4. **Trace equals Rank ($\text{Tr}(P) = \text{rank}(P) = r$):** Because the trace is the sum of eigenvalues (r ones and $n - r$ zeros)!
5. **Complementary Residual Projector ($I - P$):** $I - P$ is ALSO an orthogonal projection matrix! It projects onto the Left Nullspace $\text{Ker}(A^T) = \text{Im}(A)^\perp$, and $P(I - P) = P - P^2 = 0$.

🔢 Step-by-Step Toy Example

Problem: Let $u = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \in \mathbb{R}^3$. Construct the 3×3 projection matrix P_u onto the line spanned by u, verify its trace and eigenvalues, and find the projection of $y = \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix}$ onto u.

Step 1 — Compute u^Tu and P_u:

- $u^Tu = 1^2 + 2^2 + 2^2 = 9$.
- Outer product uu^T:

$$P_u = \frac{1}{9} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 2 & 2 \\ 2 & 4 & 4 \\ 2 & 4 & 4 \end{bmatrix}$$

Step 2 — Verify Trace & Eigenvalues:

- $\text{Tr}(P_u) = \frac{1+4+4}{9} = \frac{9}{9} = \mathbf{1}$ (matches the dimension $r = 1$ of the line!).
- Eigenvalues of P_u are $\{1, 0, 0\}$.

Step 3 — Project $y = [3, 3, 0]^T$ onto u:

- Instead of multiplying the full 3×3 matrix, use the dot-product scalar $\frac{u^Ty}{u^Tu}$:

$$\frac{u^Ty}{u^Tu} = \frac{(1)(3) + (2)(3) + (2)(0)}{9} = \frac{9}{9} = 1$$

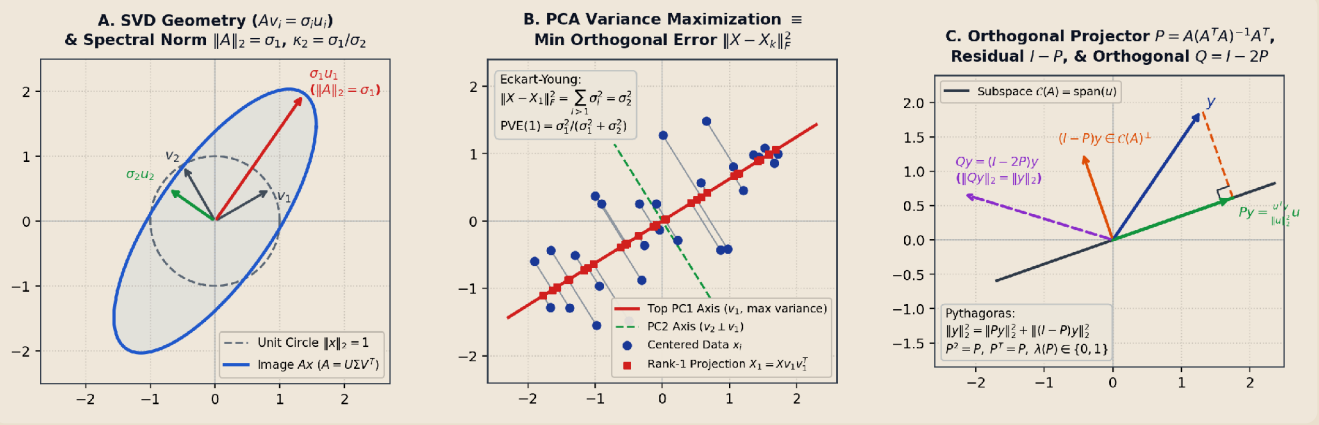
- Thus $\text{proj}_u(y) = 1 \cdot u = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$.
- Check that the residual $r = y - \text{proj}_u(y) = [2, 1, -2]^T$ is perpendicular to u: $u^Tr = (1)(2) + (2)(1) + (2)(-2) = 0$ ✔.

🎯 Exam Takeaway & Fast Shortcuts

- If an MCQ gives you a 100×4 full-rank matrix A and asks for the **trace** or **sum of eigenvalues** of $P = A(A^TA)^{-1}A^T$, do zero matrix multiplication! Use the cyclic trace property:

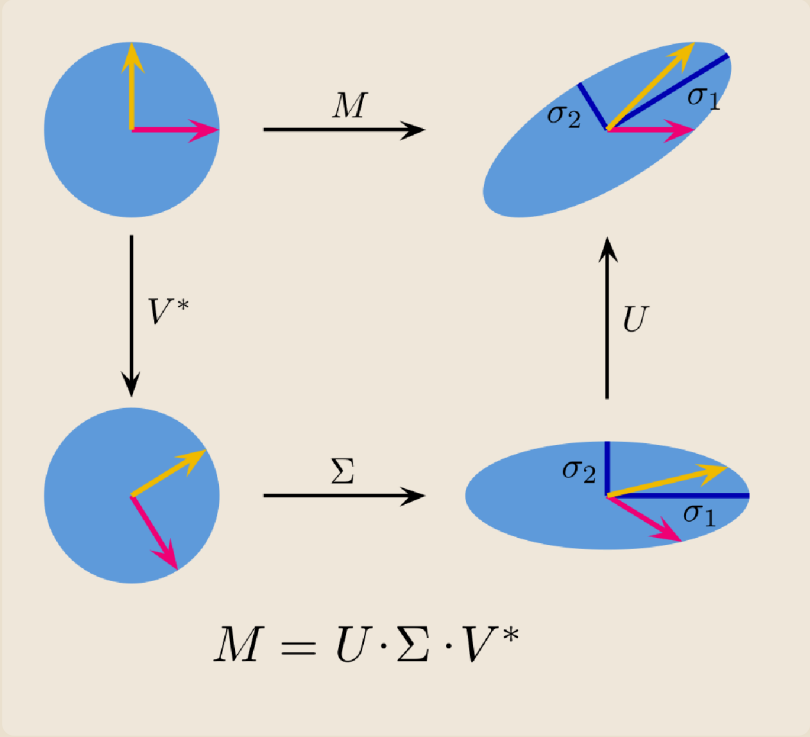
$$\mathrm{Tr}\big(A(A^TA)^{-1}A^T\big) = \mathrm{Tr}\big(\underbrace{A^TA(A^TA)^{-1}}_{I_4}\big) = \mathrm{Tr}(I_4) = \mathbf{4}$$

FIGURE SLIDE 1 OF 3



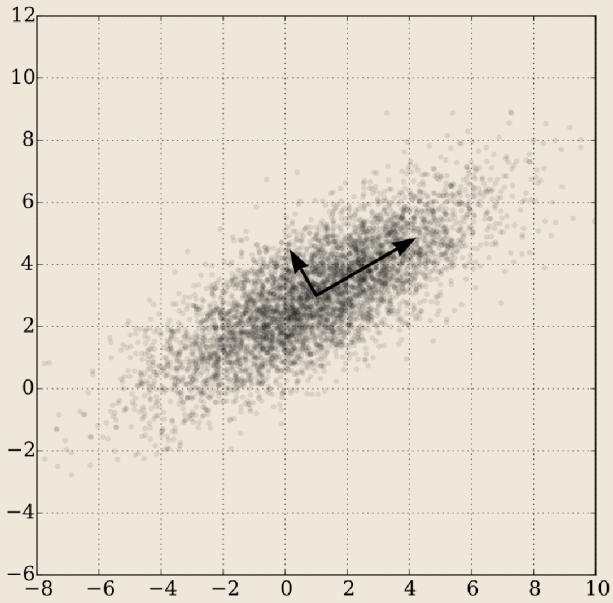
Day 1 Linear Algebra Visual Cheat-Sheet: SVD Geometry, PCA Variance Maximization, and Orthogonal Projections

FIGURE SLIDE 2 OF 3



Wikimedia Commons — Step-by-Step SVD Geometric Decomposition: Rotation V → Scaling Σ → Rotation U*

FIGURE SLIDE 3 OF 3



Wikimedia Commons — Principal Component Analysis (PCA) Eigenvector Axes on a 2D Gaussian Point Cloud

---

Day 2: Multivariable Matrix Calculus & Optimization

↑ Mục Lục

2.1 Matrix Calculus Without Memorization

💡 Intuition (Plain English First)

In single-variable calculus, you know three simple rules by heart:

1. The derivative of a linear function $f(x) = ax$ is a (and second derivative is 0).
2. The derivative of a quadratic function $f(x) = ax^2$ is $2ax$ (and second derivative is $2a$).
3. The derivative of $f(x) = (ax - b)^2$ is $2a(ax - b)$.

Matrix calculus is the exact same thing, with one single rule to remember: **vectors and matrices don't commute ($x^T A \neq Ax$), so you transpose matrices to make the dimensions match the variable you are differentiating with respect to!** In Machine Learning, we always use the **Gradient / Denominator Layout**: if $x \in \mathbb{R}^n$ is a column vector of shape $(n \times 1)$ and $f(x) \in \mathbb{R}$ is a scalar loss, then the gradient $\nabla_x f(x)$ must **also be a column vector of the exact same shape** $(n \times 1)$ so that we can subtract it in Gradient Descent ($x \leftarrow x - \eta \nabla_x f(x)$).

📐 The Math & Every Symbol Explained

Let $x, a \in \mathbb{R}^n$ be column vectors $(n \times 1)$, $b \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, and $W, M \in \mathbb{R}^{n \times m}$ (n rows, m columns):

Scalar Function f	Single-Var Analogy	Matrix Gradient ∇f	Hessian $\nabla^2 f$ & Exam Notes
$f(x) = a^T x = x^T a$	$\frac{d}{dx}(ax) = a$	$\nabla_x f = a$	$\nabla_x^2 f = 0$. Linear dot product.
$f(x) = x^T A x$	$\frac{d}{dx}(ax^2) = 2ax$	$\nabla_x f = (A + A^T)x$	$\nabla_x^2 f = A + A^T$. **Equals $2Ax$ ONLY if $A = A^T$ is symmetric!**
$f(x) = \|Ax - b\|_2^2$	$\frac{d}{dx}(ax - b)^2 = 2a(ax - b)$	$\nabla_x f = 2A^T(Ax - b)$	$\nabla_x^2 f = 2A^T A$. Setting $\nabla_x f = 0 \implies A^T Ax = A^T b$.
$f(W) = \mathrm{Tr}(M^T W)$	$\sum_{i,j} M_{ij} W_{ij}$	$\nabla_W f = M$	$\mathrm{Tr}(M^T W) = \langle M, W \rangle_F$ is just the elementwise dot product of two $n \times m$ matrices!
$f(A) = \log \lvert\det(A)\rvert$	$\frac{d}{dx} \ln x = x^{-1}$	$\nabla_A f = (A^{-1})^T$	For symmetric PD covariance Σ, $\nabla_\Sigma \log\det(\Sigma) = \Sigma^{-1}$ (used in Gaussian MLE).

🔢 Step-by-Step Toy Example

Problem: Let $A = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$ (notice $A \neq A^T$ is NOT symmetric!) and $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. Compute $\nabla_x (x^T A x)$ and verify why the naive formula "$2Ax$" is wrong.

Step 1 — Expand $f(x) = x^T A x$ into scalar algebra to see what happens:

$$f(x) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1^2 + 3x_1 x_2 + 2x_2^2$$

Taking partial derivatives with respect to x_1 and x_2:

$$\nabla_x f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 + 4x_2 \end{bmatrix}$$

Step 2 — Compare $(A + A^T)x$ vs. $2Ax$:

- Using the correct formula $(A + A^T)x$:

$$(A + A^T)x = \left(\begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 3 & 2 \end{bmatrix} \right) \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 + 4x_2 \end{bmatrix} \checkmark$$

- Whereas $2Ax = \begin{bmatrix} 2x_1 + 6x_2 \\ 4x_2 \end{bmatrix}$ is completely wrong because $x_1 x_2$ affects *both* $\frac{\partial f}{\partial x_1}$ and $\frac{\partial f}{\partial x_2}$!

🎯 Exam Takeaway & Fast Shortcuts

- Always check whether the matrix A in $x^T A x$ is specified as **symmetric** ($A = A^T$). If the question says "*For an arbitrary square matrix $A \in \mathbb{R}^{n \times n}$*", the gradient is $(A + A^T)x$ and the Hessian is $A + A^T$!

2.2 Canonical Deep Learning Backprop Derivations (Softmax + CE & Linear Layers)

💡 Intuition (Plain English First)

Two backpropagation gradients show up on almost every Deep Learning assessment:

1. **Softmax + Cross-Entropy Loss ($\frac{\partial \mathcal{L}}{\partial z} = p - y$):** Suppose a classifier outputs raw logits $z = [z_1, \ldots, z_K]^T$, converts them into probabilities $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ via Softmax, and computes Cross-Entropy loss $\mathcal{L} = -\sum_k y_k \log p_k$ against a one-hot target y. Individually, the derivative of Softmax is a messy matrix ($p_k(\delta_{ik} - p_i)$) and the derivative of $-\log p_k$ has a $1/p_k$ that can blow up. Yet when you combine them via the Chain Rule, **the p_k in the numerator and denominator cancel out cleanly**, leaving the most elegant gradient in all of Deep Learning:

$$\nabla_z \mathcal{L} = p - y \quad \text{(Predicted Probability} - \text{True Label!)}$$

2. **Linear Layer Backprop via "Dimension Matching":** In a linear layer $Y = XW + \mathbf{1}b^T$, you receive the upstream gradient $G = \frac{\partial \mathcal{L}}{\partial Y}$ from the layer above and need to find $\frac{\partial \mathcal{L}}{\partial W}$ and $\frac{\partial \mathcal{L}}{\partial X}$. You **never** need to take elementwise derivatives—because matrix multiplication is linear, $\frac{\partial \mathcal{L}}{\partial W}$ must be a product of X and G, and there is **only one way to transpose and multiply them so the matrix dimensions ($n \times m$) match the shape of W!**

📐 The Math & Every Symbol Explained

1. Softmax Jacobian & Cross-Entropy Gradient

Let $z \in \mathbb{R}^K$ be the raw logit vector, $p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ be the Softmax probabilities, and $y \in \mathbb{R}^K$ be any target distribution with $\sum_{k=1}^K y_k = 1$:

- **Softmax Jacobian ($\frac{\partial p_k}{\partial z_i}$):**

$$\frac{\partial p_k}{\partial z_i} = p_k(\delta_{ik} - p_i) = \begin{cases} p_i(1 - p_i) & \text{if } i = k \\ -p_i p_k & \text{if } i \neq k \end{cases}$$

- **Chain Rule with Cross-Entropy $\mathcal{L} = -\sum_{k=1}^K y_k \log p_k$:**

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial p_k}\frac{\partial p_k}{\partial z_i} = -\sum_{k=1}^K \left(\frac{y_k}{p_k}\right) p_k(\delta_{ik} - p_i) = -\sum_{k=1}^K y_k \delta_{ik} + p_i \underbrace{\sum_{k=1}^K y_k}_{1} = \mathbf{p_i} - \mathbf{y_i}$$

- **Numerical Stability: The Log-Sum-Exp Shift Trick (Crucial for FP16/BF16 Hardware!):** If a logit is $z_i = 100$, $e^{100} \approx 2.7 \times 10^{43}$ overflows FP16 (max value $65,504 \to \text{NaN}$). If all logits are $z_i = -100$, all $e^{-100} \to 0$, giving $0/0 = \text{NaN}$. By subtracting the maximum logit $m = \max_k z_k$ from every logit before exponentiating:

$$p_i = \frac{e^{z_i - m}}{\sum_{k=1}^K e^{z_k - m}}, \qquad \log \sum_{k=1}^K e^{z_k} = m + \log \sum_{k=1}^K e^{z_k - m}$$

The probabilities p_i are **100% mathematically identical** (since e^{-m} cancels from top and bottom), and the largest exponent is guaranteed to be $e^0 = 1$—preventing both overflow and underflow!

2. Linear Layer Backpropagation ($Y = XW + \mathbf{1}b^T$)

Let's write down the exact shape (n rows, m columns) of every tensor:

- $X \in \mathbb{R}^{N \times d_{\text{in}}}$: Input batch ($N$ sample rows, d_{in} input feature columns).
- $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$: Weight matrix ($d_{\text{in}}$ rows, d_{out} columns).
- $b \in \mathbb{R}^{d_{\text{out}}}$: Bias vector (broadcast across all N rows via $\mathbf{1} \in \mathbb{R}^N$).
- $Y \in \mathbb{R}^{N \times d_{\text{out}}}$: Output activations, and $G = \frac{\partial \mathcal{L}}{\partial Y} \in \mathbb{R}^{N \times d_{\text{out}}}$: Upstream gradient.

Now match shapes to get the three gradients:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial W}}_{(d_{\text{in}} \times d_{\text{out}})} = \underbrace{X^T}_{(d_{\text{in}} \times N)} \underbrace{G}_{(N \times d_{\text{out}})}, \qquad \underbrace{\frac{\partial \mathcal{L}}{\partial X}}_{(N \times d_{\text{in}})} = \underbrace{G}_{(N \times d_{\text{out}})} \underbrace{W^T}_{(d_{\text{out}} \times d_{\text{in}})}, \qquad \underbrace{\frac{\partial \mathcal{L}}{\partial b}}_{(d_{\text{out}} \times 1)} = \underbrace{G^T}_{(d_{\text{out}} \times N)} \underbrace{\mathbf{1}}_{(N \times 1)}$$

🔢 Step-by-Step Toy Example

Problem: A 3-class classifier outputs logits $z = [0,\ \ln 2,\ \ln 5]^T$ for a sample whose true class is Class 2 ($y = [0, 1, 0]^T$). Compute the exact gradient $\nabla_z \mathcal{L}$ of the Cross-Entropy loss with respect to the logits z.

Step 1 — Compute Softmax probabilities p:

- Exponentiate each logit: $e^0 = 1, \quad e^{\ln 2} = 2, \quad e^{\ln 5} = 5$.
- Sum of exponentials: $1 + 2 + 5 = 8$.
- Therefore:

$$p = \begin{bmatrix} 1/8 \\ 2/8 \\ 5/8 \end{bmatrix} = \begin{bmatrix} 0.125 \\ 0.250 \\ 0.625 \end{bmatrix}$$

Step 2 — Apply $\nabla_z \mathcal{L} = p - y$:

$$\nabla_z \mathcal{L} = p - y = \begin{bmatrix} 0.125 \\ 0.250 \\ 0.625 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} +0.125 \\ -0.750 \\ +0.625 \end{bmatrix}$$

- *(Intuition check: Because Gradient Descent subtracts $\eta \nabla_z \mathcal{L}$, the negative gradient -0.750 on Class 2 will **increase** z_2, while the positive gradients $+0.125$ and $+0.625$ will **push down** the wrong logits z_1 and z_3! Also notice that the sum of the gradient entries is always $\sum_i (p_i - y_i) = 1 - 1 = 0$.)*

🎯 Exam Takeaway & Fast Shortcuts

- **Zero-Sum Gradient Invariant:** Because $\sum_i p_i = 1$ and $\sum_i y_i = 1$, the entries of the Softmax + Cross-Entropy logit gradient $\nabla_z \mathcal{L} = p - y$ **always sum to** 0! On an MCQ, you can immediately eliminate any gradient vector whose entries do not sum to zero.

2.3 Convexity, Gradient Descent Step-Size Limits & Newton's Method

💡 Intuition (Plain English First)

Why do we care if a loss function $f(x)$ is **convex** (bowl-shaped)? Because on a convex surface, **every local minimum is guaranteed to be a global minimum**, and standing somewhere with zero gradient ($\nabla f(x^*) = 0$) guarantees you are at the bottom of the bowl.

Now imagine rolling a ball down a quadratic bowl $f(x) = \frac{1}{2}x^T Q x - b^T x$ using Gradient Descent ($x_{t+1} = x_t - \eta \nabla f(x_t)$):

- What happens if your learning rate η is **too big**? Instead of stepping toward the bottom of the valley, you **jump completely across the valley** to a point higher up on the opposite wall! On the next step, the steeper slope launches you even higher back across, and your loss explodes to ∞.
- Which direction of the bowl determines the maximum safe learning rate $\eta_{\max}$? The **steepest, narrowest wall** of the bowl—which corresponds to the **largest eigenvalue $\lambda_{\max}$ of the Hessian Q**! Specifically, you must keep $\eta < \frac{2}{\lambda_{\max}}$ or Gradient Descent will diverge.

What if the bowl is a **narrow canyon/ravine** where the side walls are super steep ($\lambda_{\max} = 100$) while the floor slopes very gently toward the exit ($\lambda_{\min} = 1$)?

- Because of the steep canyon walls, you are forced to use a tiny learning rate $\eta < \frac{2}{100} = 0.02$ so you don't bounce off the walls.
- But along the flat canyon floor ($\lambda_{\min} = 1$), a step size of 0.02 barely moves! This ratio $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the **condition number**: when $\kappa \gg 1$, vanilla Gradient Descent zig-zags across the steep walls while crawling slowly along the floor.
- **Momentum** fixes this by averaging velocity over time: the back-and-forth zig-zags across the steep walls (+ and - signs) **cancel each other out**, while the consistent steps along the canyon floor (all same sign) **accumulate and accelerate**!
- **Newton's Method** goes even further: it multiplies the gradient by the **inverse Hessian** $[\nabla^2 f(x)]^{-1}$, which stretches the narrow canyon back into a perfectly round circular bowl ($\kappa = 1$) and jumps to the exact bottom of any quadratic bowl in **1 single step**! (Its catch: storing and inverting an $n \times n$ Hessian for a neural network with $n = 10^9$ weights costs $\mathcal{O}(n^2)$ memory and $\mathcal{O}(n^3)$ compute, which is why Deep Learning uses diagonal approximations like Adam instead).

📐 The Math & Every Symbol Explained

1. Rules for Spotting Convex Functions ($f : \mathbb{R}^n \to \mathbb{R}$)

- **Twice-Differentiable Test:** f is convex $\iff$ its Hessian is Positive Semi-Definite ($\nabla^2 f(x) \succeq 0$) everywhere. f is **strictly convex** (unique global minimum!) if $\nabla^2 f(x) \succ 0$ everywhere.
- **3 Operations That Always Preserve Convexity:**
  1. **Non-negative weighted sums:** $\sum_i \alpha_i f_i(x)$ with $\alpha_i \geq 0$.
  2. **Affine composition:** $g(x) = f(Ax + b)$ is convex whenever f is convex.

3. **Pointwise Maximum ($\max_i f_i(x)$):** The maximum of *any* collection of convex/linear functions is **always convex**! *(This immediately proves that **ReLU** $\max(0,x)$, **SVM Hinge Loss** $\max(0,1-yw^Tx)$, L_1 **norm** $\sum_i \max(x_i,-x_i)$, and L_∞ **norm** $\max_i |x_i|$ are all convex!)*

2. Gradient Descent Stability on a Quadratic $f(x)=\frac{1}{2}x^TQx-b^Tx$ ($Q\succ 0$)

- Here $\nabla f(x)=Qx-b$ and Hessian $\nabla^2 f(x)=Q$ has eigenvalues $0<\lambda_{\min}\le\cdots\le\lambda_{\max}$.
- One step of Gradient Descent $x_{t+1}=x_t-\eta(Qx_t-b)$ shrinks the error vector $e_t=x_t-x^*$ by:

$$e_{t+1}=(I-\eta Q)e_t$$

- Along the i-th eigenvector of Q, the error is multiplied by $(1-\eta\lambda_i)$ at every step:
  - $0<\eta\le\frac{1}{\lambda_{\max}}$**:** Monotonic convergence (no overshooting).
  - $\frac{1}{\lambda_{\max}}<\eta<\frac{2}{\lambda_{\max}}$**:** Damped oscillations (jumps past x^* each step, but gets closer: $-1<1-\eta\lambda_{\max}<0$).
  - $\eta\ge\frac{2}{\lambda_{\max}}$**: Divergence!** ($|1-\eta\lambda_{\max}|\ge 1$).

$$\text{Mandatory Stability Condition:}\quad \mathbf{0<\eta<\frac{2}{\lambda_{\max}}}$$

3. Why L_1 (Lasso) Creates Exact Zeros While L_2 (Ridge) Only Shrinks

- L_2 **Penalty ($\frac{\lambda}{2}w^2$):** Derivative is λw, which shrinks toward 0 as $w\to 0$. So L_2 slows down as it approaches zero and never locks weights to exact 0.
- L_1 **Penalty ($\lambda|w|$):** Has a sharp V-shaped corner at $w=0$ with constant slope $+\lambda$ on the right and $-\lambda$ on the left. Unless the loss gradient $|\nabla f_0(0)|$ pulls harder than the threshold λ, the weight gets trapped in the V-corner at **exact** $w^*=0$!

▼ 📌 Optional Deep-Dive / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact Optimal Step Size & Formal Subdifferential ∂|0| = [-1, 1]

- **Optimal Constant Learning Rate:** To minimize $\max_i|1-\eta\lambda_i|$, make the slowest and fastest modes have equal magnitude: $1-\eta^*\lambda_{\min}=-(1-\eta^*\lambda_{\max})\implies\eta^*=\frac{2}{\lambda_{\min}+\lambda_{\max}}$, which gives per-step contraction factor $\frac{\kappa-1}{\kappa+1}$ where $\kappa=\frac{\lambda_{\max}}{\lambda_{\min}}$.
- **Formal Subdifferential of $|w|$:** At a non-differentiable corner, the subdifferential $\partial f(w)$ is the set of slopes of all lines tangent below f at w. For $|w|$, $\partial|w|=\{\text{sign}(w)\}$ for $w\neq 0$, and at $w=0$ any slope in $[-1,1]$ stays below $|w|$, so $\partial|0|=[-1,1]$. Zero is in the subdifferential of $f_0(w)+\lambda|w|$ at $w=0\iff 0\in\nabla f_0(0)+\lambda[-1,1]\iff|\nabla f_0(0)|\le\lambda$.

🔢 Step-by-Step Toy Example

Problem: Consider minimizing $f(x_1,x_2)=5x_1^2+50x_2^2$ using Gradient Descent $x_{t+1}=x_t-\eta\nabla f(x_t)$.

1. Find the Hessian matrix $Q=\nabla^2 f(x)$ and its eigenvalues $\lambda_{\min},\lambda_{\max}$ and condition number κ.
2. Find the maximum learning rate $\eta_{\max}$ before Gradient Descent diverges.

Step 1 — Compute Gradient & Hessian Q:

- $\nabla f(x) = \begin{bmatrix} 10x_1 \\ 100x_2 \end{bmatrix}$.
- Taking second derivatives gives the diagonal Hessian:

  $$Q = \nabla^2 f(x) = \begin{bmatrix} 10 & 0 \\ 0 & 100 \end{bmatrix}$$

  (Watch out: because $f(x) = \frac{1}{2}x^T Q x$, a term ax_i^2 has second derivative $2a$, so $5x_1^2 \implies \lambda_1 = 10$ and $50x_2^2 \implies \lambda_2 = 100$!)
- Condition number: $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{100}{10} = \mathbf{10}$.

Step 2 — Maximum Safe Learning Rate $\eta_{\max}$:

$$\eta < \frac{2}{\lambda_{\max}} = \frac{2}{100} = \mathbf{0.02}$$

- Let's verify why $\eta = 0.02$ is the exact tipping point: along x_2, the update is $x_2^{(t+1)} = x_2^{(t)} - \eta(100x_2^{(t)}) = (1 - 100\eta)x_2^{(t)}$. If $\eta = 0.02$, then $1 - 100(0.02) = -1$, so x_2 bounces endlessly between $+x_2^{(0)}$ and $-x_2^{(0)}$!

🎯 Exam Takeaway & Fast Shortcuts

- **The $\frac{1}{2}$ Coefficient Trap:** When given a quadratic $f(x_1, x_2) = ax_1^2 + bx_2^2$ without a $\frac{1}{2}$ in front, remember that the Hessian eigenvalues are $2a$ **and** $2b$ (not a and b!). Therefore, $\eta_{\max} = \frac{2}{\lambda_{\max}} = \frac{2}{2\max(a,b)} = \frac{1}{\max(a,b)}$.

2.4 Constrained Optimization: Lagrange Multipliers & KKT Conditions

💡 Intuition (Plain English First)

What if you want to minimize a function $f(x)$, but you aren't allowed to go anywhere you want—you must stay inside a fenced region defined by an **inequality constraint** $g(x) \leq 0$ (and/or on a curve defined by an **equality constraint** $h(x) = 0$)?

Let's visualize the two things that can happen when you minimize $f(x)$ subject to $g(x) \leq 0$:

- **Case 1 — The Unconstrained Minimum is Already Inside the Fence ($g(x^*) < 0$, "Inactive Constraint"):** If the bottom of the bowl $f(x)$ is already safely inside the allowed region, **the fence doesn't do anything**! At the optimum x^*, $\nabla f(x^*) = 0$, and the Lagrange multiplier (which measures how hard the fence has to push back against you) is $\mu^* = 0$.
- **Case 2 — The Unconstrained Minimum is Outside the Fence ($g(x^*) = 0$, "Active Constraint"):** You roll downhill until you **slam against the boundary wall** $g(x^*) = 0$. Why can't you slide any lower along the wall? Because at the optimal point x^* on the wall, the direction of steepest descent $-\nabla f(x^*)$ points **straight into the wall**, parallel to the outward normal vector $\nabla g(x^*)$! That means $-\nabla f(x^*) = \mu^* \nabla g(x^*) \implies \nabla f(x^*) + \mu^* \nabla g(x^*) = 0$ with a positive push-back force $\mu^* > 0$.

Notice that in **both** cases:

- Either $g(x^*) < 0$ and $\mu^* = 0$ (inside the fence, zero push-back force),

- Or $g(x^*) = 0$ and $\mu^* \geq 0$ (on the fence wall, positive push-back force). In either case, their product is **always zero**: $\mu^* g(x^*) = 0$. That single equation is called **Complementary Slackness**, and it is the exact mathematical reason why **Support Vector Machines (SVMs) are sparse**!

📐 The Math & Every Symbol Explained

For the constrained optimization problem:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_i(x) \leq 0 \;\; (i = 1, \ldots, m), \quad h_j(x) = 0 \;\; (j = 1, \ldots, p)$$

We form the **Lagrangian function**:

$$\mathcal{L}(x, \mu, \lambda) = f(x) + \sum_{i=1}^{m} \mu_i g_i(x) + \sum_{j=1}^{p} \lambda_j h_j(x)$$

- μ_i **(KKT / Inequality Multipliers):** Penalize violating the boundary walls $g_i(x) \leq 0$.
- λ_j **(Lagrange / Equality Multipliers):** Enforce staying on the curves $h_j(x) = 0$ (can be positive, negative, or zero).

The 4 Karush-Kuhn-Tucker (KKT) Optimality Conditions

At an optimal solution (x^*, μ^*, λ^*):

1. **Stationarity (Forces Balance):** $\nabla_x \mathcal{L}(x^*, \mu^*, \lambda^*) = \nabla f(x^*) + \sum_i \mu_i^* \nabla g_i(x^*) + \sum_j \lambda_j^* \nabla h_j(x^*) = 0$.
2. **Primal Feasibility (Obeys Constraints):** $g_i(x^*) \leq 0$ and $h_j(x^*) = 0$.
3. **Dual Feasibility (One-Way Fence):** $\mu_i^* \geq 0$ for all inequality constraints! (Equality multipliers λ_j^* have no sign restriction).
4. **Complementary Slackness:** $\mu_i^* g_i(x^*) = 0$ for every i.
   - **Connection to SVMs:** In an SVM, each training point x_i gives a margin constraint $g_i(w) = 1 - y_i(w^T x_i + b) \leq 0$ with multiplier $\alpha_i^* \geq 0$. By Complementary Slackness $\alpha_i^* \big(1 - y_i(w^T x_i + b)\big) = 0$:
     - Any training point safely outside the margin ($y_i(w^T x_i + b) > 1 \implies g_i(w) < 0$) is **forced to have** $\alpha_i^* = 0$ (it plays zero role in the final classifier $w^* = \sum_i \alpha_i^* y_i x_i$!).
     - Only the **Support Vectors** sitting on (or inside, in Soft-Margin SVM) the margin have $\alpha_i^* > 0$.

🔢 Step-by-Step Toy Example

Problem: Minimize $f(x_1, x_2) = (x_1 - 3)^2 + (x_2 - 4)^2$ (squared distance to the point $(3, 4)$) subject to staying inside the disk $x_1^2 + x_2^2 \leq 4$ (i.e., $g(x_1, x_2) = x_1^2 + x_2^2 - 4 \leq 0$).

Step 1 — Check Case 1 (Is the unconstrained minimum inside the fence?):

- The unconstrained minimum of f is $(3, 4)$.
- Plug $(3, 4)$ into g: $3^2 + 4^2 - 4 = 25 - 4 = 21 > 0$ (outside the disk of radius 2!).
- Therefore, the constraint **must be active** ($g(x^*) = 0$, **so** $x_1^2 + x_2^2 = 4$) and $\mu^* > 0$.

Step 2 — Use Stationarity ($\nabla f(x^*) + \mu^* \nabla g(x^*) = 0$):

- Geometrically, the closest point on the circle of radius 2 to $(3,4)$ (which is at distance $\sqrt{3^2+4^2}=5$ from the origin) lies along the straight ray from $(0,0)$ to $(3,4)$:

$$x^* = \frac{2}{5}\begin{bmatrix}3\\4\end{bmatrix} = \begin{bmatrix}1.2\\1.6\end{bmatrix}$$

- Check $\nabla f(x^*) + \mu^* \nabla g(x^*) = 0$ at $x^* = (1.2, 1.6)$:

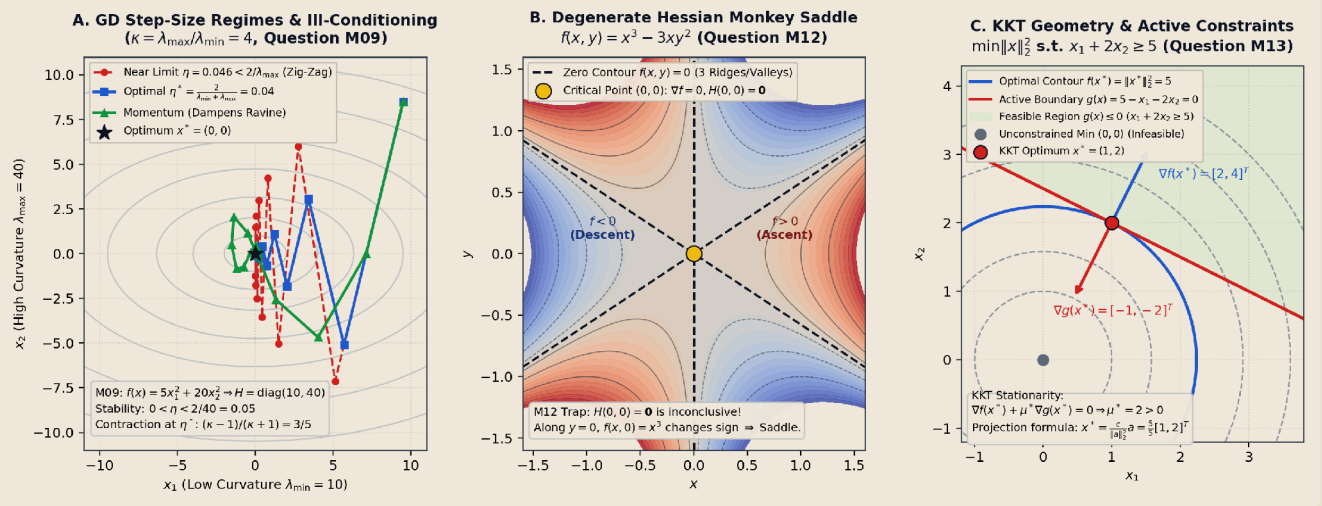
$$\nabla f(x^*) = 2\begin{bmatrix}1.2-3\\1.6-4\end{bmatrix} = \begin{bmatrix}-3.6\\-4.8\end{bmatrix}, \qquad \nabla g(x^*) = 2\begin{bmatrix}1.2\\1.6\end{bmatrix} = \begin{bmatrix}2.4\\3.2\end{bmatrix}$$

$$\begin{bmatrix}-3.6\\-4.8\end{bmatrix} + \mu^*\begin{bmatrix}2.4\\3.2\end{bmatrix} = 0 \implies \mu^* = \frac{3.6}{2.4} = \mathbf{1.5} > \mathbf{0}\ \checkmark$$

🎯 Exam Takeaway & Fast Shortcuts

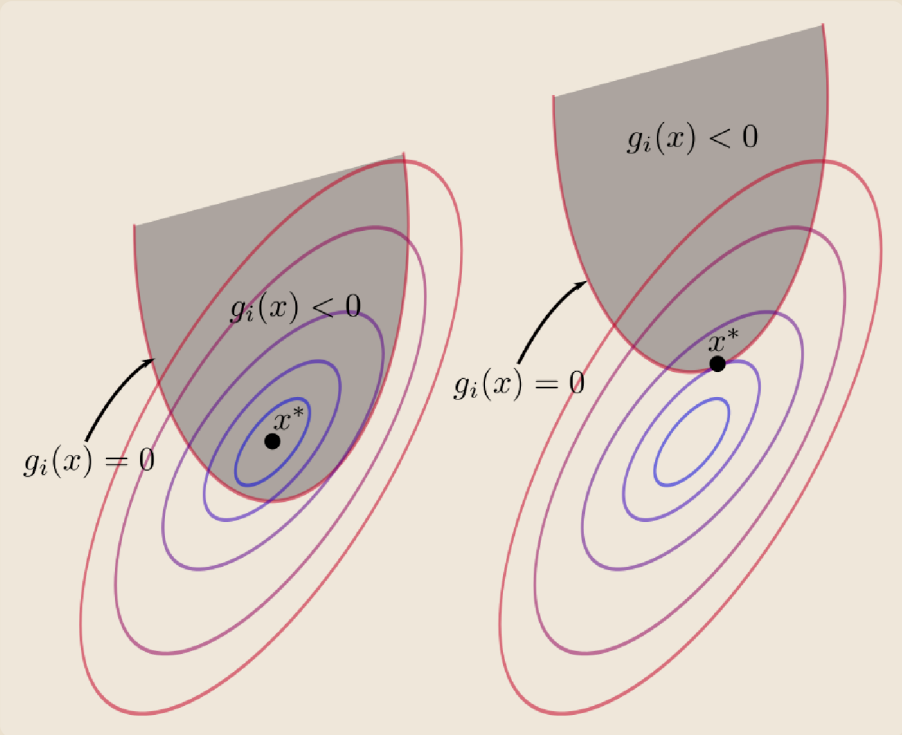
- Always check the **unconstrained optimum first** on any KKT problem! If the unconstrained minimum already satisfies $g(x) \le 0$, you are done in 5 seconds ($\mu^* = 0$). If it violates $g(x) \le 0$ (for a convex problem), you immediately know the solution lies **on the boundary** $g(x^*) = 0$.

FIGURE SLIDE 1 OF 4



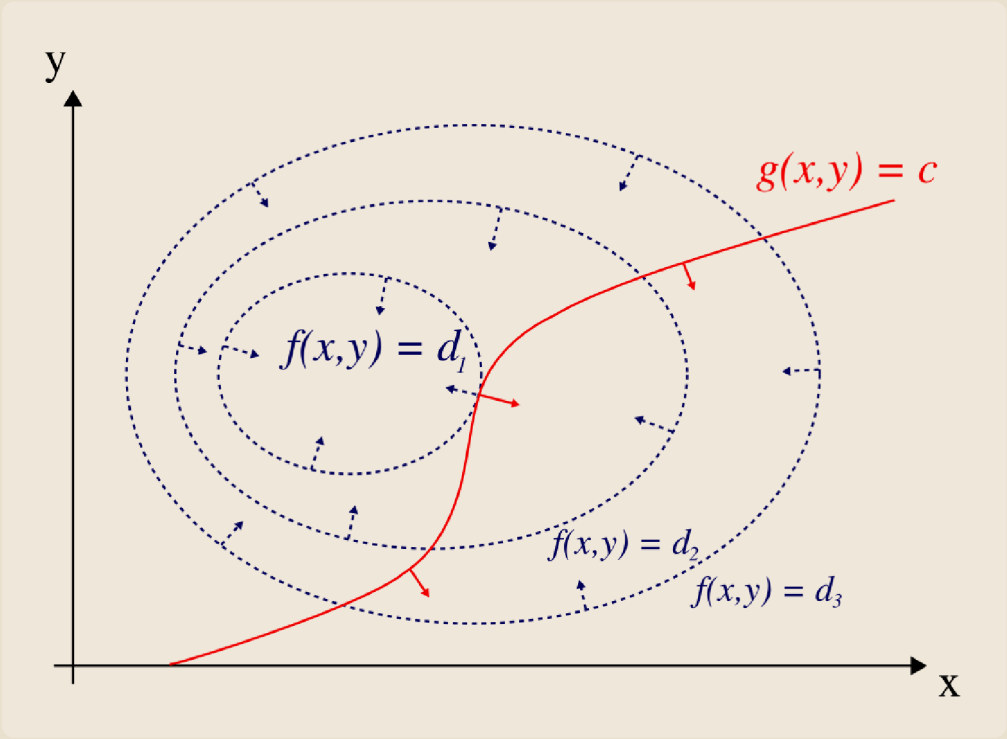
Day 2 Calculus & Optimization Visual Cheat-Sheet: GD Step-Size Regimes, Monkey Saddle, and KKT Geometry

FIGURE SLIDE 2 OF 4



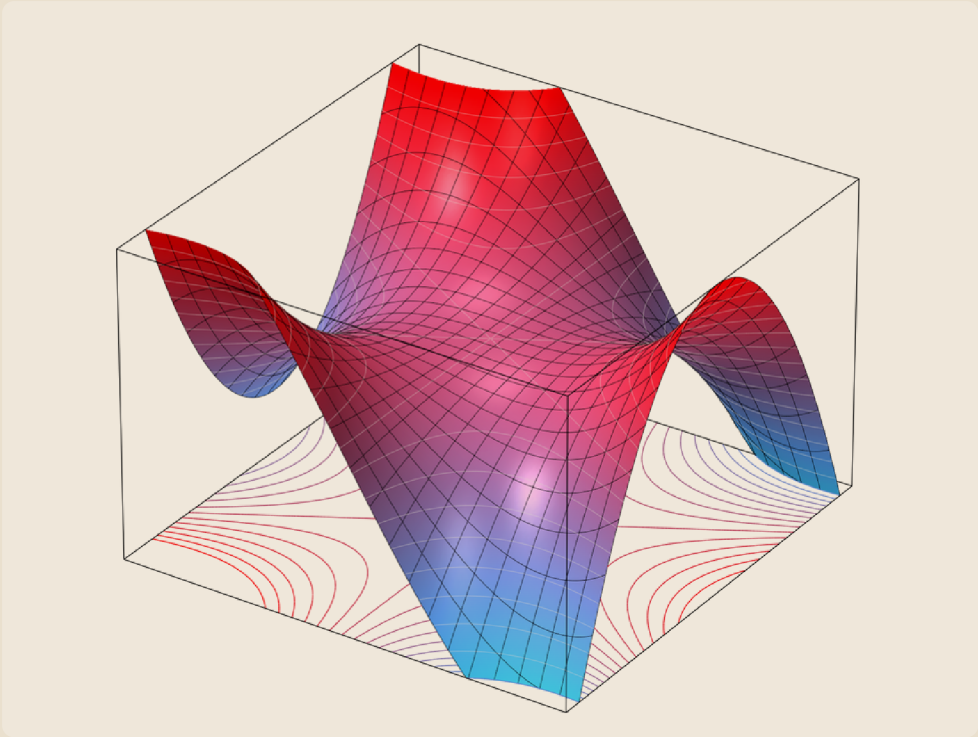
Wikimedia Commons — KKT Complementary Slackness: Inactive Interior Optimum (g_i(x) < 0, μ_i* = 0) vs. Active Boundary Optimum (g_i(x*) = 0, μ_i* > 0)*

FIGURE SLIDE 3 OF 4



Wikimedia Commons — Lagrange Multiplier Tangency: Contour f(x,y) = d_1 Tangent to Constraint Curve g(x,y) = c Where Gradients Align

FIGURE SLIDE 4 OF 4



Wikimedia Commons — 3D Shaded Monkey Saddle Surface z = x^3 - 3xy^2 with Degenerate Zero Hessian at (0,0)

Day 3: Probability, Statistics & Information Theory

↑ Mục Lục

3.1 Conditional Probability, Bayes' Theorem & The Odds-Ratio Speed Hack

💡 Intuition (Plain English First)

Why do even smart engineers get tricked by Bayes' Theorem questions (like medical test or spam filter false-positive problems)? Because human intuition forgets the **Base Rate (Prior)**: if a disease only affects 1 out of 100 people (1% prevalence), and a test has a 9% false-positive rate, then out of 100 people:

- Only 1 **person** actually has the disease (and tests positive).
- Out of the 99 **healthy people**, 9% of them (≈ 9 people) *also* get a false positive! So out of the $1 + 9 = 10$ people who got a positive test result, only 1 **out of** 10 **(10%)** is actually sick!

Instead of plugging decimals into $P(D \mid +) = \frac{P(+|D)P(D)}{P(+|D)P(D)+P(+|\neg D)P(\neg D)}$ under time pressure, use the **Odds-Ratio Form of Bayes' Rule**, which turns messy decimal division into a 10-second integer multiplication.

📐 The Math & Every Symbol Explained

1. Standard Bayes' Rule & Law of Total Probability

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)} = \frac{P(B \mid A)P(A)}{P(B \mid A)P(A) + P(B \mid A^c)P(A^c)}$$

- $P(A)$ **(Prior):** Probability of A before seeing evidence B.
- $P(B \mid A)$ **(Likelihood / Sensitivity / True Positive Rate):** Probability of observing evidence B when A is true.
- $P(B \mid A^c)$ **(False Positive Rate** $= 1 -$ Specificity**):** Probability of observing evidence B when A is false.
- $P(A \mid B)$ **(Posterior):** Updated probability that A is true after observing B.

2. The 10-Second Odds-Ratio Speed Hack

Divide $P(D \mid +)$ by $P(\neg D \mid +)$ and the denominator $P(+)$ cancels out completely:

$$\underbrace{\frac{P(D \mid +)}{P(\neg D \mid +)}}_{\text{Posterior Odds}} = \underbrace{\frac{P(+ \mid D)}{P(+ \mid \neg D)}}_{\text{Likelihood Ratio}} \times \underbrace{\frac{P(D)}{P(\neg D)}}_{\text{Prior Odds}}$$

Once you have the Posterior Odds as a ratio $a : b$, the probability is simply $P(D \mid +) = \frac{a}{a+b}$!

🔢 Step-by-Step Toy Example

Problem: A defective chip occurs in 1% of factory batches ($P(D) = 0.01$). An automated camera detects a defective batch 99% of the time ($P(+ \mid D) = 0.99$), and triggers a false alarm on a good batch 9% of the time ($P(+ \mid \neg D) = 0.09$). If the alarm goes off, what is $P(D \mid +)$?

Step 1 — Write down Prior Odds and Likelihood Ratio:

- Prior Odds: $\frac{P(D)}{P(\neg D)} = \frac{0.01}{0.99} = \frac{1}{99}$.
- Likelihood Ratio: $\frac{P(+|D)}{P(+|\neg D)} = \frac{0.99}{0.09} = 11$.

Step 2 — Multiply to get Posterior Odds:

$$\text{Posterior Odds} = 11 \times \frac{1}{99} = \frac{11}{99} = \frac{1}{9} \quad (1:9)$$

Step 3 — Convert Odds $1 : 9$ **to Probability:**

$$P(D \mid +) = \frac{1}{1+9} = \frac{1}{10} = \mathbf{10\%}$$

🎯 Exam Takeaway & Fast Shortcuts

- Watch out for wording: "**95% Specificity**" means True Negative Rate $P(- \mid \neg D) = 0.95$, which means the **False Positive Rate is** $P(+ \mid \neg D) = 1 - 0.95 = 0.05$!

3.2 Core Probability Distributions & Their ML Roles

💡 Intuition (Plain English First)

Each standard probability distribution models a specific physical process in Machine Learning:

- **Bernoulli & Binomial:** Flipping a biased coin once (Bernoulli(p)) or n times (Binomial(n, p))—used in binary classification and **Dropout masks**.
- **Poisson(λ) & Exponential(λ):** Imagine packets arriving at an NPU at an average rate of λ packets per millisecond:
  - **Poisson(λ)** counts the *discrete number of packets* arriving in 1 ms (uniquely, its **Mean equals its Variance** $= \lambda$!).
  - **Exponential(λ)** measures the *continuous waiting time* until the next packet arrives. It is the **only continuous memoryless distribution**: $P(T > s + t \mid T > s) = P(T > t)$ (having waited s seconds already gives you zero advantage—the expected remaining wait time is still $1/\lambda$!).
- **Gaussian $\mathcal{N}(\mu, \sigma^2)$ vs. Laplace(μ, b):**
  - **Gaussian** has a squared penalty $-\frac{(x-\mu)^2}{2\sigma^2}$ in the exponent—putting a Gaussian prior on weights w creates L_2 **(Ridge) regularization**. Among all continuous distributions on $\mathbb{R}$ with fixed variance σ^2, the Gaussian has the **maximum possible Shannon differential entropy**.
  - **Laplace** has an absolute-value penalty $-\frac{|x-\mu|}{b}$ in the exponent (a sharp tent peak at μ with heavier tails)—putting a Laplace prior on weights w creates L_1 **(Lasso) sparsity**!

📐 The Math & Reference Table

Distribution	PMF / PDF	Mean $\mathbb{E}[X]$	Variance $\mathrm{Var}(X)$	Key ML / Exam Property
Bernoulli(p)	$p^x(1-p)^{1-x}$ $(x \in \{0,1\})$	p	$p(1-p)$	Max variance $= 0.25$ at $p = 0.5$
Binomial(n, p)	$\binom{n}{k}p^k(1-p)^{n-k}$	np	$np(1-p)$	Sum of n i.i.d. Bernoulli(p) trials
Poisson(λ)	$\frac{\lambda^k e^{-\lambda}}{k!}$ $(k = 0, 1, 2, \dots)$	λ	λ	**Mean = Variance** $= \lambda$; sum $\mathrm{Pois}(\lambda_1) + \mathrm{Pois}(\lambda_2) = \mathrm{Pois}(\lambda_1 + \lambda_2)$
Uniform(a, b)	$\frac{1}{b-a}$ $(a \le x \le b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	Used in Xavier/He Uniform weight init ($\pm\sqrt{3 \cdot \mathrm{Var}}$)
Exponential(λ)	$\lambda e^{-\lambda x}$ $(x \ge 0)$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	**Memoryless:** $P(X > s + t \mid X > s) = P(X > t) = e^{-\lambda t}$
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2	Prior $\Longrightarrow$ L_2 **(Ridge)**; Likelihood $\Longrightarrow$ **MSE loss**
Laplace(μ, b)	$\frac{1}{2b}e^{-\frac{\lvert x-\mu\rvert}{b}}$	μ	$2b^2$	Prior $\Longrightarrow$ L_1 **(Lasso)**; Likelihood $\Longrightarrow$ **MAE (L_1) loss**

🔢 Step-by-Step Toy Example

Problem: You want to initialize a neural network weight $W \sim \text{Uniform}(-a, a)$ so that its mean is 0 and its variance is $\text{Var}(W) = \sigma^2$. Solve for the bound a in terms of σ.

Step 1 — Plug $[-a, a]$ into the Uniform variance formula $\frac{(b-a)^2}{12}$:

$$\text{Var}(W) = \frac{(a-(-a))^2}{12} = \frac{(2a)^2}{12} = \frac{4a^2}{12} = \frac{a^2}{3}$$

Step 2 — Set $\frac{a^2}{3} = \sigma^2$ and solve for a:

$$\frac{a^2}{3} = \sigma^2 \implies a^2 = 3\sigma^2 \implies \mathbf{a = \sqrt{3}\,\sigma}$$

(This is why Xavier and Kaiming/He Uniform initializers always have an extra $\sqrt{3}$ inside the square root compared to their Normal counterparts!)

🎯 Exam Takeaway & Fast Shortcuts

- Memorize the 12 **in the denominator** of Uniform variance $\frac{(b-a)^2}{12}$ and the $2b^2$ variance of $\text{Laplace}(\mu, b)$—both are frequent 30-second calculation questions.

3.3 Expectation, Variance, Correlated Ensembles & Probability Inequalities

💡 Intuition (Plain English First)

Three rules about Expectation and Variance govern almost every statistical derivation in ML:

1. **Linearity of Expectation ($\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$) Is Magic:** It holds **even when X and Y are completely dependent!** Whether X and Y are independent coin flips or 100% correlated variables, the average of a sum is *always* the sum of the averages.
2. **Variance Squares Constants—So Variances ADD Even When Subtracting!** Because variance measures squared spread $\mathbb{E}[(X-\mu)^2]$, multiplying X by a constant a scales variance by a^2: $\text{Var}(aX) = a^2\text{Var}(X)$. What happens if $a = -1$? $(-1)^2 = +1$! So if X and Y are independent, subtracting two noisy measurements $X - Y$ does **not** cancel their noise—it **adds** their variances:

   $$\text{Var}(X-Y) = (1)^2\text{Var}(X) + (-1)^2\text{Var}(Y) = \text{Var}(X) + \text{Var}(Y)$$

3. **Why Random Forests Subsample Features (Averaging Correlated Trees):** If you average n *independent* models each with variance σ^2, the ensemble variance drops to $\frac{\sigma^2}{n}$, which goes to 0 as $n \to \infty$. What if the n models (like decision trees trained on bootstrapped copies of the same dataset) have pairwise correlation $\rho > 0$? Then averaging them gives:

   $$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$$

   Notice that as you add infinitely many trees ($n \to \infty$), the second term $\frac{1-\rho}{n}\sigma^2$ goes to 0, **but the first term $\rho\sigma^2$ never goes away**! That irreducible floor $\rho\sigma^2$ is why **Random Forest** randomly restricts each split to a subset of $m \approx \sqrt{d}$ features—to de-correlate the trees (reduce ρ) and lower the variance floor!

📐 The Math & Every Symbol Explained

1. Expectation & Variance of Linear Combinations

$$\mathbb{E}\left[\sum_{i=1}^{n} a_i X_i\right] = \sum_{i=1}^{n} a_i \mathbb{E}[X_i] \quad \text{(Holds ALWAYS, even if dependent!)}$$

$$\text{Var}\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i^2 \text{Var}(X_i) + 2 \sum_{1 \le i < j \le n} a_i a_j \text{Cov}(X_i, X_j)$$

- **Covariance:** $\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$.
- **Pearson Correlation:** $\rho(X, Y) = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.
- **Independence vs. Zero Correlation Trap:**

  $$X \perp Y \implies \text{Cov}(X, Y) = 0, \qquad \text{but} \qquad \text{Cov}(X, Y) = 0 \not\implies X \perp Y!$$

  *(Classic Counterexample: Let $X \sim \mathcal{N}(0,1)$ or $X \in \{-1, 0, 1\}$ symmetrically, and let $Y = X^2$. Clearly Y is 100% determined by X, yet $\text{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0 - 0 = 0$ because correlation only measures **linear** relationships!)*

2. Law of Total Expectation & Law of Total Variance (Eve's Law)

$$\mathbb{E}[X] = \mathbb{E}_Y\big[\mathbb{E}[X \mid Y]\big], \qquad \text{Var}(X) = \mathbb{E}_Y\big[\text{Var}(X \mid Y)\big] + \text{Var}_Y\big(\mathbb{E}[X \mid Y]\big)$$

3. The 3 Essential Probability Inequalities

1. **Jensen's Inequality:**
   - If g is **convex** (e.g., $g(x) = x^2$ or e^x): $\mathbb{E}[g(X)] \ge g(\mathbb{E}[X])$. *(Notice $\mathbb{E}[X^2] \ge (\mathbb{E}[X])^2$ immediately proves $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \ge 0$!)*
   - If g is **concave** (e.g., $g(x) = \log x$): $\mathbb{E}[\log X] \le \log(\mathbb{E}[X])$. *(This is the 1-line proof that KL Divergence $D_{\text{KL}}(P \parallel Q) \ge 0$ and the VAE/Diffusion ELBO bound!)*
2. **Markov's Inequality (for any non-negative $X \ge 0$ and $a > 0$):**

   $$P(X \ge a) \le \frac{\mathbb{E}[X]}{a}$$

3. **Chebyshev's Inequality (for any X with mean μ and variance σ^2):**

   $$P(|X - \mu| \ge k\sigma) \le \frac{1}{k^2}$$

🔢 Step-by-Step Toy Example

Problem: An ensemble averages $n = 100$ regression trees. Each tree has prediction variance $\sigma^2 = 10$, and the pairwise correlation between any two trees is $\rho = 0.25$.

1. What is the variance of the 100-tree ensemble average $\bar{X}$?
2. What is the lowest possible variance achievable if you increase the number of trees to $n = \infty$?

Step 1 — Derive or Apply the Correlated Average Formula:

$$\text{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\Big[\underbrace{n\sigma^2}_{n \text{ diagonal terms}} + \underbrace{n(n-1)\rho\sigma^2}_{n(n-1) \text{ off-diagonal pairs}}\Big] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$$

Step 2 — Plug in $n = 100, \sigma^2 = 10, \rho = 0.25$:

$$\text{Var}(\bar{X}) = (0.25)(10) + \frac{1 - 0.25}{100}(10) = 2.5 + 0.075 = \mathbf{2.575}$$

Step 3 — Limit as $n \to \infty$:

$$\lim_{n\to\infty} \text{Var}(\bar{X}) = \rho\sigma^2 = (0.25)(10) = \mathbf{2.5}$$

🎯 Exam Takeaway & Fast Shortcuts

- If $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent, then $X - Y \sim \mathcal{N}(\mu_1 - \mu_2,\ \sigma_1^2 + \sigma_2^2)$. Never subtract the variances!

3.4 Maximum Likelihood (MLE) vs. MAP & Estimator Bias

💡 Intuition (Plain English First)

Suppose you have an unknown parameter θ (like the probability of heads on a coin, or the weights of a neural network) and observed data $D = \{x_1, \ldots, x_n\}$:

- **Maximum Likelihood Estimation (MLE):** Asks "*What value of θ makes the data D we just observed as probable as possible?*"

  $$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} P(D \mid \theta) = \arg\max_{\theta} \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

  MLE relies **100% on the observed data**, which can overfit badly when n is tiny (e.g., if you flip a coin $n = 2$ times and get 2 heads, MLE confidently claims the coin has $\hat{\theta}_{\text{MLE}} = 2/2 = 100\%$ chance of heads forever!).

- **Maximum A Posteriori (MAP):** Combines the data likelihood $P(D \mid \theta)$ with a **Prior belief** $P(\theta)$ via Bayes' Rule:

  $$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} P(\theta \mid D) = \arg\max_{\theta} \Big[\log P(D \mid \theta) + \log P(\theta)\Big]$$

  Notice what $\log P(\theta)$ does: it acts as a **regularization penalty** that pulls $\hat{\theta}_{\text{MAP}}$ toward the prior when n is small, and gradually fades away as $n \to \infty$!

What does it mean for an estimator $\hat{\theta}$ to be **biased**?

- **Bias:** $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$ (whether $\hat{\theta}$ systematically undershoots or overshoots θ on average).
- **Why is the Gaussian MLE variance $\hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ biased downward ($\mathbb{E}[\hat{\sigma}^2_{\text{MLE}}] = \frac{n-1}{n}\sigma^2$)?** Because the sample mean $\bar{x}$ is calculated from the *exact same data points* $x_1, \ldots, x_n$, so $\bar{x}$ sits slightly closer to those specific points x_i than the true population mean μ does! Dividing by $n-1$ instead of n (**Bessel's Correction**) compensates for the 1 degree of freedom used to estimate $\bar{x}$.

📐 The Math & Every Symbol Explained

1. Estimator MSE = Variance + Bias²

For any estimator $\hat{\theta}$ of a true parameter θ:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}\big[(\hat{\theta} - \theta)^2\big] = \text{Var}(\hat{\theta}) + \big[\text{Bias}(\hat{\theta})\big]^2$$

2. Gaussian MLE ($\mathcal{N}(\mu, \sigma^2)$) vs. Unbiased Sample Variance

$$\hat{\mu}_{\text{MLE}} = \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \;\; (\text{Unbiased: } \mathbb{E}[\bar{x}] = \mu), \qquad \hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 \;\; \left(\text{Biased: } \mathbb{E}[\hat{\sigma}^2_{\text{MLE}}] = \frac{n-1}{n}\sigma^2\right)$$

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \;\; \big(\text{Unbiased: } \mathbb{E}[S^2] = \sigma^2\big)$$

3. Uniform$(0, \theta)$ MLE (The Classic Boundary Exam Problem)

Suppose $x_1, \ldots, x_n \overset{\text{i.i.d.}}{\sim} \text{Uniform}(0, \theta)$. The density of each point is $\frac{1}{\theta}$ **only if** $\theta \geq x_i$ for every i (i.e., $\theta \geq \max_i x_i$). Thus the likelihood is:

$$L(\theta) = \frac{1}{\theta^n} \cdot \mathbb{I}\big(\theta \geq \max(x_1, \ldots, x_n)\big)$$

- **Why setting $\frac{dL}{d\theta} = 0$ fails:** $\frac{1}{\theta^n}$ is strictly decreasing! To make $\frac{1}{\theta^n}$ as large as possible while still satisfying $\theta \geq \max_i x_i$, we must make θ as small as legally allowed:

  $$\hat{\theta}_{\text{MLE}} = \max(x_1, \ldots, x_n), \qquad \mathbb{E}[\hat{\theta}_{\text{MLE}}] = \frac{n}{n+1}\theta \;\; (\text{Biased downward!})$$

4. Beta-Bernoulli Conjugate Prior (Coin Flip MAP vs. Posterior Mean)

Suppose a coin has unknown bias $\theta \in [0, 1]$ with a $\text{Beta}(\alpha, \beta)$ prior ($p(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$), and we observe k **heads in** n **flips**:

- **Posterior Distribution:** $\theta \mid D \sim \text{Beta}(\alpha + k,\ \beta + n - k)$. *(Intuition: The prior* $\text{Beta}(\alpha, \beta)$ *acts like starting with* $\alpha - 1$ ***imaginary prior heads*** *and* $\beta - 1$ ***imaginary prior tails****!)*
- **Three Estimators to Know Cold:**

$$\hat{\theta}_{\text{MLE}} = \frac{k}{n}, \qquad \hat{\theta}_{\text{MAP}} = \frac{k+\alpha-1}{n+\alpha+\beta-2} \text{ (Mode)}, \qquad \mathbb{E}[\theta \mid D] = \frac{k+\alpha}{n+\alpha+\beta} \text{ (Posterior Mean)}$$

🔢 Step-by-Step Toy Example

Problem: You place a $\text{Beta}(\alpha = 3, \beta = 3)$ prior on a coin's bias θ. You flip the coin $n = 4$ times and get $k = 4$ heads. Compute $\hat{\theta}_{\text{MLE}}$, $\hat{\theta}_{\text{MAP}}$, and the Posterior Mean $\mathbb{E}[\theta \mid D]$.

Step 1 — MLE (Data Only):

$$\hat{\theta}_{\text{MLE}} = \frac{k}{n} = \frac{4}{4} = \mathbf{1.0}$$

Step 2 — MAP (Mode of $\text{Beta}(3+4, 3+0) = \text{Beta}(7, 3)$):

- $\text{Beta}(3, 3)$ adds $\alpha - 1 = 2$ imaginary heads and $\beta - 1 = 2$ imaginary tails (4 imaginary flips total):

$$\hat{\theta}_{\text{MAP}} = \frac{4 + (3-1)}{4 + (3+3-2)} = \frac{6}{8} = \mathbf{0.75}$$

Step 3 — Posterior Mean (Expected Value of $\text{Beta}(7, 3)$):

$$\mathbb{E}[\theta \mid D] = \frac{4+3}{4+3+3} = \frac{7}{10} = \mathbf{0.70}$$

🎯 Exam Takeaway & Fast Shortcuts

- Don't confuse **MAP (Posterior Mode)** $\frac{k+\alpha-1}{n+\alpha+\beta-2}$ with **Posterior Mean** $\frac{k+\alpha}{n+\alpha+\beta}$! When $\alpha = \beta = 1$ (Uniform$(0, 1)$ flat prior), $\hat{\theta}_{\text{MAP}} = \frac{k}{n} = \hat{\theta}_{\text{MLE}}$, whereas the Posterior Mean is Laplace's Rule of Succession $\frac{k+1}{n+2}$.

3.5 Information Theory: Entropy, Cross-Entropy, KL Divergence & Mutual Information

💡 Intuition (Plain English First)

Information Theory answers a simple question: **How surprised are you when an event x happens, and how many bits does it take to transmit that event?**

- If an event is 100% certain ($P(x) = 1$), your surprise is $\log \frac{1}{1} = 0$.
- If an event is rare ($P(x) = \frac{1}{8}$), your surprise is $\log_2 \frac{1}{1/8} = 3$ bits.

From this single definition of surprise $\log \frac{1}{P(x)} = -\log P(x)$, four foundational ML quantities emerge:

1. **Shannon Entropy $H(P)$:** Your **average surprise** under the true distribution P: $H(P) = \mathbb{E}_{x \sim P}[-\log P(x)]$.
   - A deterministic coin ($P(\text{Heads}) = 1$) has $H(P) = 0$ bits (zero uncertainty).
   - A fair K-sided die ($P(x) = 1/K$) has the **maximum possible entropy** $H(P) = \log K$ (maximum uncertainty).

2. **Cross-Entropy $H(P, Q)$:** Suppose the real world follows distribution P, but your neural network predicts distribution Q. Cross-Entropy is your **average surprise when using your model Q's code to compress real-world events from P**:

$$H(P, Q) = \mathbb{E}_{x \sim P}[-\log Q(x)] = -\sum_{x} P(x) \log Q(x)$$

3. **KL Divergence $D_{\mathrm{KL}}(P \parallel Q)$:** The **extra wasted bits (penalty)** you pay because your model Q does not match the true distribution P:

$$D_{\mathrm{KL}}(P \parallel Q) = \underbrace{H(P, Q)}_{\text{Bits using model } Q} - \underbrace{H(P)}_{\text{Ideal bits using true } P} = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} \geq 0$$

*(Why do we minimize Cross-Entropy $H(P, Q)$ when training a neural network instead of $D_{\mathrm{KL}}(P \parallel Q)$? Because the true data entropy $H(P)$ does not depend on our model weights θ, so $\nabla_\theta H(P, Q) = \nabla_\theta D_{\mathrm{KL}}(P \parallel Q)$—minimizing Cross-Entropy is **100% identical** to minimizing KL Divergence!)*

4. **Mutual Information $I(X; Y)$:** How much your uncertainty about X **drops** once someone tells you Y:

$$I(X; Y) = H(X) - H(X \mid Y)$$

Unlike linear correlation $\rho(X, Y)$, Mutual Information captures **every possible non-linear dependency**: $I(X; Y) = 0$ **if and only if X and Y are strictly independent**!

📐 The Math & Every Symbol Explained

1. Core Information Theory Identities

- **Shannon Entropy:** $H(P) = -\sum_x P(x) \log P(x) \in [0, \log K]$ (by convention, $0 \log 0 = 0$).
- **Cross-Entropy Decomposition:**

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P \parallel Q) \geq H(P)$$

- **Gibbs' Inequality:** $D_{\mathrm{KL}}(P \parallel Q) \geq 0$, with equality $D_{\mathrm{KL}}(P \parallel Q) = 0 \iff P = Q$.
- **KL Divergence Is NOT a True Distance Metric:**
  1. It is **asymmetric:** $D_{\mathrm{KL}}(P \parallel Q) \neq D_{\mathrm{KL}}(Q \parallel P)$.
  2. It violates the triangle inequality.

2. Forward KL $D_{\mathrm{KL}}(P \parallel Q)$ vs. Reverse KL $D_{\mathrm{KL}}(Q \parallel P)$ (Crucial for Generative AI & RLHF!)

Suppose the true distribution $P(x)$ is bimodal (has two separate peaks) and we fit a single-peak Gaussian model $Q_\theta(x)$:

- **Forward KL $D_{\mathrm{KL}}(P \parallel Q_\theta) = \sum_x P(x) \log \frac{P(x)}{Q_\theta(x)}$ (Mode-Covering / Mean-Seeking):** Look at where $P(x) > 0$: if our model sets $Q_\theta(x) \to 0$ anywhere that $P(x) > 0$, then $\log \frac{P(x)}{0} \to +\infty$! Thus Q_θ is terrified of missing any peak of P and stretches wide to cover **all modes** of P (even putting probability mass in the empty valley between the peaks). *Used in standard MLE and Supervised Fine-Tuning (SFT).*

- **Reverse KL $D_{\mathrm{KL}}(Q_\theta \parallel P) = \sum_x Q_\theta(x) \log \frac{Q_\theta(x)}{P(x)}$ (Mode-Seeking / Zero-Forcing):** Look at where $P(x) = 0$ (the empty valley between the peaks): if $Q_\theta(x) > 0$ there, $\log \frac{Q_\theta(x)}{0} \to +\infty$! So Q_θ is forced to be 0 wherever $P(x) = 0$, and happily locks onto **one single sharp peak (mode)** of P while ignoring the other peak. *Used in Variational Inference (VAE) and RLHF/PPO KL penalty.*

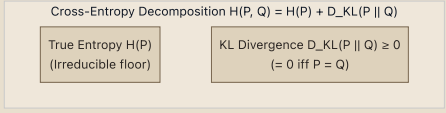
3. Mutual Information & Joint Entropy Venn Diagram Identities

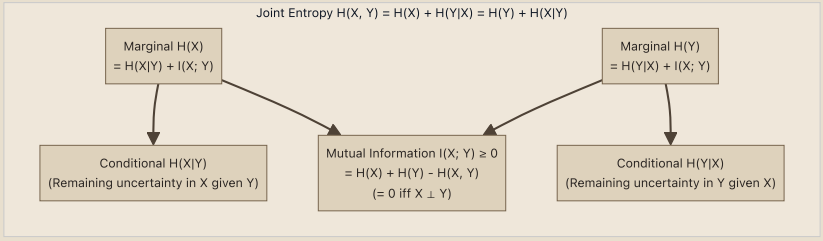
$$H(X,Y) = H(X) + H(Y \mid X) = H(Y) + H(X \mid Y)$$

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y) = D_{\mathrm{KL}}\big(P(X,Y) \parallel P(X)P(Y$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Continuous Differential Entropy Can Be Negative!**

For discrete random variables, $0 \le P(x) \le 1 \implies -\log P(x) \ge 0 \implies H(X) \ge 0$ always. However, for a **continuous** random variable with density $f(x)$, the PDF $f(x)$ can exceed 1 on a narrow interval! For example, if $X \sim \text{Uniform}(0, \frac{1}{2})$, then $f(x) = 2$ on $[0, \frac{1}{2}]$, so its **differential entropy** is $h(X) = -\int_0^{1/2} 2 \log_2(2)\, dx = -1 < 0$! (By contrast, $D_{\mathrm{KL}}(P \parallel Q) \ge 0$ and $I(X;Y) \ge 0$ remain strictly non-negative for both discrete and continuous variables).





🔢 Step-by-Step Toy Example

Problem: Let the true distribution over 2 classes be $P = [0.5, 0.5]$ and a model's predicted distribution be $Q = [0.25, 0.75]$. Using $\log_2$ (bits), compute:

1. True Entropy $H(P)$
2. Cross-Entropy $H(P, Q)$
3. KL Divergence $D_{\mathrm{KL}}(P \parallel Q)$

Step 1 — Compute True Entropy $H(P)$:

$$H(P) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = -0.5(-1) - 0.5(-1) = \mathbf{1} \text{ bit}$$

Step 2 — Compute Cross-Entropy $H(P, Q)$:

$$H(P,Q) = -0.5 \log_2 \left(\frac{1}{4}\right) - 0.5 \log_2 \left(\frac{3}{4}\right) = -0.5(-2) - 0.5(\log_2 3 - 2) = 2 - 0.5 \log_2 3 \approx 2 - 0.5(1.585) = \mathbf{1.}$$

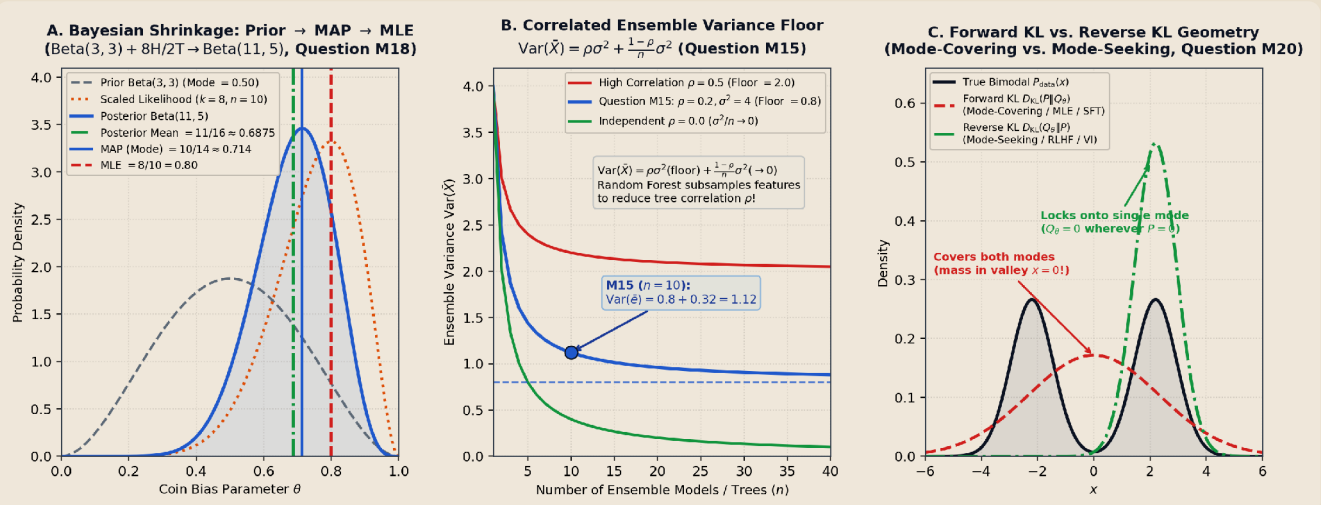
Step 3 — Compute KL Divergence $D_{\mathrm{KL}}(P \parallel Q) = H(P,Q) - H(P)$:

$$D_{\mathrm{KL}}(P \parallel Q) = 1.2075 - 1.0 = \mathbf{0.2075} \text{ bits} > 0 \checkmark$$

🎯 Exam Takeaway & Fast Shortcuts

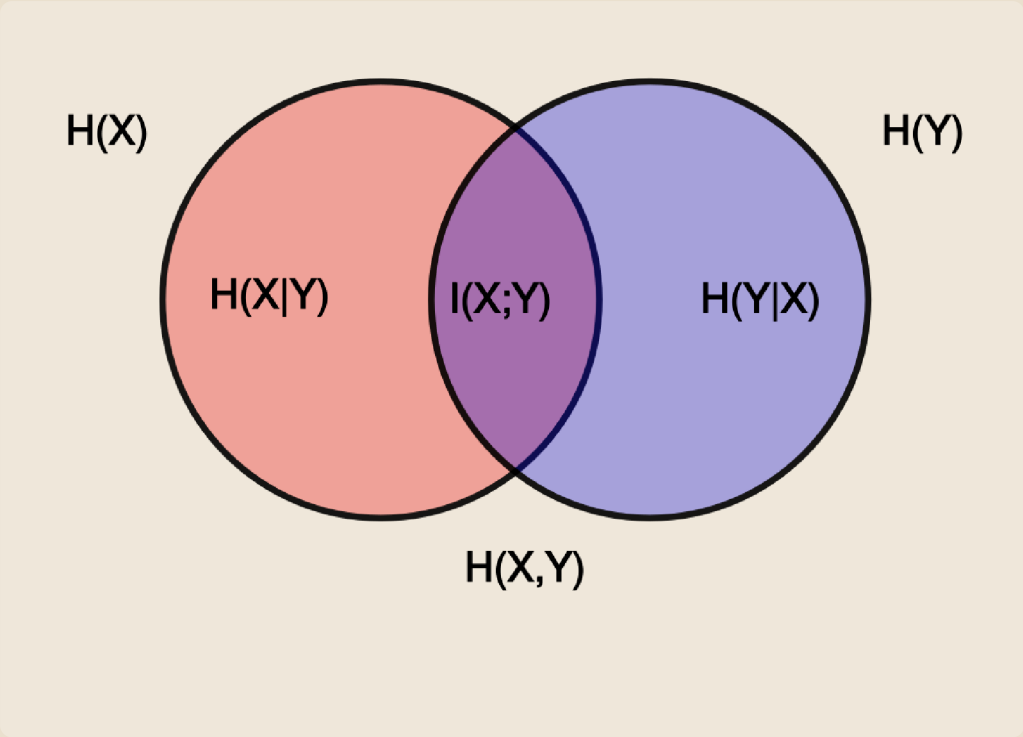
- **The One-Hot Target Shortcut:** In standard classification where the true label $P = y$ is a **one-hot vector** (e.g., $P = [0, 1, 0]$), the true distribution is 100% deterministic, so its entropy is $H(P) = 0$! Therefore, for one-hot labels, **Cross-Entropy equals KL Divergence exactly:** $H(P,Q) = D_{\mathrm{KL}}(P \parallel Q) = -\log Q(y_{\text{true}})$.

FIGURE SLIDE 1 OF 3



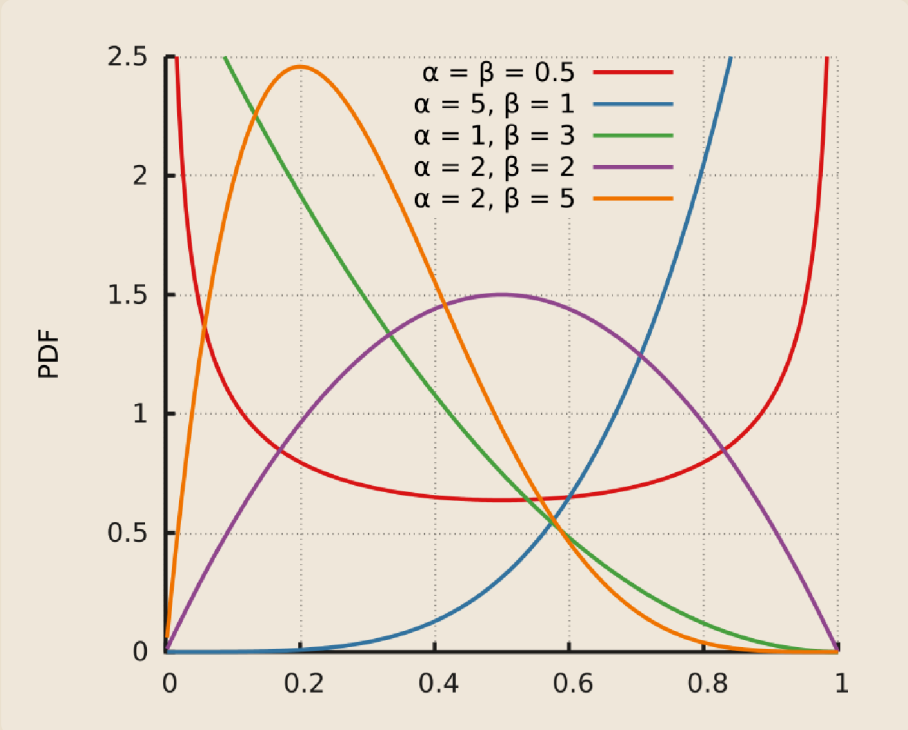
Day 3 Probability, Statistics & Information Theory Visual Cheat-Sheet: Beta-Bernoulli Shrinkage, Correlated Ensemble Floor, and Forward vs Reverse KL

FIGURE SLIDE 2 OF 3



Wikimedia Commons — Canonical Information Theory Venn Diagram: H(X), H(Y), Joint Entropy H(X,Y), Conditional Entropies H(X|Y) & H(Y|X), and Mutual Information I(X;Y)

FIGURE SLIDE 3 OF 3



Wikimedia Commons — Beta(α, β) Probability Density Function Family (Showing Interior Modes for α,β > 1 vs. Boundary Blow-Up for α = β = 0.5)

Pillar 2 (Days 4–5): Classical ML & Deep Learning Core — Step-by-Step Study Guide

↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 4–5: Nov 1 – Nov 2):** Pillar 2 is the #1 source of "**Select All That Apply**" **conceptual traps** and fast numerical exercises on HackerRank AI Residency exams (`ML01` – `ML20`). Instead of cramming dense equations into one-line bullets, every topic in this guide is broken down into four intuitive parts:
>
> 1. 💡 **Intuition (Plain English First):** What problem does this solve, and what is the physical/visual mental model?
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations where every single variable and matrix dimension ($N \times d = N$ rows/samples, d columns/features) is explicitly defined.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number calculations worked out line-by-line so you can see the mechanics in action.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and the exact traps tested in `ML01` – `ML20`.

Day 4: Classical Machine Learning & Evaluation Metrics

↑ Mục Lục

1.1 Linear, Ridge (L_2), Lasso (L_1) & ElasticNet Regression (`ML01`)

💡 Intuition (Plain English First)

Imagine you are trying to predict house prices (y) from a table of features (X) like square footage, number of bedrooms, and lot size.

- **Ordinary Least Squares (OLS)** simply draws the flat hyperplane that minimizes the sum of squared vertical distances (residuals) between the true prices and your predictions. Under ideal textbook conditions, OLS is the best unbiased linear rule you can possibly build.
- **When does OLS break?** Suppose you have **more features than data points** ($d > N$, like $1,000$ gene expression features for only 50 patients), or two features are **redundant/collinear** (e.g., Feature 1 is area in square feet and Feature 2 is area in square meters). Then there is no longer a unique best line—infinitely many weight combinations give the exact same training predictions, and the matrix $X^T X$ becomes **singular (non-invertible)**! Even if features are only *nearly* collinear, OLS weights explode to huge positive and negative numbers (e.g., $+10,000 \times \text{sq_ft} - 107,639 \times \text{sq_m}$) that cancel out on training data but fail miserably on new test data (**massive variance / overfitting**).
- **How Regularization rescues OLS:** We add a "budget penalty" on the size of the weights w so the model isn't allowed to use giant numbers unless they dramatically reduce the error:
  - **Ridge Regression (L_2 penalty):** Penalizes the sum of **squared** weights ($\sum w_j^2$). Geometrically, the weight budget is a smooth **circle/sphere**. It shrinks all weights smoothly toward zero (especially noisy, low-

variance directions) and makes $X^TX + \lambda I_d$ **100% invertible even when** $d \gg N$, but it never sets any weight to exact zero.

- **Lasso Regression (L_1 penalty):** Penalizes the sum of **absolute** weights ($\sum |w_j|$). Geometrically, the weight budget is a **diamond with sharp corners** sitting right on the coordinate axes. When the elliptical error contours expand until they touch the diamond, they almost always hit a sharp corner where one or more coordinates are **exact zero ($w_j = 0$)**! Thus, Lasso performs automatic **feature selection (sparsity)**.
- **ElasticNet ($L_1 + L_2$ penalty):** Pure Lasso gets confused when two features are identical twins ($x_1 \approx x_2$)—it randomly picks one and zeros out the other. By mixing the sharp corners of L_1 with the rounded edges of L_2, ElasticNet keeps the sparsity of Lasso while giving correlated twin features **nearly equal weights (the Grouping Effect)**.

📐 The Math & Every Symbol Explained

1. Ordinary Least Squares (OLS):

$$\mathcal{L}_{\text{OLS}}(w) = \frac{1}{2}\|Xw - y\|_2^2$$

Setting the gradient $\nabla_w \mathcal{L}_{\text{OLS}}(w) = X^T(Xw - y) = 0$ gives the **Normal Equations**:

$$\hat{w}_{\text{OLS}} = (X^TX)^{-1}X^Ty \quad (\text{requires } \text{rank}(X) = d \leq N)$$

2. Ridge Regression (L_2 Regularization):

$$\mathcal{L}_{\text{Ridge}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \implies \hat{w}_{\text{Ridge}} = (X^TX + \lambda I_d)^{-1}X^Ty$$

3. Lasso Regression (L_1 Regularization):

$$\mathcal{L}_{\text{Lasso}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$$

Where every symbol means:

- N: Number of training samples (rows of X).
- d: Number of input features (columns of X).
- $X \in \mathbb{R}^{N \times d}$: The **design matrix** with N **rows (samples) and** d **columns (features)**.
- $y \in \mathbb{R}^{N \times 1}$: The target vector of N true outputs.
- $w \in \mathbb{R}^{d \times 1}$: The weight (coefficient) vector of d parameters we want to learn.
- $X^TX \in \mathbb{R}^{d \times d}$: The unnormalized feature covariance (Gram) matrix with d **rows and** d **columns**.
- $I_d \in \mathbb{R}^{d \times d}$: The $d \times d$ identity matrix (1s on the main diagonal, 0s everywhere else).
- $\lambda \geq 0$: Scalar regularization strength controlling how heavily we penalize large weights.
- $\|w\|_2^2 = \sum_{j=1}^{d} w_j^2 = w^Tw$: Squared Euclidean ($L_2$) norm of the weights.
- $\|w\|_1 = \sum_{j=1}^{d} |w_j|$: Manhattan ($L_1$) norm of the weights.

Key Mathematical Properties to Know:

- **High-Dimensional Singularity ($d > N$):** Because the rank of a product cannot exceed the smallest dimension, $\text{rank}(X^TX) = \text{rank}(X) \leq \min(N, d)$. Whenever $d > N$, $\text{rank}(X^TX) \leq N < d$, which means the

$d \times d$ matrix $X^T X$ is **rank-deficient (singular / non-invertible)** and must have at least $d - N$ zero eigenvalues!

- **Gauss-Markov Theorem:** If the true data follows $y = Xw^* + \epsilon$ with zero-mean, uncorrelated, equal-variance (homoscedastic) noise ($\mathbb{E}[\epsilon] = 0$, $\text{Cov}(\epsilon) = \sigma^2 I_N$), then $\hat{w}_{\text{OLS}}$ is **BLUE** (**B**est **L**inear **U**nbiased **E**stimator)—it has the smallest variance among all linear unbiased estimators.
- **Why Ridge is Always Invertible (Eigenvalue Shift):** Since $X^T X$ is symmetric positive semi-definite ($X^T X \succeq 0$), all of its eigenvalues satisfy $\mu_i \geq 0$. Adding λI_d shifts every eigenvalue up by λ:

  $$\mu_i(X^T X + \lambda I_d) = \mu_i + \lambda \geq \lambda > 0 \quad (\text{for any } \lambda > 0)$$

  Because no eigenvalue can ever be zero, $X^T X + \lambda I_d$ is **strictly positive definite and invertible** even when $d \gg N$ or features are 100% collinear!
- **SVD Shrinkage & Bias-Variance Tradeoff:** If we take the Singular Value Decomposition $X = U\Sigma V^T$ (with singular values σ_i), Ridge scales each principal component coordinate by:

  $$\text{Shrinkage Factor} = \frac{\sigma_i^2}{\sigma_i^2 + \lambda} < 1$$

  Directions with tiny variance ($\sigma_i^2 \ll \lambda$) get crushed toward 0. Introducing $\lambda > 0$ intentionally adds a small amount of **bias** (pulling $\mathbb{E}[\hat{w}_{\text{Ridge}}]$ toward 0) in exchange for a **much larger drop in variance**.

🔢 Step-by-Step Worked Toy Example

Suppose we have $N = 2$ houses and $d = 2$ perfectly collinear features (Feature 2 is an exact copy of Feature 1), with targets $y = [2, 4]^T$:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \quad (2 \times 2), \qquad y = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \quad (2 \times 1)$$

Step 1: Try Ordinary Least Squares (OLS) and watch it fail. First, compute $X^T X$ (2×2) and $X^T y$ (2×1):

$$X^T X = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 1^2 + 2^2 & 1^2 + 2^2 \\ 1^2 + 2^2 & 1^2 + 2^2 \end{bmatrix} = \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 1(2) + 2(4) \\ 1(2) + 2(4) \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$$

Now check the determinant of $X^T X$:

$$\det(X^T X) = (5)(5) - (5)(5) = 25 - 25 = 0$$

The eigenvalues of $X^T X$ are $\mu_1 = 10$ and $\mu_2 = 0$. Because $\det(X^T X) = 0$, $(X^T X)^{-1}$ **does not exist!** Any weights satisfying $w_1 + w_2 = 2$ (like $[2, 0]^T$, $[0, 2]^T$, or $[1002, -1000]^T$) give zero training error.

Step 2: Apply Ridge Regression with $\lambda = 1$. Add $\lambda I_2 = 1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ to $X^T X$:

$$X^T X + \lambda I_2 = \begin{bmatrix} 5 + 1 & 5 \\ 5 & 5 + 1 \end{bmatrix} = \begin{bmatrix} 6 & 5 \\ 5 & 6 \end{bmatrix}$$

Check the new determinant and eigenvalues:

$$\det(X^TX + I_2) = (6)(6) - (5)(5) = 36 - 25 = 11 > 0 \quad \text{(eigenvalues shifted to } \mu_1 + 1 = 11,\ \mu_2 + 1 = 1)$$

Invert the 2×2 matrix using $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$:

$$(X^TX + I_2)^{-1} = \frac{1}{11}\begin{bmatrix} 6 & -5 \\ -5 & 6 \end{bmatrix}$$

Multiply by $X^Ty = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$ to find the unique Ridge weights:

$$\hat{w}_{\text{Ridge}} = \frac{1}{11}\begin{bmatrix} 6 & -5 \\ -5 & 6 \end{bmatrix}\begin{bmatrix} 10 \\ 10 \end{bmatrix} = \frac{1}{11}\begin{bmatrix} 60 - 50 \\ -50 + 60 \end{bmatrix} = \begin{bmatrix} 10/11 \\ 10/11 \end{bmatrix} \approx \begin{bmatrix} 0.909 \\ 0.909 \end{bmatrix}$$

Notice how Ridge cured the singularity AND automatically split the weight **equally** between the two identical features!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact Soft-Thresholding Operator & ElasticNet Equations**

- **Lasso Soft-Thresholding Operator (for orthonormal features $X^TX = I_d$):** When features are orthonormal, each weight w_j decouples. If the OLS estimate for feature j is $\hat{w}_{j,\text{OLS}}$, then:
  - **Ridge** scales it multiplicatively: $\hat{w}_{j,\text{Ridge}} = \frac{\hat{w}_{j,\text{OLS}}}{1+\lambda}$ (never zero unless $\hat{w}_{j,\text{OLS}} = 0$).
  - **Lasso** shifts it toward zero by a constant amount λ and clips at zero (**Soft-Thresholding**):

    $$\hat{w}_{j,\text{Lasso}} = \text{sign}(\hat{w}_{j,\text{OLS}})\max\left(0,\ |\hat{w}_{j,\text{OLS}}| - \lambda\right)$$

    Concrete check: If $\hat{w}_{j,\text{OLS}} = +0.40$ and $\lambda = 0.50$, Ridge outputs $\frac{0.40}{1.50} \approx +0.267$, whereas Lasso outputs $+1 \cdot \max(0,\ 0.40 - 0.50) = \mathbf{0}$!
- **ElasticNet Objective:**

  $$\mathcal{L}_{\text{ElasticNet}}(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\left(\alpha\|w\|_1 + \frac{1-\alpha}{2}\|w\|_2^2\right)$$

  where $\alpha \in [0, 1]$ mixes Lasso ($\alpha = 1$) and Ridge ($\alpha = 0$).

🎯 Exam Takeaway & Common Traps (ML01)

- **Trap 1 ($d > N$ Invertibility):** OLS $(X^TX)^{-1}$ **always fails** when $d > N$. Ridge $(X^TX + \lambda I_d)^{-1}$ **always succeeds** for any $\lambda > 0$ because its smallest eigenvalue is at least $\lambda > 0$.
- **Trap 2 (Sparsity vs. Shrinkage):** Ridge (L_2) shrinks weights asymptotically toward 0 but **never sets them to exact** 0. Lasso (L_1) produces **exact 0s** because MSE ellipses hit the sharp axis-aligned corners of the L_1 diamond.
- **Trap 3 (Bias-Variance of Regularization):** Increasing λ **increases bias** and **decreases variance**. (The Gauss-Markov theorem only guarantees OLS is best among *unbiased* estimators; a slightly *biased* Ridge estimator often achieves a lower total MSE than OLS!).

1.2 Logistic Regression, Probability Calibration & Support Vector Machines (`ML02`, `ML03`)

💡 Intuition (Plain English First)

How do we move from predicting continuous house prices to classifying emails as Spam ($y = 1$) vs. Not Spam ($y = 0$)?

- **Logistic Regression (`ML02`):** Instead of using the raw linear score $z = w^T x + b \in (-\infty, +\infty)$ directly as a probability (which could be -5 or $+12$), we pass z through the S-shaped **Sigmoid** squishification function $\sigma(z) = \frac{1}{1+e^{-z}}$, which smoothly maps any real number into $(0, 1)$.
  - *The Linear Separability Paradox:* You might think a dataset where a line separates 100% of positive and negative points is the easiest case for Logistic Regression. **Wrong!** Because $\sigma(z)$ only *approaches* 1 as $z \to +\infty$ and 0 as $z \to -\infty$, an unregularized Logistic Regression model will keep multiplying its weights w by $10\times$, $100\times$, $1000\times \to \infty$ forever trying to make the steepness infinite and push the Cross-Entropy loss all the way to 0!
- **Support Vector Machines (SVM) (`ML03`):** Unlike Logistic Regression (which cares about the probability of *every* data point, even points miles away from the boundary), an SVM acts like a civil engineer building the **widest possible safety street (margin)** between the closest positive and negative points.
  - The points sitting right on the curb of the street (or inside the street if we allow some slack) are called **Support Vectors**. If you delete every other training point far away from the street, the SVM decision boundary **does not move by a single millimeter**!
  - **What if the data isn't separable by a flat line?** Instead of explicitly computing thousands of polynomial features, the **Kernel Trick** lets us compute dot products in a high-dimensional (or even *infinite*-dimensional, for RBF) feature space using a simple similarity formula $K(x, z)$ on the original vectors.

📐 The Math & Every Symbol Explained

1. Logistic Regression & Log-Odds (`ML02`):

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = w^T x + b$$

Taking the natural log of the ratio of positive to negative probability gives the **Log-Odds (Logit)**, which is strictly linear in x:

$$\ln\left(\frac{P(y = 1 \mid x)}{P(y = 0 \mid x)}\right) = \ln\left(\frac{\sigma(z)}{1 - \sigma(z)}\right) = w^T x + b$$

- **Binary Cross-Entropy (BCE) Loss** (for labels $y_i \in \{0, 1\}$ and predicted probability $\hat{p}_i = \sigma(w^T x_i + b)$):

  $$\mathcal{L}_{\text{BCE}}(w, b) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \Big]$$

  *(Equivalently, if labels are coded as $y_i \in \{-1, +1\}$, this is the **Logistic Loss**: $\frac{1}{N} \sum_{i=1}^{N} \ln\left(1 + e^{-y_i (w^T x_i + b)}\right)$).*

2. Hard-Margin & Soft-Margin SVM Primal (`ML03`): For binary labels $y_i \in \{-1, +1\}$ and decision boundary $w^T x + b = 0$:

- **Signed Geometric Distance** from point x_i to the hyperplane $w^T x + b = 0$:

$$d_{\text{signed}}(x_i) = \frac{w^T x_i + b}{\|w\|_2}$$

- **Total Geometric Margin Width** between the positive margin curb ($w^T x + b = +1$) and negative margin curb ($w^T x + b = -1$):

$$\text{Margin Width} = \frac{2}{\|w\|_2}$$

- **Soft-Margin Primal Problem & Unconstrained Hinge Loss Form:**

$$\min_{w,b,\xi \geq 0} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N} \xi_i \quad \text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0$$

Substituting the smallest valid slack $\xi_i = \max\left(0,\ 1 - y_i(w^T x_i + b)\right)$ yields the **Hinge Loss** objective:

$$\min_{w,b} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N} \underbrace{\max\left(0,\ 1 - y_i(w^T x_i + b)\right)}_{\text{Hinge Loss}}$$

3. SVM Dual Problem, KKT Conditions & Kernels (`ML03`):

$$\max_{0 \leq \alpha_i \leq C} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{subject to } \sum_{i=1}^{N} \alpha_i y_i = 0$$

Once solved, the optimal weight vector in feature space is a sparse linear combination of **only the Support Vectors** ($\alpha_i > 0$):

$$w^* = \sum_{i \in \text{SV}} \alpha_i y_i \phi(x_i)$$

Where every symbol means:

- $x_i \in \mathbb{R}^{d \times 1}$: Feature vector of sample i (d rows, 1 column).
- $y_i \in \{-1, +1\}$ (for SVM) or $\{0, 1\}$ (for BCE): Ground-truth binary class label.
- $w \in \mathbb{R}^{d \times 1}$, $b \in \mathbb{R}$: Normal vector to the hyperplane and scalar bias offset.
- $\xi_i \geq 0$: **Slack variable** for sample i, measuring how far point x_i encroaches past its margin curb $y_i(w^T x_i + b) = 1$.
- $C > 0$: **Regularization / Slack Penalty hyperparameter** (inverse of λ).
  - **Large $C \to \infty$:** Refuses to tolerate slack $\xi_i \implies$ **narrow margin**, fits training points tightly (**low bias, high variance / overfitting**).
  - **Small $C \to 0$:** Tolerates many margin violations $\implies$ **wide margin**, smooth boundary (**high bias, low variance**).
- $\alpha_i \in [0, C]$: **Dual Lagrange multiplier** for sample i. By the Karush-Kuhn-Tucker (KKT) complementary slackness conditions, every training point falls into one of three exact buckets:
  1. $\alpha_i = 0 \implies y_i(w^T x_i + b) > 1$ **($\xi_i = 0$): Non-Support Vector** strictly outside the street. Deleting this point before training changes nothing!

2. $0 < \alpha_i < C \implies y_i(w^T x_i + b) = 1$ **($\xi_i = 0$): Free (Unbounded) Support Vector** sitting **exactly on the margin curb**. Used to solve for b^*.
3. $\alpha_i = C \implies y_i(w^T x_i + b) \leq 1$ **($\xi_i \geq 0$): Bounded Support Vector** that violates the margin (if $0 < \xi_i < 1$, it is inside the street but still on the correct side of the center line; if $\xi_i > 1$, it is **misclassified** on the wrong side of $w^T x + b = 0$).

- $K(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{H}}$: **Kernel function**. By **Mercer's Theorem**, a symmetric function $K(x, z)$ is a valid kernel if and only if its $N \times N$ Gram matrix $G_{ij} = K(x_i, x_j)$ is **Positive Semi-Definite ($G \succeq 0$)** for any finite set of points.
- **RBF (Gaussian) Kernel:** $K(x, z) = \exp\left(-\gamma \|x - z\|_2^2\right)$. Because the Taylor series $e^{2\gamma x^T z} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (x^T z)^k$ contains all polynomial powers up to ∞, the RBF kernel maps inputs into an **infinite-dimensional** feature space! Large γ = narrow spiky Gaussian around each point = **high variance / overfitting**.

🔢 Step-by-Step Worked Toy Example

Part 1: Watching Unregularized Logistic Regression Explode on Separable Data (`ML02`). Suppose we have a tiny 1D dataset with two points that are cleanly separated at $x = 0$:

- Point 1: $x_1 = -1,\ y_1 = 0$
- Point 2: $x_2 = +1,\ y_2 = 1$

Let bias $b = 0$, and let's test what happens to the Binary Cross-Entropy loss as we scale the weight $w > 0$ larger and larger:

1. **Try $w = 1$:**
   - For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(1 \cdot 1) = \frac{1}{1+e^{-1}} \approx 0.731 \implies \text{Loss}_2 = -\ln(0.731) \approx 0.313$.
2. **Try $w = 5$:**
   - For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(5 \cdot 1) = \frac{1}{1+e^{-5}} \approx 0.9933 \implies \text{Loss}_2 = -\ln(0.9933) \approx 0.0067$.
3. **Try $w = 20$:**
   - For $x_2 = +1$ ($y_2 = 1$): $\hat{p}_2 = \sigma(20) = \frac{1}{1+e^{-20}} \approx 0.999999998 \implies \text{Loss}_2 \approx 2 \times 10^{-9}$.
4. **Why it never stops:** The gradient with respect to w is $\frac{\partial \mathcal{L}}{\partial w} = -(1 - \sigma(w)) < 0$, which is **never zero for any finite w**! Thus $\|w\|_2 \to \infty$ unless we add an L_2 penalty $\frac{\lambda}{2} w^2$ (whose positive derivative λw balances the gradient at a finite w^*).

Part 2: Computing SVM Margin Width, Hinge Loss & KKT Categories (`ML03`). Suppose a trained 2D Soft-Margin SVM with $C = 1.0$ has weight vector $w = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and bias $b = -2$.

1. **Compute the Euclidean norm $\|w\|_2$ and total geometric margin width:**

$$\|w\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

$$\text{Total Margin Width} = \frac{2}{\|w\|_2} = \frac{2}{5} = \mathbf{0.40}$$

2. **Classify three training points using functional margin $m_i = y_i(w^T x_i + b)$ and Hinge Loss $\xi_i = \max(0, 1 - m_i)$:**
   - **Point A:** $x_A = [2, 1]^T,\ y_A = +1$

- Score: $w^T x_A + b = 3(2) + 4(1) - 2 = 8$.
- Functional margin: $m_A = (+1)(8) = 8 > 1$.
- Slack / Hinge Loss: $\xi_A = \max(0, 1 - 8) = \mathbf{0}$.
- **KKT Status:** Strictly outside the street $\implies$ $\alpha_{\mathbf{A}} = \mathbf{0}$ **(Non-Support Vector)**.

- **Point B:** $x_B = [1, 0]^T,\ y_B = +1$
  - Score: $w^T x_B + b = 3(1) + 4(0) - 2 = 1$.
  - Functional margin: $m_B = (+1)(1) = \mathbf{1}$.
  - Slack / Hinge Loss: $\xi_B = \max(0, 1 - 1) = \mathbf{0}$.
  - **KKT Status:** Lies **exactly on the margin curb** $\implies$ $\mathbf{0 < \alpha_B < 1.0}$ **(Free Support Vector)**.

- **Point C:** $x_C = [0, 1]^T,\ y_C = -1$
  - Score: $w^T x_C + b = 3(0) + 4(1) - 2 = +2$ (predicts positive, but true label is $y_C = -1$!).
  - Functional margin: $m_C = (-1)(+2) = -2 < 1$.
  - Slack / Hinge Loss: $\xi_C = \max(0, 1 - (-2)) = \mathbf{3.0} > 1$ (misclassified!).
  - **KKT Status:** Violates the margin $\implies$ $\alpha_{\mathbf{C}} = \mathbf{C} = \mathbf{1.0}$ **(Bounded Support Vector)**.

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Implicit Bias of GD & Probability Calibration (Platt vs. Isotonic)**

- **Implicit Bias of Unregularized GD (Soudry et al.):** Even though $\|w_t\|_2 \to \infty$ when training unregularized Logistic Regression via gradient descent on linearly separable data, the *unit direction vector* $\frac{w_t}{\|w_t\|_2}$ converges to the exact **Hard-Margin SVM direction** at rate $\mathcal{O}(1/\ln t)$!
- **Probability Calibration:**
  - **Why Logistic Regression is Naturally Well-Calibrated:** Log-loss (Cross-Entropy) and Brier Score $\frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$ are **strictly proper scoring rules**—their mathematical expectation is minimized if and only if the model outputs the true conditional probability $p_i = P(y_i = 1 \mid x_i)$.
  - **Why SVMs, Random Forests & Boosted Trees Need Calibration:**
    - **Random Forests** average predictions across many high-variance trees; to get 0.0 or 1.0, almost every single tree must agree, which pushes predicted probabilities **away from** 0 **and** 1 (under-confident; S-shaped calibration curve).
    - **Boosted Trees (GBDT) and Deep Neural Nets** push probabilities **too close to** 0 **and** 1 (over-confident).
  - **Platt Scaling (Parametric):** Fits a 1D logistic sigmoid $P(y = 1 \mid f) = \frac{1}{1+\exp(Af+B)}$ on a held-out validation set (2 scalar parameters A, B; in multiclass deep nets with $B = 0$, this is **Temperature Scaling** $\text{softmax}(z/T)$). Best when validation data is small ($N_{\text{cal}} < 1,000$).
  - **Isotonic Regression (Non-Parametric):** Fits a free-form piecewise-constant monotonically non-decreasing step function; more flexible than Platt scaling, but easily overfits unless $N_{\text{cal}} > 1,000$.

🎯 **Exam Takeaway & Common Traps (`ML02`, `ML03`)**

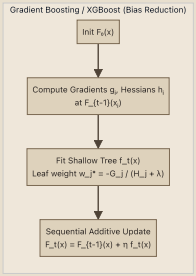
- **Trap 1 (`ML02` — Separable Data in Logistic Regression):** Without regularization ($\lambda = 0$), linearly separable data causes Logistic Regression weights to **diverge to infinity ($\|w\|_2 \to \infty$)** as BCE loss approaches 0. Adding L_2 regularization ($\lambda > 0$) guarantees a unique finite solution.

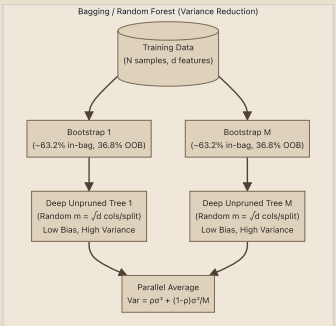
- **Trap 2 (`ML03` — Margin Width Formula):** The distance from the center line $w^T x + b = 0$ to one margin curb is $\frac{1}{\|w\|_2}$, so the **total margin width between $+1$ and -1 is $\frac{2}{\|w\|_2}$** (don't forget the factor of 2, and don't divide by $\|w\|_2^2$!).
- **Trap 3 (`ML03` — C and γ Overfitting Direction):** Increasing $C \uparrow$ OR increasing RBF $\gamma \uparrow$ makes the SVM boundary tighter and more complex (**lower bias, higher variance / more overfitting**).

1.3 Decision Trees, Random Forests & Gradient Boosting (XGBoost Math) (`ML04`, `ML05`)

💡 Intuition (Plain English First)

- **How a Single Decision Tree Thinks (`ML04`):** A decision tree plays "20 Questions" with your features (e.g., "*Is Age* > 30?"). At each step, it searches every feature and threshold to find the split that makes the two child groups as **pure** (homogeneous) as possible—meaning each child node contains mostly one class instead of a $50/50$ coin-flip mix.
- **Why One Deep Tree Isn't Enough:** A deep, unpruned decision tree can memorize any training set (**low bias**), but if you perturb a few rows of training data, the top split can change and produce a completely different tree (**huge variance / overfitting**).
- **Two Opposite Ways to Combine Trees into an Ensemble (`ML05`):**
  1. **Bagging / Random Forest = "Wisdom of Independent Crowds" (Slashes Variance):** Take M deep, noisy, low-bias trees. Train each tree **independently in parallel** on a random bootstrap sample of rows, and force each split to only look at a random subset of $m \approx \sqrt{d}$ columns so the trees don't all pick the same dominant feature. When you average M independent noisy estimates, their random errors cancel out!
  2. **Gradient Boosting (GBDT / XGBoost) = "Relay Team of Specialists" (Slashes Bias):** Start with a weak, shallow tree (high bias, low variance, e.g., depth 3–6). Tree 2 does *not* train on the original labels y—it trains on the **mistakes (residuals / negative gradients)** left behind by Tree 1! Each subsequent tree sequentially patches the remaining errors of the whole team so far.





📐 The Math & Every Symbol Explained

1. Node Impurity & Information Gain (`ML04`): Suppose a node contains N_{node} samples across K classes with class proportions $p_1, p_2, \ldots, p_K$ (where $\sum_{k=1}^{K} p_k = 1$):

- **Gini Impurity:** Probability that a randomly chosen sample in the node would be misclassified if we labeled it randomly according to the node's class distribution:

$$G = \sum_{k=1}^{K} p_k(1 - p_k) = 1 - \sum_{k=1}^{K} p_k^2$$

  - **3-Second Binary Shortcut ($K = 2$ with positive fraction p and negative fraction $1 - p$):**

$$G(p) = 1 - p^2 - (1-p)^2 = 2p(1-p)$$

    (Maximum Gini impurity for binary classification is 0.50 at a $50/50$ split $p = 0.5$; minimum is 0.00 at a pure node $p = 0$ or 1).

- **Shannon Entropy:**

$$H = -\sum_{k=1}^{K} p_k \log_2(p_k) \quad (\text{binary max is } \mathbf{1.0} \text{ bit at } p = 0.5)$$

- **Information Gain / Gini Gain** when splitting a Parent node (N samples) into Left (N_L samples) and Right (N_R samples):

$$\text{Gain} = I(\text{Parent}) - \left(\frac{N_L}{N} I(\text{Left}) + \frac{N_R}{N} I(\text{Right}) \right)$$

  where $I(\cdot)$ is either Gini G or Entropy H.

- **Regression Trees:** Impurity I is the node variance (MSE). A leaf predicts the **sample mean** $\bar{y}$ of its training points when minimizing MSE (L_2 loss), or the **sample median** when minimizing MAE (L_1 loss).

2. Random Forest Math: Out-of-Bag (OOB) & Ensemble Variance:

- **Out-of-Bag (OOB) Fraction:** In bootstrap sampling, we draw N samples *with replacement* from N rows. The probability that a specific row x_i is **never picked** in all N draws is:

$$P(\text{row } i \text{ is OOB}) = \left(1 - \frac{1}{N}\right)^N \xrightarrow{N \to \infty} e^{-1} \approx \mathbf{0.368} \ (\mathbf{36.8\%})$$

  Thus, every tree trains on $\sim 63.2\%$ **unique in-bag rows** and leaves $\sim 36.8\%$ **OOB rows** available as a built-in validation set!

- **Variance of an Average of M Correlated Trees:** If each tree has variance σ^2 and the average pairwise correlation between any two trees is $\rho \in [0, 1]$, the variance of the ensemble average $\bar{f}(x) = \frac{1}{M}\sum_{m=1}^{M} f_m(x)$ is:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1 - \rho}{M}\sigma^2$$

- σ^2: Variance of a single deep tree.
- M: Number of trees in the forest. As $M \to \infty$, the second term $\frac{1-\rho}{M}\sigma^2 \to 0$, so **adding more trees to a Random Forest NEVER causes overfitting**!
- ρ: Pairwise correlation between trees. Even with $M = \infty$ trees, ensemble variance is floored at $\rho\sigma^2$. Restricting each split to $m \approx \sqrt{d}$ random features **decorrelates the trees ($\rho \downarrow$)**, breaking that floor!

3. Standard GBDT vs. XGBoost (`ML05`): At boosting step t, we update our additive prediction via $F_t(x_i) = F_{t-1}(x_i) + \eta f_t(x_i)$ (where η is the learning rate / shrinkage):

- **Standard GBDT (First-Order):** Fits tree f_t to the **negative gradients (pseudo-residuals)** evaluated at the previous step's predictions:

$$r_{i,t} = -g_i = -\left[\frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i}\right]_{\hat{y}_i = F_{t-1}(x_i)}$$

- **XGBoost (Second-Order + Explicit Tree Regularization):** Approximates the loss using a **2nd-order Taylor expansion** around $F_{t-1}(x_i)$ and adds an explicit structural penalty $\Omega(f_t)$ on the tree:

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^{N}\left[g_i f_t(x_i) + \frac{1}{2}h_i f_t(x_i)^2\right] + \underbrace{\gamma T + \frac{1}{2}\lambda\sum_{j=1}^{T} w_j^2}_{\Omega(f_t)}$$

  where:
  - $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$: **First-order gradient** of the loss for sample i.
  - $h_i = \partial^2_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$: **Second-order derivative (Hessian / curvature)** of the loss for sample i.
  - T: Number of terminal leaves in tree f_t, penalized by complexity cost $\gamma \geq 0$ (acts as automatic pre-pruning!).
  - w_j: Scalar prediction output of leaf j, penalized by L_2 regularization $\lambda \geq 0$.

🔢 Step-by-Step Worked Toy Example

Example 1: Computing Binary Gini Impurity & Gini Gain in 15 Seconds (`ML04`). Suppose a Parent node has $N = 10$ samples (5 Positive, 5 Negative). A candidate split divides them into:

- **Left Child ($N_L = 5$):** 4 Positive, 1 Negative ($p_L = 4/5 = 0.80$).
- **Right Child ($N_R = 5$):** 1 Positive, 4 Negative ($p_R = 1/5 = 0.20$).

Let's compute the Gini Impurity of each node and the Gini Gain line by line using $G(p) = 2p(1-p)$:

1. **Parent Gini ($p = 0.50$):**

$$G(\text{Parent}) = 2(0.50)(1 - 0.50) = 2(0.25) = \mathbf{0.50}$$

2. **Left Child Gini ($p_L = 0.80$):**

$$G(\text{Left}) = 2(0.80)(1 - 0.80) = 2(0.80)(0.20) = \mathbf{0.32}$$

3. **Right Child Gini ($p_R = 0.20$):**

$$G(\text{Right}) = 2(0.20)(1 - 0.20) = 2(0.20)(0.80) = \mathbf{0.32}$$

4. **Weighted Child Gini & Gini Gain:**

$$\text{Weighted Child Gini} = \frac{5}{10}(0.32) + \frac{5}{10}(0.32) = 0.32$$

$$\text{Gini Gain} = 0.50 - 0.32 = \mathbf{0.18}$$

Example 2: Why Feature Subsampling ($m \approx \sqrt{d}$) Matters in Random Forests. Suppose each deep tree has variance $\sigma^2 = 10.0$, and we build a forest of $M = 100$ trees:

- **Case A (Bagging without feature subsampling, high correlation $\rho = 0.60$):**

$$\text{Var}(\bar{f}) = (0.60)(10.0) + \frac{1 - 0.60}{100}(10.0) = 6.00 + 0.04 = \mathbf{6.04}$$

- **Case B (Random Forest with $m \approx \sqrt{d}$ feature subsampling, lower correlation $\rho = 0.20$):**

$$\text{Var}(\bar{f}) = (0.20)(10.0) + \frac{1 - 0.20}{100}(10.0) = 2.00 + 0.08 = \mathbf{2.08}$$

Notice that with $M = 100$ trees, the $\frac{1-\rho}{M}\sigma^2$ term is already tiny (0.04 vs. 0.08); almost **all** of the variance reduction ($6.04 \to 2.08$) comes from **lowering tree correlation** ρ via random feature subsampling!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Exact XGBoost Optimal Leaf Weight & Split Gain Equations**

Let $I_j = \{i \mid x_i \in \text{leaf } j\}$ be the set of training samples falling into leaf j, and define the summed leaf gradient $G_j = \sum_{i \in I_j} g_i$ and summed leaf Hessian $H_j = \sum_{i \in I_j} h_i$. Rewriting the 2nd-order objective leaf-by-leaf gives $\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2\right] + \gamma T$. Setting $\frac{\partial \tilde{\mathcal{L}}^{(t)}}{\partial w_j} = 0$ yields:

- **Optimal Leaf Weight (Damped Newton Step):**

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

- **Minimum Leaf Objective Value:**

$$\tilde{\mathcal{L}}_j^* = -\frac{1}{2}\frac{G_j^2}{H_j + \lambda}$$

- **Exact Split Gain (splitting parent $G_L + G_R$ into Left L and Right R):**

$$\text{Gain}_{\text{XGB}} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

(If the bracketed reduction in loss is smaller than the new-leaf complexity cost γ, then $\text{Gain}_{\text{XGB}} < 0$ and XGBoost prunes the split!)

🎯 **Exam Takeaway & Common Traps (`ML04`, `ML05`)**

- **Trap 1 (`ML04` — Binary Gini Numbers to Memorize):** $50/50$ split $\implies G = 0.50$; $80/20$ split $\implies G = 2(0.8)(0.2) = \mathbf{0.32}$; $90/10$ split $\implies G = 2(0.9)(0.1) = \mathbf{0.18}$; pure node $\implies G = 0.00$.
- **Trap 2 (`ML05` — Bagging vs. Boosting Bias/Variance):** Random Forest starts with **deep (low-bias, high-variance)** trees and averages them in parallel to **reduce variance** (cannot overfit as $M \to \infty$). Gradient Boosting starts with **shallow (high-bias, low-variance)** trees and adds them sequentially to **reduce bias** (CAN overfit if M is too large!).
- **Trap 3 (`ML05` — GBDT vs. XGBoost):** Standard GBDT uses only **1st-order gradients (g_i)**; XGBoost uses a **2nd-order Taylor expansion (both gradients g_i and Hessians h_i)** plus explicit L_2 leaf weight (λ) and leaf count (γ) regularization.
- **Trap 4 (Feature Importance Bias):** Default tree impurity importance (**MDI**, mean decrease in Gini on training splits) is heavily biased toward **high-cardinality continuous features or ID columns**. Always use **Permutation Importance** on OOB or held-out validation data.

1.4 Unsupervised Learning: K-Means, GMM (EM) & Dimensionality Reduction (`ML06`)

💡 Intuition (Plain English First)

What if we have no labels y at all (or we want to compress high-dimensional data before visualizing/modeling)?

- **K-Means vs. Gaussian Mixture Models (GMM):**
  - **K-Means** is a "winner-take-all" clustering algorithm. Every point belongs 100% to its single closest centroid (hard assignment), and because it measures plain Euclidean distance $\|x_i - \mu_k\|_2^2$, it implicitly assumes all clusters are **equal-radius spheres**.
  - **Why K-Means++?** If you pick initial centroids completely at random, two centroids might land in the same cluster while a faraway cluster gets zero centroids. **K-Means++** fixes this by picking the first centroid at random, and then picking each next centroid with probability proportional to its **squared distance** $D(x)^2$ from the closest already-chosen centroid!
  - **GMM (trained via Expectation-Maximization / EM)** upgrades K-Means into a **soft, probabilistic** model where each cluster k is a full Gaussian bell curve $\mathcal{N}(\mu_k, \Sigma_k)$ with its own covariance matrix Σ_k (allowing stretched, tilted **elliptical** clusters of different sizes) and each point gets a fractional responsibility $\gamma_{ik} \in (0, 1)$. In fact, K-Means is just the limiting case of GMM when all covariances are fixed to $\Sigma_k = \sigma^2 I$ and $\sigma^2 \to 0$!
- **PCA vs. LDA vs. t-SNE (`ML06`):**
  - **PCA (Principal Component Analysis): Unsupervised.** Ignores class labels completely and finds orthogonal axes of maximum total variance (spread).
  - **LDA (Linear Discriminant Analysis): Supervised.** Uses class labels to find a linear projection that simultaneously pushes **different class centers as far apart as possible** (maximizing between-class scatter S_B) while keeping **points within the same class tightly packed** (minimizing within-class scatter S_W).
    - *Why can LDA output at most $C - 1$ dimensions?* Think geometry! If you have $C = 2$ class centers (two points in 50D space), a single straight line ($1\text{D} = 2 - 1$) connects them. If you have $C = 3$ class centers (three points in 50D space), they always define a flat triangle/plane ($2\text{D} = 3 - 1$)! Any direction perpendicular to that $(C - 1)$-dimensional subspace has **zero distance between class centers**.

- **t-SNE: Non-linear manifold visualization.** In high dimensions, there is plenty of room for many neighbors at moderate distances, but when you squash them into a 2D plot, they crush together in the center (**the Crowding Problem**). t-SNE solves this by using a **heavy-tailed Student-t distribution ($\nu = 1$, Cauchy)** in 2D so moderately distant points can spread much farther apart in the 2D plot.

📐 The Math & Every Symbol Explained

1. K-Means Objective & K-Means++ Initialization:

$$\min_{\{z_{ik}\},\{\mu_k\}} \text{WCSS} = \sum_{i=1}^{N}\sum_{k=1}^{K} z_{ik}\|x_i - \mu_k\|_2^2$$

- $z_{ik} \in \{0, 1\}$: Hard cluster assignment indicator (1 if point x_i is assigned to cluster k, 0 otherwise).
- $\mu_k \in \mathbb{R}^{d\times 1}$: Centroid (mean vector) of cluster k.
- **K-Means++ Sampling Probability:** Given existing centroids $\{\mu_1, \ldots, \mu_m\}$, let $D(x_i) = \min_{1\le c\le m} \|x_i - \mu_c\|_2$ be the distance from x_i to its nearest existing centroid. The next centroid is sampled with probability:

  $$P(\text{select } x_i) = \frac{D(x_i)^2}{\sum_{j=1}^{N} D(x_j)^2}$$

2. Gaussian Mixture Models (GMM) & EM Algorithm: Model density: $p(x_i \mid \theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$, where $\pi_k \in [0, 1]$ are mixing weights ($\sum_{k=1}^{K} \pi_k = 1$).

- **E-Step (Expectation — compute soft responsibilities γ_{ik}):**

  $$\gamma_{ik} = P(z_i = k \mid x_i, \theta^{(t)}) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

- **M-Step (Maximization — update parameters using effective cluster count $N_k = \sum_{i=1}^{N} \gamma_{ik}$):**

  $$\pi_k^{\text{new}} = \frac{N_k}{N}, \qquad \mu_k^{\text{new}} = \frac{1}{N_k}\sum_{i=1}^{N} \gamma_{ik} x_i, \qquad \Sigma_k^{\text{new}} = \frac{1}{N_k}\sum_{i=1}^{N} \gamma_{ik}(x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^T$$

- **Monotonicity & The Singularity Trap:** By Jensen's Inequality (maximizing the Evidence Lower Bound, ELBO), every EM iteration is guaranteed to monotonically non-decrease the log-likelihood: $\log p(X \mid \theta^{(t+1)}) \ge \log p(X \mid \theta^{(t)})$. However, if a Gaussian component collapses onto a single data point ($\mu_k = x_i$ and $\sigma_k^2 \to 0$), the Gaussian peak $\frac{1}{\sqrt{2\pi\sigma_k^2}} \to +\infty$, causing an **infinite likelihood singularity**! We prevent this in practice by adding a small diagonal ridge $\Sigma_k \leftarrow \Sigma_k + \epsilon I_d$.

3. Linear Discriminant Analysis (LDA) Dimensionality Cap (`ML06`): Given N samples across C classes with overall mean $\mu \in \mathbb{R}^{d\times 1}$ and class means $\mu_c \in \mathbb{R}^{d\times 1}$:

- **Within-Class Scatter Matrix ($d \times d$):** $S_W = \sum_{c=1}^{C} \sum_{x_i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$
- **Between-Class Scatter Matrix ($d \times d$):** $S_B = \sum_{c=1}^{C} N_c (\mu_c - \mu)(\mu_c - \mu)^T$
- **Rayleigh Quotient Objective:** $\max_W \frac{\det(W^T S_B W)}{\det(W^T S_W W)}$, solved by the top eigenvectors of $S_W^{-1} S_B$.
- **Exact Rank Bound:** Each term $(\mu_c - \mu)(\mu_c - \mu)^T$ in S_B is an outer product of a $d \times 1$ vector with itself, which has **rank at most** 1. Because the C centered vectors satisfy the linear constraint $\sum_{c=1}^{C} N_c(\mu_c - \mu) = 0$, at most $C - 1$ of them are linearly independent:

$$\text{rank}(S_B) \leq \min(d,\ C-1) \implies \textbf{Max LDA Output Dimensions} = \min(\mathbf{d},\ \mathbf{C-1})$$

🔢 Step-by-Step Worked Toy Example

Example 1: K-Means++ $D(x)^2$ Sampling Probabilities. Suppose we have 3 data points on a 1D line at $x_1 = 0,\ x_2 = 1,\ x_3 = 4$, and K-Means++ randomly picks the first centroid at $\mu_1 = x_1 = 0$. What is the exact probability of selecting x_2 vs. x_3 as the second centroid μ_2?

1. **Compute distance $D(x_i) = |x_i - \mu_1|$ to the nearest existing centroid:**
   - $D(x_1) = |0 - 0| = 0$
   - $D(x_2) = |1 - 0| = 1$
   - $D(x_3) = |4 - 0| = 4$
2. **Square each distance $D(x_i)^2$:**
   - $D(x_1)^2 = 0^2 = 0$
   - $D(x_2)^2 = 1^2 = 1$
   - $D(x_3)^2 = 4^2 = 16$
3. **Normalize by the sum $0 + 1 + 16 = 17$:**

   $$P(\mu_2 = x_2) = \frac{1}{17} \approx 5.9\%, \qquad P(\mu_2 = x_3) = \frac{16}{17} \approx \mathbf{94.1\%}$$

   Notice how squaring the distance gives the faraway point $x_3 = 4$ a $16\times$ **higher chance** of being picked than $x_2 = 1$!

Example 2: LDA vs. PCA Output Dimensions (`ML06`). Suppose a biomedical dataset has $N = 500$ patient scans, each with $d = 50$ numerical features, labeled into $C = 3$ disease classes (Healthy, Mild, Severe):

- **If you run PCA:** You can extract up to $\min(N-1, d) = \min(499, 50) = \mathbf{50}$ principal components.
- **If you run LDA:** Because $C = 3$ class centroids in $\mathbb{R}^{50}$ span at most a 2D plane ($\text{rank}(S_B) \leq 3 - 1 = 2$), LDA can output **at most** $\min(50, 3-1) = \mathbf{2}$ **linear discriminants**! Asking `LinearDiscriminantAnalysis(n_components=3)` in scikit-learn will immediately raise a `ValueError`.

🎯 Exam Takeaway & Common Traps (`ML06`)

- **Trap 1 (`ML06` — LDA Max Components):** For C classes and d features, LDA produces at most $\min(d, C-1)$ components (for binary classification $C = 2$, LDA always projects onto **1D**, regardless of whether $d = 10$ or $d = 10,000$!).
- **Trap 2 (K-Means & GMM Properties):** Both K-Means and GMM EM are **non-convex** and converge only to a **local** optimum. However, GMM MLE can diverge to **infinite log-likelihood ($+\infty$)** if a component's variance $\sigma_k^2 \to 0$ on a single data point; K-Means cannot diverge because its objective is bounded below by 0.
- **Trap 3 (t-SNE Interpretation):** t-SNE uses a heavy-tailed **Student-t ($\nu = 1$ Cauchy)** distribution in low-D to solve the Crowding Problem and minimizes asymmetric Forward KL $D_{\text{KL}}(P \parallel Q)$. **Never** interpret cluster diameters or distances between separated clusters on a t-SNE plot as true metric distances!

1.5 Bias-Variance Decomposition, Cross-Validation & Imbalanced Metrics (`ML07` – `ML10`)

💡 Intuition (Plain English First)

- **The Dartboard Mental Model for Bias-Variance (`ML09`):** Imagine throwing darts at a bullseye $f(x)$, where each dart is a model $\hat{f}_D(x)$ trained on a fresh random training dataset D:
  - **Bias:** How far the *average center* of your dart cloud $\mathbb{E}_D[\hat{f}(x)]$ is from the true bullseye $f(x)$. High bias = **underfitting** (your model is too simple/rigid, like fitting a straight line to a sine wave).
  - **Variance:** How widely your darts scatter around their own average center when the training set changes slightly. High variance = **overfitting** (your model memorizes random training noise).
  - **Irreducible Noise (σ_ϵ^2):** Random wind gusts inherent to the target $y = f(x) + \epsilon$ itself. Even the true function $f(x)$ suffers an expected squared error of σ_ϵ^2!
- **Why ROC-AUC Lies on Rare Events & Why PR-AUC Tells the Truth (`ML07`):** Suppose you are inspecting $10,000$ silicon wafers on a Qualcomm assembly line: 100 **are Defective (Positive)** and $9,900$ **are Normal (Negative)**. A mediocre detector catches 60 defective wafers ($TP = 60, FN = 40$) but triggers 90 false alarms ($FP = 90, TN = 9,810$):
  - **What ROC-AUC sees:** It compares $\text{TPR} = \frac{60}{100} = 60\%$ against $\text{FPR} = \frac{90}{9,900} = 0.91\%$. Because the denominator of FPR contains all $9,900$ true negatives, 90 false alarms look like a tiny drop in the ocean! The ROC curve hugs the top-left corner and reports a glowing score near 1.0.
  - **What PR-AUC sees:** It replaces FPR with $\text{Precision} = \frac{TP}{TP+FP} = \frac{60}{60+90} = \mathbf{40\%}$. Notice that $TN = 9,810$ appears **nowhere** in Precision or Recall! PR-AUC immediately exposes that 60% **of the wafers your detector flags are actually false alarms**.
- **What does ROC-AUC $= 0.85$ actually mean in plain English (`ML08`)?** It does *not* mean 85% accuracy! It means: if you pick one random positive sample x^+ and one random negative sample x^-, there is an 85% **probability that your model assigns a higher raw score to the positive sample than to the negative sample**: $P(\hat{s}(x^+) > \hat{s}(x^-)) = 0.85$. Because it only depends on **which score is bigger (pairwise rank order)**, applying *any* strictly increasing function (like $3s - 10$ or $\sigma(s)$) changes **zero** rankings and leaves ROC-AUC and PR-AUC 100% identical!

📐 The Math & Every Symbol Explained

1. Exact MSE Bias-Variance Decomposition (`ML09`): Assume true target $y = f(x) + \epsilon$ where $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma_\epsilon^2$. Over random training datasets D, the expected squared test error at point x decomposes into three orthogonal terms:

$$\text{MSE}(x) = \mathbb{E}_{D,\epsilon}\left[\left(y - \hat{f}_D(x)\right)^2\right] = \underbrace{\left(\mathbb{E}_D[\hat{f}_D(x)] - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D\left[\left(\hat{f}_D(x) - \mathbb{E}_D[\hat{f}_D(x)]\right)^2\right]}_{\text{Variance}} + \underbrace{\sigma_\epsilon^2}_{\text{Irreducible Noise}}$$

- **k-Nearest Neighbors (k-NN) Complexity Inversion:** For k-NN regression, averaging k neighbors gives $\text{Var}(\hat{f}_{k\text{-NN}}(x)) = \frac{\sigma_\epsilon^2}{k}$. Thus $k = 1$ **is the most complex model (lowest bias, highest variance σ_ϵ^2)**, and increasing $k \to N$ makes the boundary smoother (**increases bias, decreases variance**)!

2. Confusion Matrix Rates, F_β, MCC & Multiclass Averaging (`ML07`, `ML08`, `ML10`):

$$\text{Precision (PPV)} = \frac{TP}{TP + FP}, \qquad \text{Recall (TPR / Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{FPR (Fall-out)} = \frac{FP}{FP + TN} = 1 - \text{Specificity}, \qquad \text{Specificity (TNR)} = \frac{TN}{TN + FP}$$

- **Generalized F_β Score & Matthews Correlation Coefficient (MCC):**

$$F_\beta = (1 + \beta^2)\frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}, \qquad \text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \in [-$$

  - In F_β, Recall is weighted β **times as heavily** as Precision ($\beta = 1 \implies F_1$ harmonic mean; $\beta = 2 \implies$ favors Recall; $\beta = 0.5 \implies$ favors Precision).
  - **MCC** is the Pearson correlation coefficient between true and predicted binary labels; it uses all four cells of the confusion matrix symmetrically and equals 0 for any random or constant-class classifier.

- **Probabilistic Ranking Identity of ROC-AUC (`ML08`):**

$$\text{ROC-AUC} = P\Big(\hat{s}(x^+) > \hat{s}(x^-)\Big) + \frac{1}{2}P\Big(\hat{s}(x^+) = \hat{s}(x^-)\Big)$$

- **Multiclass Macro-F1 vs. Micro-F1 (`ML10`):** For a C-class single-label problem (where every sample has exactly 1 true class and 1 predicted class):

$$\text{Macro-}F_1 = \frac{1}{C}\sum_{c=1}^{C} F_{1,c} \quad \text{(treats all } C \text{ classes equally, even tiny rare classes!)}$$

$$\text{Micro-}F_1 = \frac{2\sum_{c=1}^{C} TP_c}{2\sum_{c=1}^{C} TP_c + \sum_{c=1}^{C} FP_c + \sum_{c=1}^{C} FN_c} = \textbf{Overall Accuracy}$$

  - *Why does* Micro-F_1 = Accuracy*?* In single-label classification, if sample i is really Class A but predicted as Class B, that single mistake counts as **one False Negative for Class A** AND **one False Positive for Class B**. Globally across all classes, $\sum_c FP_c = \sum_c FN_c = N_{\text{wrong}}$ and $\sum_c TP_c = N_{\text{correct}}$, so Micro-Precision = Micro-Recall = Micro-F_1 = $\frac{N_{\text{correct}}}{N}$ = Accuracy!

🔢 Step-by-Step Worked Toy Example

Example 1: Exact MSE Bias-Variance Calculation (`ML09`). At a test point x_0, the true noiseless function value is $f(x_0) = 10.0$, and the observation noise has standard deviation $\sigma_\epsilon = 2.0$. Across many random training sets D, our regression estimator has mean prediction $\mathbb{E}_D[\hat{f}(x_0)] = 8.5$ and prediction variance $\text{Var}_D(\hat{f}(x_0)) = 3.25$. What is the expected test MSE at x_0?

1. **Compute** Bias $= \mathbb{E}_D[\hat{f}(x_0)] - f(x_0)$ **and square it:**

$$\text{Bias} = 8.5 - 10.0 = -1.5 \implies \text{Bias}^2 = (-1.5)^2 = \mathbf{2.25}$$

2. **Identify Variance:**

$$\text{Variance} = \mathbf{3.25}$$

3. **Square the noise standard deviation $\sigma_\epsilon = 2.0$ to get Irreducible Noise variance σ_ϵ^2:**

$$\text{Irreducible Noise} = \sigma_\epsilon^2 = 2.0^2 = \mathbf{4.00}$$

4. **Sum all three components:**

$$\text{MSE}(x_0) = 2.25 + 3.25 + 4.00 = \mathbf{9.50}$$

Example 2: Pairwise Ranking & Monotonic Transform Invariance of ROC-AUC (`ML08`). Suppose we have 2 Positive samples (x_1^+, x_2^+) and 2 Negative samples (x_1^-, x_2^-) with raw model scores:

$$\hat{s}(x_1^+) = 0.8, \quad \hat{s}(x_2^+) = 0.4, \qquad \hat{s}(x_1^-) = 0.6, \quad \hat{s}(x_2^-) = 0.2$$

1. **Compute ROC-AUC by checking all $2 \times 2 = 4$ (Positive, Negative) pairs:**
   - Pair 1 (x_1^+, x_1^-): Is $0.8 > 0.6$? **Yes (1)**
   - Pair 2 (x_1^+, x_2^-): Is $0.8 > 0.2$? **Yes (1)**
   - Pair 3 (x_2^+, x_1^-): Is $0.4 > 0.6$? **No (0)**
   - Pair 4 (x_2^+, x_2^-): Is $0.4 > 0.2$? **Yes (1)**

   $$\text{ROC-AUC} = \frac{1+1+0+1}{4} = \frac{3}{4} = \mathbf{0.75}$$

2. **Apply a strictly increasing transformation $g(s) = 3s - 10$ to every score:**

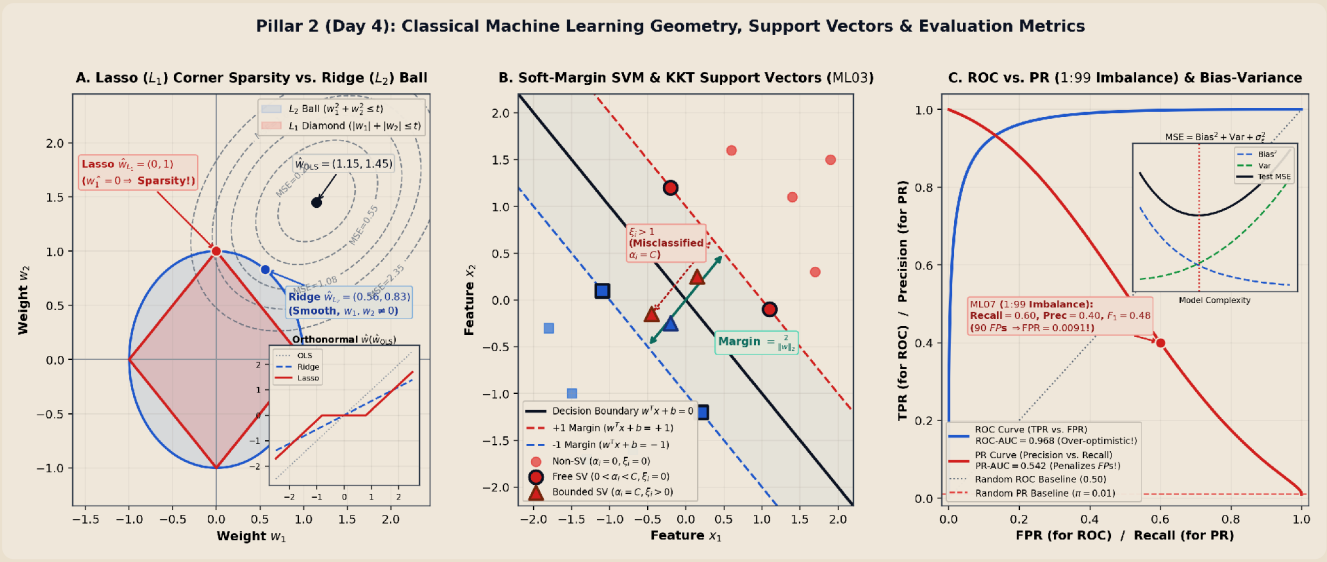
   $$g(x_1^+) = -7.6, \quad g(x_2^+) = -8.8, \qquad g(x_1^-) = -8.2, \quad g(x_2^-) = -9.4$$

   Check the 4 pairs again: $(-7.6 > -8.2)$ ✅, $(-7.6 > -9.4)$ ✅, $(-8.8 > -8.2)$ ❌, $(-8.8 > -9.4)$ ✅. The new ROC-AUC is still $\frac{3}{4} = 0.75$! *(If instead we applied a strictly decreasing function $h(s) = -3s + 10$, every inequality would flip and ROC-AUC would become $1 - 0.75 = \mathbf{0.25}$).*

🎯 Exam Takeaway & Common Traps (`ML07` – `ML10`)

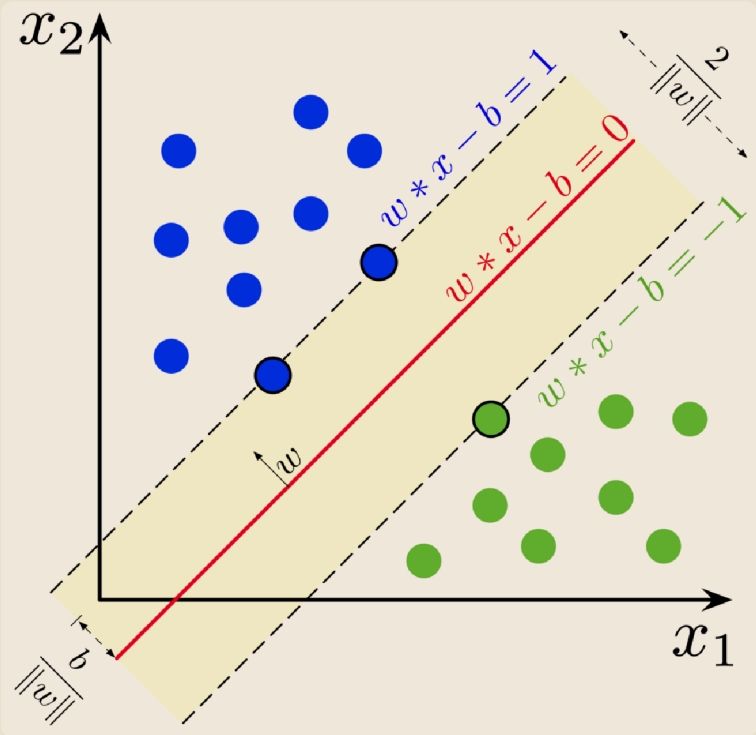
- **Trap 1 (`ML07` — Extreme Class Imbalance):** When negatives vastly outnumber positives ($TN \gg TP$), **FPR and ROC-AUC are overly optimistic** because huge TN crushes $\text{FPR} = \frac{FP}{FP+TN} \approx 0$. Always choose **PR-AUC** to evaluate rare-positive detection.
- **Trap 2 (`ML08` — Monotonic Transform Invariance):** Any strictly increasing transform $g(s)$ (e.g., $3s - 10$, e^s, $\sigma(s)$) leaves **both ROC-AUC and PR-AUC** 100% **unchanged** (though it *will* change fixed-threshold F1 and calibration!).
- **Trap 3 (`ML09` — Noise Standard Deviation vs. Variance):** If an exam problem gives noise **standard deviation** $\sigma_\epsilon = 2$, remember to **square it ($\sigma_\epsilon^2 = 4$)** before adding $\text{Bias}^2 + \text{Variance} + \sigma_\epsilon^2$! Also remember: fitting `StandardScaler`, PCA, or SMOTE *before* `KFold` splitting causes **data leakage**.
- **Trap 4 (`ML10` — Macro-F1 vs. Micro-F1):** On imbalanced multiclass data, **Micro-F1 equals Overall Accuracy** (dominated by majority classes), whereas **Macro-F1 weights every class equally** ($1/C$) and drops sharply if rare classes fail.

FIGURE SLIDE 1 OF 4



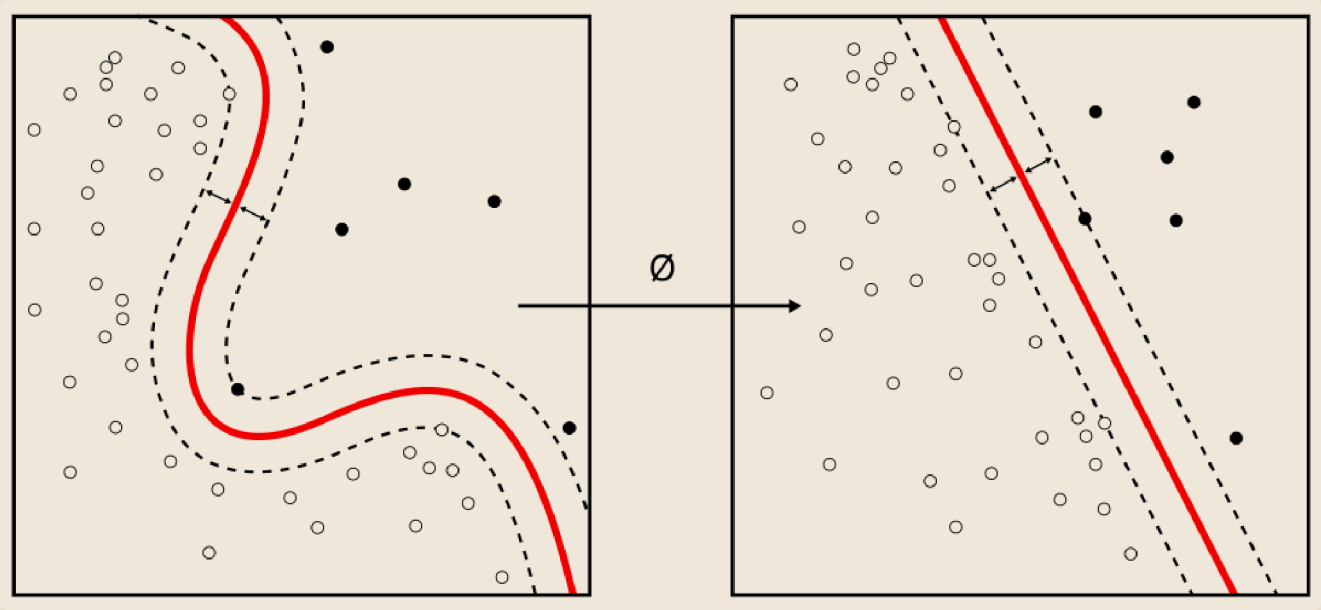
Pillar 2 Day 4 Classical ML Visuals

FIGURE SLIDE 2 OF 4



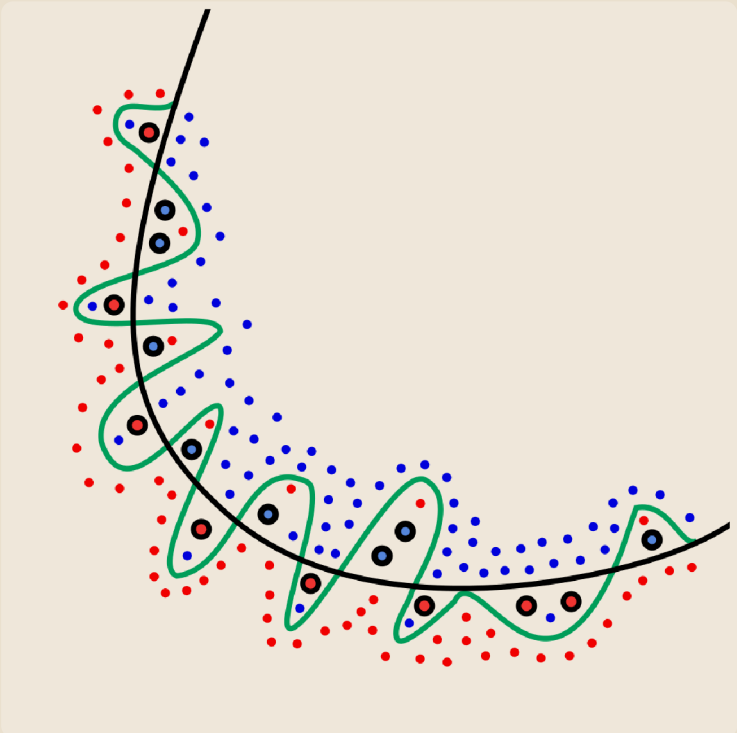
Canonical SVM Margin Geometry (Wikimedia)

FIGURE SLIDE 3 OF 4



Kernel Machine Feature Space Mapping (Wikimedia)

FIGURE SLIDE 4 OF 4



High-Variance Overfitting vs Smooth Decision Boundary (Wikimedia)

Day 5: Deep Learning Core Mechanics (Activations, Norms, Init, Optimizers)

↑ Mục Lục

2.1 Activation Functions & Their Failure Modes (ML11)

💡 Intuition (Plain English First)

Why do we need activation functions at all? Because if you stack 50 linear layers without a non-linearity in between, matrix multiplication is associative: $W_{50}(\dots W_2(W_1x)) = (W_{50}\cdots W_2W_1)x = W_{\text{single}}x$. Your 50-layer deep network collapses into a single flat linear regression! However, choosing the wrong non-linear activation function introduces three famous training pathologies (ML11):

1. **Vanishing Gradients (Sigmoid & Tanh Saturation):** Sigmoid squashes $(-\infty, +\infty)$ into $(0, 1)$. Not only does its slope flatten to ≈ 0 at the tails ($|x| > 4$), even at its **steepest point ($x = 0$), Sigmoid's derivative is only** 0.25! During backpropagation, multiplying by ≤ 0.25 at every layer cuts the gradient by at least $4\times$ per layer.
2. **Zig-Zagging Weight Updates (Why Non-Zero-Centered Outputs Hurt):** Sigmoid outputs are strictly positive ($x_i \in (0, 1)$). When those positive numbers feed into the *next* linear layer $z = w^Tx + b$, the gradient on that layer's weight vector is $\nabla_w \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial z}\right) x$. Because every entry of x is positive, **every weight gradient $\frac{\partial \mathcal{L}}{\partial w_i}$ is forced to have the exact same sign**! The optimizer can only move all weights up together $(+, +)$ or all weights down together $(-, -)$, forcing a slow diagonal zig-zag whenever the true minimum requires increasing w_1 while decreasing w_2.
3. **Dying ReLU:** $\text{ReLU}(x) = \max(0, x)$ fixes positive saturation because its slope is a clean 1.0 for all $x > 0$. But for $x < 0$, its output and slope are **exact** 0. If a large learning-rate step pushes a neuron's bias b strongly negative so that $w^Tx + b < 0$ for *every* sample in the dataset, the neuron outputs 0 forever, receives 0 gradient forever, and never wakes up!

📐 The Math & Every Symbol Explained

Activation	Forward Equation $f(x)$	Derivative $f'(x)$ & Range	Zero-Centered?	Key Properties & Exam Traps (ML11)
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x)) \in (0, \mathbf{0.25}]$	❌ No $\in (0, 1)$	Max derivative is $\sigma'(0) = 0.25 \implies \geq 4\times$ gradient shrinkage per layer; all-positive output causes **zig-zagging gradients** on next layer's W!
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$	$1 - \tanh^2(x) \in (0, \mathbf{1.0}]$	✅ Yes $\in (-1, 1)$	$\tanh'(0) = 1.0$ and zero-centered (fixes zig-zagging); still saturates as $\lvert x\rvert \to \infty$.
ReLU	$\max(0, x)$	$\begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases} \in \{0, 1\}$	❌ No $\in [0, \infty)$	Non-saturating for $x > 0$; suffers from **Dying ReLU** if $w^Tx + b < 0$ for all inputs (0 gradient forever).
LeakyReLU / PReLU	$\max(\alpha x, x)$	$\begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases} \in \{\alpha, 1\}$	Nearly	Fixes Dying ReLU via small negative slope α (e.g. 0.01 in LeakyReLU, or learnable parameter in PReLU).
GELU	$x\Phi(x) \approx x\sigma(1.702x)$	Smooth non-monotonic	Nearly	Standard in BERT/GPT/ViT ($\Phi(x)$ is standard Gaussian CDF); smooth negative trough near $x \approx -0.75$ gives non-zero gradient.

Activation	Forward Equation $f(x)$	Derivative $f'(x)$ & Range	Zero-Centered?	Key Properties & Exam Traps (ML11)
Swish / SiLU	$x\sigma(x)$	$\sigma(x) + x\sigma(x)(1-\sigma(x))$	Nearly	Smooth self-gated activation used inside **SwiGLU** FFNs in modern LLMs (LLaMA, Qwen).

The Zig-Zag Gradient Equation: For a single neuron $z = w^T x + b = \sum_{i=1}^{d} w_i x_i + b$ receiving activation vector $x \in \mathbb{R}^{d\times 1}$ from the previous layer:

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\left(\frac{\partial \mathcal{L}}{\partial z}\right)}_{\text{scalar } \delta \in \mathbb{R}} \cdot \underbrace{x}_{\text{vector } \in \mathbb{R}^{d\times 1}} \implies \frac{\partial \mathcal{L}}{\partial w_i} = \delta \cdot x_i$$

If $x_i > 0$ for all $i = 1, \ldots, d$, then sign $\left(\frac{\partial \mathcal{L}}{\partial w_i}\right) = \text{sign}(\delta)$ is **identical for every weight** w_i!

🔢 Step-by-Step Worked Toy Example

Part 1: Watching Sigmoid Crush Gradients by $1,024\times$ in 5 Layers. Suppose we have a 5-layer network using Sigmoid activations where every layer's pre-activation is right at the origin $z^{(l)} = 0$ (the **best possible case** where Sigmoid's slope is at its maximum) and weights are $w^{(l)} = 1.0$:

1. Evaluate Sigmoid and its derivative at $z = 0$:

$$\sigma(0) = \frac{1}{1+e^0} = 0.5, \qquad \sigma'(0) = \sigma(0)\big(1-\sigma(0)\big) = 0.5 \times 0.5 = \mathbf{0.25}$$

2. Propagate a unit gradient $\frac{\partial \mathcal{L}}{\partial a^{(5)}} = 1.0$ backward through 5 Sigmoid layers via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial z^{(1)}} = (0.25)^5 = \left(\frac{1}{4}\right)^5 = \frac{1}{1,024} \approx \mathbf{0.000977}$$

Even with zero saturation ($z = 0$ everywhere!), the first layer's gradient is **over $1,000\times$ smaller** than the last layer's gradient!

Part 2: Concrete Numerical Proof of the Zig-Zag Weight Update (ML11). Suppose a neuron has current weights $w = [1.0,\ 1.0]^T$ and the true optimal weights are $w^* = [2.0,\ 0.0]^T$. To reach w^* in a straight line, we need weight change $\Delta w^* = [+1.0,\ -1.0]^T$ (w_1 must **increase**, w_2 must **decrease**). Now suppose the input from the previous Sigmoid layer is strictly positive: $x = [0.6,\ 0.8]^T > 0$, with learning rate $\eta = 1.0$:

- **If upstream scalar gradient $\delta = \frac{\partial \mathcal{L}}{\partial z} = -1.0$:**

$$\nabla_w \mathcal{L} = \delta \cdot x = [-0.6,\ -0.8]^T \implies \Delta w = -\eta \nabla_w \mathcal{L} = [+0.6,\ +0.8]^T \quad \text{(BOTH weights increase!)}$$

- **If upstream scalar gradient $\delta = \frac{\partial \mathcal{L}}{\partial z} = +1.0$:**

$$\nabla_w \mathcal{L} = \delta \cdot x = [+0.6,\ +0.8]^T \implies \Delta w = -\eta \nabla_w \mathcal{L} = [-0.6,\ -0.8]^T \quad \text{(BOTH weights decrease!)}$$

Because $x_1 > 0$ and $x_2 > 0$, **no single gradient step can ever move in the $[+,-]$ quadrant**—the optimizer must overshoot in $[+,+]$ on one step and then correct in $[-,-]$ on the next step (zig-zagging)!

🎯 Exam Takeaway & Common Traps (ML11)

- **Trap 1 (`ML11` — Sigmoid vs. Tanh Max Slope):** Sigmoid's maximum derivative is $\sigma'(0) = 0.25$ (not 1.0!), whereas Tanh's maximum derivative is $\tanh'(0) = 1.0$ and Tanh is zero-centered.
- **Trap 2 (`ML11` — Why Non-Zero-Centered Activations Zig-Zag):** All-positive activations $x > 0$ force the gradient $\nabla_w \mathcal{L} = \frac{\partial \mathcal{L}}{\partial z} x$ of the **subsequent** linear layer to lie strictly in the all-positive or all-negative orthant.
- **Trap 3 (`ML11` — Dying ReLU vs. GELU/LeakyReLU):** Standard ReLU has an exact 0 **derivative for all** $x < 0$, permanently killing neurons whose pre-activations become negative across the training set. LeakyReLU (slope α for $x < 0$) and GELU/SiLU (smooth negative trough) keep gradients alive.

2.2 Normalization Layers & Inverted Dropout: Shapes, Buffers & Traps (`ML12`, `ML13`)

💡 Intuition (Plain English First)

- **Why Normalize Internal Layers? (`ML12`):** As weights in Layer 1 update during training, the distribution of inputs arriving at Layer 2 shifts and scales wildly, forcing Layer 2 to constantly chase a moving target. Normalization layers pin each layer's activations back to **mean** 0 **and variance** 1, then apply a learnable scale γ and shift β so the network can undo the normalization if it wants to.
- **The Visual Classroom Analogy for `BatchNorm` vs. `LayerNorm`:** Imagine a spreadsheet where **rows are** N **students (samples in a batch)** and **columns are** C **exam questions (channels/features)**:
  - `BatchNorm` **normalizes DOWN each column (across all** N **students for Question** c**):** "*How did Student* n *score on Question* c *compared to the rest of the classroom?*" That's why BatchNorm needs a big enough classroom ($N \geq 8$) during training AND must save a `running_mean` and `running_var` notebook so that when a single student ($N = 1$) shows up alone at test time (`model.eval()`), it knows what the classroom average was!
  - `LayerNorm` **normalizes ACROSS each row (across all** D **questions for a single Student** n**):** "*How did Student* n *score on Question* d *compared to Student* n*'s own average across all questions?*" Because LayerNorm never looks at other students in the batch, it works identically at batch size $N = 1$, needs **zero running buffers**, and has the **exact same formula in `model.train()` and `model.eval()`**!
- **Why "Inverted" Dropout? (`ML13`):** During training, Dropout randomly turns off a fraction p of neurons (keeping $1 - p$ of them) to prevent co-adaptation. If you turn off 25% of neurons ($p = 0.25$) during training, the next layer only receives 75% of its usual total input energy. Instead of multiplying weights by 0.75 at test time, **Inverted Dropout divides the surviving neurons by** $(1 - p) = 0.75$ **during training** so their expected value stays 100% unchanged and `model.eval()` **becomes a pure no-op (identity function)**!

📐 The Math & Every Symbol Explained

Let's define the input tensor shapes first:

- **Computer Vision (4D Tensor):** $X \in \mathbb{R}^{N \times C \times H \times W}$, where N **= batch size (samples)**, C **= channels**, H **= spatial height**, W **= spatial width**.
- **Transformers / NLP (3D Tensor):** $X \in \mathbb{R}^{N \times L \times D}$, where N **= batch size (sequences)**, L **= sequence length (tokens)**, D **= hidden embedding dimension (features)**.

General Normalization Equation:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y = \gamma \odot \hat{x} + \beta$$

Normalization Layer	Tensor Shape	Axes Averaged for μ, σ^2	Learnable Params (γ, β)	Running Buffers (`mean`, `var`)	Key Behavior & Use Case (`ML12`)
`nn.BatchNorm2d(C)`	(N, C, H, W)	(N, H, W) per channel c ($N \cdot H \cdot W$ values)	$2C$ ($\gamma, \beta \in \mathbb{R}^C$)	$2C$ (EMA updated in `train()`, frozen in `eval()`)	Standard in CNNs; fails if $N = 1$; set preceding Conv `bias=False`!
`nn.LayerNorm(D)`	(N, L, D)	(D) per token (n, l) (D values)	$2D$ ($\gamma, \beta \in \mathbb{R}^D$)	0 (None)	Standard in Transformers; identical in `train()` and `eval()`.
`RMSNorm(D)`	(N, L, D)	(D) per token (root-mean-square only, no μ)	$1D$ ($\gamma \in \mathbb{R}^D$ only, no β)	0 (None)	Used in LLaMA/Mistral/Qwen: $y = \frac{x}{\sqrt{\frac{1}{D}\sum x_i^2 + \epsilon}} \odot \gamma$.
`nn.InstanceNorm2d(C)`	(N, C, H, W)	(H, W) per (n, c) ($H \cdot W$ values)	$2C$ (if `affine=True`)	0 (by default)	Style Transfer; equivalent to `GroupNorm(G=C, C)`.
`nn.GroupNorm(G, C)`	(N, C, H, W)	$(C/G, H, W)$ per (n, g)	$2C$ ($\gamma, \beta \in \mathbb{R}^C$)	0 (None)	Batch-size independent CV norm; $G = 1 \iff$ LayerNorm, $G = C \iff$ InstanceNorm.

Why Preceding `bias=False` Before `BatchNorm` (`ML12`): Suppose a `Linear` or `Conv2d` layer computes $z = Wx + b$. When `BatchNorm` immediately normalizes z, the mean is $\mathbb{E}[z] = \mathbb{E}[Wx + b] = \mathbb{E}[Wx] + b$. Subtracting the mean gives:

$$z - \mathbb{E}[z] = (Wx + b) - (\mathbb{E}[Wx] + b) = Wx - \mathbb{E}[Wx]$$

The bias b **cancels out algebraically to exact zero**, and BatchNorm's own learnable shift β plays the role of the bias afterward!

Inverted Dropout Equation (`ML13`): Given drop probability $p \in [0, 1)$ (so keep probability is $q = 1 - p$) and binary mask $m_i \sim \text{Bernoulli}(1 - p)$:

- **During `model.train()`:**

$$\tilde{a}_i = \frac{m_i \cdot a_i}{1 - p} = \begin{cases} \dfrac{a_i}{1 - p} & \text{with probability } 1 - p \\ 0 & \text{with probability } p \end{cases}$$

Checking expectation: $\mathbb{E}[\tilde{a}_i] = (1-p)\left(\frac{a_i}{1-p}\right) + p(0) = a_i$.

- **During `model.eval()`:**

$$\tilde{a}_i = \mathbf{1.0} \cdot \mathbf{a_i} = \mathbf{a_i} \quad \text{(exact identity — no scaling at test time!)}$$

🔢 Step-by-Step Worked Toy Example

Example 1: `BatchNorm` vs. `LayerNorm` on a 2×2 Matrix (`ML12`). Suppose we have a mini-batch of $N = 2$ samples (rows) and $D = 2$ features/channels (columns), with $\gamma = 1, \beta = 0, \epsilon = 0$:

$$X = \begin{bmatrix} 2 & 6 \\ 4 & 10 \end{bmatrix} \begin{matrix} \leftarrow \text{Sample 1} \\ \leftarrow \text{Sample 2} \end{matrix}$$

1. **`BatchNorm1d(2)` (Normalizes DOWN each column across the $N = 2$ samples):**
   - **Column 1 ($[2, 4]^T$):** Mean $\mu_1 = \frac{2+4}{2} = 3$, Variance $\sigma_1^2 = \frac{(2-3)^2+(4-3)^2}{2} = 1 \implies \sigma_1 = 1$. Normalized Column 1: $\left[\frac{2-3}{1}, \frac{4-3}{1}\right]^T = [-1, +1]^T$.
   - **Column 2 ($[6, 10]^T$):** Mean $\mu_2 = \frac{6+10}{2} = 8$, Variance $\sigma_2^2 = \frac{(6-8)^2+(10-8)^2}{2} = 4 \implies \sigma_2 = 2$. Normalized Column 2: $\left[\frac{6-8}{2}, \frac{10-8}{2}\right]^T = [-1, +1]^T$.
   - **BatchNorm Output:** $\hat{X}_{\text{BN}} = \begin{bmatrix} -1 & -1 \\ +1 & +1 \end{bmatrix}$ (Sample 1 is below the batch average on both features; Sample 2 is above on both).
2. **`LayerNorm(2)` (Normalizes ACROSS each row within each individual sample):**
   - **Row 1 ($[2, 6]$):** Mean $\mu^{(1)} = \frac{2+6}{2} = 4$, Variance $(\sigma^{(1)})^2 = \frac{(2-4)^2+(6-4)^2}{2} = 4 \implies \sigma^{(1)} = 2$. Normalized Row 1: $\left[\frac{2-4}{2}, \frac{6-4}{2}\right] = [-1, +1]$.
   - **Row 2 ($[4, 10]$):** Mean $\mu^{(2)} = \frac{4+10}{2} = 7$, Variance $(\sigma^{(2)})^2 = \frac{(4-7)^2+(10-7)^2}{2} = 9 \implies \sigma^{(2)} = 3$. Normalized Row 2: $\left[\frac{4-7}{3}, \frac{10-7}{3}\right] = [-1, +1]$.
   - **LayerNorm Output:** $\hat{X}_{\text{LN}} = \begin{bmatrix} -1 & +1 \\ -1 & +1 \end{bmatrix}$ (For both samples, Feature 2 is larger than Feature 1).

Example 2: Counting Parameters & Averages in `nn.BatchNorm2d(64)` (`ML12`). Given input tensor shape $(N = 16,\ C = 64,\ H = 32,\ W = 32)$:

- **How many scalar values are averaged to compute a single channel's μ_c?**

$$N \times H \times W = 16 \times 32 \times 32 = \mathbf{16,384} \text{ values per channel}$$

- **How many learnable parameters (γ, β)?** $2 \times C = 2 \times 64 = \mathbf{128}$ (independent of H and W!).
- **How many non-trainable running state buffers?** $2 \times C = 2 \times 64 = \mathbf{128}$ (`running_mean` $\in \mathbb{R}^{64}$ and `running_var` $\in \mathbb{R}^{64}$).

Example 3: Inverted Dropout Calculation (`ML13`). Suppose `nn.Dropout(p=0.25)` receives activation $a = 6.0$:

- **In `model.train()`:** If the neuron is not dropped (happens with probability $1 - 0.25 = 0.75$), its output is scaled up to:

$$\tilde{a} = \frac{6.0}{1 - 0.25} = \frac{6.0}{0.75} = \mathbf{8.0}$$

- **In `model.eval()`:** Dropout does nothing at all, outputting $\tilde{a} = \mathbf{6.0}$.

🎯 Exam Takeaway & Common Traps (`ML12`, `ML13`)

- **Trap 1 (`ML12` — `train()` vs. `eval()` Mode):** Forgetting `model.eval()` at inference time causes `BatchNorm` to normalize using current test-batch statistics (catastrophic if test batch size $N = 1$) and keeps `Dropout` randomly zeroing activations! `LayerNorm` and `GroupNorm` have **zero running buffers** and behave identically in `train()` and `eval()`.
- **Trap 2 (`ML12` — GroupNorm Equivalences):** `GroupNorm(G=1, C)` $\equiv$ `LayerNorm` over (C, H, W); `GroupNorm(G=C, C)` $\equiv$ `InstanceNorm2d(C)`.
- **Trap 3 (`ML13` — Inverted Dropout Multiplier):** Scaling by $\frac{1}{1-p}$ happens **only during training**; test-time multiplier is 1.0.

2.3 Weight Initialization Derivations & Symmetry Breaking (`ML14`, `ML15`)

💡 Intuition (Plain English First)

- **The Whisper-Down-the-Hallway Analogy (`ML14`):** Imagine a 100-layer neural network as a line of 100 people passing a signal forward (and passing gradients backward).
  - If each layer's weights are slightly too large so that activation variance doubles at every layer ($\text{Var}(y) = 2\,\text{Var}(x)$), then by Layer 100 the variance is $2^{100} \approx 10^{30}$ (**Exploding Activations/Gradients** $\rightarrow$ `NaN`).
  - If each layer's weights are slightly too small so that activation variance halves at every layer ($\text{Var}(y) = 0.5\,\text{Var}(x)$), then by Layer 100 the variance is $0.5^{100} \approx 10^{-30}$ (**Vanishing Activations/Gradients** $\rightarrow$ **zero learning**).
  - **Xavier (Glorot) and Kaiming (He) Initialization** solve for the exact starting weight variance $\text{Var}(w)$ that keeps signal variance **constant** ($\text{Var}(y) = \text{Var}(x)$) from Layer 1 to Layer 100!
- **Why Does ReLU Need $2\times$ the Weight Variance of Linear/Tanh? (`ML14`):** A symmetric activation like $\tanh(z) \approx z$ near zero lets both positive and negative signals pass through. **ReLU ($\max(0, z)$) chops off the entire negative half of the bell curve and sets it to zero**, cutting the signal variance in half ($\frac{1}{2}$)! To compensate for ReLU throwing away half the variance at every layer, **Kaiming (He) Initialization doubles the weight variance** from $\frac{1}{n_{\text{in}}}$ to $\frac{2}{n_{\text{in}}}$.
- **Why Can't We Just Initialize All Weights to $W = 0$? (`ML15` — The Symmetry Trap):** In convex logistic regression (1 layer), initializing $w = 0$ works fine. In a multi-layer neural network, if you initialize a hidden layer's weight matrix to $W = 0$ (or to any constant number like 0.5), **every neuron in that hidden layer has the exact same incoming weights, computes the exact same forward activation, and receives the exact same backward gradient**! Even after $10,000$ steps of SGD, all 512 neurons in that layer remain identical clones of a single neuron. Random initialization (Xavier/Kaiming) is mandatory to **break symmetry**!

📐 The Math & Every Symbol Explained

Consider a single linear layer neuron $y = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ (with bias $b = 0$), where weights w_i and inputs x_i are mutually independent and weights have zero mean ($\mathbb{E}[w_i] = 0$):

$$\text{Var}(y) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \cdot \text{Var}(w) \cdot \mathbb{E}[x^2]$$

Where every symbol means:

- n_{in} (fan_in): Number of inputs feeding *into* each neuron of the layer (for `nn.Linear(in_features, out_features)`, n_{in} = in_features; for `nn.Conv2d(C_in, C_out, k, k)`, $n_{\text{in}} = C_{\text{in}} \times k \times k$).
- n_{out} (fan_out): Number of output neurons in the layer.
- $\text{Var}(w) = \sigma_w^2$: Variance of each randomly initialized weight w_i.
- $\mathbb{E}[x^2]$: Second moment (average squared value) of the input activation x coming out of the previous layer's activation function.

Case 1: Linear / Tanh / Symmetric Activation ($\mathbb{E}[x] = 0 \implies \mathbb{E}[x^2] = \text{Var}(x)$):

- Forward variance preservation ($\text{Var}(y) = \text{Var}(x)$) requires $n_{\text{in}} \text{Var}(w) = 1 \implies \text{Var}(w) = \frac{1}{n_{\text{in}}}$.
- Backward gradient variance preservation requires $n_{\text{out}} \text{Var}(w) = 1 \implies \text{Var}(w) = \frac{1}{n_{\text{out}}}$.
- Compromising via the harmonic mean gives **Xavier / Glorot Initialization**:

$$\text{Var}(w)_{\text{Xavier}} = \frac{2}{n_{\text{in}} + n_{\text{out}}} \implies \text{Normal Std: } \sigma_w = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}, \qquad \text{Uniform } [-a, a] : a = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$$

Case 2: ReLU Activation ($x = \max(0, z)$ where $z \sim \mathcal{N}(0, \text{Var}(z))$) (`ML14`):

- Because ReLU zeroes out the negative half ($z < 0$) of the zero-mean symmetric pre-activation z:

$$\mathbb{E}[x^2] = \int_{-\infty}^{0} 0^2 p(z)\, dz + \int_{0}^{\infty} z^2 p(z)\, dz = \frac{1}{2} \text{Var}(z)$$

- Substituting $\mathbb{E}[x^2] = \frac{1}{2} \text{Var}(z)$ into the forward variance equation gives $\text{Var}(y) = n_{\text{in}} \text{Var}(w) \left(\frac{1}{2} \text{Var}(z)\right)$. Setting $\text{Var}(y) = \text{Var}(z)$ requires $n_{\text{in}} \text{Var}(w) \cdot \frac{1}{2} = 1$, which gives **Kaiming / He Initialization**:

$$\text{Var}(w)_{\text{Kaiming}} = \frac{2}{n_{\text{in}}} \implies \text{Normal Std: } \sigma_w = \sqrt{\frac{2}{n_{\text{in}}}}, \qquad \text{Uniform } [-a, a] : a = \sqrt{\frac{6}{n_{\text{in}}}}$$

(Why does Uniform have a 6 instead of a 2 inside the square root? Because the variance of $\text{Uniform}(-a, a)$ *is* $\frac{(2a)^2}{12} = \frac{a^2}{3}$*, so setting* $\frac{a^2}{3} = \text{Var}(w)$ *gives* $a = \sqrt{3 \cdot \text{Var}(w)}$*!)*

🔢 Step-by-Step Worked Toy Example

Example 1: Computing Kaiming He Normal & Uniform Initialization for a ReLU Layer (`ML14`). Suppose you have a PyTorch layer `nn.Linear(in_features=512, out_features=256)` followed by a `ReLU` activation:

1. Identify $n_{\text{in}} = 512$ and $n_{\text{out}} = 256$.
2. Compute Kaiming (He) fan-in variance:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}} = \frac{2}{512} = \frac{1}{256}$$

3. Take the square root to get the **Kaiming Normal standard deviation** σ_w:

$$\sigma_w = \sqrt{\frac{1}{256}} = \frac{1}{16} = \mathbf{0.0625} \implies w_i \sim \mathcal{N}(0,\ 0.0625^2)$$

4. Multiply by $\sqrt{3}$ to get the **Kaiming Uniform bound** $[-a, a]$:

$$a = \sqrt{3 \cdot \mathrm{Var}(w)} = \sqrt{\frac{6}{512}} = \frac{\sqrt{3}}{16} \approx \mathbf{0.10825} \implies w_i \sim \mathrm{Uniform}(-0.10825,\ +0.10825)$$

Example 2: Line-by-Line Proof of the All-Zero Weight Symmetry Trap (`ML15`). Suppose a 2-neuron hidden layer $\mathrm{ReLU}(W^{(1)}x + b^{(1)})$ feeds into a 1-neuron output layer $\hat{y} = w^{(2)T}h$, and we mistakenly initialize **all weights to zero**:

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad w^{(2)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ (even if output weights are non-zero!)}$$

Let input $x = [3,\ 5]^T$ and true label $y = 10$ with loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$:

1. **Forward Pass:**
   - Neuron 1 pre-activation: $z_1 = 0(3) + 0(5) + 0 = 0 \implies h_1 = 0$.
   - Neuron 2 pre-activation: $z_2 = 0(3) + 0(5) + 0 = 0 \implies h_2 = 0$.
   - Output: $\hat{y} = 1(0) + 1(0) = 0 \implies \frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = -10$.
2. **Backward Pass (assuming LeakyReLU/Linear slope $= 1$ at 0 so gradients aren't even blocked by zero slope):**
   - Gradient arriving at Neuron 1: $\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_1^{(2)} = -10 \times 1 = -10$.
   - Gradient arriving at Neuron 2: $\frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_2^{(2)} = -10 \times 1 = -10$.
   - Weight gradient for Row 1 (Neuron 1): $\nabla_{w_1^{(1)}} \mathcal{L} = -10 \cdot [3,\ 5] = [-30,\ -50]$.
   - Weight gradient for Row 2 (Neuron 2): $\nabla_{w_2^{(1)}} \mathcal{L} = -10 \cdot [3,\ 5] = [-30,\ -50]$.
3. **Result after SGD step ($\eta = 0.01$):**

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 0.30 & 0.50 \\ 0.30 & 0.50 \end{bmatrix}$$

Row 1 and Row 2 are still 100% identical! Neuron 1 and Neuron 2 are permanently locked together as clones forever. *(Note: Initializing **biases** $b = 0$ is completely fine and standard as long as W is initialized randomly!)*

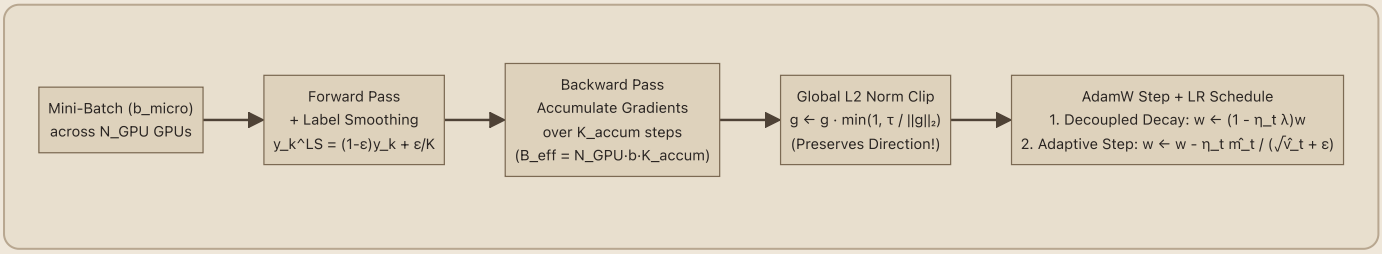
🎯 Exam Takeaway & Common Traps (`ML14`, `ML15`)

- **Trap 1 (`ML14` — ReLU Factor of 2):** Always use **Kaiming / He** ($\sigma_w = \sqrt{2/n_{\text{in}}}$) for ReLU networks, and **Xavier / Glorot** ($\sigma_w = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$) for Tanh/Sigmoid/Linear networks.
- **Trap 2 (`ML15` — Zero Weights vs. Zero Biases):** Initializing **weights** $W = 0$ (or any identical constant) fails to break symmetry and locks all hidden neurons in a layer into identical updates forever. Initializing **biases** $b = 0$ (while W is random) is completely safe and standard!

2.4 Modern Optimizers, Decoupled Weight Decay & Training Schedules (`ML16` – `ML20`)

💡 Intuition (Plain English First)

- **How Optimizers Evolve from Vanilla SGD to Adam (`ML16`):** Imagine rolling a ball down a long, narrow canyon whose side walls are super steep (high curvature) while the canyon floor slopes gently toward the minimum:
  - **Vanilla SGD** bounces wildly back and forth between the steep canyon walls while barely inching along the canyon floor.
  - **SGD with Momentum** gives the ball physical mass (v_t): the opposite wall-bounces cancel out, while the consistent downhill pushes along the canyon floor build up speed! (**Nesterov Momentum** is even smarter: it first takes the momentum step to a "lookahead" position and evaluates the gradient *there*, braking before it overshoots).
  - **AdaGrad & RMSProp** adjust the step size for each coordinate individually by dividing by the square root of past squared gradients $\sqrt{v_t}$: steep canyon walls (large g_i^2) get divided by a big number (smaller steps), while flat directions (tiny g_i^2) get divided by a small number (bigger steps). *(AdaGrad sums g_t^2 from $t = 1 \to \infty$ without forgetting, so its learning rate eventually dies to 0; **RMSProp** fixes this with a leaky moving average).*
  - **Adam (Adaptive Moment Estimation)** combines **Momentum (1st moment m_t) + RMSProp (2nd moment v_t) + Bias Correction ($\hat{m}_t, \hat{v}_t$)** (because initializing $m_0 = 0, v_0 = 0$ biases both moving averages toward zero during the first few steps!).
- **Why L_2 Regularization $\neq$ Weight Decay in Adam (`AdamW`, `ML17`):** In vanilla SGD, adding $\frac{\lambda}{2}\|w\|_2^2$ to the loss adds λw to the gradient, which subtracts $\eta\lambda w$ from the weights—identical to decaying weights by $(1 - \eta\lambda)w$. **In Adam, they are NOT equivalent!** If you add $\frac{\lambda}{2}\|w\|_2^2$ to the loss, the penalty λw gets mixed into the gradient g_t and then **divided by the adaptive preconditioner** $\sqrt{\hat{v}_t}$! That means weights with large historical gradients get almost *zero* weight decay, while weights with tiny gradients get heavily decayed. **AdamW** fixes this by **decoupling** weight decay from the gradient and shrinking w directly by $(1 - \eta_t\lambda)w$!
- **Label Smoothing (`ML18`), Effective Batch Size (`ML19`) & LR Schedules/Clipping (`ML20`):**
  - **Label Smoothing:** Instead of forcing softmax to chase hard $[0, 1, 0]$ targets (which drives logits $z_k \to +\infty$ and makes the model over-confident), we shave a tiny ϵ (e.g., 0.1) off the 1.0 and spread it evenly ($\frac{\epsilon}{K}$) across all K classes.
  - **Effective Batch Size:** If each GPU can only fit $b_{\text{micro}} = 8$ samples in VRAM, we can simulate a massive batch size by running across N_{GPU} GPUs in parallel AND accumulating gradients over K_{accum} forward/backward passes before calling `optimizer.step()`.
  - **Global L_2 Norm Clipping vs. Value Clamping:** If a bad mini-batch produces a giant gradient vector g, clamping each coordinate independently to $[-\tau, +\tau]$ **changes the angle/direction** of the vector! Rescaling the whole vector by $\frac{\tau}{\|g\|_2}$ shrinks its length to τ while **preserving its exact direction**.



📐 The Math & Every Symbol Explained

1. Momentum vs. Nesterov Accelerated Gradient (NAG):

- **Standard Momentum:** $v_{t+1} = \mu v_t - \eta \nabla \mathcal{L}(w_t), \qquad w_{t+1} = w_t + v_{t+1}$ (gradient evaluated at **current** w_t).
- **Nesterov (NAG):** $v_{t+1} = \mu v_t - \eta \nabla \mathcal{L}(w_t + \mu v_t), \qquad w_{t+1} = w_t + v_{t+1}$ (gradient evaluated at **lookahead** $w_t + \mu v_t$).

2. Adam & AdamW Update Equations (`ML16`, `ML17`): Given gradient $g_t = \nabla_w \mathcal{L}_t(w_{t-1})$ at step $t \geq 1$ (initialized with $m_0 = 0,\ v_0 = 0$):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad \text{(1st moment EMA of gradient, default } \beta_1 = 0.9\text{)}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \qquad \text{(2nd moment EMA of squared gradient, default } \beta_2 = 0.999\text{)}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \qquad \text{(Bias-corrected 1st and 2nd moments at step } t\text{)}$$

$$w_t = \underbrace{(1 - \eta_t \lambda) w_{t-1}}_{\text{Decoupled Weight Decay (AdamW)}} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

3. Label Smoothing Cross-Entropy (`ML18`): For K classes, smoothing parameter $\epsilon \in [0, 1]$, and hard one-hot label $y_k \in \{0, 1\}$:

$$y_k^{\text{LS}} = (1 - \epsilon) y_k + \frac{\epsilon}{K} = \begin{cases} (1 - \epsilon) + \dfrac{\epsilon}{K} & \text{if } k = \text{true class } c \\ \dfrac{\epsilon}{K} & \text{if } k \neq c \end{cases}$$

4. Effective Batch Size (`ML19`), Cosine Annealing & Gradient Clipping (`ML20`):

- **Effective Batch Size:**

  $$B_{\text{eff}} = N_{\text{GPU}} \times b_{\text{micro}} \times K_{\text{accum}}$$

  (When scaling B_{eff} by $k\times$, gradient variance drops by $1/k$; scale LR linearly $\eta \leftarrow k\eta$ for SGD, or by $\sqrt{k}$ for AdamW).

- **Cosine Annealing LR Schedule** (over $t \in [0, T]$ steps):

  $$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$$

  - Derivative $\frac{d\eta_t}{dt} \propto -\sin\left(\frac{t}{T}\pi\right)$: decay rate is 0 **(flat)** at $t = 0$ and $t = T$, and **steepest in the middle at** $t = T/2$ (where $\eta_{T/2} = \frac{\eta_{\max} + \eta_{\min}}{2}$)!

- **Global L_2 Norm Clipping (`torch.nn.utils.clip_grad_norm_`):**

  $$g \leftarrow g \cdot \min\left(1,\ \frac{\tau}{\|g\|_2}\right) = \begin{cases} g & \text{if } \|g\|_2 \leq \tau \\ \dfrac{\tau}{\|g\|_2} g & \text{if } \|g\|_2 > \tau \end{cases}$$

🔢 Step-by-Step Worked Toy Example

Example 1: The Adam Step $t = 1$ Exam Shortcut (`ML16`). Suppose a scalar weight starts at $w_0 = 2.0$ with $\eta = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 0$, and the first gradient at $t = 1$ is $g_1 = +5.0$ (with $m_0 = 0, v_0 = 0$). What is w_1?

1. **Compute biased moments m_1 and v_1:**

$$m_1 = 0.9(0) + (1 - 0.9)(5.0) = 0.1 \times 5.0 = \mathbf{0.5}$$

$$v_1 = 0.999(0) + (1 - 0.999)(5.0^2) = 0.001 \times 25.0 = \mathbf{0.025}$$

(Notice how tiny $m_1 = 0.5$ and $v_1 = 0.025$ are compared to the true $g_1 = 5.0$ and $g_1^2 = 25.0$—that's why bias correction is needed!)

2. **Apply Bias Correction at $t = 1$ ($1 - \beta_1^1 = 0.1$ and $1 - \beta_2^1 = 0.001$):**

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{0.5}{0.1} = \mathbf{5.0} = g_1$$

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^1} = \frac{0.025}{0.001} = \mathbf{25.0} = g_1^2$$

3. **Compute the parameter update Δw_1:**

$$\Delta w_1 = -\eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = -0.01 \times \frac{5.0}{\sqrt{25.0}} = -0.01 \times \frac{5.0}{5.0} = -\mathbf{0.01} = -\eta \cdot \text{sign}(g_1)$$

$$w_1 = w_0 + \Delta w_1 = 2.0 - 0.01 = \mathbf{1.99}$$

> **3-Second Exam Rule (`ML16`):** At step $t = 1$, $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$ for **any** $\beta_1, \beta_2 \in (0, 1)$, so the very first Adam step is **always** $\Delta w_1 = -\eta \cdot \text{sign}(g_1)$, completely independent of β_1, β_2, or the magnitude of g_1!

Example 2: Label Smoothing Target Vector (`ML18`). For $K = 5$ classes, smoothing $\epsilon = 0.10$, and true class index $c = 2$ (hard one-hot $y = [0, 1, 0, 0, 0]$):

- **True class ($k = 2,\ y_2 = 1$):**

$$y_2^{\text{LS}} = (1 - 0.10)(1) + \frac{0.10}{5} = 0.90 + 0.02 = \mathbf{0.92}$$

- **Each of the 4 wrong classes ($k \neq 2,\ y_k = 0$):**

$$y_k^{\text{LS}} = (1 - 0.10)(0) + \frac{0.10}{5} = \mathbf{0.02}$$

- **Smoothed target vector:** $y^{\text{LS}} = [0.02,\ \mathbf{0.92},\ 0.02,\ 0.02,\ 0.02]$ (check sum: $0.92 + 4(0.02) = 1.00$).

Example 3: Effective Batch Size (`ML19`) & Global Norm Clipping vs. Value Clamping (`ML20`).

- **Effective Batch Size:** Training on $N_{\text{GPU}} = 4$ GPUs with micro-batch size $b_{\text{micro}} = 8$ per GPU and `gradient_accumulation_steps` $K_{\text{accum}} = 16$:

$$B_{\text{eff}} = 4 \times 8 \times 16 = \mathbf{512}$$

- **Global L_2 Norm Clipping vs. Element-Wise Clamping:** Suppose gradient vector $g = [6.0,\ 8.0]^T$ (note the direction ratio $\frac{g_2}{g_1} = \frac{8}{6} = \frac{4}{3}$) and threshold $\tau = 5.0$:
  1. Compute L_2 norm: $\|g\|_2 = \sqrt{6.0^2 + 8.0^2} = \sqrt{36 + 64} = \sqrt{100} = 10.0 > 5.0$.
  2. **Global Norm Clip (`clip_grad_norm_`):** Scale entire vector by $\frac{\tau}{\|g\|_2} = \frac{5.0}{10.0} = 0.5$:

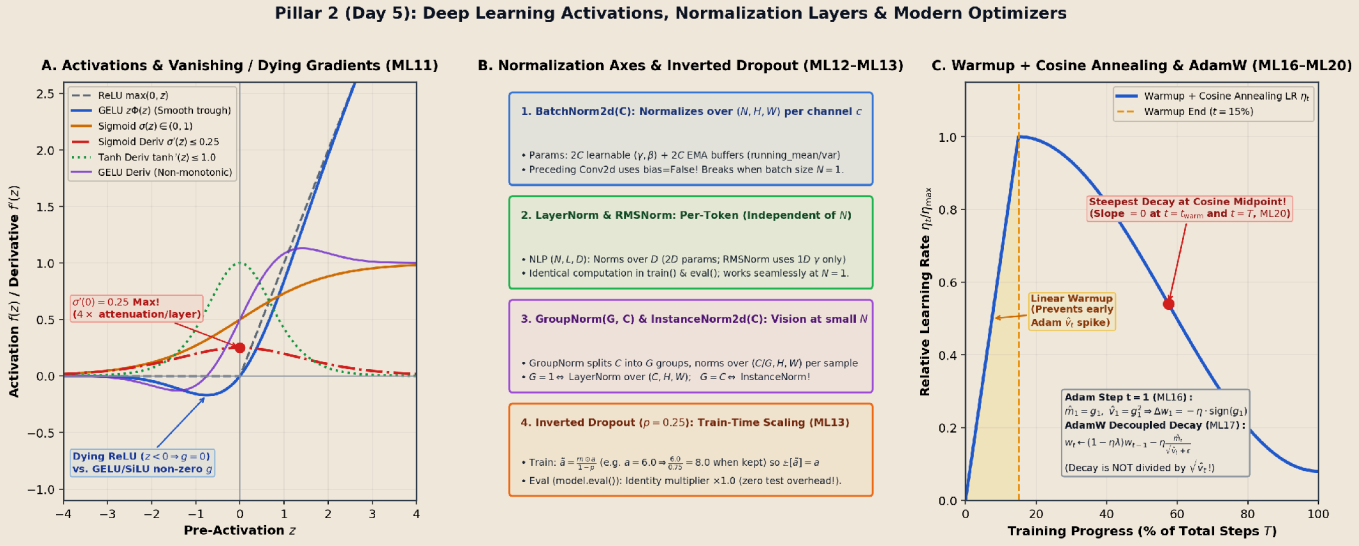
$$g_{\text{norm-clipped}} = 0.5 \times [6.0,\ 8.0]^T = [\mathbf{3.0},\ \mathbf{4.0}]^{\mathbf{T}} \quad \left(\text{norm} = 5.0,\ \text{direction ratio } \frac{4}{3} \textbf{ preserved!}\right)$$

  3. **Element-Wise Value Clamp (`clip_grad_value_`):** Clamp each coordinate independently to $[-5.0, +5.0]$:

$$g_{\text{val-clamped}} = [\min(6.0, 5.0),\ \min(8.0, 5.0)]^T = [\mathbf{5.0},\ \mathbf{5.0}]^{\mathbf{T}} \quad \left(\text{direction ratio changed to } \frac{5}{5} = 1 \text{ — disto}$$

🎯 Exam Takeaway & Common Traps (`ML16` – `ML20`)

- **Trap 1 (`ML16` — Adam Step $t = 1$):** Because bias correction divides by $(1 - \beta_1^1)$ and $(1 - \beta_2^1)$, Adam's first update is always $-\eta \cdot \text{sign}(g_1)$. Without bias correction, early steps are biased **toward** 0 (not ∞!).
- **Trap 2 (`ML17` — Adam vs. AdamW):** In Adam, L_2 loss regularization $\frac{\lambda}{2}\|w\|_2^2$ is **not** equivalent to weight decay because the penalty λw gets divided by $\sqrt{\hat{v}_t}$ (giving large-gradient weights weaker decay). AdamW decouples weight decay directly onto w.
- **Trap 3 (`ML20` — Warmup, Cosine & Clipping):**
  - **Linear Warmup** prevents early divergence in Adam/Transformers when $\hat{v}_t$ is estimated from only a handful of mini-batches and has high variance.
  - **Cosine Annealing** has slope 0 at $t = 0$ and $t = T$, and drops **steepest at the midpoint** $t = T/2$.
  - **Global L_2 Norm Clipping** rescales $g \leftarrow \frac{\tau}{\|g\|_2} g$ and **preserves gradient direction**, whereas element-wise value clamping distorts direction.



Pillar 2 Day 5 Deep Learning Core Visuals

Pillar 3 (Days 6–8): Computer Vision (CV) — Step-by-Step Intuitive Study Textbook

↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 6–8: Nov 3 – Nov 5):** Computer Vision questions on the Qualcomm AI Residency Phase 1 MCQ split evenly between **exact tensor, parameter, and receptive-field arithmetic** (where a single off-by-one error in padding, bias counting, or dilation costs the point) and **architectural design reasoning** (why MobileNetV2 removes ReLU in its bottleneck, why RoI Align beats RoI Pooling, why DETR needs no NMS, how ViT interpolates positional embeddings, and how Latent Diffusion & Classifier-Free Guidance work).
>
> Every section in this textbook follows a **4-part learning flow**:
>
> 1. 💡 **Intuition (Plain English First):** What problem does this solve, why was it invented, and what is happening visually?
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations with every variable and matrix shape ($n \times m$, i.e., n rows and m columns) defined explicitly.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number calculations worked out line by line.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and exact traps tested in `CV01`–`CV20`.

Day 6: Convolution Arithmetic, FLOPs & Efficient Edge CNNs

↑ Mục Lục

1.1 The 3 Master Spatial Dimension Formulas (Standard, Dilated & Transposed Convolutions — `CV01`, `CV03`)

💡 Intuition (Plain English First)

Imagine sliding a small magnifying glass (a **kernel** of size $K \times K$) across an image of height H_{in} and width W_{in}:

- **Padding (P)** adds a border of zeros around the edges of the input image so the kernel can center itself on border pixels without the output shrinking at every layer.
- **Stride (S)** controls how many pixels the magnifying glass jumps after each step. A stride of $S = 1$ slides pixel-by-pixel; a stride of $S = 2$ skips every other pixel, cutting the output height and width roughly in half.
- **Dilation (D, "Atrous Convolution")** pokes empty holes (spaces) between the kernel weights so the filter covers a wider spatial area **without adding any extra learnable parameters**. Standard convolution has $D = 1$ (weights sit right next to each other). When $D = 2$, we insert 1 zero-gap between every adjacent weight, turning a 3×3 weight grid into a 5×5 footprint while still only storing $3 \times 3 = 9$ numbers!
- **Transposed Convolution (`nn.ConvTranspose2d`)** runs the spatial math backward to **upsample** a small feature map into a larger one (e.g., doubling $16 \times 16 \to 32 \times 32$ in a decoder or generator). However, if the kernel size K is not cleanly divisible by the stride S (for example, $K = 3, S = 2$), some output pixels get hit

by more overlapping kernel weights than their neighbors, creating ugly grid-like **checkerboard artifacts**. Modern architectures fix this using **Sub-Pixel Convolution (`nn.PixelShuffle`)**, which generates extra channels via a standard convolution and rearrangements them into higher spatial resolution like laying down mosaic tiles—with zero uneven overlap!

📐 The Math & Every Symbol Explained

Let an input feature map have spatial dimensions $H_{\text{in}} \times W_{\text{in}}$ (an $n \times m$ spatial grid with $n = H_{\text{in}}$ rows and $m = W_{\text{in}}$ columns). For any single spatial dimension I (representing either H_{in} or W_{in}):

- I: Input spatial size (H_{in} or W_{in}) in pixels.
- O: Output spatial size (H_{out} or W_{out}) in pixels.
- K: Physical kernel size (number of weight taps along one dimension, e.g., $K = 3$ for a 3×3 filter).
- P: Zero-padding added to **each** side of the input (top/bottom or left/right), so total padding along one axis is $2P$.
- S: Stride (step size in pixels between consecutive kernel positions).
- D: Dilation rate (spacing between adjacent kernel elements; $D = 1$ is a standard contiguous kernel, and $D - 1$ is the number of skipped pixels between adjacent weights).
- K_{eff}: Effective kernel footprint on the input grid after accounting for dilation gaps.
- P_{out}: Output padding (`output_padding` in `nn.ConvTranspose2d`, default 0), used only to resolve ambiguity when multiple input sizes map to the same strided output size.
- r: Upscaling factor in Sub-Pixel Convolution (`nn.PixelShuffle(r)`).

1. **Effective Kernel Size (with Dilation $D \geq 1$):**

$$K_{\text{eff}} = K + (K - 1)(D - 1) = D(K - 1) + 1$$

(Why? A kernel of K weights has $K - 1$ gaps between them, and each gap spans D steps instead of 1.)

2. **Standard / Dilated Conv2d & Pooling Output Size:**

$$O = \left\lfloor \frac{I + 2P - K_{\text{eff}}}{S} \right\rfloor + 1 = \left\lfloor \frac{I + 2P - D(K - 1) - 1}{S} \right\rfloor + 1$$

   - **"Same" Padding Condition ($O = I$ when $S = 1$):**

$$I = I + 2P - D(K - 1) \implies P = \frac{D(K - 1)}{2}$$

3. **Transposed Convolution (`nn.ConvTranspose2d`) Output Size:**

$$O = (I - 1) \times S - 2P + D(K - 1) + P_{\text{out}} + 1$$

   - For standard settings ($D = 1, P_{\text{out}} = 0$), this simplifies to:

$$O = (I - 1)S - 2P + K$$

4. **Sub-Pixel Convolution (`nn.PixelShuffle(r)`) Tensor Reshape:**

$$\text{Shape}: \quad (B,\, C_{\text{out}} \cdot r^2,\, H,\, W) \longrightarrow (B,\, C_{\text{out}},\, H \cdot r,\, W \cdot r)$$

where B is batch size and C_{out} is the target output channel count.

🔢 Step-by-Step Worked Toy Example

- **Example A (Dilated Convolution Output Size & Parameter Count — `CV01`):** Suppose we feed an input tensor of shape ($C_{\text{in}} = 16$, $H_{\text{in}} = 32$, $W_{\text{in}} = 32$) into a 2D convolution layer with $C_{\text{out}} = 32$ filters, kernel size $K = 3$, stride $S = 1$, padding $P = 1$, dilation $D = 2$, and `bias=False`.
  1. **Step 1 (Compute Effective Kernel Size K_{eff}):**

     $$K_{\text{eff}} = D(K-1) + 1 = 2(3-1) + 1 = 5$$

     So our 3×3 kernel spans a 5×5 spatial patch on the input!
  2. **Step 2 (Compute Output Spatial Size O):**

     $$O = \left\lfloor \frac{32 + 2(1) - 5}{1} \right\rfloor + 1 = \left\lfloor \frac{29}{1} \right\rfloor + 1 = 30$$

     The output shape is $(32, 30, 30)$. Notice that $P = 1$ did **not** preserve $O = 32$ because dilation expanded K_{eff} to 5! To get "Same" output ($O = 32$) with $K = 3, D = 2, S = 1$, we would need $P = \frac{2(3-1)}{2} = 2$.
  3. **Step 3 (Count Learnable Parameters):** Does dilation $D = 2$ increase the number of weights to $5 \times 5 = 25$ per channel? **No!** The gaps are empty zeros, not learnable weights:

     $$\text{Parameters} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} = 3^2 \times 16 \times 32 = 9 \times 512 = 4,608$$

- **Example B (Transposed Convolution Doubling Resolution — `CV03`):** We want to upsample a 16×16 feature map to 32×32 ($O = 2I$) using `nn.ConvTranspose2d` with $D = 1, P_{\text{out}} = 0$.
  - Try $K = 4, S = 2, P = 1$:

    $$O = (16-1) \times 2 - 2(1) + 4 = 30 - 2 + 4 = 32 = 2 \times 16 \quad \checkmark$$

    Moreover, since $K = 4$ is cleanly divisible by $S = 2$ ($4/2 = 2$), every output pixel receives contributions from exactly $2 \times 2 = 4$ kernel taps—avoiding checkerboard artifacts!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV01`):** Dilation $D > 1$ changes $K_{\text{eff}} = D(K-1) + 1$ and shrinks the output spatial size O (unless you increase padding to $P = \frac{D(K-1)}{2}$), **but it adds ZERO learnable parameters**.
- **Trap 2 (`CV03`):** Memorize the golden upsampling triplet for `ConvTranspose2d`: $K = 4, S = 2, P = 1$ always gives $O = 2I$ cleanly without checkerboard artifacts.
- **Trap 3:** Always apply the floor function $\lfloor \cdot \rfloor$ **before** adding $+1$ when $S > 1$!

1.2 Exact Parameter & MACs/FLOPs Formulas Across Convolution Variants (`CV02`, `CV04`)

💡 Intuition (Plain English First)

Why is a standard 2D convolution so computationally expensive on mobile and edge chips (like Qualcomm Snapdragon NPUs)? Because a standard $K \times K$ convolution tries to do **two completely different jobs at the exact same time**:

1. **Spatial filtering:** Looking at a $K \times K$ local neighborhood of pixels within each channel.
2. **Cross-channel mixing:** Combining all C_{in} input channels together to produce each of the C_{out} output channels.

Efficient edge architectures decouple or restrict this joint operation in four smart ways:

- 1×1 **Pointwise Convolution (`Network-in-Network`):** Shrinks the spatial window to a single pixel (1×1). It does **zero spatial filtering** across neighboring pixels; instead, at every (h, w) pixel location, it takes the vector of C_{in} channels and multiplies it by an $n \times m$ ($n = C_{\text{out}}$ rows, $m = C_{\text{in}}$ columns) weight matrix W. It is mathematically identical to running a **Fully Connected (Linear/MLP) layer independently on every pixel**!
- **Grouped Convolution (`groups = G`):** Splits the C_{in} input channels into G separate, non-talking groups of $\frac{C_{\text{in}}}{G}$ channels each, and assigns $\frac{C_{\text{out}}}{G}$ filters to each group. Because each filter only looks at $\frac{1}{G}$ of the input channels, both parameters and compute drop by an exact factor of G!
- **Depthwise Convolution (`groups = C_in`):** Pushes grouped convolution to the extreme by setting $G = C_{\text{in}} = C_{\text{out}}$. Every single input channel gets its own dedicated $K \times K$ spatial filter, with **zero mixing between different channels**.
- **Depthwise Separable Convolution (`DW + PW`, used in MobileNet/Xception):** Splits a standard convolution into two lightweight steps:
  1. A $K \times K$ **Depthwise Conv** to filter spatial patterns inside each channel independently.
  2. A 1×1 **Pointwise Conv** to mix information across all channels. For a 3×3 filter, this two-step factorization slashes parameters and MACs by $\sim 8\times$ **to** $9\times$ with almost no accuracy loss!

📐 The Math & Every Symbol Explained

Let the input tensor have shape $(C_{\text{in}}, H_{\text{in}}, W_{\text{in}})$ and the output tensor have shape $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ using a $K \times K$ spatial kernel:

- C_{in}: Number of input channels (depth of input feature map).
- C_{out}: Number of output channels (number of filters).
- $H_{\text{out}}, W_{\text{out}}$: Output feature map height and width (forming an $H_{\text{out}} \times W_{\text{out}}$ grid of H_{out} rows and W_{out} columns, containing $H_{\text{out}} W_{\text{out}}$ total output spatial locations).
- K: Spatial kernel size ($K \times K$ weights per channel slice).
- G: Number of channel groups in Grouped Convolution (must divide both C_{in} and C_{out} evenly).
- MACs: Multiply-Accumulate operations. Producing one output value from a dot product of length M requires M multiplications and M additions $= M$ MACs $= 2M$ FLOPs.

Convolution Variant	Learnable Parameters (`bias=False`)	Bias Parameters (`bias=True`)	Total MACs (Multiply-Accumulates)	Cost Ratio vs. Standard Conv
Standard Conv2d	$K^2 C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}} C_{\text{out}}$	$1.0\times$
1×1 Pointwise Conv	$C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} C_{\text{in}} C_{\text{out}}$	$\frac{1}{K^2}$
Grouped Conv (G groups, `CV04`)	$\frac{1}{G} K^2 C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$ *(NOT $\frac{C_{\text{out}}}{G}$!)*	$\frac{1}{G} H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}} C_{\text{out}}$	$\frac{1}{G}$
Depthwise Conv ($G = C_{\text{in}} = C_{\text{out}}$)	$K^2 C_{\text{in}}$	$+C_{\text{in}}$	$H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}}$	$\frac{1}{C_{\text{out}}}$
Depthwise Separable (`DW + PW`, `CV02`)	$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$	$+C_{\text{in}} + C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} (K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}})$	$\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$
Spatially Separable ($K \times 1$ then $1 \times K$)	$2K C_{\text{in}} C_{\text{out}}$ *(if $mid = C_{\text{in}}$)*	$+C_{\text{in}} + C_{\text{out}}$	$2K H_{\text{out}} W_{\text{out}} C_{\text{in}} C_{\text{out}}$	$\frac{2}{K}$ *(Rank-1 filters only)*

Explicitly, the **Depthwise Separable Reduction Ratio** relative to a Standard $K \times K$ Conv is:

$$\frac{\text{Cost(DW + PW)}}{\text{Cost(Standard)}} = \frac{K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{K^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Standard Conv vs. Depthwise Separable Conv — `CV02`):** Let $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, kernel size $K = 3$, output spatial grid $H_{\text{out}} \times W_{\text{out}} = 32 \times 32$ ($1,024$ pixels), and `bias=False`.
  1. **Standard 3×3 Conv:**

     $$\text{Params}_{\text{std}} = 3^2 \times 64 \times 128 = 9 \times 8,192 = 73,728$$

     $$\text{MACs}_{\text{std}} = 32 \times 32 \times 73,728 = 75,497,472 \text{ MACs}$$

  2. **Depthwise Separable Conv (3×3 DW followed by 1×1 PW):**
     - **Step 2a (Depthwise 3×3 stage):** Each of the 64 input channels gets one 3×3 filter:

       $$\text{Params}_{\text{DW}} = 3^2 \times 64 = 576$$

     - **Step 2b (Pointwise 1×1 stage):** Mixes 64 channels into 128 channels via an $n \times m$ (128×64) weight matrix:

       $$\text{Params}_{\text{PW}} = 1^2 \times 64 \times 128 = 8,192$$

     - **Step 2c (Total DW + PW Parameters & MACs):**

       $$\text{Params}_{\text{DW+PW}} = 576 + 8,192 = 8,768$$

       $$\text{MACs}_{\text{DW+PW}} = 32 \times 32 \times 8,768 = 8,978,432 \text{ MACs}$$

3. **Reduction Factor Check:**

$$\frac{73,728}{8,768} \approx 8.41\times \text{ fewer parameters and MACs!} \quad \left(\text{Formula check: } \frac{1}{128}+\frac{1}{9}=\frac{137}{1152}=\frac{1}{8.409}\right)$$

- **Example B (Grouped Convolution Bias Trap — `CV04`):** Suppose a `Conv2d` layer has $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, $K = 3$, `groups = 4`, and `bias = True`. How many total learnable parameters does it have?
  1. **Kernel Weights:** Divided by $G = 4$:

$$\text{Weights} = \frac{3^2 \times 64 \times 128}{4} = \frac{73,728}{4} = 18,432$$

  2. **Bias Parameters:** Every one of the $C_{\text{out}} = 128$ output channels still gets its own scalar bias term (biases are **never** divided by G!):

$$\text{Biases} = C_{\text{out}} = 128$$

  3. **Total Parameters:** $18,432 + 128 = \mathbf{18,560}$ (a common distractor is $18,432 + 128/4 = 18,464$—don't fall for it!).

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV04` Grouped Conv Bias Rule):** `groups = G` divides the **weight** count by G, but if `bias=True`, the bias vector always has C_{out} **parameters** (not C_{out}/G).
- **Trap 2 (`CV02` DW-Sep Bias Rule):** If both the Depthwise ($K \times K$) and Pointwise (1×1) layers have `bias=True`, total biases equal $C_{\text{in}} + C_{\text{out}}$ (in practice, BatchNorm follows each conv, so `bias=False` is used).
- **Trap 3 (MACs vs. FLOPs):** 1 MAC = 2 FLOPs. If an exam question asks for **FLOPs** instead of **MACs**, multiply the MAC count by 2!
- **Trap 4 (Spatially Separable Limitation):** Factoring a $K \times K$ matrix (an $n \times m$ grid with $n = K$ rows and $m = K$ columns) into a column vector ($K \times 1$) times a row vector ($1 \times K$) only works without approximation if the 2D filter matrix has **Rank** $= 1$ (like a Sobel or Gaussian filter). Depthwise Separable Conv splits across the **channel** axis instead, which works for arbitrary 2D spatial filters!

1.3 Landmark CNN & Edge Architectures: VGG, ResNet, MobileNetV2, GAP & SENet (`CV05`, `CV06`, `CV07`, `CV18`, `CV19`)

💡 Intuition (Plain English First)

How did CNN backbones evolve from heavy desktop models into ultra-fast mobile vision engines? Each landmark architecture introduced one foundational engineering trick:

1. **VGG (3×3 Stacking Principle, `CV05`):** Early CNNs (like AlexNet) used huge 7×7 or 11×11 filters in single layers. VGG showed that stacking **three** 3×3 **convolutions** back-to-back covers the **exact same** 7×7 **receptive field** as a single 7×7 layer, while using 45% **fewer parameters** and inserting **three ReLU non-linearities** instead of one!
2. **ResNet (Residual Connections & Bottleneck Blocks, `CV06`, `CV07`):**

- *Why did plain 56-layer networks perform worse than 20-layer networks even on the **training set**?* Not because of overfitting! It is an **optimization degradation problem**: multiplying Jacobians across 50+ layers causes gradients to vanish or become ill-conditioned, making it nearly impossible for plain layers to even learn an identity mapping $y = x$. By wiring an **identity skip connection** $y = \mathcal{F}(x) + x$, if a layer isn't needed, the network can easily push its weights toward zero ($\mathcal{F}(x) \to 0$) and preserve $y = x$, allowing gradients to flow unimpeded through the $+$ highway!
- *Why the ResNet-50 Bottleneck ($1 \times 1 \to 3 \times 3 \to 1 \times 1$)?* Running a 3×3 conv directly on $C = 256$ channels is expensive. So ResNet-50 first uses a 1×1 **conv to squeeze** channels by $4\times$ ($256 \to 64$), runs the 3×3 **spatial conv in the narrow** 64**-channel space**, and uses a second 1×1 **conv to expand** back ($64 \to 256$).

3. **MobileNetV2 (Inverted Residuals & Linear Bottlenecks,** `CV06`**):**
   - MobileNetV2 flips ResNet's hourglass inside-out! Instead of Wide $\to$ Narrow $\to$ Wide, MobileNetV2 connects skip connections between **narrow** bottleneck tensors (e.g., $C = 32$), uses a 1×1 conv to **expand** channels by $t = 6\times$ ($32 \to 192$), runs a cheap 3×3 **Depthwise Conv** in the wide 192-channel space, and projects back to 32 channels with a 1×1 conv.
   - *Crucial Insight — Why is the final 1×1 projection **LINEAR** (NO ReLU)?* ReLU zeroes out all negative numbers ($\max(0, x)$). In a high-dimensional expanded space (192 channels), there is enough redundancy across channels that zeroing negative activations doesn't destroy the underlying feature manifold. But when you compress down into a tight low-dimensional bottleneck (32 channels), every coordinate carries dense information—applying ReLU there irreversibly crushes half the feature space!

4. **Global Average Pooling (GAP) vs. Flatten + FC (** `CV18` **):**
   - VGG flattened its final $(512, 7, 7)$ feature map into a $25,088$-dimensional vector and connected it to a 4096 or 1000-unit Fully Connected layer—creating tens of millions of parameters that overfit easily and lock the network to a fixed 224×224 input size.
   - **GAP** simply takes the spatial average of each $H \times W$ (7×7) feature map, collapsing $(C, H, W) \to (C, 1, 1)$ with **zero learnable parameters**!

5. **Squeeze-and-Excitation (SENet Channel Attention,** `CV19` **):**
   - Standard convolutions weight all feature channels equally when summing them. An **SE Block** asks: "*For this specific image, which channels are most important?*" It **Squeezes** each channel's $H \times W$ spatial map into a single number via GAP, passes that C-dimensional vector through a tiny 2-layer MLP (**Excitation**) with a Sigmoid gate to produce per-channel importance scores $s_c \in (0, 1)$, and **Scales** each channel by s_c.

📐 The Math & Every Symbol Explained

1. **VGG Stacked 3×3 vs. Single $K_{\text{large}} \times K_{\text{large}}$ Layer (** `CV05` **):**
   - Stacking k layers of 3×3 stride-1 convolutions yields an effective Receptive Field of:

     $$RF = 2k + 1$$

   - For $k = 3$ (matching one 7×7 layer across C channels, `bias=False`):

     $$\text{Params}_{3\times(3\times3)} = 3 \times (3^2C^2) = 27C^2 \quad \text{vs.} \quad \text{Params}_{1\times(7\times7)} = 7^2C^2 = 49C^2$$

2. **ResNet Basic Block vs. Bottleneck Block (** `CV06` **,** `CV07` **):**

- **Residual Mapping:** $y = \mathcal{F}(x, \{W_i\}) + x$ (or $y = \mathcal{F}(x) + W_s x$ using a 1×1 stride-S projection matrix W_s when spatial size or channel count changes).
- **Basic Block (ResNet-18/34, $C \to C \to C$):** Two 3×3 convs $\implies 2 \times 9C^2 = 18C^2$ parameters.
- **Bottleneck Block (ResNet-50/101/152, $C \to \frac{C}{4} \to \frac{C}{4} \to C$):**

$$\text{Params}_{\text{Bottleneck}} = \underbrace{\left(1^2 \cdot C \cdot \frac{C}{4}\right)}_{1\times1 \text{ Reduce}} + \underbrace{\left(3^2 \cdot \frac{C}{4} \cdot \frac{C}{4}\right)}_{3\times3 \text{ Conv}} + \underbrace{\left(1^2 \cdot \frac{C}{4} \cdot C\right)}_{1\times1 \text{ Expand}} = \frac{1}{4}C^2 + \frac{9}{16}C^2 + \frac{1}{4}C^2 = \frac{17}{16}C^2 = 1.06$$

3. **MobileNetV2 Inverted Residual Block (Expansion Ratio t, `CV06`):** For input/output bottleneck channel width C and expansion factor t (typically $t = 6$):

$$C \xrightarrow[\text{ReLU6}]{1\times1 \text{ Expand}} tC \xrightarrow[\text{ReLU6}]{3\times3 \text{ Depthwise}} tC \xrightarrow[\text{LINEAR (No ReLU!)}]{1\times1 \text{ Project}} C$$

$$\text{Params}_{\text{MBConv}} = (C \cdot tC) + (3^2 \cdot tC) + (tC \cdot C) = 2tC^2 + 9tC$$

4. **Global Average Pooling (GAP) vs. Flatten + FC (`CV18`):** For a final feature map of shape (C, H, W) feeding a classifier with K_{cls} classes (`bias=False`):

$$\text{Params}_{\text{Flatten+FC}} = (C \cdot H \cdot W) \cdot K_{\text{cls}} \quad \text{vs.} \quad \text{Params}_{\text{GAP+FC}} = C \cdot K_{\text{cls}} \quad (\text{Reduction factor} = H \times W)$$

5. **Squeeze-and-Excitation (SENet) Block (`CV19`):** Given feature map $X \in \mathbb{R}^{C \times H \times W}$ and bottleneck reduction ratio r (typically $r = 16$):
   - **Squeeze (GAP):** $z_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} X_c(i, j) \in \mathbb{R}^C$.
   - **Excitation (2-Layer Bottleneck MLP):**

$$s = \sigma(W_2 \, \text{ReLU}(W_1 z)) \in (0, 1)^C$$

     where $W_1 \in \mathbb{R}^{(C/r) \times C}$ is an $n \times m$ matrix with $n = C/r$ rows and $m = C$ columns, $W_2 \in \mathbb{R}^{C \times (C/r)}$ is an $n \times m$ matrix with $n = C$ rows and $m = C/r$ columns, and $\sigma(\cdot)$ is the element-wise Sigmoid function.
   - **SE Parameter Count (`bias=False`):**

$$\text{Params}_{\text{SE}} = \left(\frac{C}{r} \cdot C\right) + \left(C \cdot \frac{C}{r}\right) = \frac{2C^2}{r}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (VGG 3×3 Stacking vs. 7×7 Conv — `CV05`):** Let $C_{\text{in}} = C_{\text{out}} = 64$ (`bias=False`).
  - **One 7×7 layer:** $7^2 \times 64 \times 64 = 49 \times 4,096 = \mathbf{200,704}$ parameters.
  - **Three 3×3 layers:** $3 \times (3^2 \times 64 \times 64) = 27 \times 4,096 = \mathbf{110,592}$ parameters.
  - **Savings:** $\frac{200,704 - 110,592}{200,704} = \frac{22}{49} \approx \mathbf{44.9\%}$ fewer parameters for the exact same 7×7 receptive field!
- **Example B (ResNet Bottleneck vs. Standard 3×3 Conv — `CV07`):** Suppose a residual stage operates at $C = 256$ channels (`bias=False`).
  1. **Direct 3×3 Conv ($256 \to 256$):**

$$\text{Params}_{\text{direct}} = 3^2 \times 256 \times 256 = 9 \times 65,536 = \mathbf{589,824}$$

2. **ResNet Bottleneck ($256 \xrightarrow{1\times1} 64 \xrightarrow{3\times3} 64 \xrightarrow{1\times1} 256$):**
   - 1×1 Reduce ($256 \to 64$): $1^2 \times 256 \times 64 = 16,384$
   - 3×3 Spatial ($64 \to 64$): $3^2 \times 64 \times 64 = 36,864$
   - 1×1 Expand ($64 \to 256$): $1^2 \times 64 \times 256 = 16,384$
   - **Total Bottleneck Params:** $16,384 + 36,864 + 16,384 = \mathbf{69,632}$ (which matches $\frac{17}{16} \times 256^2 = 69,632$).
3. **Reduction Ratio:** $\frac{589,824}{69,632} = \frac{9}{17/16} = \frac{144}{17} \approx \mathbf{8.47}\times$ fewer parameters!

- **Example C (GAP vs. Flatten+FC & SENet Parameter Audit — `CV18`, `CV19`):**
  1. **GAP vs. Flatten (`CV18`):** Given a $(C, H, W) = (512, 7, 7)$ feature map and $K_{\text{cls}} = 1000$ classes (`bias=False`):
     - **Flatten + FC:** $(512 \times 7 \times 7) \times 1000 = 25,088 \times 1000 = \mathbf{25,088,000}$ parameters.
     - **GAP + FC:** GAP produces $(512, 1, 1)$ with 0 parameters, followed by $512 \times 1000 = \mathbf{512,000}$ parameters—an exact $7 \times 7 = \mathbf{49}\times$ reduction!
  2. **SENet Block (`CV19`):** Given $C = 256$ channels and reduction ratio $r = 16$ (`bias=False`):
     - Bottleneck dimension: $\frac{C}{r} = \frac{256}{16} = 16$.
     - Total SE parameters: $\frac{2\times256^2}{16} = 2 \times 256 \times 16 = \mathbf{8,192}$ parameters.

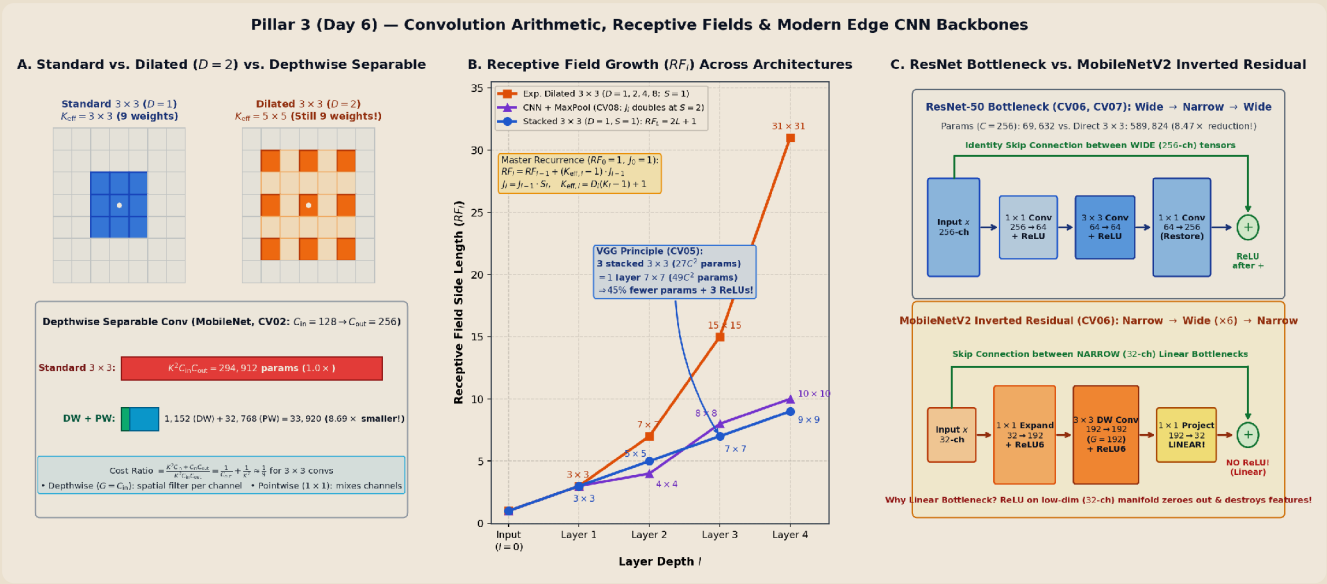
🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV06` ResNet vs. MobileNetV2):** ResNet Bottleneck is **Wide** → **Narrow** → **Wide** (skip connection on wide channels); MobileNetV2 Inverted Residual is **Narrow** → **Wide** → **Narrow** (skip connection on narrow channels, and **no ReLU** on the final 1×1 projection).
- **Trap 2 (`CV06` Why ResNet Works):** Skip connections solve **training optimization degradation** (vanishing gradients through deep layers), **not** overfitting!
- **Trap 3 (`CV18` GAP Benefits):** GAP adds **zero** learnable parameters, reduces classifier head parameters by $H \times W$, and allows the network to accept **any input image resolution** at test time.

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): ConvNeXt Modernization Details**

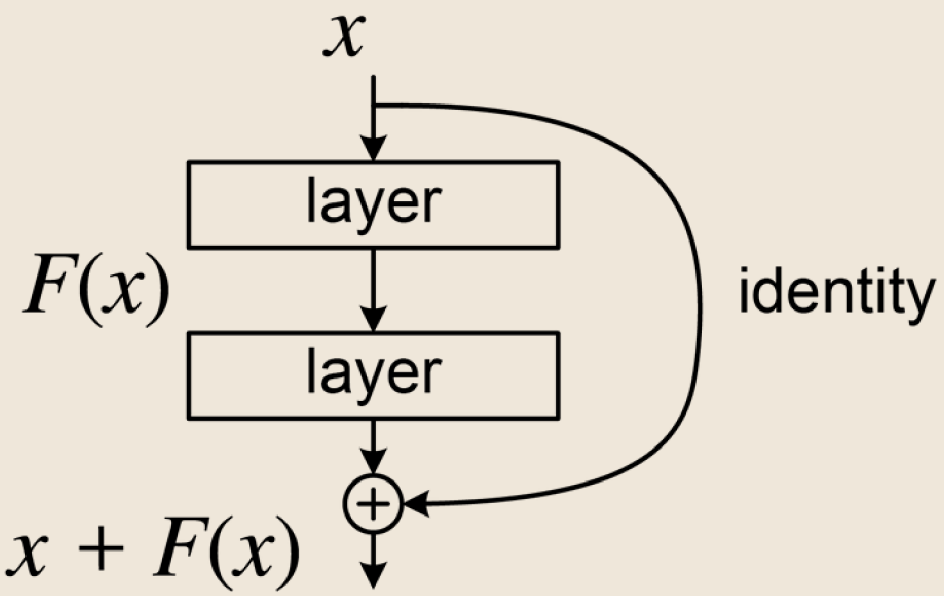
- **How ConvNeXt Modernizes a Standard ResNet-50 to Match Swin Transformer Accuracy:**
  1. **Macro Design:** Replaces the 7×7 stride-2 + MaxPool stem with a ViT-style 4×4 **non-overlapping stride-4 "patchify" stem**, and changes stage block ratios from $(3, 4, 6, 3)$ to $(3, 3, 9, 3)$.
  2. **Depthwise Separable + Large 7×7 Kernel:** Adopts ResNeXt/MobileNet depthwise convolutions and expands the spatial kernel from 3×3 to 7×7 (matching Swin's 7×7 window size).
  3. **Inverted Bottleneck ($C \to 4C \to C$):** Moves the 7×7 depthwise conv to the front of the block (`7x7 DW -> 1x1 Expand (4C) -> GELU -> 1x1 Project (C)`), keeping the heavy 7×7 conv at the narrower channel width C!
  4. **Micro-Design (Transformer Style):** Replaces ReLU with **GELU** (using only **one** activation per block instead of three) and replaces BatchNorm with **LayerNorm** (using only one norm per block).

FIGURE SLIDE 1 OF 3



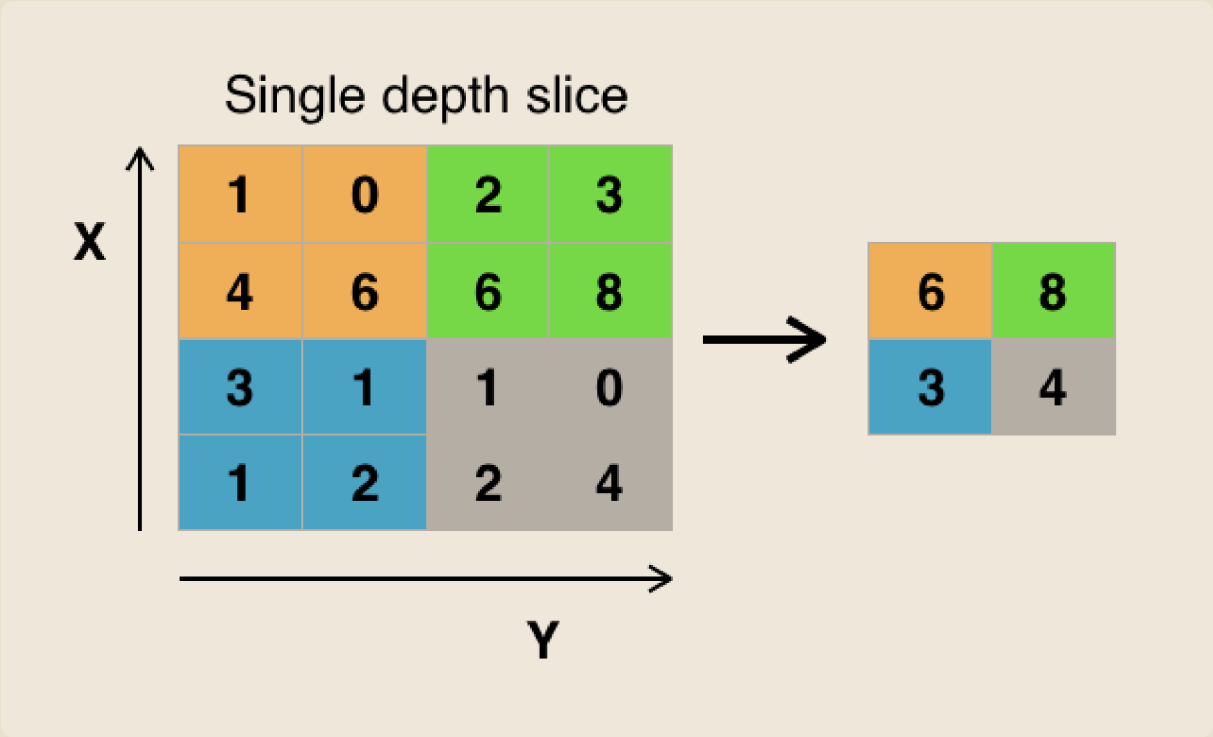
Day 6 Convolution Arithmetic, Receptive Fields & Modern Edge CNN Backbones

FIGURE SLIDE 2 OF 3



Canonical Residual Block Identity Shortcut

FIGURE SLIDE 3 OF 3



2x2 Stride-2 Max Pooling Operation

Day 7: Receptive Field Recursion, Object Detection & Segmentation

↑ Mục Lục

2.1 Iterative Receptive Field Calculation (`CV01` , `CV08`)

💡 Intuition (Plain English First)

Pick a single pixel deep inside layer L of a CNN and ask: "*How big of a square patch on the original raw input image could possibly affect this one neuron's value?*" That square footprint is the **Receptive Field (RF_L)**.

- At the raw input image (Layer 0), each pixel only sees itself, so $RF_0 = 1$ and the distance ("jump" J_0) between two adjacent pixels is 1 pixel.
- When you apply a 3×3 convolution at Layer 1, each output pixel looks at 3 adjacent input pixels ($1 + 2 = 3$).
- What happens if Layer 2 is a 2×2 MaxPool with stride $S_2 = 2$? Not only does each pooled pixel merge 2×2 neighbors from Layer 1, it also **doubles the step size ("jump" $J_2 = 2$)** between adjacent features! Because every pixel in Layer 2 now sits 2 raw input pixels apart, any 3×3 convolution placed *after* that pooling layer will reach across twice as many input pixels!

📐 The Math & Every Symbol Explained

To compute the input Receptive Field after L sequential layers (Convolution or Pooling):

- $l \in \{1, 2, \dots, L\}$: Layer index ($l = 0$ is the raw input image).
- K_l: Physical kernel size of layer l.
- D_l: Dilation rate of layer l ($D_l = 1$ for standard conv or pooling).

- $K_{\text{eff},l} = D_l(K_l - 1) + 1$: Effective kernel size of layer l.
- S_l: Stride of layer l.
- J_l: Cumulative stride (the "jump" or spacing on the input image between two adjacent pixels in layer l's feature map).
- RF_l: Receptive field width/height (in raw input image pixels) of a single neuron at the output of layer l.

1. **Initialization at the Input Image ($l = 0$):**

$$RF_0 = 1, \qquad J_0 = 1$$

2. **Layer-by-Layer Forward Recursion ($l = 1, 2, \ldots, L$):**

$$RF_l = RF_{l-1} + (K_{\text{eff},l} - 1) \times J_{l-1}$$

$$J_l = J_{l-1} \times S_l = \prod_{i=1}^{l} S_i$$

*(Notice that RF_l uses the **previous** layer's jump J_{l-1}, and then J_l updates for the next layer!)*

🔢 Step-by-Step Worked Toy Example

Let's trace the exact Receptive Field RF_l and Jump J_l through a 4-layer network (`CV08` , `CV01`):

- **Layer 1:** 3×3 Conv, stride $S_1 = 1$, padding $P_1 = 1$, dilation $D_1 = 1$.
- **Layer 2:** 2×2 MaxPool, stride $S_2 = 2$, padding $P_2 = 0$.
- **Layer 3:** 3×3 Dilated Conv, stride $S_3 = 1$, padding $P_3 = 2$, dilation $D_3 = 2$.
- **Layer 4:** 1×1 Pointwise Conv, stride $S_4 = 1$, padding $P_4 = 0$, dilation $D_4 = 1$.

Let's compute step by step starting from $RF_0 = 1, J_0 = 1$:

1. **After Layer 1 ($K_1 = 3, D_1 = 1 \implies K_{\text{eff},1} = 3, S_1 = 1$):**

$$RF_1 = RF_0 + (3 - 1) \times J_0 = 1 + 2 \times 1 = \mathbf{3}, \qquad J_1 = 1 \times 1 = \mathbf{1}$$

2. **After Layer 2 ($K_2 = 2, D_2 = 1 \implies K_{\text{eff},2} = 2, S_2 = 2$):**

$$RF_2 = RF_1 + (2 - 1) \times J_1 = 3 + 1 \times 1 = \mathbf{4}, \qquad J_2 = 1 \times 2 = \mathbf{2}$$

(The 2×2 MaxPool only added 1 pixel to RF_2, but it doubled the jump J_2 to 2!)

3. **After Layer 3 ($K_3 = 3, D_3 = 2 \implies K_{\text{eff},3} = 2(3 - 1) + 1 = 5, S_3 = 1$):**

$$RF_3 = RF_2 + (5 - 1) \times J_2 = 4 + 4 \times 2 = \mathbf{12}, \qquad J_3 = 2 \times 1 = \mathbf{2}$$

(Because $J_2 = 2$ and dilation made $K_{\text{eff},3} = 5$, Layer 3 added $4 \times 2 = 8$ pixels to the receptive field!)

4. **After Layer 4 ($K_4 = 1, D_4 = 1 \implies K_{\text{eff},4} = 1, S_4 = 1$):**

$$RF_4 = RF_3 + (1 - 1) \times J_3 = 12 + 0 \times 2 = \mathbf{12}, \qquad J_4 = 2 \times 1 = \mathbf{2}$$

(A 1×1 stride-1 conv never changes RF or J!)

🎯 Exam Takeaway & Common Traps

- **Trap 1: Padding P_l NEVER affects the Receptive Field RF_l!** Padding only shifts border coordinates and changes the output spatial dimensions $H_{\text{out}} \times W_{\text{out}}$. Ignore P_l completely when computing RF_l.
- **Trap 2: 1×1 Convolutions ($S = 1$) NEVER change RF_l** because $K_{\text{eff}} - 1 = 1 - 1 = 0$.
- **Trap 3 (`CV08`):** Don't multiply RF_{l-1} by 2 when hitting a 2×2 stride-2 MaxPool! A 2×2 MaxPool adds $(2-1) \times J_{l-1}$ to RF_{l-1} and doubles J_l for the *subsequent* layers.

2.2 Object Detection Part 1: IoU, GIoU, NMS & Feature Pyramid Networks (`CV09`, `CV10`)

💡 Intuition (Plain English First)

In object detection, we need to draw a tight rectangular bounding box around every object and classify it:

- **Intersection over Union (IoU):** Measures how well a predicted box A lines up with a ground-truth box B by dividing their **overlapping area** by their **total combined (union) area**. If they match perfectly, $\text{IoU} = 1$; if they don't touch at all, $\text{IoU} = 0$.
- **Why IoU Fails as a Loss Function (and how GIoU Fixes It, `CV09`):** Suppose your predicted box A and ground-truth box B do not overlap yet ($|A \cap B| = 0$). Then $\text{IoU} = 0$ whether the boxes are 1 pixel apart or $1,000$ pixels apart! Because the loss $\mathcal{L}_{\text{IoU}} = 1 - \text{IoU} = 1$ is completely flat everywhere outside overlap, its **gradient is zero ($\nabla \text{IoU} = 0$)** and the network has no idea which direction to move the box! **Generalized IoU (GIoU)** fixes this by drawing the smallest enclosing box C around both A and B, and subtracting the fraction of "empty wasted space" inside C. Even when A and B don't touch, moving A closer to B shrinks the enclosing box C, giving a smooth, non-zero gradient!
- **Non-Maximum Suppression (NMS, `CV10`):** A detector often fires 5 or 10 slightly shifted boxes on the exact same car. **Greedy NMS** cleans up these duplicates **per class**: it grabs the highest-scoring box B^*, keeps it, and deletes every other box of the same class whose overlap with B^* exceeds a threshold ($\text{IoU} > \theta_{\text{NMS}}$). However, in a crowded street where two real pedestrians stand right in front of each other ($\text{IoU} > \theta_{\text{NMS}}$), hard NMS accidentally deletes the second pedestrian! **Soft-NMS** fixes this by smoothly decaying the overlapping box's confidence score instead of immediately zeroing it out.
- **Feature Pyramid Network (FPN):** Early CNN layers have sharp spatial resolution (great for tiny objects) but weak semantic meaning; deep CNN layers have rich semantic meaning (great for recognizing object categories) but coarse spatial resolution. **FPN** builds a top-down bridge that upsamples deep semantic maps and adds them to shallow high-resolution maps (via 1×1 lateral channel projections), giving the detector rich semantics at **every** spatial scale!

📐 The Math & Every Symbol Explained

Let two axis-aligned 2D bounding boxes be defined by their top-left and bottom-right coordinates $A = [x_{1A}, y_{1A}, x_{2A}, y_{2A}]$ and $B = [x_{1B}, y_{1B}, x_{2B}, y_{2B}]$:

- $|A| = (x_{2A} - x_{1A})(y_{2A} - y_{1A})$: Area of box A (and similarly $|B|$ for box B).
- $|A \cap B|$: Area of the intersection rectangle:

$$w_{\cap} = \max(0,\, \min(x_{2A}, x_{2B}) - \max(x_{1A}, x_{1B}))\,, \quad h_{\cap} = \max(0,\, \min(y_{2A}, y_{2B}) - \max(y_{1A}, y_{1B}))$$

$$|A \cap B| = w_\cap \times h_\cap$$

- $|A \cup B| = |A| + |B| - |A \cap B|$: Area of the union of boxes A and B (we subtract $|A \cap B|$ so the overlapping region isn't double-counted!).
- C: The smallest axis-aligned enclosing rectangle that completely covers both A and B:

$$x_{1C} = \min(x_{1A}, x_{1B}), \quad y_{1C} = \min(y_{1A}, y_{1B}), \quad x_{2C} = \max(x_{2A}, x_{2B}), \quad y_{2C} = \max(y_{2A}, y_{2B})$$

$$|C| = (x_{2C} - x_{1C})(y_{2C} - y_{1C})$$

1. **Intersection over Union (**$\text{IoU} \in [0, 1]$**):**

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

2. **Generalized IoU (**$\text{GIoU} \in (-1, 1]$**,** `CV09`**):**

$$\text{GIoU}(A, B) = \text{IoU}(A, B) - \frac{|C \setminus (A \cup B)|}{|C|} = \text{IoU}(A, B) - \frac{|C| - |A \cup B|}{|C|}$$

   - **Range limits:** $\text{GIoU} = 1$ when $A = B$ perfectly; $\text{GIoU} \to -1$ as the distance between non-overlapping boxes approaches infinity ($\frac{|A \cup B|}{|C|} \to 0$).

3. **Greedy NMS vs. Linear Soft-NMS Score Update (** `CV10` **):** Given highest-scoring box B^* and a remaining candidate box B_i with confidence score s_i:

$$s_i^{\text{Hard-NMS}} = \begin{cases} s_i & \text{if } \text{IoU}(B^*, B_i) \leq \theta_{\text{NMS}} \\ 0 & \text{if } \text{IoU}(B^*, B_i) > \theta_{\text{NMS}} \end{cases}, \qquad s_i^{\text{Soft-NMS}} = \begin{cases} s_i & \text{if } \text{IoU}(B^*, B_i) \leq \theta_{\text{NMS}} \\ s_i \left(1 - \text{IoU}(B^*, B_i)\right) & \text{if } \text{IoU}(B^*, B_i) > \theta_{\text{NMS}} \end{cases}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Overlapping Boxes IoU & GIoU —** `CV09` **):** Let Box $A = [0, 0, 4, 4]$ and Box $B = [2, 2, 6, 6]$.
  1. **Individual Areas:** $|A| = (4 - 0)(4 - 0) = 16$, and $|B| = (6 - 2)(6 - 2) = 16$.
  2. **Intersection Area** $|A \cap B|$**:**
     - Overlap along x: $[\max(0, 2),\ \min(4, 6)] = [2, 4] \implies w_\cap = 4 - 2 = 2$.
     - Overlap along y: $[\max(0, 2),\ \min(4, 6)] = [2, 4] \implies h_\cap = 4 - 2 = 2$.
     - $|A \cap B| = 2 \times 2 = \mathbf{4}$.
  3. **Union Area** $|A \cup B|$**:**

$$|A \cup B| = |A| + |B| - |A \cap B| = 16 + 16 - 4 = \mathbf{28}$$

  4. **Exact IoU:**

$$\text{IoU}(A, B) = \frac{4}{28} = \frac{1}{7} \approx \mathbf{0.1429}$$

  5. **Smallest Enclosing Box** C **and GIoU:**

    - $C = [\min(0,2), \min(0,2), \max(4,6), \max(4,6)] = [0,0,6,6] \implies |C| = 6 \times 6 = \mathbf{36}$.
    - Empty area inside C: $|C| - |A \cup B| = 36 - 28 = 8$.
    - $\text{GIoU}(A,B) = \frac{1}{7} - \frac{8}{36} = \frac{1}{7} - \frac{2}{9} = -\frac{5}{63} \approx \mathbf{-0.0794}$.
- **Example B (Disjoint Boxes Showing Why GIoU Beats IoU — `CV09`):** Let $A = [0,0,2,2]$ ($|A| = 4$) and $B = [4,4,6,6]$ ($|B| = 4$).
    - They do not touch: $|A \cap B| = 0 \implies |A \cup B| = 4 + 4 - 0 = 8$, so IoU $= 0$ (zero gradient!).
    - Enclosing box $C = [0,0,6,6] \implies |C| = 36$.
    - GIoU $= 0 - \frac{36-8}{36} = -\frac{28}{36} = \mathbf{-0.7778}$ (non-zero gradient that pulls A and B together!).
- **Example C (Greedy NMS vs. Soft-NMS Step-by-Step — `CV10`):** Suppose a `'car'` detector outputs 3 boxes: B_1 ($s_1 = 0.95$), B_2 ($s_2 = 0.80$), and B_3 ($s_3 = 0.70$), with $\text{IoU}(B_1, B_2) = 0.60$, $\text{IoU}(B_1, B_3) = 0.20$, and $\text{IoU}(B_2, B_3) = 0.10$. Let $\theta_{\text{NMS}} = 0.50$.
    - **Round 1:** Select highest-scoring box $B^* = B_1$ (0.95) and move it to the keep list.
    - **Compare remaining boxes against B_1:**
        - B_2 has $\text{IoU}(B_1, B_2) = 0.60 > 0.50$: **Hard NMS** deletes B_2 ($s_2 \leftarrow 0$); **Linear Soft-NMS** decays B_2's score to $s_2 \leftarrow 0.80 \times (1 - 0.60) = \mathbf{0.32}$.
        - B_3 has $\text{IoU}(B_1, B_3) = 0.20 \leq 0.50$: Kept untouched ($s_3 = 0.70$).
    - **Final Survivors (Hard NMS):** $[B_1, B_3]$.

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV09`):** Standard IoU is in $[0,1]$ and has **zero gradient** for non-overlapping boxes; **GIoU** is in $(-1,1]$ and always provides a usable gradient by penalizing the empty area $|C| - |A \cup B|$ of the enclosing box C.
- **Trap 2 (`CV10`):** Standard NMS runs **independently per class** in $\mathcal{O}(M^2)$ pairwise comparisons for M candidate boxes, and its main failure mode is **suppressing true-positive objects that overlap heavily in crowded scenes** (fixed by Soft-NMS).
- **Trap 3 (FPN Lateral Convs):** FPN uses 1×1 **lateral convolutions** to project different backbone stages (C_3, C_4, C_5 with $512, 1024, 2048$ channels) to a **single uniform channel width ($d = 256$)** so the top-down $2\times$ upsampled features can be combined via **element-wise addition**, followed by a 3×3 anti-aliasing conv.

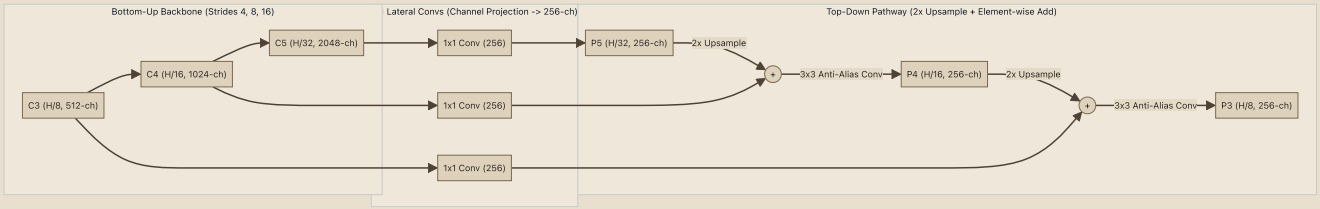
▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): DIoU & CIoU Exact Penalty Formulas**

- **Why GIoU Can Still Be Slow:** When one box is completely inside another ($A \subset B$), $|C| = |B| = |A \cup B|$, so the GIoU penalty term $\frac{|C| - |A \cup B|}{|C|} = 0$ vanishes and GIoU degenerates back into standard IoU!
- **DIoU (Distance-IoU):** Directly minimizes the normalized squared Euclidean distance $\rho^2(\mathbf{b}, \mathbf{b}^{gt})$ between the center points of the predicted box $\mathbf{b}$ and ground-truth box $\mathbf{b}^{gt}$, divided by the squared diagonal length c^2 of the smallest enclosing box C:

$$\mathcal{L}_{\text{DIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$$

- **CIoU (Complete-IoU):** Adds an aspect-ratio consistency penalty αv where $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$



2.3 Object Detection Part 2: Two-Stage vs. One-Stage, RoI Align (2D Bilinear Interpolation), Focal Loss, DETR & mAP (`CV11`, `CV12`)

💡 Intuition (Plain English First)

Modern object detectors fall into three architectural families:

1. **Two-Stage Detectors (Faster R-CNN / Mask R-CNN, `CV12`):**
   - *Stage 1:* A **Region Proposal Network (RPN)** scans the feature map and asks a simple class-agnostic question: "*Could there be ANY object inside this region?*"
   - *Stage 2:* For each promising Region of Interest (RoI), we crop out a fixed-size feature grid (e.g., 7×7) and feed it to a classification and bounding-box refinement head.
   - *Why RoI Align Beats RoI Pooling (`CV12`):* Suppose an RPN proposal has width $w = 135$ pixels on the image, and our backbone has stride $S = 16$. On the feature map, the box width is $\frac{135}{16} = 8.4375$ cells. **RoI Pooling** brutally rounds that floating-point coordinate down to an integer ($\lfloor 8.4375 \rfloor = 8$), and then rounds *again* when dividing the 8-cell region into 7×7 bins ($\lfloor 8/7 \rfloor = 1$)! At stride 16, a 0.5-cell rounding error shifts the box by 8 pixels on the original image—destroying pixel-level mask accuracy. **RoI Align** bans all integer rounding (`floor`), keeps exact floating-point coordinates, and uses **2D Bilinear Interpolation** to smoothly sample feature values between the 4 nearest pixel centers!
2. **One-Stage Detectors (YOLO, SSD, RetinaNet, FCOS, `CV11`):**

- Skip the RPN proposal step and predict class probabilities and boxes directly at every grid cell in a single pass.
- *The Extreme Class Imbalance Problem & Focal Loss (`CV11`):* A dense one-stage grid evaluates $\sim 100,000$ candidate anchor locations per image, but only ~ 10 contain real foreground objects—the other $99,990$ are easy background sky/road/wall! Even if each easy background box has a tiny cross-entropy loss of 0.01, summing $100,000 \times 0.01 = 1,000$ completely drowns out the few hard foreground objects! **Focal Loss** multiplies standard cross-entropy by $(1-p_t)^\gamma$: when the model is already 99% sure an anchor is background ($p_t = 0.99$), $(1-0.99)^2 = 0.0001$ silences its gradient by $10,000\times$, letting the optimizer focus purely on the hard mistakes!
- *Anchor-Based vs. Anchor-Free:* Anchor-based models (RetinaNet, YOLOv3) place k predefined reference boxes of fixed scales/aspect ratios at each grid cell and predict offsets $(\Delta x, \Delta y, \Delta w, \Delta h)$. **Anchor-Free** models (FCOS, CenterNet, YOLOv8) drop predefined anchors completely and regress the 4 distances (l, t, r, b) from each feature point directly to the left, top, right, and bottom box edges.

3. **End-to-End Transformer Detectors (DETR, `CV12`):**
   - Why do traditional detectors need hand-designed Anchor Boxes and post-processing NMS? Because they allow *many* neighboring anchors to predict the same object! **DETR** feeds CNN features into a Transformer Encoder-Decoder with $N = 100$ learnable **Object Queries**. During training, it runs the **Hungarian Algorithm** to find the single best **1-to-1 bipartite matching** between the 100 queries and the ground-truth objects (assigning remaining queries to $\varnothing$ "no object"). Because the loss strictly penalizes duplicate predictions during training, DETR outputs exactly one box per object and **needs ZERO Anchor Boxes and ZERO NMS**!

📐 The Math & Every Symbol Explained

1. **2D Bilinear Interpolation (Used Inside RoI Align, `CV12`):** Suppose we want to evaluate a feature map at a continuous sub-pixel coordinate (x, y) that lies inside a unit grid cell whose four surrounding integer grid corners are $(x_0, y_0), (x_1, y_0), (x_0, y_1), (x_1, y_1)$ with known feature values $Q_{00}, Q_{10}, Q_{01}, Q_{11}$. Let the fractional offsets be $\Delta x = x - x_0 \in [0, 1]$ and $\Delta y = y - y_0 \in [0, 1]$:
   - **Step 1 (Interpolate horizontally along top y_0 and bottom y_1):**

$$f(x, y_0) = (1 - \Delta x)\, Q_{00} + \Delta x\, Q_{10}, \qquad f(x, y_1) = (1 - \Delta x)\, Q_{01} + \Delta x\, Q_{11}$$

   - **Step 2 (Interpolate vertically along y):**

$$f(x, y) = (1 - \Delta y)\, f(x, y_0) + \Delta y\, f(x, y_1)$$

$$f(x, y) = (1 - \Delta x)(1 - \Delta y) Q_{00} + \Delta x (1 - \Delta y) Q_{10} + (1 - \Delta x)\Delta y\, Q_{01} + \Delta x\, \Delta y\, Q_{11}$$

2. **Focal Loss (`CV11`):** Let $p_t \in (0, 1]$ be the model's predicted probability for the **ground-truth class** (i.e., $p_t = p$ if $y = 1$, and $p_t = 1 - p$ if $y = 0$):

$$\mathrm{CE}(p_t) = -\log(p_t) \quad \longrightarrow \quad \mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

   - $\alpha_t \in [0, 1]$: Class-balancing weight (typically $\alpha = 0.25$ for foreground).
   - $\gamma \geq 0$: Focusing parameter (default $\gamma = 2$; when $\gamma = 0$, Focal Loss reduces to α-balanced Cross-Entropy).

- $(1-p_t)^\gamma$: Modulating factor that exponentially suppresses well-classified examples ($p_t \to 1$).

3. **DETR Hungarian Bipartite Matching (`CV12`):** Given N predicted outputs $\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ and ground-truth set y padded with $\varnothing$ ("no object") to size N, DETR builds an $N \times N$ (N rows, N columns) pairwise matching cost matrix $\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ and finds the optimal permutation $\hat{\sigma} \in \mathfrak{S}_N$ via the Hungarian algorithm:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^{N} \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

4. **Mean Average Precision (mAP):**
   - At a given IoU threshold τ_{IoU} (e.g., 0.50), sort all predictions for class c by confidence score descending. Each prediction is a **True Positive (TP)** if IoU $\geq \tau_{\text{IoU}}$ with an unmatched ground-truth box of class c; otherwise (or if it is a **duplicate** detection on an already-matched GT box!) it is a **False Positive (FP)**. Unmatched GT boxes are **False Negatives (FN)**.
   - Precision $= \frac{\text{TP}}{\text{TP}+\text{FP}}$, Recall $= \frac{\text{TP}}{\text{TP}+\text{FN}}$, $\text{AP}_c = \int_0^1 P_c(R)\, dR$, $\text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_c$.
   - **PASCAL VOC `mAP@0.50`:** Evaluates at a single fixed threshold $\tau_{\text{IoU}} = 0.50$.
   - **COCO `mAP@[.50:.05:.95]`:** Averages mAP across **10 IoU thresholds** ($\tau_{\text{IoU}} \in \{0.50, 0.55, 0.60, \ldots, 0.95\}$) to strictly reward tight localization.

🔢 Step-by-Step Worked Toy Example

- **Example A (RoI Align 2D Bilinear Interpolation at a Sub-Pixel Point — `CV12`):** Suppose an RoI Align sampling point lands at continuous coordinate $(x, y) = (0.2, 0.6)$ inside the unit square $[0,1] \times [0,1]$, where the four corner feature values are:
  - Top-left $Q_{00} = f(0,0) = 10$, Top-right $Q_{10} = f(1,0) = 20$
  - Bottom-left $Q_{01} = f(0,1) = 30$, Bottom-right $Q_{11} = f(1,1) = 40$

  1. **Step 1 (Horizontal interpolation at $y_0 = 0$ and $y_1 = 1$ using $\Delta x = 0.2$):**

  $$f(0.2, 0) = (1-0.2)(10) + (0.2)(20) = 8 + 4 = \mathbf{12}$$

  $$f(0.2, 1) = (1-0.2)(30) + (0.2)(40) = 24 + 8 = \mathbf{32}$$

  2. **Step 2 (Vertical interpolation at $\Delta y = 0.6$ between 12 and 32):**

  $$f(0.2, 0.6) = (1-0.6)(12) + (0.6)(32) = 4.8 + 19.2 = \mathbf{24.0}$$

  (Notice how smooth and differentiable this is compared to nearest-neighbor rounding, which would snap $(0.2, 0.6) \to (0, 1)$ and output 30!)

- **Example B (Focal Loss Modulating Factor Comparison — `CV11`):** Let $\gamma = 2$ and $\alpha_t = 1$. Compare the Focal Loss modulating factor $(1-p_t)^\gamma$ for an **easy background anchor** ($p_t = 0.99$) vs. a **hard foreground anchor** ($p_t = 0.10$):

  1. **Easy Background ($p_t = 0.99$):**

  $$(1-0.99)^2 = (0.01)^2 = \mathbf{10^{-4}} = 0.0001 \quad \text{(loss scaled down by } 10,000\times!)$$

2. **Hard Foreground ($p_t = 0.10$):**

$$(1 - 0.10)^2 = (0.90)^2 = \mathbf{0.81} \quad \text{(keeps 81\% of its full cross-entropy weight!)}$$

3. **Relative Emphasis Ratio:** $\frac{0.81}{0.0001} = \mathbf{8,100}\times$ more relative weight on the hard example compared to standard cross-entropy!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV12` RoI Align vs. RoI Pooling):** RoI Pooling uses integer rounding (`floor`), causing misalignment; RoI Align uses **exact floating-point coordinates + 2D bilinear interpolation** (with **zero** learnable parameters!).
- **Trap 2 (`CV11` Focal Loss γ):** When $\gamma = 0$, Focal Loss equals standard Cross-Entropy. As γ increases (e.g., $\gamma = 2$), easy examples ($p_t \to 1$) are exponentially down-weighted.
- **Trap 3 (`CV12` DETR):** DETR uses **Hungarian 1-to-1 bipartite matching** + N learnable **Object Queries**, eliminating both **Anchor Boxes** and **NMS**.
- **Trap 4 (mAP Duplicate Detections):** If a model predicts **two** boxes with IoU ≥ 0.50 on the **same** ground-truth object, only the highest-scoring box counts as a **TP**—the second box is penalized as a **False Positive (FP)**!

2.4 Semantic, Instance & Panoptic Segmentation (`CV13`, `CV14`)

💡 Intuition (Plain English First)

What is the difference between the three core pixel-level segmentation tasks (`CV13`)?

- **Semantic Segmentation:** Colors every pixel by its **category** (`'road'`, `'sky'`, `'person'`, `'car'`), but does **not** separate individual instances of the same class. If 5 people stand next to each other, all their pixels just say `'person'`—you can't tell where Person #1 ends and Person #2 begins!
- **Instance Segmentation (Mask R-CNN):** Detects only countable foreground objects ("**things**" like `'person'` or `'car'`) and gives each individual object its own separate binary mask (`'Person #1'`, `'Person #2'`), while ignoring uncountable background regions ("**stuff**" like `'sky'` or `'grass'`).
- **Panoptic Segmentation:** Unifies both! Every single pixel in the image gets a semantic category label (covering both "stuff" and "things") **AND** an instance ID if it belongs to a countable object.

How do segmentation networks preserve sharp pixel boundaries while understanding global context, and how are they trained (`CV13`, `CV14`)?

- **FCN vs. U-Net vs. DeepLabv3+ (`CV13`):**
  - **FCN (Fully Convolutional Network):** Fuses upsampled decoder features with encoder skip features via **element-wise addition (`+`)**, keeping channel count unchanged.
  - **U-Net:** Fuses upsampled decoder features with encoder skip features via **channel-wise concatenation (`torch.cat` along `dim=1`)**, which **doubles the channel count** entering the decoder convolution so the network can learn how to blend fine edge details with deep semantics!
  - **DeepLabv3+ (ASPP — Atrous Spatial Pyramid Pooling):** Runs multiple **dilated convolutions in parallel** at different dilation rates (e.g., $D = 6, 12, 18$) plus global average pooling to capture small,

medium, and large spatial context simultaneously without downsampling.

- **Mask R-CNN's Decoupled Mask Head Trick (`CV13`):** Instead of forcing all K classes to compete at every pixel via a K-way Softmax, Mask R-CNN outputs K separate binary masks (one per class) and applies **per-pixel Sigmoid + Binary Cross-Entropy (BCE) Loss ONLY on the true class k^*'s mask channel**! Once the box head already knows the object is a `'dog'`, the mask head just traces the foreground silhouette without cross-class competition.
- **Dice Loss vs. Pixel-Wise BCE (`CV14`):** Suppose you are segmenting a tiny tumor or thin blood vessel that occupies only 0.5% of the image (99.5% background). A model that lazily predicts "100% background everywhere" gets 99.5% accuracy and a tiny Pixel-Wise BCE loss! **Soft Dice Loss** ignores the massive pool of true negatives (TN) and directly optimizes the **F1 / overlap ratio** between predicted foreground and true foreground!

📐 The Math & Every Symbol Explained

For binary segmentation of a foreground pixel set P (predicted) against ground-truth foreground pixel set G in terms of confusion matrix pixel counts TP (True Positives), FP (False Positives), and FN (False Negatives):

1. **IoU (Jaccard Index) vs. Dice Coefficient (F1 Score, `CV14`):**

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{|P \cap G|}{|P| + |G| - |P \cap G|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

$$\text{Dice} = \frac{2\,|P \cap G|}{|P| + |G|} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = F_1$$

2. **Exact Algebraic Conversion Between IoU and Dice (`CV14`):**

$$\text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}} \qquad \Longleftrightarrow \qquad \text{IoU} = \frac{\text{Dice}}{2 - \text{Dice}}$$

(Important inequality: For all $\text{IoU} \in (0, 1)$*, we always have* $\text{Dice} > \text{IoU}$*, with equality* $\text{Dice} = \text{IoU}$ *only at* 0 *and* 1*!)*

3. **Differentiable Soft Dice Loss (for continuous probabilities $p_i \in [0, 1]$ and binary labels $g_i \in \{0, 1\}$):**

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_{i=1}^{M} p_i g_i + \epsilon}{\sum_{i=1}^{M} p_i + \sum_{i=1}^{M} g_i + \epsilon}$$

where $M = H \times W$ is total pixels in the $H \times W$ (H rows, W columns) mask and $\epsilon \approx 10^{-6}$ prevents division by zero when both P and G are empty.

🔢 Step-by-Step Worked Toy Example

- **Example A (Converting Between IoU and Dice — `CV14`):** Suppose a segmentation model achieves $\text{IoU} = 0.60$ on the validation set. What is its exact Dice (F1) score?

$$\text{Dice} = \frac{2 \times 0.60}{1 + 0.60} = \frac{1.20}{1.60} = \mathbf{0.75}$$

Conversely, if a paper reports $\text{Dice} = 0.80$, what is the exact IoU?

$$\text{IoU} = \frac{0.80}{2 - 0.80} = \frac{0.80}{1.20} = \frac{2}{3} \approx \mathbf{0.6667}$$

- **Example B (Computing TP, FP, FN, IoU & Dice on a 4×4 Mask Grid — `CV14`):** Suppose an image has $4 \times 4 = 16$ pixels. The ground-truth mask G has 5 foreground pixels ($|G| = 5$), the model predicts 4 foreground pixels ($|P| = 4$), and 3 of those predicted pixels overlap with the ground truth ($|P \cap G| = 3$):
  1. **Confusion Matrix Counts:**
     - $\text{TP} = |P \cap G| = \mathbf{3}$
     - $\text{FP} = |P| - \text{TP} = 4 - 3 = \mathbf{1}$
     - $\text{FN} = |G| - \text{TP} = 5 - 3 = \mathbf{2}$
     - $\text{TN} = 16 - (3 + 1 + 2) = \mathbf{10}$ (Notice that neither IoU nor Dice uses TN = 10!)
  2. **IoU & Dice:**

$$\text{IoU} = \frac{3}{3 + 1 + 2} = \frac{3}{6} = \mathbf{0.50}, \qquad \text{Dice} = \frac{2(3)}{2(3) + 1 + 2} = \frac{6}{9} = \frac{2}{3} \approx \mathbf{0.6667}$$

(Check conversion formula: $\frac{2(0.50)}{1+0.50} = \frac{1.0}{1.5} = \frac{2}{3}$✓)

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV13` U-Net Skip Connections):** If an encoder stage has $C = 128$ channels and the upsampled decoder feature map has 128 channels, U-Net's concatenation (`torch.cat`) produces a **256-channel tensor** entering the decoder's 3×3 convolution (whereas FCN / ResNet addition keeps it at 128).
- **Trap 2 (`CV13` Mask R-CNN Loss):** Mask R-CNN uses **per-pixel Sigmoid + BCE only on the ground-truth class channel**, NOT per-pixel multi-class Softmax!
- **Trap 3 (`CV14` Dice vs. IoU):** Memorize Dice $= \frac{2\,\text{IoU}}{1+\text{IoU}}$. Because the denominator $1 + \text{IoU} < 2$ whenever $\text{IoU} < 1$, **Dice is always strictly greater than IoU** for any partial overlap.

Day 8: Vision Transformers (ViT, Swin), Multimodal (CLIP) & Diffusion ↑ Mục Lục

3.1 Vision Transformer (ViT) & Swin Transformer (`CV15`, `CV16`)

💡 Intuition (Plain English First)

Transformers revolutionized NLP by treating a sentence as a sequence of word tokens and computing all-to-all Self-Attention. Can we feed an image directly into a standard Transformer?

- **Why We Can't Use Individual Pixels as Tokens:** A small 224×224 image has $224 \times 224 = 50,176$ pixels. Self-attention compares every token to every other token, creating an $N \times N$ attention matrix of $50,176^2 \approx$ **2.5** billion entries per layer—completely impossible on GPU memory!
- **How ViT Solves This ("An Image is Worth 16×16 Words", `CV15`):**
  1. Chop the $H \times W \times C$ ($224 \times 224 \times 3$) image into a grid of non-overlapping $P \times P$ (16×16) square tiles called **patches**. That immediately shrinks the sequence length from $50,176$ pixels down to just $\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = \mathbf{196}$ patches!

2. Flatten each $16 \times 16 \times 3$ patch into a vector of 768 numbers and multiply by a learnable linear projection matrix E (implemented in practice as a single `Conv2d` with kernel size $P = 16$ and stride $S = 16$).
3. Prepend one extra learnable `[CLS]` **(classification) token** at the front of the sequence (making the total sequence length $L = N + 1 = 197$), and add a learnable **1D Positional Embedding** to every token so the Transformer knows which patch came from the top-left vs. bottom-right!

- **Inductive Bias & High-Resolution Fine-Tuning (`CV16`):**
  - Unlike CNNs—which hard-code **2D spatial locality** (pixels only talk to immediate neighbors first) and **translation equivariance** (weight sharing across the grid) into every layer—a standard ViT has **almost zero spatial inductive bias** (only the initial patch cut is local; every self-attention layer is global). That's why ViT underperforms ResNets when trained from scratch on small datasets, but scales better on massive datasets ($>$ 10M–100M images).
  - *What happens when you fine-tune a pretrained* 224×224 *ViT on higher-resolution* 384×384 *images?* Keep the patch size $P = 16$ identical! That way, each patch still has $16 \times 16 \times 3 = 768$ values, so the patch projection matrix E and all Transformer weights need **zero modification**. Only the number of patches increases ($196 \to 576$), which we handle by reshaping the 14×14 pretrained patch positional embeddings into a 2D grid and applying **2D bicubic interpolation** to 24×24!
- **Why Swin Transformer Was Invented (`CV16`):** Even with patches, global ViT attention scales **quadratically** $\mathcal{O}(N^2)$ with image area—making it too slow for high-resolution object detection and segmentation (1024×1024). **Swin Transformer** restricts self-attention to small non-overlapping local **Windows of** $M \times M$ **patches** (e.g., $M = 7$), dropping complexity to **linear** $\mathcal{O}(M^2 N)$ in image size! To let information travel across window borders, every second layer **shifts the window grid** by $\lfloor M/2 \rfloor = 3$ patches (**SW-MSA**), while **Patch Merging** layers (2×2 spatial concatenation + linear channel reduction) build a multi-scale ResNet/FPN-like feature pyramid.

📐 The Math & Every Symbol Explained

Given an input image $X \in \mathbb{R}^{H \times W \times C}$ (H rows, W columns, C channels) and patch size $P \times P$:

- N: Total number of image patches:

$$N = \frac{H}{P} \times \frac{W}{P} = \frac{H \cdot W}{P^2}$$

- D: Transformer hidden embedding dimension (e.g., $D = 768$ for ViT-Base).
- $E \in \mathbb{R}^{(P^2 C) \times D}$: Linear patch projection matrix—an $n \times m$ matrix with $n = P^2 C$ rows (flattened patch size) and $m = D$ columns (embedding dimension).

$$\text{Params}_{\text{patch_proj}} = (P^2 C) \cdot D + D \quad \text{(with bias)}$$

- $L = N + 1$: Total token sequence length entering the Transformer Encoder after prepending the learnable class token $x_{\text{class}} \in \mathbb{R}^{1 \times D}$ (1 row, D columns).
- $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$: Learnable 1D positional embedding matrix—an $n \times m$ matrix with $n = N + 1$ rows and $m = D$ columns, adding $(N + 1)D$ learnable parameters.
- **Single ViT Encoder Block Parameter Count (with bias, MLP expansion ratio** $r_{\text{MLP}} = 4$**):**
  - **Multi-Head Self-Attention (MHA):** Q, K, V projections ($3 \times (D^2 + D)$) + output projection W_O ($D^2 + D$) $= \mathbf{4D^2 + 4D}$. *(Independent of head count* h *because* $h \times (D \times \frac{D}{h}) = D^2$*!)*

- **Feed-Forward MLP ($D \to 4D \to D$):** $W_1 \in \mathbb{R}^{D \times 4D}$ $(4D^2 + 4D)$ + $W_2 \in \mathbb{R}^{4D \times D}$ $(4D^2 + D) = \mathbf{8D^2 + 5D}$.

- **Global Self-Attention (ViT) vs. Windowed Self-Attention (Swin, Window Size $M \times M$):**

$$\Omega(\text{Global MSA}) = 4ND^2 + 2N^2D = \mathcal{O}(N^2), \qquad \Omega(\text{W-MSA}) = 4ND^2 + 2M^2ND = \mathcal{O}(M^2N)$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Full ViT-Base Tokenization & Parameter Audit — `CV15`):** Let input image be $H = 224, W = 224, C = 3$, patch size $P = 16$, and hidden dimension $D = 768$.
  1. **Number of Patches (N) & Sequence Length (L):**

$$N = \frac{224 \times 224}{16^2} = 14 \times 14 = \mathbf{196} \text{ patches}, \qquad L = N + 1 = \mathbf{197} \text{ tokens}$$

  2. **Flattened Patch Dimension & Patch Projection Parameters (with bias):**
     - Each patch has $P^2C = 16^2 \times 3 = 256 \times 3 = \mathbf{768}$ pixel values.
     - Weight matrix E has shape 768×768 plus a 768-dim bias vector:

$$\text{Params}_{\text{patch_proj}} = (768 \times 768) + 768 = 589,824 + 768 = \mathbf{590,592}$$

  3. **Class Token & Positional Embedding Parameters:**
     - Class token x_{class} (1×768): 768 parameters.
     - Positional embedding E_{pos} (197×768): $197 \times 768 = \mathbf{151,296}$ parameters.
  4. **MHA vs. MLP Parameters in One Transformer Block ($D = 768$, without bias for quick ratio):**
     - MHA $\approx 4D^2 = 4 \times 768^2 = 2,359,296$ ($\frac{1}{3}$ of block parameters).
     - MLP $\approx 8D^2 = 8 \times 768^2 = 4,718,592$ ($\frac{2}{3}$ of block parameters—the MLP holds twice as many weights as self-attention!).

- **Example B (High-Resolution Fine-Tuning & Swin Window Speedup — `CV16`):**
  1. **Fine-tuning ViT-B/16 at $H' = W' = 384$:**
     - Patch size stays $P = 16 \implies$ new patch grid is $\frac{384}{16} \times \frac{384}{16} = 24 \times 24 = \mathbf{576}$ patches, so total sequence length is $L' = 576 + 1 = \mathbf{577}$ tokens.
     - We interpolate the 14×14 patch positional embeddings to 24×24 via 2D bicubic interpolation (leaving the 1×768 `[CLS]` positional embedding untouched).
  2. **Global Attention vs. Swin Window Attention ($M = 7$) on a 56×56 patch grid ($N = 3,136$):**
     - Global ViT pairwise token comparisons: $N^2 = 3,136^2 = \mathbf{9,834,496}$.
     - Swin W-MSA pairwise token comparisons (64 windows of $7 \times 7 = 49$ tokens each): $64 \times 49^2 = M^2N = 49 \times 3,136 = \mathbf{153,664}$—a $64\times$ **reduction** in attention matrix compute!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV15` Don't Forget the `+1` `[CLS]` Token!):** When an exam question asks for the sequence length entering the ViT encoder or the first dimension of the positional embedding matrix E_{pos}, it is $N + 1 = \frac{HW}{P^2} + 1$ (not just N).

- **Trap 2 (`CV15` Multi-Head Parameter Invariance):** Splitting $D = 768$ into $h = 12$ heads of $d_k = 64$ vs. $h = 8$ heads of $d_k = 96$ changes **zero** learnable parameters in MHA!
- **Trap 3 (`CV16` Swin Patch Merging):** A Swin Patch Merging layer groups 2×2 neighboring patches (multiplying channels by 4: $C \to 4C$) and applies a linear layer to reduce $4C \to 2C$—halving spatial resolution $(H/2, W/2)$ while doubling channels $2C$, exactly mirroring a CNN downsampling stage!

3.2 CLIP (Contrastive Language-Image Pre-training) & Modern Augmentation/SSL (`CV17`, `CV20`)

💡 Intuition (Plain English First)

- **CLIP (`CV17`):** Traditional image classifiers train a fixed $1,000$-way linear head on manually labeled classes—meaning they cannot recognize a `'skateboard'` if `'skateboard'` wasn't one of the $1,000$ training indices. **CLIP** learns visual concepts directly from raw internet `(Image, Caption)` pairs! Given a batch of N image-caption pairs, CLIP runs an Image Encoder and a Text Encoder to get N image vectors and N text vectors, normalizes them to unit length, and computes an $N \times N$ **grid of cosine similarities**. Along the main diagonal ($i = j$) are the N true matching pairs; everywhere off the diagonal ($i \neq j$) are $N^2 - N$ mismatched pairs ("in-batch negatives"). CLIP simply applies Cross-Entropy Loss across every row (picking the right caption for each image) and every column (picking the right image for each caption)! At test time, you can do **zero-shot classification** on any dataset by turning class names into text prompts (`"a photo of a {class}"`) and picking whichever prompt has the highest cosine similarity with the image!
- **Mixup vs. CutMix (`CV20`):**
  - **Mixup** averages two images pixel-by-pixel across the entire canvas ($\tilde{x} = \lambda x_A + (1 - \lambda) x_B$), which looks unnatural and blurry locally.
  - **CutMix** cuts a clean rectangular patch out of image B and pastes it directly over image A, and mixes the one-hot class labels in **exact proportion to the pixel area** of each image ($\lambda = 1 - \frac{\text{patch area}}{\text{image area}}$). Because every local patch still looks like a sharp, natural image, CutMix significantly outperforms Mixup on localization and detection tasks!
- **Masked Autoencoders (MAE, `CV20`):** How did BERT train self-supervised NLP models? By masking 15% of words and predicting them. Why do **Masked Autoencoders (MAE)** mask a whopping 75% **of image patches** instead of 15%? Because images have high spatial redundancy—if you only mask 15%, a model can trivially guess a missing patch by copying the color of its immediate neighbors without learning any semantic object structure! Even better, MAE **completely drops the 75% masked patches from the heavy ViT Encoder**, feeding **only the 25% visible patches** through the encoder (cutting encoder compute and memory by $4\times$!), and only inserts learnable `[MASK]` tokens right before a tiny lightweight decoder to reconstruct raw pixels via MSE loss.

📐 The Math & Every Symbol Explained

1. **CLIP Symmetric Contrastive (InfoNCE) Loss (`CV17`):** Given a minibatch of N matched image-text pairs $\{(I_i, T_i)\}_{i=1}^N$:
   - $\hat{I}_i = \frac{f_I(I_i)}{\|f_I(I_i)\|_2} \in \mathbb{R}^d$, $\hat{T}_j = \frac{f_T(T_j)}{\|f_T(T_j)\|_2} \in \mathbb{R}^d$: L_2-normalized image and text embeddings.
   - $S \in \mathbb{R}^{N \times N}$ (N rows, N columns): Scaled cosine similarity logit matrix with learnable temperature $\tau > 0$:

$$S_{ij} = \frac{1}{\tau}\hat{I}_i \cdot \hat{T}_j$$

- **Number of Positive & Negative Pairs in Batch N:**

$$\text{Positives} = N \quad (\text{diagonal } i = j), \qquad \text{Negatives} = N^2 - N = N(N-1) \quad (\text{off-diagonal } i \neq j)$$

- **Symmetric InfoNCE Loss:**

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I}\right) = -\frac{1}{2N}\sum_{i=1}^{N}\left[\ln\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ij})} + \ln\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ji})}\right]$$

- **Step-0 Initialization Sanity Check:** When all logits S_{ij} are equal (e.g., $S_{ij} = c$), every softmax probability is $\frac{1}{N}$, so:

$$\mathcal{L}_{\text{CLIP}}^{(0)} = -\ln\left(\frac{1}{N}\right) = \ln(N)$$

2. **Mixup & CutMix Label Mixing (`CV20`):**
   - **Mixup:** Given $\lambda \sim \text{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda x_A + (1-\lambda)x_B, \qquad \tilde{y} = \lambda y_A + (1-\lambda)y_B$$

   - **CutMix:** Given a binary rectangular mask $M \in \{0,1\}^{H\times W}$ (H rows, W columns) of cut-out box dimensions $r_h \times r_w$ where $M = 0$ inside the pasted box and $M = 1$ outside:

$$\tilde{x} = M \odot x_A + (\mathbf{1} - M) \odot x_B, \qquad \lambda = 1 - \frac{r_w \cdot r_h}{W \cdot H}, \qquad \tilde{y} = \lambda y_A + (1-\lambda)y_B$$

🔢 Step-by-Step Worked Toy Example

- **Example A (CLIP Batch Matrix Audit & Step-0 Loss — `CV17`):**
  1. **Toy Minibatch ($N = 4$):**
     - The similarity matrix S has shape $4 \times 4 = 16$ total entries.
     - **Positive pairs:** $N = \mathbf{4}$ (diagonal entries $S_{11}, S_{22}, S_{33}, S_{44}$).
     - **Negative pairs:** $N(N-1) = 4 \times 3 = \mathbf{12}$ off-diagonal entries.
     - **Step-0 uniform initialization loss:** $\mathcal{L}_0 = \ln(4) \approx \mathbf{1.3863}$.
  2. **Production Minibatch ($N = 256$):**
     - **Positive pairs:** 256.
     - **Negative pairs:** $256^2 - 256 = 65,536 - 256 = \mathbf{65,280}$.
     - **Step-0 uniform initialization loss:** $\mathcal{L}_0 = \ln(256) \approx \mathbf{5.5452}$.
- **Example B (CutMix Label Calculation & MAE Visible Token Count — `CV20`):**
  1. **CutMix Label Vector:** Suppose image x_A (class 0: $[1,0,0]$) and image x_B (class 1: $[0,1,0]$) both have resolution $H \times W = 100 \times 100$ ($10,000$ pixels). We cut a $r_h \times r_w = 40 \times 50$ rectangular patch ($2,000$ pixels) from x_B and paste it onto x_A:

$$\lambda = 1 - \frac{50 \times 40}{100 \times 100} = 1 - \frac{2,000}{10,000} = 1 - 0.20 = \mathbf{0.80}$$

$$\tilde{y} = 0.80\,[1, 0, 0] + 0.20\,[0, 1, 0] = [\mathbf{0.80},\ \mathbf{0.20},\ \mathbf{0.00}]$$

2. **MAE Encoder Token Count:** For a 224×224 image with $P = 16$ ($N = 196$ total patches) and masking ratio $m = 75\%$:
   - Only $N_{\text{vis}} = (1 - 0.75) \times 196 = 0.25 \times 196 = \mathbf{49}$ visible patch tokens enter the ViT Encoder!
   - Before the lightweight decoder, we append $196 - 49 = \mathbf{147}$ shared learnable `[MASK]` tokens (plus positional embeddings) to restore the full 196-patch grid and compute MSE reconstruction loss **only on the** 147 **masked patches**.

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV17` CLIP Loss Symmetry):** CLIP's InfoNCE loss is the **average of two N-way Cross-Entropy losses** (image-to-text across rows, and text-to-image across columns), NOT N^2 independent binary cross-entropy losses!
- **Trap 2 (`CV20` MAE Asymmetric Encoder-Decoder):** MAE never feeds `[MASK]` tokens into the heavy ViT Encoder—feeding `[MASK]` tokens into the encoder would waste $4\times$ compute and create a distribution shift between pretraining and fine-tuning!

3.3 Diffusion Models & Generative Vision: DDPM, DDIM, Latent Diffusion & CFG

💡 Intuition (Plain English First)

How do modern image generators like DDPM and Stable Diffusion conjure photorealistic images out of pure static noise?

1. **The Forward Process (Destroying an Image with Noise) & The $\mathcal{O}(1)$ Shortcut:**
   - Start with a clean training photo x_0. At each timestep $t = 1, 2, \ldots, T$ (typically $T = 1000$), mix in a tiny bit of fresh Gaussian static noise until, at step T, x_T is indistinguishable from pure random static $\mathcal{N}(0, I)$.
   - *The "Nice Property" ($\mathcal{O}(1)$ Forward Jump):* During training, do we have to run a slow 500-step `for` loop just to generate a noisy image x_{500}? **No!** Because the sum of independent Gaussian random variables is still a single Gaussian, we can jump directly from the clean image x_0 to **any** noisy timestep x_t in **one single formula**:

     $$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

     where $\bar{\alpha}_t \in (0, 1)$ measures how much original image signal remains at step t (near 1 when t is small; near 0 when $t \to T$).

2. **The Reverse Process (Why Predict the Noise ϵ Instead of x_0?):**
   - To generate a new image, we start from pure noise $x_T \sim \mathcal{N}(0, I)$ and train a neural network (a time-conditioned U-Net or Diffusion Transformer `DiT`) to peel away a little bit of noise at each step $t \to t - 1$.
   - Ho et al. (DDPM) showed that training the network $\epsilon_\theta(x_t, t)$ with simple **Mean Squared Error (MSE) loss to predict the exact noise vector** ϵ that was added to x_0 is vastly more stable than asking it to output the

clean image directly, and is mathematically equivalent to **Score Matching** (estimating the gradient of the log data density $\nabla_{x_t} \log q(x_t)$)!

3. **DDPM vs. DDIM (Why 20 Steps Instead of 1000?):**
   - Standard **DDPM** reverse sampling is a stochastic Markov chain that injects fresh random noise $\sigma_t z$ at every step, requiring all $1,000$ sequential steps.
   - **DDIM (Denoising Diffusion Implicit Models)** constructs a **non-Markovian deterministic process** (a probability-flow ODE) that has the **exact same training marginal distribution** $q(x_t \mid x_0)$ as DDPM. That means you can take an **already-trained DDPM checkpoint with ZERO retraining** and sample high-quality images deterministically in just 20 **to** 50 **steps**!
4. **Latent Diffusion (Stable Diffusion) & Classifier-Free Guidance (CFG):**
   - Running diffusion directly in $512 \times 512 \times 3$ pixel space is painfully slow. **Latent Diffusion** first uses a frozen pretrained **VAE Encoder** to compress the image by $f = 8\times$ along height and width into a tiny $64 \times 64 \times 4$ latent tensor z_0—reducing spatial area (and attention tokens) by $8^2 = 64\times$!
   - How does the text prompt `"an astronaut riding a horse"` steer the image? Via **Cross-Attention** inside the U-Net/DiT: the noisy image latent z_t provides the **Queries (Q)**, while the CLIP/T5 text embeddings provide the **Keys (K) and Values (V)**.
   - Finally, **Classifier-Free Guidance (CFG)** makes the image follow the text prompt much more sharply: at each denoising step, we run the network twice—once **with** the text prompt $\epsilon_\theta(z_t, c)$ and once **without** the prompt $\epsilon_\theta(z_t, \varnothing)$—and **extrapolate** in the direction pointing from unconditional to conditional by a guidance weight $w > 1$ (e.g., $w = 7.5$).

📐 The Math & Every Symbol Explained

- x_0: Clean training image (or clean VAE latent z_0).
- $t \in \{1, 2, \dots, T\}$: Diffusion timestep ($t = 0$ is clean data; $t = T$ is pure isotropic Gaussian noise).
- $\beta_t \in (0, 1)$: Forward variance schedule at step t ($\beta_1 < \beta_2 < \cdots < \beta_T$, e.g., 10^{-4} to 0.02).
- $\alpha_t = 1 - \beta_t$: Single-step signal retention factor.
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$: Cumulative signal variance retention from step 0 to step t.
- $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: Ground-truth standard Gaussian noise sampled with the exact same tensor shape as x_0.
- $\epsilon_\theta(x_t, t, c)$: Neural network (U-Net or DiT with parameters θ) that takes noisy input x_t, timestep t, and conditioning c, and predicts the noise ϵ.

1. **Single-Step & Closed-Form $\mathcal{O}(1)$ Forward Diffusion Process:**

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}\Big)$$

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$

(Notice that the signal variance weight $\bar{\alpha}_t$ and noise variance weight $1 - \bar{\alpha}_t$ always sum to 1: $(\sqrt{\bar{\alpha}_t})^2 + (\sqrt{1-\bar{\alpha}_t})^2 = 1$, preserving unit variance if x_0 has unit variance!)

2. **Simplified Noise-Prediction Training Loss (DDPM):**

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[1,T],\, x_0,\, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \Big[\big\| \epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t\big) \big\|_2^2 \Big]$$

3. **Classifier-Free Guidance (CFG) Extrapolation Formula:** Let $\epsilon_\theta(z_t, c)$ be the conditional noise prediction (given text prompt c), $\epsilon_\theta(z_t, \varnothing)$ be the unconditional noise prediction (given an empty/null token $\varnothing$, trained by randomly dropping the prompt 10–20% of the time during training), and $w \geq 1$ be the guidance scale:

$$\tilde{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t, \varnothing) + w \cdot (\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing)) = w\,\epsilon_\theta(z_t, c) + (1 - w)\,\epsilon_\theta(z_t, \varnothing)$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Closed-Form DDPM Forward Noise Calculation at Step t):** Suppose at timestep $t = 400$, the cumulative signal product is $\bar{\alpha}_{400} = 0.36$. For a single pixel with clean value $x_0 = 2.0$ and sampled Gaussian noise $\epsilon = -0.5$, what are the mean, variance, and exact noisy value x_{400}?
  1. **Signal coefficient:** $\sqrt{\bar{\alpha}_t} = \sqrt{0.36} = \mathbf{0.60}$.
  2. **Noise standard deviation:** $\sqrt{1 - \bar{\alpha}_t} = \sqrt{1 - 0.36} = \sqrt{0.64} = \mathbf{0.80}$ (variance is $1 - \bar{\alpha}_t = 0.64$).
  3. **Exact noisy pixel x_{400}:**

     $$x_{400} = (0.60)(2.0) + (0.80)(-0.5) = 1.20 - 0.40 = \mathbf{0.80}$$

     (And if we know $x_{400} = 0.80$ and our network perfectly predicts $\hat{\epsilon} = -0.5$, we can immediately recover the clean pixel estimate: $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} = \frac{0.80 - 0.80(-0.5)}{0.60} = \frac{1.20}{0.60} = 2.0$!)
- **Example B (Classifier-Free Guidance & Latent Diffusion Compression):**
  1. **CFG Extrapolation:** At step t, suppose the unconditional noise prediction for a latent coordinate is $\epsilon_\theta(z_t, \varnothing) = 0.20$, the text-conditioned prediction is $\epsilon_\theta(z_t, c) = 0.60$, and the user sets guidance scale $w = 7.5$:

     $$\tilde{\epsilon} = 0.20 + 7.5 \times (0.60 - 0.20) = 0.20 + 7.5 \times 0.40 = 0.20 + 3.00 = \mathbf{3.20}$$

     (Check when $w = 1.0$: $\tilde{\epsilon} = 0.20 + 1.0(0.40) = 0.60$, which is standard conditional generation without extra guidance; when $w = 0$: $\tilde{\epsilon} = 0.20$, which is purely unconditional!)
  2. **Latent Diffusion Spatial Token Reduction:** A $512 \times 512 \times 3$ image has $512 \times 512 = 262,144$ spatial pixels. With a VAE downsampling factor of $f = 8$, the latent z_0 has spatial grid $64 \times 64 = 4,096$ locations —an exact $8^2 = \mathbf{64}\times$ reduction in spatial positions (and a $64^2 = 4,096\times$ **reduction** in all-to-all self-attention matrix size!).

🎯 Exam Takeaway & Common Traps

- **Trap 1 (Forward Variance Sum):** In $x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1 - \bar{\alpha}_t}\,\epsilon$, remember the square roots! The **standard deviations** are $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1 - \bar{\alpha}_t}$; the **variances** are $\bar{\alpha}_t$ and $1 - \bar{\alpha}_t$.
- **Trap 2 (DDIM Retraining Myth):** DDIM does **NOT** require retraining a DDPM model—it uses the exact same trained noise-predictor weights $\epsilon_\theta(x_t, t)$ with a deterministic non-Markovian update rule.
- **Trap 3 (Cross-Attention Q, K, V Sources in Text-to-Image Diffusion):** In Stable Diffusion's Cross-Attention layers, **Queries (Q)** always come from the **noisy image latent** z_t (so the output keeps the image's spatial $H \times W$ shape), while **Keys (K) and Values (V)** come from the **text prompt embeddings** c!

▼ 📌 Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Full DDPM, Score-Matching, DDIM & CFG Mathematical Derivations

1. **Why $q(x_t \mid x_0)$ Collapses in Closed Form (Reparameterization Induction):** Since $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}$ and $x_{t-1} = \sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-2}$, substituting recursively gives:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}}_{\text{Sum of two independent Gaussians } \sim \mathcal{N}(0,\, (1-\alpha_t\alpha_{t-1})\mathbf{I})}$$

   By induction down to $t = 0$ with $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, we obtain $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$.

2. **Exact Equivalence Between Noise Prediction ϵ_θ and Score Matching $\nabla_{x_t} \log q(x_t)$:** For $q(x_t \mid x_0) = \mathcal{N}(x_t;\, \sqrt{\bar{\alpha}_t} x_0,\, (1-\bar{\alpha}_t)\mathbf{I})$, taking the gradient of the log-density with respect to x_t yields:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1-\bar{\alpha}_t} = -\frac{\sqrt{1-\bar{\alpha}_t}\, \epsilon}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}} \implies \epsilon_\theta(x_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\, \nabla_{x_t} \log q(x_t)$$

3. **Why Classifier-Free Guidance Works (Implicit Classifier Derivation):** By Bayes' rule on the score of an implicit classifier $p(c \mid z_t) \propto \frac{p(z_t \mid c)}{p(z_t)}$:

$$\nabla_{z_t} \log p(c \mid z_t) = \nabla_{z_t} \log p(z_t \mid c) - \nabla_{z_t} \log p(z_t) \propto -\left(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing)\right)$$

   Therefore, sampling with $\tilde{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t, \varnothing) + w(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing))$ is equivalent to following the score of the sharpened distribution $\tilde{p}(z_t \mid c) \propto p(z_t) \cdot p(c \mid z_t)^w$ without needing a separate noisy-image classifier!

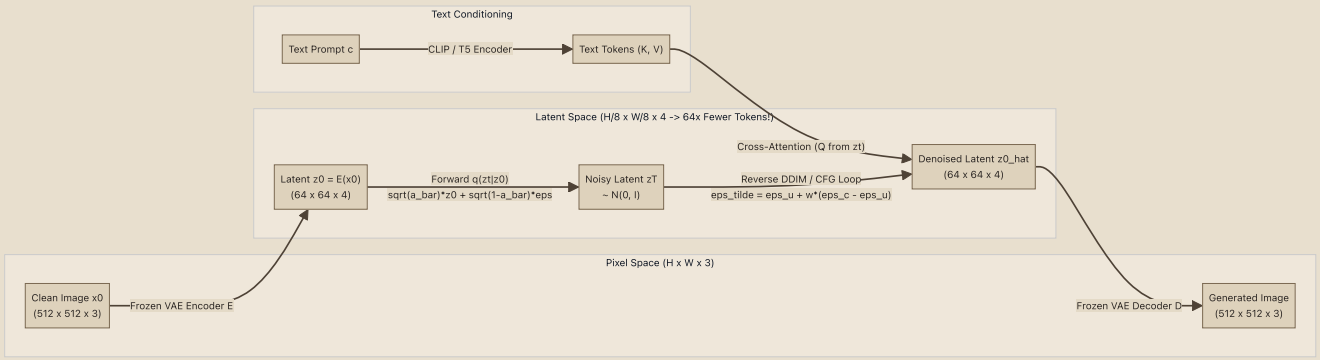
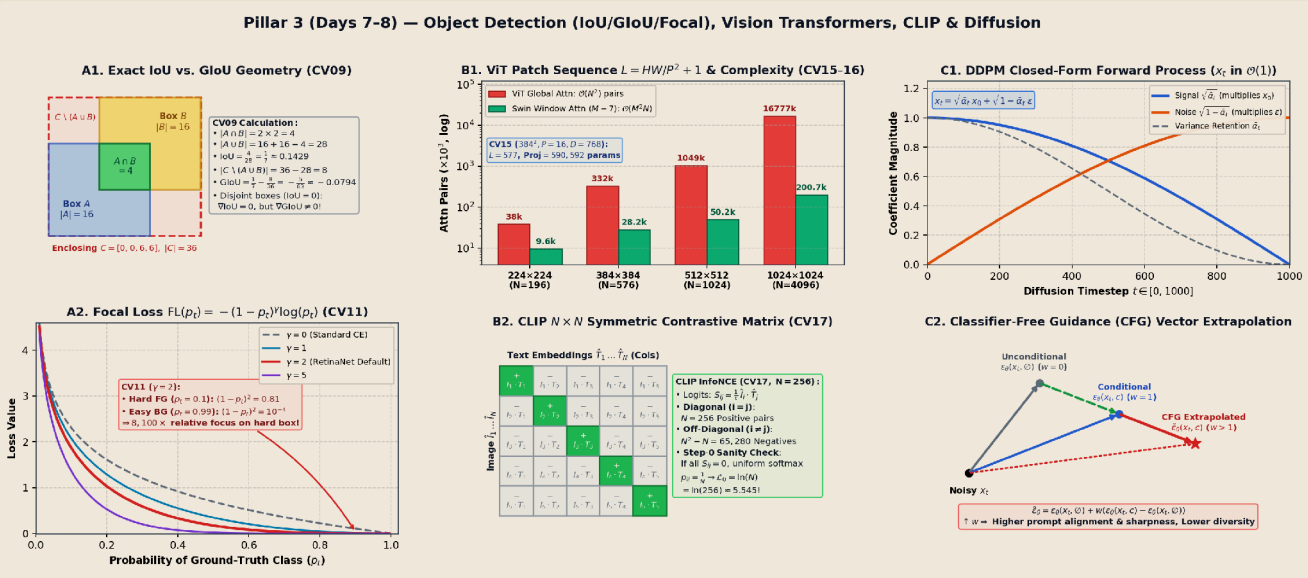
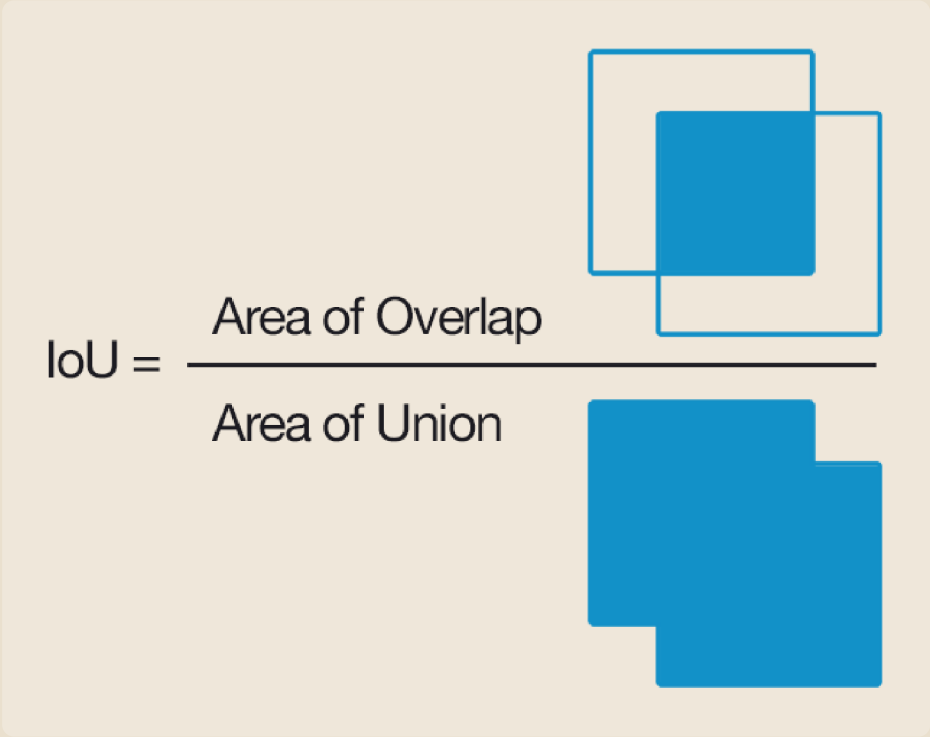


FIGURE SLIDE 1 OF 2



Days 7-8 Object Detection, ViT, CLIP & Diffusion Visuals

FIGURE SLIDE 2 OF 2



Canonical Intersection over Union Visual Equation

Pillar 4 (Days 9–10): NLP & Sequence Models — Intuitive Step-by-Step Study Textbook

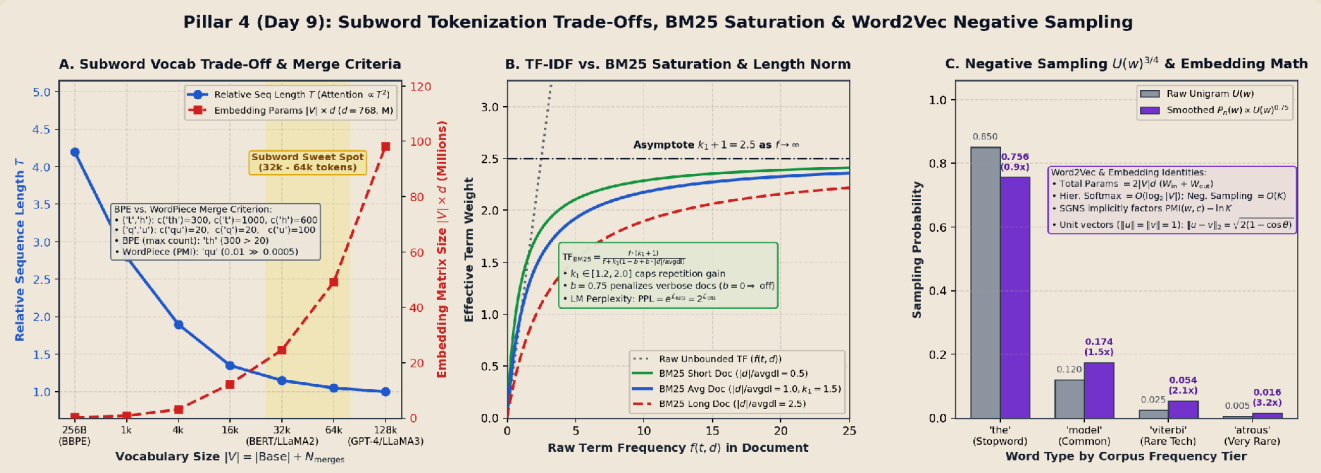
↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 9–10: Nov 6 – Nov 7):** Classical and neural NLP questions on HackerRank focus heavily on **tokenization merge rules & vocabulary math**, **exact RNN/LSTM/GRU/BiLSTM parameter counts**, **Word2Vec/GloVe/FastText training objectives**, **sequence labeling (Linear-Chain CRF Viterbi & CTC collapse rules)**, **decoding algorithms (Beam Search length penalty)**, and **exact metric calculations (Perplexity, BLEU, ROUGE, TF-IDF/BM25, Levenshtein Edit Distance)**.
>
> Every section in this guide is structured in four parts:
>
> 1. 💡 **Intuition (Plain English First):** The mental model and why the method was invented.
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations with every letter, subscript, and matrix dimension ($n \times m = n$ rows, m columns) defined explicitly.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number or toy-string calculations worked out line by line.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rules of thumb tested in `NLP01` – `NLP20`.

Day 9: Tokenization, Statistical NLP & Word Embeddings

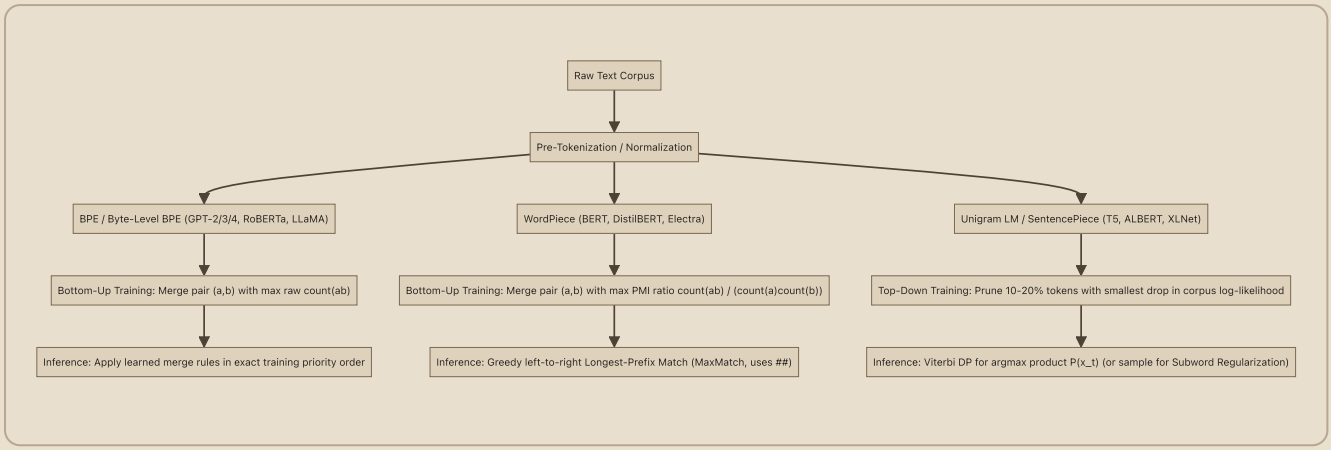
↑ Mục Lục

FIGURE SLIDE 1 OF 2



Pillar 4 Day 9 Visual Summary: Subword Tokenization Trade-Offs, BM25 Saturation & Word2Vec Negative Sampling

FIGURE SLIDE 2 OF 2



1.1 Subword Tokenization: BPE vs. Byte-Level BPE vs. WordPiece vs. Unigram LM

💡 Intuition (Plain English First)

Before a neural network can read text, we have to chop raw strings into discrete chunks called **tokens** and map each token to an integer ID in a vocabulary V. How big should those chunks be?

- **The Word-Level Trap:** If every distinct English word gets its own token (`"play"`, `"playing"`, `"played"`, `"unplayable"`), our vocabulary explodes past $|V| > 500,000$ words. Worse, the model has no idea that `"play"` and `"unplayable"` share a root, and any new word, typo, or rare medical term at test time becomes an unhelpful Out-Of-Vocabulary `<UNK>` token.
- **The Character-Level Trap:** What if we just tokenize into individual letters (`'p'`, `'l'`, `'a'`, `'y'`)? Now our vocabulary is tiny, and we never see `<UNK>`, *but* individual letters carry almost no meaning on their own, and our sequence length T becomes 4–6× longer! Because Transformer self-attention scales quadratically as $O(T^2)$, making a sequence $5\times$ longer makes attention $25\times$ **slower and more memory-hungry**.
- **The Subword Sweet Spot:** Subword tokenizers (**BPE**, **WordPiece**, **Unigram LM**) get the best of both worlds. Common words like `"play"` stay intact as single tokens, while rare or complex words like `"unplayable"` get split into meaningful morphemes (`"un"` + `"play"` + `"able"`).
  - **BPE (Byte-Pair Encoding)** builds its vocabulary *bottom-up* like Lego bricks: it starts with individual characters and repeatedly glues together whichever adjacent pair (a, b) appears **most frequently** in the training corpus.
  - **WordPiece** also builds *bottom-up*, but instead of picking the pair with the highest raw count, it picks the pair (a, b) that occurs together much more often than chance would predict (**Pointwise Mutual Information / PMI**). For instance, `'q'` and `'u'` may not be the most common letters overall, but whenever `'q'` shows up, `'u'` is almost guaranteed to follow!
  - **Unigram LM (SentencePiece)** works in the *exact opposite direction (top-down)*: it starts with a giant candidate list of substrings, fits a probabilistic unigram model, and iteratively **prunes (throws away)** the 10–20% of tokens whose removal hurts the overall corpus likelihood the least.

📐 The Math & Every Symbol Explained

1. **Exact BPE Vocabulary Size Formula:**

$$|V|_{\text{BPE}} = |\text{Base Alphabet}| + N_{\text{merges}} + |\text{Special Tokens}|$$

- $|V|_{\text{BPE}}$: Final total number of tokens in the BPE vocabulary.
- $|\text{Base Alphabet}|$: Number of initial atomic characters (or 256 raw UTF-8 bytes in **Byte-Level BPE**).
- N_{merges}: Number of merge operations run during training. **Why?** Each merge glues an existing pair (a, b) into a brand-new token ab and adds ab to the vocabulary *without deleting* a or b (since a and b might still appear separately in other words!). Thus, 1 merge $= +1$ vocabulary token.
- $|\text{Special Tokens}|$: Control tokens added manually, such as `<pad>`, `<bos>`, `<eos>`, or `[CLS]`.

2. **BPE Training Merge Criterion (Max Raw Co-Occurrence Count):**

$$(a^*, b^*) = \arg\max_{(a,b) \in V \times V} \text{count}(ab)$$

- $\text{count}(ab)$: Total number of times token a is immediately followed by token b across all words in the training corpus.

3. **WordPiece Training Merge Criterion (Max Likelihood / PMI Ratio):**

$$(a^*, b^*) = \arg\max_{(a,b) \in V \times V} \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}$$

- $\text{count}(a), \text{count}(b)$: Individual frequencies of token a and token b in the corpus. Dividing by $\text{count}(a) \cdot \text{count}(b)$ normalizes for how common the parts are individually, favoring strongly coupled pairs.

4. **Unigram LM (SentencePiece) Viterbi Segmentation Objective:**

$$x^* = \arg\max_{x \in S(W)} \prod_{t=1}^{|x|} P(x_t) = \arg\max_{x \in S(W)} \sum_{t=1}^{|x|} \ln P(x_t)$$

- W: The input word or string to be tokenized.
- $S(W)$: The set of all valid subword segmentations $x = (x_1, \ldots, x_{|x|})$ whose concatenation reconstructs W.
- $P(x_t)$: Learned unigram probability of subword token x_t (where $\sum_{v \in V} P(v) = 1$).

Algorithm	Used By	Training Direction	Exact Merge / Pruning Criterion	Inference Segmentation Algorithm
BPE (Byte-Pair Encoding)	GPT-2/3/4, RoBERTa, LLaMA	Bottom-up (starts with base chars/bytes)	Merges adjacent pair (a, b) with **highest raw count**: $\arg\max_{(a,b)} \text{count}(ab)$	Applies learned merge rules greedily in **exact training priority order** (NOT test-string frequency!)
WordPiece	BERT, DistilBERT, Electra	Bottom-up (uses `##` prefix for continuation subwords)	Merges pair (a, b) maximizing **PMI / Likelihood ratio**: $\arg\max_{(a,b)} \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}$	Greedy left-to-right **Longest-Prefix Matching** (`MaxMatch`) against vocabulary V
Unigram LM (SentencePiece)	T5, ALBERT, XLNet, mBART	Top-down (starts with huge candidate vocab, prunes 10–20% per step)	Computes unigram probabilities via EM; **removes** tokens whose deletion causes the smallest drop in $\sum \ln P(x_i)$	**Viterbi Dynamic Programming** to find $\arg\max_{x \in S(W)} \prod_t P(x_t)$ (supports **Subword Regularization** sampling)

- **Byte-Level BPE (`BBPE` — GPT-2/4, RoBERTa, LLaMA):** Uses all 256 **raw UTF-8 bytes** as the base alphabet ($|\text{Base Alphabet}| = 256$). Because every possible Unicode string (including emojis and rare scripts) is a sequence of UTF-8 bytes, Byte-Level BPE achieves **100% open-vocabulary coverage with ZERO `<UNK>` tokens** without exploding the base alphabet to $\sim 150,000$ Unicode characters.
- **SentencePiece Lossless Space Encoding:** Treats whitespace as a normal visible meta-symbol `_` (`U+2581`, e.g., `"_Hello_world"`), bypassing language-specific whitespace pre-tokenizers (critical for languages without spaces like Chinese, Japanese, and Thai) and guaranteeing lossless detokenization `detokenize(tokenize(s)) == s`.

🔢 Step-by-Step Worked Toy Example

Part A: 2 Steps of BPE Training on `"low"`, `"lower"`, `"newest"` Suppose our toy training corpus has three words with the following frequencies (each ending with an end-of-word marker `</w>` so the model knows where words end):

- `"l o w </w>"` : count $= 5$
- `"l o w e r </w>"` : count $= 2$
- `"n e w e s t </w>"` : count $= 6$

Initial Base Alphabet: `{'l', 'o', 'w', 'e', 'r', 'n', 's', 't', '</w>'}` (9 base tokens).

- **Merge Step 1:** Count all adjacent symbol pairs across the corpus:
  - ('e', 's'): appears in `"n e w e s t </w>"` (6 times) $\implies$ count $= 6$
  - ('s', 't'): appears in `"n e w e s t </w>"` (6 times) $\implies$ count $= 6$
  - ('w', 'e'): appears in `"l o w e r </w>"` (2) + `"n e w e s t </w>"` (6) $\implies$ count $= \mathbf{8}$ *(Winner!)*
  - ('l', 'o'): appears in `"l o w </w>"` (5) + `"l o w e r </w>"` (2) $\implies$ count $= 7$
  - ('o', 'w'): appears in `"l o w </w>"` (5) + `"l o w e r </w>"` (2) $\implies$ count $= 7$
  - **Action:** Merge ('w', 'e') $\rightarrow$ 'we'. Add `'we'` to vocabulary ($|V| = 9 + 1 = 10$).
  - **Updated Corpus:** `"l o w </w>"` (5), `"l o we r </w>"` (2), `"n e we s t </w>"` (6).
- **Merge Step 2:** Recount adjacent pairs in the updated corpus:
  - ('l', 'o'): $5 + 2 = \mathbf{7}$
  - ('o', 'w'): now only in `"l o w </w>"` (5), because `"l o we r"` has `'we'` instead of `'w'`!
  - ('e', 'we'), ('we', 's'), ('s', 't'): 6 each.
  - **Action:** Merge ('l', 'o') $\rightarrow$ 'lo'. Add `'lo'` to vocabulary ($|V| = 10 + 1 = 11$).
  - **Updated Corpus:** `"lo w </w>"` (5), `"lo we r </w>"` (2), `"n e we s t </w>"` (6).

> 💡 **TIP**
>
> **Part B: Worked 30-Second BPE vs. WordPiece Comparison Trap (`NLP01`):** Suppose in a corpus: count("t") $= 1000$, count("h") $= 600$, count("th") $= 300$, whereas count("q") $= 20$, count("u") $= 100$, count("qu") $= 20$. Which pair gets merged next?
>
> - **BPE chooses `"th"`** because raw co-occurrence count count("th") $= 300 >$ count("qu") $= 20$.
> - **WordPiece chooses `"qu"`** because the PMI likelihood ratio is:
>
> $$\text{Score}_{\text{WP}}(\text{"qu"}) = \frac{\text{count("qu")}}{\text{count("q")} \cdot \text{count("u")}} = \frac{20}{20 \times 100} = 0.01 \gg \text{Score}_{\text{WP}}(\text{"th"}) = \frac{300}{1000 \times 600} = 0.0($$
>
> Why? Because `"q"` *never* appears without `"u"`, giving `"qu"` a $20\times$ higher mutual information score!

🎯 Exam Takeaway & Common Traps

- **BPE Inference Priority Trap (`NLP01`):** At test time, BPE does **not** count pair frequencies in your test sentence! It looks up its list of learned merge rules and applies them in the **exact priority order they were learned during training** (Merge Rule #1 first, then Merge Rule #2, etc.).
- **Vocabulary Math Trap (`NLP02`):** If you start with 256 bytes (Byte-Level BPE), run $30,000$ merges, and add 4 special tokens, the final vocabulary size is $256 + 30,000 + 4 = 30,260$ (never subtract old characters!).
- **WordPiece `##` Prefix:** In BERT's WordPiece, `"playing"` $\rightarrow$ `["play", "##ing"]`. The `##` means "*glue this token to the previous token without a space.*" If a test word contains even *one* unknown character that prevents `MaxMatch` from covering the full word, WordPiece replaces the **entire word** with `[UNK]`.

1.2 N-Gram Language Models, Smoothing Hierarchy & Perplexity

💡 Intuition (Plain English First)

A **Language Model (LM)** answers one question: *Given the words seen so far, what is the probability distribution of the next word?*

- **Why N-Grams Need the Markov Assumption:** Conditioning on an entire book's history $P(w_t \mid w_1, w_2, \ldots, w_{t-1})$ is impossible by counting because almost every long sentence is unique. An **N-gram model** makes a simplifying **Markov assumption**: only the last $N-1$ words matter! For a **Bigram ($N=2$)** model, we only look at the single previous word w_{t-1} and estimate $P(w_t \mid w_{t-1})$ by simply dividing how often `"w_{t-1} w_t"` appeared by how often `"w_{t-1}"` appeared.
- **The Zero-Probability Catastrophe & Smoothing:** What if our training corpus had `"ate pizza"` and `"ate pasta"`, so count("ate") $= 100$, but at test time we see the perfectly valid phrase `"ate sushi"`, which happened to have count("ate sushi") $= 0$ in training? Raw Maximum Likelihood Estimation (MLE) assigns $P(\text{"sushi"} \mid \text{"ate"}) = \frac{0}{100} = 0$. Since sentence probability is a **product** of word probabilities, a single 0 makes the probability of the **entire test sentence equal to** 0 (and its log $P = -\infty$, so Perplexity blows up to ∞)! **Smoothing (Laplace / Add-k)** fixes this by "stealing" a tiny bit of probability mass from frequent words and giving a non-zero pseudocount k to every possible word in the vocabulary V.

- **What Perplexity (`PPL`) Really Means:** Loss functions like average Cross-Entropy (Negative Log-Likelihood, NLL) are measured in logarithmic units (*nats* or *bits*), which are hard to picture intuitively. **Perplexity** simply exponentiates the average cross-entropy loss: $\text{PPL} = e^{\text{NLL}}$. Intuitively, **Perplexity is the model's "effective branching factor"**—if a model has a Perplexity of 10, it is just as confused at each step as if it had to roll a fair 10-sided die to pick the next word! Lower Perplexity = better, more confident model.

📐 The Math & Every Symbol Explained

1. **N-Gram Chain Rule Approximation & Bigram MLE:**

$$P(w_1, w_2, \ldots, w_T) \approx \prod_{t=1}^{T} P(w_t \mid w_{t-N+1}^{t-1}), \qquad P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

   - T: Total number of tokens in the sequence $W = (w_1, \ldots, w_T)$.
   - N: Order of the N-gram model ($N = 1$ Unigram, $N = 2$ Bigram, $N = 3$ Trigram).
   - w_{t-N+1}^{t-1}: The context window of the $N - 1$ immediately preceding tokens.
   - **Parameter Table Size:** Storing all N-gram probabilities over a vocabulary of size $|V|$ requires a conceptual table of size $|V|^N$ entries, with $|V|^{N-1}(|V| - 1)$ independent parameters (since each conditional distribution over $|V|$ next words sums to 1).

2. **Laplace (Add-1) & Add-k Smoothing Formula (`NLP03`):**

$$P_{\text{Add-}k}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t) + k}{\text{count}(w_{t-1}) + k \cdot |V|}$$

   - k: Pseudocount added to every possible next word ($k = 1$ for Laplace smoothing; $0 < k < 1$ for Lidstone / Add-k smoothing).
   - $|V|$: Total number of unique token types in the vocabulary.
   - **Why is the denominator $+k|V|$ and NOT $+k$?** Because for a fixed history w_{t-1}, the conditional probabilities across all $|V|$ candidate next words $w \in V$ must sum to 1:

$$\sum_{w \in V} P_{\text{Add-}k}(w \mid w_{t-1}) = \frac{\sum_{w \in V} \text{count}(w_{t-1}, w) + \sum_{w \in V} k}{\text{count}(w_{t-1}) + k|V|} = \frac{\text{count}(w_{t-1}) + k|V|}{\text{count}(w_{t-1}) + k|V|} = 1$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Interpolated Kneser-Ney Smoothing Formula**

Why is **Interpolated Kneser-Ney** the gold standard of classical N-gram smoothing? Consider the famous "**San Francisco" Trap**: suppose the word `"Francisco"` appears 500 times in a newspaper corpus, so its raw unigram count count("Francisco") is very high. However, it *only* ever appears after the word `"San"`. If we encounter an unseen bigram after `"ate"` (like `"ate ___"`), standard backoff uses the raw unigram frequency and predicts `"ate Francisco"`!

Kneser-Ney fixes this in two ways: (1) **Absolute Discounting** subtracts a small constant $d \in (0, 1)$ (typically $d \approx 0.75$) from observed bigram counts, and (2) replaces the raw unigram backoff probability with the **Continuation Probability** $P_{\text{continuation}}(w_t)$, which measures *how many distinct preceding words w' have ever been seen before w_t*:

$$P_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max(\text{count}(w_{t-1}, w_t) - d, 0)}{\text{count}(w_{t-1})} + \lambda(w_{t-1}) P_{\text{continuation}}(w_t)$$

$$P_{\text{continuation}}(w_t) = \frac{|\{w' \in V : \text{count}(w', w_t) > 0\}|}{\sum_{w'' \in V} |\{w' \in V : \text{count}(w', w'') > 0\}|}$$

Because `"Francisco"` only ever follows 1 word (`"San"`), its continuation numerator $|\{w' : \text{count}(w', \text{"Francisco"}) > 0\}| = 1$ is tiny, preventing `"Francisco"` from polluting unseen contexts!

3. **Perplexity (`PPL`) Exact Formula (`NLP04`):**

$$\text{PPL}(W) = P(w_1, \ldots, w_T)^{-1/T} = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \ln P(w_t \mid w_{<t})\right) = \exp(\mathcal{L}_{\text{CE, nats}})$$

   - $P(w_1, \ldots, w_T)^{-1/T}$: The inverse geometric mean of the per-token probabilities assigned to the test sequence W of length T.
   - $\mathcal{L}_{\text{CE, nats}} = -\frac{1}{T} \sum_{t=1}^{T} \ln P(w_t \mid w_{<t})$: Average token-level Cross-Entropy (Negative Log-Likelihood) computed using the natural logarithm ln (base e, units = **nats**).
   - **Base-2 (Bits) Form:** If Cross-Entropy is computed using $\log_2$ (units = **bits per token**), you must exponentiate with base 2:

$$\text{PPL}(W) = 2^{\mathcal{L}_{\text{CE, bits}}}$$

   - **Uniform Baseline Identity:** A completely clueless model that guesses uniformly at random ($P(w_t \mid w_{<t}) = \frac{1}{|V|}$ for every step) achieves $\mathcal{L}_{\text{CE, nats}} = \ln |V|$ and $\text{PPL} = e^{\ln |V|} = |V|$.

🔢 Step-by-Step Worked Toy Example

Example 1: Laplace (Add-1) Smoothing Calculation (`NLP03`) Suppose our vocabulary has $|V| = 2,000$ unique words. In our training corpus, the word `"machine"` appears count("machine") = 500 times, `"machine learning"` appears 150 times, and `"machine dancing"` appears 0 times.

- **Unsmoothed MLE:**

$$P_{\text{MLE}}(\text{"learning"} \mid \text{"machine"}) = \frac{150}{500} = 0.30, \qquad P_{\text{MLE}}(\text{"dancing"} \mid \text{"machine"}) = \frac{0}{500} = 0$$

- **Laplace (Add-1, $k = 1$) Smoothing:** We add 1 to the numerator and $1 \times |V| = 2,000$ to the denominator:

$$P_{\text{Add-1}}(\text{"learning"} \mid \text{"machine"}) = \frac{150+1}{500+2000} = \frac{151}{2500} = \mathbf{0.0604}$$

$$P_{\text{Add-1}}(\text{"dancing"} \mid \text{"machine"}) = \frac{0+1}{500+2000} = \frac{1}{2500} = \mathbf{0.0004}$$

(Notice how a large vocabulary $|V| = 2,000$ pulls `"learning"` down sharply from 0.30 to 0.0604—which is why in practice people use a small $k = 0.01$ or Kneser-Ney instead of $k = 1$!)

Example 2: Perplexity from Average NLL & Token Probabilities (`NLP04`)

- **Case A (From Average NLL in Nats):** A language model evaluates a validation set and reports an average Negative Log-Likelihood loss of $\mathcal{L}_{\text{CE, nats}} = 2.0$ nats/token. What is its Perplexity?

$$\text{PPL} = e^{2.0} \approx \mathbf{7.389}$$

- **Case B (From Average NLL in Bits):** A model reports a cross-entropy of $\mathcal{L}_{\text{CE, bits}} = 3.0$ bits/token. What is its Perplexity?

$$\text{PPL} = 2^{3.0} = \mathbf{8.0}$$

- **Case C (From a 3-Token Sequence):** Suppose for a $T = 3$ token sentence, the model predicts $P(w_1) = 0.5$, $P(w_2 \mid w_1) = 0.25$, and $P(w_3 \mid w_1, w_2) = 0.125$.

$$\text{PPL} = (0.5 \times 0.25 \times 0.125)^{-1/3} = \left(\frac{1}{2} \times \frac{1}{4} \times \frac{1}{8}\right)^{-1/3} = \left(\frac{1}{64}\right)^{-1/3} = 64^{1/3} = \mathbf{4.0}$$

🎯 Exam Takeaway & Common Traps

- **Add-k Denominator Trap (`NLP03`):** Always check that the denominator is $\text{count}(w_{t-1}) + k|V|$, **never** $\text{count}(w_{t-1}) + k$ and **never** $\text{count}(w_{t-1}) + N_{\text{total tokens}}$.
- **Nats vs. Bits Trap (`NLP04`):** Check the units in the question stem! "**nats**" or "ln" $\implies e^{\mathcal{L}}$; "**bits**" or "$\log_2$" $\implies 2^{\mathcal{L}}$.
- **Cross-Tokenizer Incomparability Trap (`NLP04`):** You **cannot** compare per-token Perplexity between two models that use different tokenizers ($|V|_A = 32$k vs. $|V|_B = 128$k)! A character-level model ($|V| = 100$) will automatically have a tiny per-token PPL (~ 3), while a word-level model ($|V| = 100$k) has a much larger per-token PPL (~ 80), simply because predicting 1 letter is easier than predicting a whole word. To compare models across different tokenizers fairly, convert to **Bits-Per-Byte (BPB)**:

$$\text{BPB} = \frac{T_{\text{tokens}}}{T_{\text{UTF-8 bytes}}} \cdot \mathcal{L}_{\text{CE, bits}}$$

1.3 TF-IDF, BM25 & Levenshtein Edit Distance

💡 Intuition (Plain English First)

How does a search engine (or the lexical retriever inside a modern Hybrid RAG pipeline) match a user's query q against millions of documents d, and how do spell-checkers match misspelled strings?

1. **Why TF-IDF Exists:** If you search for `"the quantum compiler"`, simply counting how many query words appear in a document fails miserably: the stopword `"the"` appears 500 times in every long article, drowning out the rare, informative words `"quantum"` and `"compiler"`. **TF-IDF** multiplies two numbers for each query word t:
   - **Term Frequency (TF):** How often does word t appear in *this specific document* d? (Higher = more relevant to this document).
   - **Inverse Document Frequency (IDF):** How *rare* is word t across the entire collection of N documents? If `"the"` appears in 100% of documents, its rarity is $\log(N/N) = \log(1) = 0$, automatically wiping out stopwords!
2. **Why BM25 Beats Raw TF-IDF:** Raw TF-IDF has two big flaws:
   - *Linear TF Explosion:* If `"quantum"` appears 20 times in a document, raw TF scores it 20× higher than a document where `"quantum"` appears once—even though after seeing a word 3 or 4 times, we already know the document is about `"quantum"`! **BM25** passes term frequency through a saturating curve $\frac{f \cdot (k_1+1)}{f+k_1}$ that hits an upper ceiling of $k_1 + 1$.
   - *Long Document Bias:* A 10,000-word encyclopedic article naturally repeats words more often than a concise 200-word paragraph. **BM25** divides the term frequency by the relative document length $\frac{|d|}{\text{avgdl}}$, modulated by a hyperparameter $b \in [0, 1]$.
3. **Why Levenshtein Edit Distance Uses Dynamic Programming:** What if the user types `"INTENTION"` instead of `"EXECUTION"`? To measure how close two strings s_1 and s_2 are at the character level, **Levenshtein Edit Distance** counts the minimum number of single-character **Insertions**, **Deletions**, and **Substitutions** needed to turn s_1 into s_2. Instead of checking exponentially many alignments, the **Wagner-Fischer algorithm** builds an $(M+1) \times (N+1)$ grid where cell $D[i, j]$ solves the edit distance between the first i characters of s_1 and the first j characters of s_2 using the 3 neighboring cells above, left, and diagonal.

📐 The Math & Every Symbol Explained

1. **TF-IDF Formula:**

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t), \qquad \text{IDF}(t) = \log\left(\frac{N}{\text{DF}(t)}\right)$$

   - $\text{TF}(t, d) = f(t, d)$: Raw count (or relative frequency) of term t in document d.
   - N: Total number of documents in the corpus.
   - $\text{DF}(t)$: **Document Frequency**—the number of distinct documents in the corpus that contain term t at least once.
   - *Smoothed IDF (Scikit-Learn default):* $\text{IDF}_{\text{smooth}}(t) = \ln\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$ (prevents division-by-zero for unseen words and keeps words appearing in all N documents at a baseline weight of 1).
2. **BM25 (Best Matching 25) Formula (`NLP05`):**

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}_{\text{RSJ}}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

   - q: User query consisting of query terms $t \in q$.
   - $f(t, d)$: Raw count of term t in document d.

- $|d|$: Length (total word count) of document d.
- avgdl: Average document length across all N documents in the corpus.
- $k_1 \in [1.2, 2.0]$: **Term Frequency Saturation** hyperparameter.
  - As $f(t,d) \to \infty$, the fraction $\frac{f(t,d)(k_1+1)}{f(t,d)+\text{const}}$ asymptotically approaches $\lim_{f\to\infty} = k_1 + 1$.
  - Setting $k_1 = 0$ turns term frequency into a binary indicator (1 if $f > 0$, else 0).
- $b \in [0, 1]$ (typically 0.75): **Document Length Normalization** hyperparameter.
  - If $|d| >$ avgdl (verbose document), the denominator grows larger, penalizing the score.
  - Setting $b = 0$ makes $1 - b + b\frac{|d|}{\text{avgdl}} = 1$, **completely disabling document length normalization**.
- $\text{IDF}_{\text{RSJ}}(t) = \ln\left(1 + \frac{N - \text{DF}(t) + 0.5}{\text{DF}(t) + 0.5}\right)$: Robertson-Spärck Jones IDF weight.

3. **Levenshtein Edit Distance Dynamic Programming Recurrence (`NLP18`):** Let string s_1 have length M and string s_2 have length N. We construct a DP matrix $D \in \mathbb{R}^{(M+1)\times(N+1)}$ ($M+1$ **rows,** $N+1$ **columns**), where row 0 and column 0 represent the empty string ϵ:
   - **Base Cases:** $D[i, 0] = i$ for $0 \le i \le M$ (i deletions to turn $s_1[1..i]$ into ϵ), and $D[0, j] = j$ for $0 \le j \le N$ (j insertions to turn ϵ into $s_2[1..j]$).
   - **Recurrence for** $1 \le i \le M$, $1 \le j \le N$**:**

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 & \text{(Deletion of } s_1[i]\text{)} \\ D[i,j-1] + 1 & \text{(Insertion of } s_2[j]\text{)} \\ D[i-1,j-1] + c_{\text{sub}}(s_1[i], s_2[j]) & \text{(Match / Substitution)} \end{cases}$$

   - $c_{\text{sub}}(s_1[i], s_2[j])$: Substitution cost, equal to 0 if $s_1[i] = s_2[j]$ (characters match!), and 1 if $s_1[i] \neq s_2[j]$ under standard Levenshtein distance (or 2 under Jurafsky & Martin's alignment convention where substitution = 1 deletion + 1 insertion).

🔢 Step-by-Step Worked Toy Example

Example 1: TF-IDF & BM25 Saturation on a 3-Document Corpus (`NLP05`) Suppose we have a corpus of $N = 3$ documents with average length avgdl $= 100$ words:

- **Doc 1 ($|d_1| = 100$):** Contains `"the"` (5 times), `"neural"` (2 times), `"compiler"` (1 time).
- **Doc 2 ($|d_2| = 100$):** Contains `"the"` (8 times), `"neural"` (1 time), `"compiler"` (0 times).
- **Doc 3 ($|d_3| = 100$):** Contains `"the"` (4 times), `"neural"` (0 times), `"compiler"` (0 times).

1. **Compute Unsmoothed Base-10** $\text{IDF}(t) = \log_{10}(N/\text{DF}(t))$**:**
   - `"the"` appears in all 3 docs (DF = 3): IDF("the") $= \log_{10}(3/3) = \log_{10}(1) = \mathbf{0}$. Even though `"the"` appears 8 times in Doc 2, its TF-IDF $= 8 \times 0 = \mathbf{0}$!
   - `"neural"` appears in 2 docs (DF = 2): IDF("neural") $= \log_{10}(3/2) \approx \mathbf{0.176}$.
   - `"compiler"` appears in only 1 doc (DF = 1): IDF("compiler") $= \log_{10}(3/1) \approx \mathbf{0.477}$.
2. **Watch BM25 Saturation in Action ($k_1 = 1.5$, $|d| =$ avgdl)**: Since $|d| =$ avgdl $= 100$, the length factor is $1 - b + b(1) = 1$, so the BM25 TF weight is $\frac{f\cdot(1.5+1)}{f+1.5} = \frac{2.5f}{f+1.5}$:
   - If $f = 1$: BM25 TF weight $= \frac{2.5(1)}{1+1.5} = \mathbf{1.00}$.
   - If $f = 6$: BM25 TF weight $= \frac{2.5(6)}{6+1.5} = \frac{15}{7.5} = \mathbf{2.00}$ (a 6× increase in word count only doubles the score!).
   - If $f = 1000$: BM25 TF weight $= \frac{2.5(1000)}{1001.5} \approx \mathbf{2.496} \le k_1 + 1 = \mathbf{2.50}$ (hard ceiling at 2.50!).

Example 2: Levenshtein Edit Distance DP Table (`NLP18`) Let's transform $s_1 =$ "CAT" ($M = 3$) into $s_2 =$ "CUT" ($N = 3$) and also check the classic exam pair `"INTENTION"` $\rightarrow$ `"EXECUTION"`:

- For `"CAT"` $\rightarrow$ `"CUT"`, we build a $(3+1) \times (3+1) = 4 \times 4$ matrix (4 rows, 4 columns):
  - Row 1 (`'C'` vs `'C','U','T'`): `'C'` matches `'C'` ($D[1,1] = D[0,0] + 0 = 0$), then $D[1,2] = 1, D[1,3] = 2$.
  - Row 2 (`'A'` vs `'C','U','T'`): `'A'` $\neq$ `'U'`, so $D[2,2] = \min(D[1,2]+1, D[2,1]+1, D[1,1]+1) = \min(2, 2, 0+1) = \mathbf{1}$ (1 substitution `'A'` $\rightarrow$ `'U'`).
  - Row 3 (`'T'` vs `'C','U','T'`): `'T'` matches `'T'`, so $D[3,3] = D[2,2] + 0 = \mathbf{1}$.
- **Canonical Exam Pair (`'INTENTION'` $\rightarrow$ `'EXECUTION'`, `NLP18`):** Both strings have length 9 and share the identical 4-character suffix `'TION'` (0 edits needed for the last 4 characters!). We only need to transform the 5-character prefix `'INTEN'` into `'EXECU'`:
  - Under **standard Levenshtein distance (substitution cost $= 1$)**: Replacing all 5 prefix positions takes at most 5 **edits** (specifically: delete `'I'`, `'N'`$\rightarrow$`'E'`, `'T'`$\rightarrow$`'X'`, insert `'E'`, `'N'`$\rightarrow$`'C'`, `'U'`? Even simpler: 5 direct position operations $= 5$ **edits**).
  - Under **Jurafsky & Martin's alternate alignment cost (substitution cost $= 2$):** Because `'INTEN'` and `'EXECU'` share one `'E'` that aligns, 1 deletion (`'I'`) + 1 insertion (`'C'`) + 3 substitutions ($3 \times 2 = 6$) $= 8$ **edits**.

🎯 Exam Takeaway & Common Traps

- **Zero-IDF Stopword Rule:** In unsmoothed TF-IDF, any word appearing in **all N documents** has $\text{DF}(t) = N \implies \text{IDF}(t) = \log(N/N) = \mathbf{0}$.
- **BM25 Asymptote Trap (`NLP05`):** If an MCQ asks for the maximum possible term-frequency multiplier in BM25 as term count $f(t,d) \rightarrow \infty$ (for an average-length document), the answer is $k_1 + 1$ (NOT ∞ and NOT k_1).
- **Edit Distance Cost Convention (`NLP18`):** Always read whether the problem specifies **unit substitution cost (1)** ($\text{Edit}(\text{"INTENTION"}, \text{"EXECUTION"}) = 5$) or **substitution cost $= 2$** ($= 8$). Time complexity is always $O(M \cdot N)$, and space complexity can be optimized from $O(M \cdot N)$ to $O(\min(M, N))$ if you only need the final distance value (since row i only depends on row $i-1$).

1.4 Static vs. Contextual Word Embeddings & Vector Space Geometry

💡 Intuition (Plain English First)

One-hot vectors treat every word as completely orthogonal: in a one-hot vocabulary of size $|V| = 50,000$, the dot product between `"cat"` and `"kitten"` is 0—just as unrelated as `"cat"` and `"refrigerator"`. How do we learn dense vectors in $\mathbb{R}^d$ (say $d = 300$) where semantically similar words point in similar directions?

1. **The Distributional Hypothesis & Word2Vec:** Linguist J.R. Firth famously said, "*You shall know a word by the company it keeps.*" Words like `"cat"` and `"kitten"` appear surrounded by the same neighbor words (`"pet"`, `"purr"`, `"milk"`, `"vet"`). **Word2Vec (Mikolov et al., 2013)** turns unsupervised text into a self-supervised prediction game over a sliding window of size m:
   - **CBOW (Continuous Bag-of-Words):** Takes the surrounding $2m$ context words, **averages** their vectors together, and tries to predict the missing **center word** in the middle (like a fill-in-the-blank quiz). Because

it averages the context into 1 vector, CBOW runs **1 update per window** (fast!) and smooths over noise, making it great for **frequent words**.

- **Skip-Gram:** Does the exact reverse—takes the **single center word** w_c and tries to independently predict each of the $2m$ **surrounding context words** one by one ($2m$ updates per window). Because a rare word doesn't get averaged away with its neighbors and gets $2m$ direct gradient updates every time it appears, Skip-Gram is much better for **rare words**.

2. **Why Word2Vec Needs TWO Matrices ($2|V|d$ Parameters):** Why doesn't Word2Vec just use a single $|V| \times d$ matrix? If a word w used the exact same vector v_w when acting as a center word and when acting as a context word, the model would need $v_w^T v_w = \|v_w\|_2^2$ (which is huge!) in the softmax denominator, representing the absurd event of a word appearing next to *itself*! To keep the math clean and stable, Word2Vec maintains **two separate lookup tables**: a **Center (Input) matrix** W_{in} and a **Context (Output) matrix** W_{out}.
3. **Why Full Softmax Is Too Slow & How SGNS Fixes It:** Computing a standard Softmax over $|V| = 100,000$ words requires computing $100,000$ dot products in the denominator for *every single training step*!
   - **Hierarchical Softmax** arranges all $|V|$ words as leaves of a binary Huffman tree, turning a $100,000$-way choice into $\approx \log_2(100,000) \approx 17$ binary left/right coin flips along a tree path.
   - **Skip-Gram with Negative Sampling (SGNS)** is even simpler: instead of asking "*Which of the 100,000 words is the true neighbor?*", it asks a binary yes/no question: "*Is* `(cat, purr)` *a real pair from the text (label 1), and are these* $K = 5$ *randomly drawn fake pairs* `(cat, carburetor)`, `(cat, democracy)`, ... *noise (label 0)?*" That reduces the cost from $O(|V|)$ to just $O(K)$ (6 dot products instead of $100,000$)!
4. **From Static Word2Vec to FastText & Contextual BERT:** Word2Vec and GloVe assign **1 fixed vector per word type** in a static lookup table. That creates two huge problems: (1) any unseen typo or new word at test time fails (`<UNK>`), and (2) **polysemy**—the word `"bank"` gets the exact same vector in `"river bank"` and `"investment bank"`!
   - **FastText** fixes the `<UNK>` / morphology problem by breaking every word into **character** n**-grams** (`"<wh"`, `"whe"`, `"her"`, `"ere"`, `"re>"`) and summing their vectors—so even a misspelled or unseen word gets a vector!
   - **ELMo & BERT** fix the **polysemy** problem by running the full sentence through a deep bidirectional encoder (BiLSTM in ELMo, Transformer in BERT) so the representation h_t of `"bank"` dynamically changes depending on the rest of the sentence.

📐 The Math & Every Symbol Explained

1. **Word2Vec Parameter Count & Full Softmax Objective (`NLP06`):**

$$\text{Total Learnable Parameters} = |W_{\text{in}}| + |W_{\text{out}}| = |V| \cdot d + |V| \cdot d = \mathbf{2|V|d}$$

$$P(w_o \mid w_c) = \frac{\exp(u_o^T v_c)}{\sum_{w=1}^{|V|} \exp(u_w^T v_c)}$$

- $|V|$: Vocabulary size (number of unique words).
- d: Embedding dimension (e.g., $d = 300$).
- $W_{\text{in}} \in \mathbb{R}^{|V| \times d}$ ($|V|$ **rows,** d **columns**): Center/Input embedding matrix; row c is the center word vector $v_c \in \mathbb{R}^d$.

- $W_{\text{out}} \in \mathbb{R}^{|V| \times d}$ ($|V|$ **rows,** d **columns**): Context/Output embedding matrix; row o is the context word vector $u_o \in \mathbb{R}^d$.
- *(Note: Word2Vec linear projections have* ***no bias terms****! After training,* W_{in} *is either used alone or averaged/concatenated with* W_{out}*).*

2. **Skip-Gram with Negative Sampling (SGNS) Objective (`NLP07`):**

$$\mathcal{J}_{\text{SGNS}}(w_c, w_o) = \ln \sigma(u_o^T v_c) + \sum_{k=1}^{K} \mathbb{E}_{w_k \sim P_n(w)} \left[\ln \sigma(-u_k^T v_c) \right]$$

- $\sigma(z) = \frac{1}{1+e^{-z}}$: Standard logistic sigmoid function (note that $\sigma(-z) = 1 - \sigma(z)$).
- $u_o^T v_c$: Dot product between true context vector u_o and center vector v_c (we want this to be large and positive so $\sigma(u_o^T v_c) \to 1$).
- K: Number of negative (noise) samples drawn per positive pair (typically $K \in [5, 20]$ for small datasets, $K \in [2, 5]$ for massive datasets).
- u_k: Context vector of the k-th negative sample w_k drawn from noise distribution $P_n(w)$ (we want $u_k^T v_c$ to be negative so $\sigma(-u_k^T v_c) \to 1$).

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): 3/4 Unigram Noise Power & Levy-Goldberg SPPMI Theorem**

- **Why does Word2Vec sample negative words from** $P_n(w) \propto U(w)^{3/4}$ **instead of raw unigram frequency** $U(w)$**?** If we sampled noise words proportionally to raw frequency $U(w)$, ultra-frequent stopwords like `"the"` would dominate the negative samples, while rare words would almost never be trained as negative examples. Raising every probability to the 0.75 power dampens big numbers and gives rare words a huge relative boost! For example, compare a common word ($U(\text{"is"}) = 0.05$) with a rare word ($U(\text{"aardvark"}) = 0.0001$):
  - Raw ratio: $0.05 / 0.0001 = 500\times$ more likely.
  - Smoothed 3/4-power ratio: $0.05^{0.75} / 0.0001^{0.75} = 0.1057 / 0.001 = 105.7\times$—giving `"aardvark"` nearly a $5\times$ **boost** in sampling probability!
- **Levy & Goldberg (2014) Theorem:** Optimizing Skip-Gram with K Negative Samples is mathematically equivalent to implicitly factorizing the **Shifted Pointwise Mutual Information (SPPMI)** matrix:

$$v_c^T u_o = \text{PMI}(w_c, w_o) - \ln K = \ln \frac{P(w_c, w_o)}{P(w_c)P(w_o)} - \ln K$$

3. **Unit-Normalized Cosine Similarity vs. Euclidean (L_2) Distance Identity (`NLP08`):** For any two vectors $u, v \in \mathbb{R}^d$ that have been L_2-normalized to unit length ($\|u\|_2 = \|v\|_2 = 1$):

$$\cos\theta = \frac{u^T v}{\|u\|_2 \|v\|_2} = u^T v$$

$$\|u - v\|_2^2 = \|u\|_2^2 + \|v\|_2^2 - 2u^T v = 1 + 1 - 2\cos\theta = 2(1 - \cos\theta) \implies \|u - v\|_2 = \sqrt{2(1 - \cos\theta)}$$

- Because $\sqrt{2(1-\cos\theta)}$ is a strictly decreasing function of $\cos\theta$ on $[-1, 1]$, **ranking unit-normalized embeddings by smallest Euclidean distance** $\|u - v\|_2$ **produces the exact same nearest-neighbor ranking as sorting by largest Cosine Similarity** $\cos\theta$**!**

Model	Granularity	Handles Unseen OOV Words?	Polysemy (`"river bank"` vs. `"investment bank"`)?	Core Mechanism / Objective
Word2Vec	Word type	No (`<UNK>`)	No (1 static vector per word)	Local context window prediction (SGNS / CBOW)
GloVe	Word type	No (`<UNK>`)	No (1 static vector per word)	Global log-bilinear regression on co-occurrence counts $\ln X_{ij}$
FastText	Character n-grams ($n \in [3, 6]$)	**Yes!** (sums subword n-grams)	No (static sum of char n-grams)	Represents word as $v_w + \sum_{g \in \mathcal{G}(w)} z_g$ (ideal for typos & rich morphology)
ELMo / BERT	Subword + Full Sequence	Yes (via subwords)	**Yes!** (contextual hidden state h_t)	Deep bidirectional sequence encoding (BiLSTM LM in ELMo; Masked LM in BERT)

> 📌 **NOTE**
>
> **BERT Masked Language Modeling (MLM) 80-10-10 Rule (`NLP19`):** During BERT pretraining, 15% of tokens in each sequence are selected for MLM loss. Of those selected tokens: 80% **are replaced with** `[MASK]`, 10% **are replaced with a random vocabulary token**, and 10% **are kept unchanged**. Why not 100% `[MASK]`? Because the `[MASK]` token never appears during downstream fine-tuning (creating a **pretrain-finetune discrepancy**)! If BERT only ever had to predict `[MASK]` tokens, it could lazily ignore non-masked tokens. Including 10% random and 10% unchanged tokens forces the Transformer encoder to build rich, denoised contextual representations for *every* input token, because any token might be evaluated in the loss!

🔢 Step-by-Step Worked Toy Example

Example 1: Word2Vec Parameter Count & Per-Step Softmax Speedup (`NLP06` , `NLP07`) Suppose we train a Skip-Gram model on a vocabulary of size $|V| = 65,536 = 2^{16}$ with embedding dimension $d = 300$ and $K = 5$ negative samples:

1. **Total Learnable Parameters:**

$$\text{Params} = 2 \times |V| \times d = 2 \times 65,536 \times 300 = \mathbf{39,321,600} \quad (\text{NOT } 65,536 \times 300 = 19.66\text{M!})$$

2. **Dot Products per Context Prediction Step:**
   - **Full Softmax:** $O(|V|) = \mathbf{65,536}$ dot products.
   - **Hierarchical Softmax:** $O(\log_2 |V|) = \log_2(2^{16}) = \mathbf{16}$ dot products ($4,096\times$ speedup!).
   - **Negative Sampling ($K = 5$):** $O(K + 1) = 5 + 1 = \mathbf{6}$ dot products ($10,922\times$ speedup!).

Example 2: Unit-Vector Euclidean Distance from Cosine Similarity (`NLP08`) Two unit-normalized word embeddings $u, v \in \mathbb{R}^{768}$ ($\|u\|_2 = \|v\|_2 = 1$) have a cosine similarity of $\cos\theta = 0.68$. What is their exact Euclidean distance $\|u - v\|_2$?

- Step 1: Plug into the squared L_2 identity:

$$\|u - v\|_2^2 = 2(1 - \cos\theta) = 2(1 - 0.68) = 2(0.32) = 0.64$$

- Step 2: Take the square root:

  $$\|u - v\|_2 = \sqrt{0.64} = \mathbf{0.80}$$

- *(Bonus check: What if u and v are orthogonal, $\cos\theta = 0$? Then $\|u - v\|_2 = \sqrt{2(1-0)} = \sqrt{2} \approx 1.414$. What if they point in opposite directions, $\cos\theta = -1$? Then $\|u - v\|_2 = \sqrt{2(1-(-1))} = \sqrt{4} = 2.0$.)*

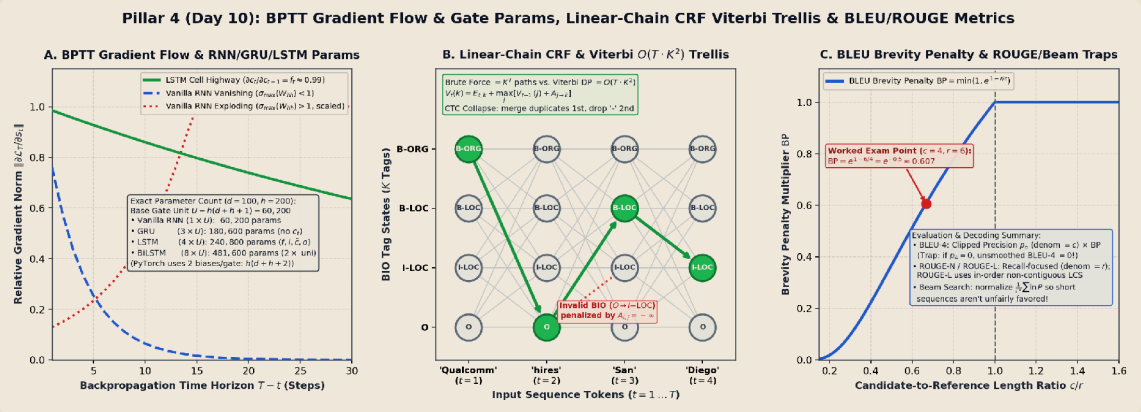
🎯 Exam Takeaway & Common Traps

- **The $2|V|d$ Parameter Trap (`NLP06`):** Whenever an exam asks for the number of trainable parameters in Word2Vec (Skip-Gram or CBOW), remember to multiply $|V| \times d$ by 2 because of the separate input (W_{in}) and output (W_{out}) embedding matrices!
- **CBOW vs. Skip-Gram Rare Word Trap: Skip-Gram** is superior for **rare words** (no context averaging; $2m$ independent updates per window), whereas **CBOW** trains faster and is slightly better for **frequent words**.
- **FastText vs. BERT Polysemy Trap:** FastText solves **OOV / typos** via character n-grams, but it is still a **static** embedding model—it assigns the exact same vector to `"bank"` regardless of context! Only **contextual encoders (ELMo, BERT, GPT)** resolve **polysemy**.

Day 10: Sequence Models (RNN/LSTM/GRU), Sequence Labeling & NLP Metrics

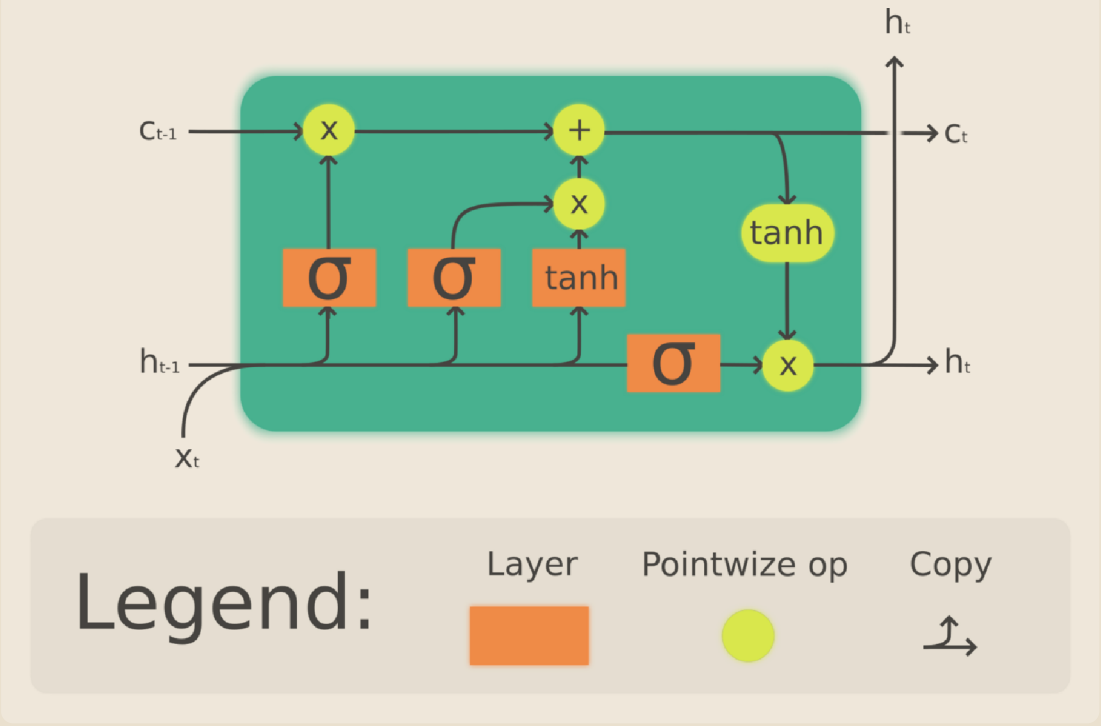
↑ Mục Lục

FIGURE SLIDE 1 OF 5



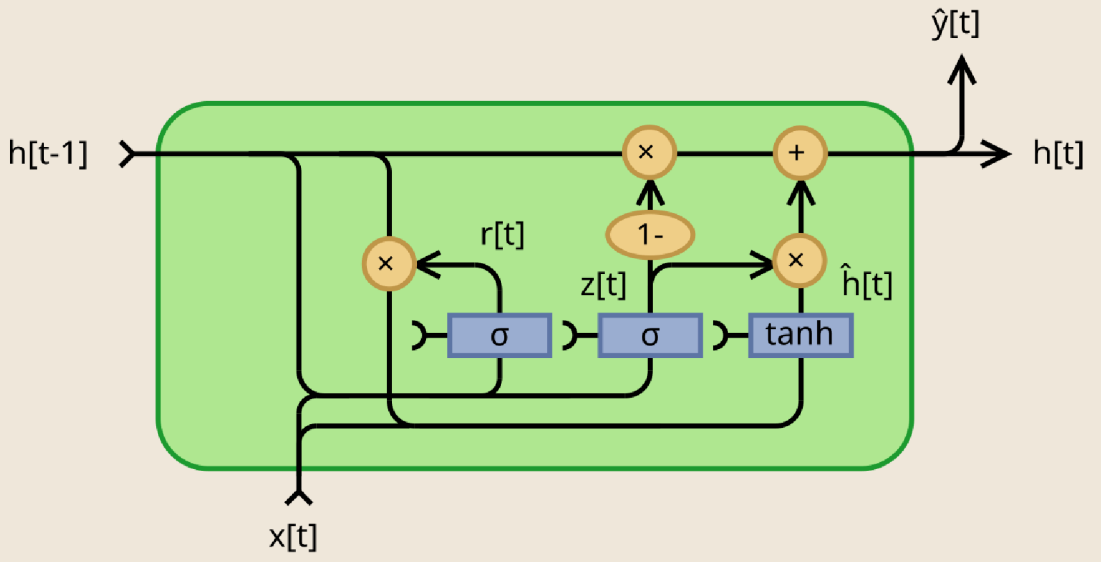
Pillar 4 Day 10 Visual Summary: BPTT Gradient Flow & Gate Params, Linear-Chain CRF Viterbi Trellis & BLEU/ROUGE Metrics

FIGURE SLIDE 2 OF 5



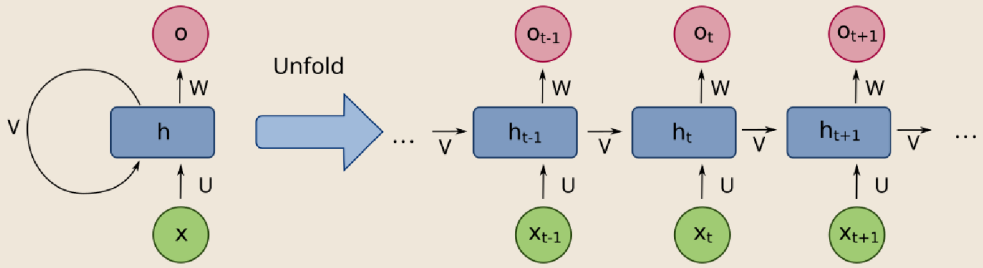
Canonical LSTM Cell Architecture (Wikimedia Commons)

FIGURE SLIDE 3 OF 5



Canonical GRU Cell Architecture (Wikimedia Commons)

FIGURE SLIDE 4 OF 5



Recurrent Neural Network Unfolding Through Time (Wikimedia Commons)

FIGURE SLIDE 5 OF 5



2.1 Vanilla RNN, LSTM, GRU & BiLSTM: Equations, Parameter Counts & BPTT Gradients

💡 Intuition (Plain English First)

How do we process a variable-length sentence $x_1, x_2, \ldots, x_T$ while keeping a memory of what came earlier?

1. **Why Vanilla RNN Gradients Vanish or Explode:** A **Vanilla RNN** maintains a hidden state vector h_t and updates it at every step by multiplying the previous state h_{t-1} by a weight matrix W_{hh}, adding the new input $W_{xh}x_t$, and squashing the result through $\tanh$:

   $$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

   Notice what happens during **Backpropagation Through Time (BPTT)**: to send an error signal from the end of a 50-word sentence ($t = 50$) back to the first word ($t = 1$), the chain rule multiplies by the Jacobian $\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(1 - h_k^2)W_{hh}^T$ **50 times in a row**!

   - Just like multiplying $0.8^{50} = 0.000014$, if the largest singular value $\sigma_{\max}(W_{hh}) < 1$ (or if $\tanh$ saturates so $1 - h_k^2 \to 0$), the gradient **vanishes exponentially** to zero—the network completely forgets the subject at the start of the sentence!
   - Conversely, like multiplying $1.2^{50} = 9,100$, if $\sigma_{\max}(W_{hh}) > 1$, the gradient **explodes exponentially** to `NaN`!

2. **How LSTM's Additive Cell State Creates a "Gradient Highway":** Why did **Long Short-Term Memory (LSTM, Hochreiter & Schmidhuber 1997)** solve the vanishing gradient problem? Instead of forcing the memory through a matrix multiplication and a squashing $\tanh$ at every single step, LSTM introduces a separate **Cell State conveyor belt** c_t whose update is **purely element-wise addition**:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Look at the derivative along this highway: $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ (plus indirect terms through the gates). There is **no matrix multiplication** W_{hh} and **no** $\tanh'$ **squashing factor** blocking the main path! Whenever the **forget gate** $f_t \approx 1$, the error gradient flows straight back across 100+ time steps completely undiminished (just like residual connections $y = x + F(x)$ in ResNets!).

3. **How Each LSTM Gate Works (The Mental Model):** Every gate is a vector of numbers between 0 and 1 (produced by σ) that acts like a dimmer switch via element-wise multiplication $\odot$:
   - **Forget Gate ($f_t \in (0,1)^h$):** "*What fraction of my old long-term memory c_{t-1} should I keep vs. erase?*" (e.g., when a sentence switches to a new subject, $f_t \to 0$ to wipe out the old subject's gender/number).
   - **Input Gate ($i_t \in (0,1)^h$) & Candidate Memory ($\tilde{c}_t \in (-1,1)^h$):** "*What new information $\tilde{c}_t$ is arriving right now, and how much of it (i_t) should I write onto the conveyor belt?*"
   - **Output Gate ($o_t \in (0,1)^h$):** "*Out of everything stored in my long-term cell state c_t, what part is relevant to expose right now as my short-term working state $h_t = o_t \odot \tanh(c_t)$?*"
4. **How GRU Simplifies LSTM: Gated Recurrent Unit (Cho et al., 2014)** asks: *Do we really need two separate state vectors (c_t and h_t) and four gates?* GRU merges c_t and h_t into a **single state** h_t and uses only **3 gate blocks** (**Reset gate** r_t, **Update gate** z_t, and **Candidate** $\tilde{h}_t$), coupling the forget and input decisions together via a convex combination $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. This cuts parameter count and memory by 25% compared to LSTM while keeping the additive gradient highway!

📐 The Math & Every Symbol Explained

Let input at step t be $x_t \in \mathbb{R}^d$ ($d \times 1$ column vector) and hidden state be $h_t \in \mathbb{R}^h$ ($h \times 1$ column vector). When we concatenate $[h_{t-1}; x_t]$, we get a vector of length $h + d$.

1. **Single-Gate Parameter Block Size (U):** Every gate takes $x_t \in \mathbb{R}^d$ and $h_{t-1} \in \mathbb{R}^h$ and outputs a vector in $\mathbb{R}^h$. This requires:
   - Input-to-hidden weight matrix $W_{xh} \in \mathbb{R}^{h \times d}$ (h **rows,** d **columns** $\implies h \cdot d$ parameters).
   - Hidden-to-hidden weight matrix $W_{hh} \in \mathbb{R}^{h \times h}$ (h **rows,** h **columns** $\implies h \cdot h = h^2$ parameters).
   - Bias vector $b \in \mathbb{R}^h$ (h **rows,** 1 **column** $\implies h$ parameters in standard textbook formulation; or **two** bias vectors $b_{ih}, b_{hh} \in \mathbb{R}^h \implies 2h$ parameters in PyTorch's `nn.RNN` / `nn.GRU` / `nn.LSTM` implementation):

     $$U_{\text{1-bias}} = h(d + h + 1), \qquad U_{\text{PyTorch (2-bias)}} = h(d + h + 2)$$

2. **LSTM Exact Gate Equations (4 Gate Blocks $\implies 4 \times U$ Parameters, `NLP10`):**

$$f_t = \sigma\left(W_f [h_{t-1}; x_t] + b_f\right) \qquad (\text{Forget Gate}, f_t \in \mathbb{R}^h)$$

$$i_t = \sigma\left(W_i [h_{t-1}; x_t] + b_i\right) \qquad (\text{Input Gate}, i_t \in \mathbb{R}^h)$$

$$\tilde{c}_t = \tanh\left(W_c [h_{t-1}; x_t] + b_c\right) \qquad (\text{Candidate Cell State}, \tilde{c}_t \in \mathbb{R}^h)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad (\text{Additive Cell State Highway}, c_t \in \mathbb{R}^h)$$

$$o_t = \sigma\left(W_o[h_{t-1}; x_t] + b_o\right) \qquad (\text{Output Gate, } o_t \in \mathbb{R}^h)$$

$$h_t = o_t \odot \tanh(c_t) \qquad (\text{Exposed Hidden State, } h_t \in \mathbb{R}^h)$$

- $W_f, W_i, W_c, W_o \in \mathbb{R}^{h \times (h+d)}$ (h **rows,** $h+d$ **columns**): Combined weight matrices for the 4 gates.
- $\odot$: Hadamard (element-wise) vector multiplication.
- **Positive Forget-Gate Bias Trick:** Initializing $b_f = +1.0$ (Jozefowicz et al., 2015) makes $f_t \approx \sigma(1.0) \approx 0.73$ at the start of training, ensuring the cell state remembers long-range dependencies by default instead of immediately zeroing them out.

3. **GRU Exact Gate Equations (3 Gate Blocks $\implies 3 \times U$ Parameters):**

$$r_t = \sigma\left(W_r[h_{t-1}; x_t] + b_r\right) \qquad (\text{Reset Gate, } r_t \in \mathbb{R}^h)$$

$$z_t = \sigma\left(W_z[h_{t-1}; x_t] + b_z\right) \qquad (\text{Update Gate, } z_t \in \mathbb{R}^h)$$

$$\tilde{h}_t = \tanh\left(W_h[r_t \odot h_{t-1}; x_t] + b_h\right) \qquad (\text{Candidate Hidden State, } \tilde{h}_t \in \mathbb{R}^h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \qquad (\text{Interpolated Hidden State, } h_t \in \mathbb{R}^h)$$

4. **BPTT Gradient Flow Equation (`NLP11`):**

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{k=2}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{k=2}^{T} \text{diag}(1 - h_k^2) W_{hh}^T$$

- **Global Gradient Norm Clipping (Fix for Exploding Gradients):** If $\|g\|_2 > \tau$, rescale the entire gradient vector $g \leftarrow \frac{\tau}{\|g\|_2} g$ so its direction is preserved while its norm is capped at threshold τ.

Architecture	Gates / States	Theoretical Params (1 bias/gate)	PyTorch Params (2 biases/gate)	Worked Example ($d = 100, h = 200$, 1 bias, `NLP09`)
Vanilla RNN	1 (tanh state h_t)	$1 \times h(d+h+1)$	$1 \times h(d+h+2)$	$200(100+200+1) = \mathbf{60,200}$
GRU	3 ($r_t, z_t, \tilde{h}_t$; no c_t)	$3 \times h(d+h+1)$	$3 \times h(d+h+2)$	$3 \times 60,200 = \mathbf{180,600}$
LSTM	4 ($f_t, i_t, \tilde{c}_t, o_t$; has c_t, h_t)	$4 \times h(d+h+1)$	$4 \times h(d+h+2)$	$4 \times 60,200 = \mathbf{240,800}$ ($241,600$ in PyTorch)
BiLSTM (1 Layer)	$2 \times 4 = 8$ ($\overrightarrow{\text{LSTM}} + \overleftarrow{\text{LSTM}}$)	$8 \times h(d+h+1)$	$8 \times h(d+h+2)$	$2 \times 240,800 = \mathbf{481,600}$ (output dim $= 2h = 400$)

🔢 Step-by-Step Worked Toy Example

Let's walk through a tiny toy network with input dimension $d = 4$ and hidden state dimension $h = 8$ (using standard textbook 1 bias per gate) to see how every single parameter is counted:

1. **Step 1: Compute 1 Gate Block U:**
   - Input-to-hidden matrix W_{xh} has shape 8×4 (8 rows, 4 columns) $= 32$ weights.

- Hidden-to-hidden matrix W_{hh} has shape 8×8 (8 rows, 8 columns) $= 64$ weights.
- Bias vector b has shape $8 \times 1 = 8$ biases.
- **Single-Gate Block Total:** $U = 32 + 64 + 8 = 8(4 + 8 + 1) = 8 \times 13 = \mathbf{104}$ parameters.

2. **Step 2: Multiply by Number of Gates:**
   - **Vanilla RNN (1 gate):** $1 \times 104 = \mathbf{104}$ parameters.
   - **GRU (3 gates: $r_t, z_t, \tilde{h}_t$):** $3 \times 104 = \mathbf{312}$ parameters.
   - **LSTM (4 gates: $f_t, i_t, \tilde{c}_t, o_t$):** $4 \times 104 = \mathbf{416}$ parameters *(in PyTorch with 2 biases/gate:* $4 \times 8(4 + 8 + 2) = \mathbf{448}$ *parameters)*.
   - **1-Layer Bidirectional LSTM (Forward LSTM + Backward LSTM):** $2 \times 416 = \mathbf{832}$ parameters. At each time step t, the forward LSTM outputs $\overrightarrow{h}_t \in \mathbb{R}^8$ and the backward LSTM outputs $\overleftarrow{h}_t \in \mathbb{R}^8$, which are concatenated into $h_t^{(1)} = [\overrightarrow{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{\mathbf{16}}$.
3. **Step 3: What if we stack a 2nd BiLSTM Layer on top? (The Stacked BiLSTM Trap!)** Because Layer 1 outputs a concatenated vector of size $2h = 16$, **Layer 2's input dimension is $d_2 = 2h = 16$ (NOT 4 and NOT 8!)**:
   - Layer 2 single-gate block: $U_2 = h(2h + h + 1) = 8(16 + 8 + 1) = 8 \times 25 = 200$.
   - Layer 2 BiLSTM (8 gates total): $8 \times 200 = \mathbf{1,600}$ parameters!
   - Total 2-Layer BiLSTM parameters $= 832$ (Layer 1) $+ 1,600$ (Layer 2) $= \mathbf{2,432}$ parameters.

🎯 Exam Takeaway & Common Traps

- **Memorize the Multiplier Rule (`NLP09`):** Once you compute $U = h(d + h + 1)$, **RNN** $= 1U$, **GRU** $= 3U$, **LSTM** $= 4U$, **1-Layer BiLSTM** $= 8U$. Always check if a PyTorch-specific question gives an option that is $+4h$ larger for LSTM (due to PyTorch's redundant second bias b_{hh}).
- **Stacked BiLSTM Layer-2 Input Trap:** In a multi-layer BiLSTM with hidden size h per direction, Layer 1 outputs dimension $2h$, so Layer 2 has input size $d_2 = 2h$!
- **Vanishing vs. Exploding Fix Trap (`NLP11`): Gradient Norm Clipping fixes exploding gradients ONLY**—it does *nothing* to help vanishing gradients! Vanishing gradients require architectural changes (**LSTM/GRU additive cell state updates**, **residual connections**, or **Transformer direct attention paths**).

2.2 Sequence Labeling: BIO Tagging, Linear-Chain CRF & CTC Loss

💡 Intuition (Plain English First)

In sequence labeling tasks like Named Entity Recognition (NER), Part-of-Speech (POS) tagging, or Automatic Speech Recognition (ASR), we want to assign a sequence of labels to a sequence of inputs.

1. **Why We Need BIO Tagging & Span-Level F1:** Suppose a sentence says `"Per-Olof flew from New York Los Angeles yesterday"` (with a missing `"to"`). If we only labeled words as `LOC` (Location) or `O` (Outside), both `"New York"` and `"Los Angeles"` would become `LOC LOC LOC LOC`—making it impossible to tell whether that is one giant 4-word city or two separate 2-word cities! **BIO Tagging** prefixes the **first word** of every entity with `B-` (Beginning, e.g., `B-LOC`) and continuation words with `I-` (Inside, e.g., `I-LOC`), so `"New York Los Angeles"` becomes `B-LOC I-LOC B-LOC I-LOC`.

- *Why Token-Level Accuracy is Useless for NER:* In normal English text, $> 92\%$ of words are not named entities (`O`). A lazy model that predicts `O` for every word in the test set gets 92% **token-level accuracy** while finding **zero** entities! That is why NER is always evaluated using **Exact Span-Level F1** (a predicted entity only counts as a True Positive if its start index, end index, *and* entity type match the ground truth 100%).

2. **Why Token-Level Softmax Fails on BIO & How Linear-Chain CRF Fixes It (`NLP15`):** If you put an independent Softmax classifier on top of each BiLSTM/BERT token state h_t, the decision at word t has **no idea what tag was actually chosen at word** $t-1$! If word 1 has 51% probability for `O` and 49% for `B-PER`, while word 2 has 51% probability for `I-PER`, independent greedy Softmax outputs **`O -> I-PER`** — which is **grammatically illegal** in BIO tagging (you cannot be *Inside* a Person entity without first *Beginning* one!).
   - A **Linear-Chain Conditional Random Field (CRF)** fixes this by learning a $K \times K$ **tag transition score matrix** A where entry $A_{i,j}$ rewards or penalizes transitioning from tag i at step $t-1$ to tag j at step t. The CRF very quickly learns $A_{\text{O,I-PER}} \approx -\infty$ and scores the **entire sequence of tags** $y_{1:T}$ **globally**. Because checking all K^T tag sequences by brute force is impossible (10^{50} paths for $K = 10, T = 50$), CRF uses the **Forward Algorithm** during training and **Viterbi Dynamic Programming** at inference to find the exact best path in $O(T \cdot K^2)$ time!

3. **Why CTC Loss Exists for Speech & OCR (`NLP20`):** In NER, 10 input words map cleanly to 10 output tags ($T = U$). But in speech recognition, you might have $T = 300$ audio frames (10 ms each) for the 3-letter word `"too"` ($U = 3$), and nobody labeled which exact millisecond corresponds to `'t'` vs. `'o'`! **Connectionist Temporal Classification (CTC)** lets the network output one character (or a special blank symbol `'-'`) at every single frame, and then collapses the T-frame output into a word using a simple 2-step rule: **(1) merge consecutive identical characters first, then (2) delete all blanks** `'-'`. Because consecutive `'o'`s get merged into a single `'o'`, to spell a word with a true double letter like `"too"`, the model **must** emit at least one blank `'-'` between the two `'o'`s (e.g., `'t-o-o'`)!

📐 The Math & Every Symbol Explained

1. **Linear-Chain CRF Global Sequence Score & Probability (`NLP15`):**

$$s(X, y_{1:T}) = \sum_{t=1}^{T} (E_{t,y_t} + A_{y_{t-1},y_t})$$

$$P(y_{1:T} \mid X) = \frac{\exp\left(s(X, y_{1:T})\right)}{Z(X)}, \qquad Z(X) = \sum_{y' \in \mathcal{Y}^T} \exp\left(s(X, y')\right)$$

   - T: Sequence length (number of tokens).
   - $K = |\mathcal{Y}|$: Number of distinct BIO tags in the tagset $\mathcal{Y}$.
   - $E \in \mathbb{R}^{T \times K}$ (T **rows,** K **columns**): **Emission score matrix** output by the encoder (e.g., BiLSTM or BERT linear head), where $E_{t,k}$ is the local logit score for assigning tag k to token t.
   - $A \in \mathbb{R}^{K \times K}$ (K **rows,** K **columns**): Learnable **Transition score matrix**, where $A_{i,j}$ is the score of transitioning from tag i at step $t-1$ to tag j at step t.
   - $Z(X)$: **Partition function** summing $\exp(s(X, y'))$ over all K^T possible tag sequences $y' \in \mathcal{Y}^T$. During training, we minimize the negative log-likelihood $-\ln P(y^*_{1:T} \mid X) = \ln Z(X) - s(X, y^*_{1:T})$, computing

$\ln Z(X)$ in $O(T \cdot K^2)$ time via the **Forward Algorithm** (which replaces max with logsumexp).

2. **Viterbi Dynamic Programming Recurrence (Exact Inference in $O(T \cdot K^2)$ Time):**

$$V_t(k) = E_{t,k} + \max_{j \in \{1,\ldots,K\}} (V_{t-1}(j) + A_{j,k})$$

   - $V_t(k)$: Score of the highest-scoring partial tag path ending in tag k at time step t (stored in a trellis matrix $V \in \mathbb{R}^{T \times K}$ of T **rows,** K **columns**).
   - At each step $t \in \{1, \ldots, T\}$ and for each target tag $k \in \{1, \ldots, K\}$, we check all K previous tags $j \in \{1, \ldots, K\}$ ($\implies K \times K = K^2$ operations per step, or $O(T \cdot K^2)$ total!).

3. **CTC Alignment Collapse Function $\mathcal{B}(\pi)$ (`NLP20`):**

$$\mathcal{B} : (\Sigma \cup \{\text{`-'}\})^T \to \Sigma^{\leq T}$$

   - **Step 1:** Collapse any contiguous run of identical symbols into a single symbol (e.g., `'tt--oo'` → `'t-o'`).
   - **Step 2:** Remove all blank tokens `'-'` (e.g., `'t-o'` → `'to'`).

<details>
<summary>📌 Optional / Speech & OCR Bonus (Skip unless reviewing NLP20): Worked CTC Collapse Examples</summary>

Why is the order of operations in CTC (**Step 1: merge duplicates** → **Step 2: drop blanks**) so important? Let's trace three 6-frame alignments for target words `"to"`, `"too"`, and `"tto"`:

- **Alignment 1:** `'tt--oo'`
  - Step 1 (merge consecutive duplicates): `'tt'` → `'t'`, `'--'` → `'-'`, `'oo'` → `'o'` $\implies$ `'t-o'`
  - Step 2 (drop all blanks `'-'`): **`'to'`** *(Notice that `'oo'` without a blank between them collapsed into a single `'o'`!)*
- **Alignment 2:** `'tto-o-'`
  - Step 1 (merge consecutive duplicates): `'tt'` → `'t'` $\implies$ `'to-o-'`
  - Step 2 (drop all blanks `'-'`): **`'too'`** *(Because there was a blank `'-'` separating the two `'o'`s, Step 1 did NOT merge them, so Step 2 leaves `'oo'` intact!)*
- **Alignment 3:** `'t-t-oo'`
  - Step 1 (merge consecutive duplicates): `'oo'` → `'o'` $\implies$ `'t-t-o'`
  - Step 2 (drop all blanks `'-'`): **`'tto'`**
</details>

🔢 Step-by-Step Worked Toy Example

Viterbi vs. Greedy Decoding on a 2-Step Toy NER Trellis (`NLP15`) Suppose our input is a $T = 2$ word phrase: $x_1 =$ "New", $x_2 =$ "York", and we have $K = 3$ BIO tags: `{1: B-LOC, 2: I-LOC, 3: O}`. Suppose the encoder produces the following **Emission Matrix** $E \in \mathbb{R}^{2 \times 3}$ **(2 rows, 3 columns)** (assuming uniform start transitions $A_{\text{START},k} = 0$):

Time Step / Token	$E_{t,\text{B-LOC}}$	$E_{t,\text{I-LOC}}$	$E_{t,\text{O}}$
$t = 1$ (`"New"`)	2.0	0.0	**2.2** *(slightly prefers `'O'` because `"New"` is a common adjective!)*
$t = 2$ (`"York"`)	1.0	**3.0**	0.5

And suppose the CRF has learned the **Transition Matrix** $A \in \mathbb{R}^{3\times3}$ **(3 rows = from-tag** i**, 3 columns = to-tag** j **)**:

- $A_{\text{B-LOC}\to\text{I-LOC}} = +\mathbf{2.0}$ (strongly rewards valid multi-word location continuation!)
- $A_{\text{O}\to\text{I-LOC}} = -\mathbf{10.0}$ (heavily penalizes the illegal `O -> I-LOC` transition!)
- All other transitions ($A_{\text{B-LOC}\to\text{B-LOC}}, A_{\text{B-LOC}\to\text{O}}, A_{\text{O}\to\text{O}}, A_{\text{O}\to\text{B-LOC}}$) $= 0.0$.

1. **What does Independent Token-Level Greedy Decoding do?**
   - At $t = 1$ (`"New"`), it looks only at row 1 of E: $\arg\max(2.0, 0.0, 2.2) = \text{O}$ (score 2.2).
   - At $t = 2$ (`"York"`), it looks only at row 2 of E: $\arg\max(1.0, 3.0, 0.5) = \text{I-LOC}$ (score 3.0).
   - **Greedy Output:** `O -> I-LOC` (an **illegal** BIO sequence! Its true global CRF score is $2.2 - 10.0 + 3.0 = -4.8$).
2. **What does CRF Viterbi Dynamic Programming do?**
   - **Step** $t = 1$ **(`"New"`):** Initialize $V_1(k) = E_{1,k}$:

     $$V_1(\text{B-LOC}) = 2.0, \qquad V_1(\text{I-LOC}) = 0.0, \qquad V_1(\text{O}) = 2.2$$

   - **Step** $t = 2$ **(`"York"`):** Compute $V_2(\text{I-LOC}) = E_{2,\text{I-LOC}} + \max_j[V_1(j) + A_{j\to\text{I-LOC}}]$:
     - From `j = B-LOC`: $V_1(\text{B-LOC}) + A_{\text{B-LOC}\to\text{I-LOC}} = 2.0 + 2.0 = \mathbf{4.0}$ *(Best predecessor!)*
     - From `j = I-LOC`: $V_1(\text{I-LOC}) + A_{\text{I-LOC}\to\text{I-LOC}} = 0.0 + 0.0 = 0.0$
     - From `j = O`: $V_1(\text{O}) + A_{\text{O}\to\text{I-LOC}} = 2.2 + (-10.0) = -7.8$
     - Add emission $E_{2,\text{I-LOC}} = 3.0$:

       $$V_2(\text{I-LOC}) = 3.0 + 4.0 = \mathbf{7.0}$$

   - Compare with $V_2(\text{B-LOC}) = 1.0 + \max(2.0 + 0, 0, 2.2 + 0) = 3.2$ and $V_2(\text{O}) = 0.5 + 2.2 = 2.7$.
   - **Viterbi Backtrace:** The highest score at $t = 2$ is $V_2(\text{I-LOC}) = 7.0$, and its backpointer points to `B-LOC` at $t = 1$. Thus Viterbi outputs the globally optimal, grammatically valid span `B-LOC -> I-LOC`!

🎯 Exam Takeaway & Common Traps

- **CRF Complexity Trap (`NLP15`):** With T tokens and K tags, brute-force search is $O(K^T)$, whereas both **Forward training** and **Viterbi inference** take $O(T \cdot K^2)$ time (quadratic in the number of tags K, linear in sequence length T!).
- **CTC Double-Letter Trap (`NLP20`):** Because CTC merges consecutive identical tokens *before* deleting blanks `'-'`, emitting a true repeated character (like `'ll'` in `"hello"` or `'oo'` in `"too"`) **strictly requires at least one blank `'-'` between the two identical letters**.

2.3 Seq2Seq, Attention Variants, Decoding & NLP Evaluation Metrics

💡 Intuition (Plain English First)

How do Encoder-Decoder (Seq2Seq) models translate a French sentence into English or summarize an article, how do we search for the best output sentence, and how do we grade the result?

1. **The Seq2Seq Bottleneck & How Bahdanau/Luong Attention Fixed It (`NLP13`):** In early RNN Seq2Seq models (Sutskever et al., 2014), the Encoder read a 50-word source sentence $x_1, \ldots, x_{T_x}$ and squashed its *entire meaning* into the **single final hidden state vector** h_{T_x}—like trying to write the summary of a whole novel on the back of a postage stamp! By the time the Decoder was generating word 30, details from word 1 had vanished.
   - **The Attention Fix (Bahdanau 2014, Luong 2015):** Instead of throwing away intermediate encoder states, keep **all** encoder hidden states $h_1, h_2, \ldots, h_{T_x}$ on a shelf! At every decoder step t, the decoder uses its current state s_t as a "search query" to compare against every encoder state h_i, turns those compatibility scores $e_{t,i}$ into a probability distribution $\alpha_{t,i}$ via Softmax, and computes a fresh **context vector** $c_t = \sum_i \alpha_{t,i} h_i$ focused on whichever source words are being translated *right now*.
   - **Bahdanau (Additive) vs. Luong (Multiplicative):** Bahdanau passes s_t and h_i through a small 1-layer neural network ($v_a^T \tanh(W_1 s_t + W_2 h_i)$), whereas Luong computes a fast matrix/dot product ($s_t^T W_a h_i$ or $s_t^T h_i$). Later, **Vaswani et al. (2017)** divided the dot product by $\sqrt{d_k}$ so large dot products wouldn't push Softmax into flat, zero-gradient saturation!

2. **Why Teacher Forcing Causes Exposure Bias (`NLP12`):** When training an autoregressive decoder, what token should we feed as input at step t? If we feed the model's own (initially terrible) predictions, a mistake at word 1 derails the rest of the sentence, making training slow and unstable. **Teacher Forcing** fixes this during training by always feeding the **ground-truth human word** y^*_{t-1} as the input to step t.
   - **The Catch (Exposure Bias):** At test time, there is no human teacher! The model is forced to feed its **own predicted words** $\hat{y}_{t-1}$ back into itself. Because the model was *only ever exposed* to perfect human prefixes during training, a single slightly odd word choice at test time pushes the model into an unfamiliar state, causing errors to snowball rapidly over long generations.

3. **Why Beam Search Needs a Length Penalty (`NLP14`):** At each decoding step, **Greedy Decoding ($B = 1$)** picks the single highest-probability word right now, which can easily walk into a grammatical trap two steps later. **Beam Search** keeps the top B (say $B = 4$) most promising partial sentences alive at every step.
   - **Why Unnormalized Beam Search Loves Short Sentences:** Every word probability $P(y_t \mid y_{<t}, X)$ is a number ≤ 1, so its log-probability $\ln P(y_t \mid \ldots)$ is **strictly negative** (e.g., -1.5). Therefore, every time a candidate sentence adds one more word, its total score $\sum_{t=1}^{L} \ln P$ **gets more negative (decreases)**! Without correction, a short 3-word fragment beats a complete 15-word translation every time. Dividing the cumulative log-probability by L^α ($\alpha \in [0.6, 1.0]$) computes the *average* log-probability per token, leveling the playing field between short and long sentences.

4. **Why BLEU Needs Clipped Precision & a Brevity Penalty (`NLP16`):** How do we automatically score a machine translation C against a human reference R?
   - **Why Clipped Precision?** Suppose the reference is `"the cat is on the mat"` (6 words) and a broken model outputs `"the the the the"` (4 words). Naive unigram precision says: "*Every `'the'` in the candidate appears in the reference, so precision* $= 4/4 = 100\%$*!*" **Clipped Precision** stops this exploit by capping each word's numerator credit at the maximum number of times it appears in the reference ($\text{count}_R(\text{"the"}) = 2 \implies 2/4 = 50\%$).
   - **Why the Brevity Penalty (BP)?** Even with clipping, what if the model outputs a 1-word translation: just `"the"`? Now clipped count $= 1$ and candidate length $c = 1$, giving $p_1 = 1/1 = 100\%$ precision! Why doesn't BLEU just use Recall like F1? Because in translation, you often have *multiple valid human references* (`"the cat is on the mat"` vs. `"there is a feline atop the rug"`), and a good translation shouldn't be forced to recall words from *all* references at once! Instead, BLEU replaces recall

with a **Brevity Penalty** $\text{BP} = \exp(1 - r/c)$ whenever the candidate is shorter than the reference ($c \leq r$). If $c > r$, $\text{BP} = 1$ (since clipped precision already penalizes overly long outputs).

5. **Why Summarization Uses ROUGE-L & Semantic Eval Uses BERTScore (`NLP17`):**
   - In summarization, we care most about **Recall**: *Did the summary capture the key facts from the reference?* **ROUGE-N** divides matched n-grams by the **Reference length** (not Candidate length!), and **ROUGE-L** measures the **Longest Common Subsequence (LCS)**—words appearing in the same relative order, even if other words are inserted in between!
   - Both BLEU and ROUGE are purely lexical (exact string match): if the reference says `"automobile"` and the model writes `"car"`, BLEU and ROUGE give 0 **credit**! **BERTScore** fixes this by matching token embeddings via cosine similarity in a pretrained Transformer space.

📐 The Math & Every Symbol Explained

1. **Seq2Seq Attention Equations & Alignment Variants (`NLP13`):** Let encoder hidden states be $h_1, \ldots, h_{T_x} \in \mathbb{R}^{d_h}$ and the decoder state at step t be $s_t \in \mathbb{R}^{d_s}$:

$$e_{t,i} = \text{score}(s_t, h_i), \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T_x} \exp(e_{t,j})}, \qquad c_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i$$

   - $e_{t,i} \in \mathbb{R}$: Raw unnormalized alignment score between decoder step t and encoder position i.
   - $\alpha_{t,i} \in (0, 1)$: Normalized attention weight ($\sum_{i=1}^{T_x} \alpha_{t,i} = 1$).
   - $c_t \in \mathbb{R}^{d_h}$: Dynamic context vector at decoder step t.

Attention Mechanism	Alignment Score Formula $\text{score}(s_t, h_i)$	Learnable Parameter Matrix Shapes ($n \times m$ = n rows, m cols) & Properties
Bahdanau (Additive, 2014)	$v_a^T \tanh(W_1 s_t + W_2 h_i)$	$W_1 \in \mathbb{R}^{d_a \times d_s}$ (d_a rows, d_s cols), $W_2 \in \mathbb{R}^{d_a \times d_h}$ (d_a rows, d_h cols), $v_a \in \mathbb{R}^{d_a \times 1}$. Works even when $d_s \neq d_h$!
Luong General (Multiplicative, 2015)	$s_t^T W_a h_i$	$W_a \in \mathbb{R}^{d_s \times d_h}$ (d_s rows, d_h cols). Fast bilinear matrix product; handles $d_s \neq d_h$.
Luong Dot-Product (2015)	$s_t^T h_i$	**Zero** learnable attention parameters! Strictly requires $d_s = d_h$.
Scaled Dot-Product (Vaswani 2017)	$\frac{s_t^T h_i}{\sqrt{d_k}}$	Divides by $\sqrt{d_k}$ so that if s_t, h_i have unit variance, $\text{Var}(s_t^T h_i) = d_k$ is rescaled back to 1, preventing Softmax saturation.

2. **Beam Search Length-Normalized Scoring Objective (`NLP14`):**

$$\text{Score}_{\text{norm}}(y_{1:L}) = \frac{1}{L^\alpha} \sum_{t=1}^{L} \ln P(y_t \mid y_{<t}, X)$$

   - B: Beam width (number of active partial hypotheses kept at each step; $B = 1$ reduces to **Greedy Decoding**).
   - $|V|$: Vocabulary size; at each step $t \in \{1, \ldots, T\}$, Beam Search expands $B \times |V|$ candidates and keeps the top B, taking $O(T \cdot B \cdot |V|)$ time.
   - L: Length (number of tokens) of candidate hypothesis $y_{1:L}$.

- $\alpha \in [0, 1]$: Length normalization exponent ($\alpha = 0$ is unnormalized; $\alpha = 1.0$ is exact per-token average log-probability; Google NMT uses $\frac{(5+L)^\alpha}{(5+1)^\alpha}$ with $\alpha \in [0.6, 0.7]$).

3. **BLEU-N (Bilingual Evaluation Understudy) Exact Formula (`NLP16`):**

$$\text{BLEU-}N = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \ln p_n\right) = \text{BP} \cdot \prod_{n=1}^{N} p_n^{w_n}, \qquad \text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{if } c \leq r \end{cases}$$

$$p_n = \frac{\sum_{g \in \text{Unique } n\text{-grams in } C} \min\left(\text{count}_C(g),\ \max_{R \in \text{Refs}} \text{count}_R(g)\right)}{\text{Total number of } n\text{-grams in Candidate } C}$$

- $c = |C|$: Token length of the Candidate translation.
- $r = |R|$: Effective Reference translation length.
- p_n: Clipped n-gram **Precision** (denominator is the **Candidate** n-gram count $c - n + 1$).
- w_n: Positive weights summing to 1 (for standard **BLEU-4**, $N = 4$ and $w_1 = w_2 = w_3 = w_4 = \frac{1}{4} = 0.25$).

4. **ROUGE-N & ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation, `NLP17`):**

$$\text{ROUGE-}N = \frac{\sum_{g \in R} \min(\text{count}_C(g), \text{count}_R(g))}{\text{Total number of } n\text{-grams in Reference } R}$$

$$R_{\text{LCS}} = \frac{\text{LCS}(R, C)}{|R|}, \qquad P_{\text{LCS}} = \frac{\text{LCS}(R, C)}{|C|}, \qquad F_{\text{LCS}} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}$$

- $\text{LCS}(R, C)$: Length of the **Longest Common Subsequence** between Reference R and Candidate C (tokens must appear in the same relative order, **not necessarily contiguous**!).
- For balanced F1 score ($\beta = 1$), $F_{\text{LCS}} = \frac{2 R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + P_{\text{LCS}}}$.

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): ChrF++ & Self-BLEU Diversity Metrics**

- **ChrF / ChrF++ (Popović, 2015/2017):** Computes character n-gram (typically $n \in [1, 6]$) Precision and Recall F-score (with word n-grams added in ChrF++). Because it operates at the character level, ChrF++ rewards morphologically related words (e.g., `"translating"` vs. `"translated"`) and is completely immune to differences in subword tokenization.
- **Self-BLEU (Zhu et al., 2018):** How do you measure whether a generative LLM is suffering from **mode collapse** (repeating the same generic phrases over and over)? Generate M sample outputs, treat each output s_i as a candidate, and compute its BLEU score against the remaining $M - 1$ generated outputs as references! **Lower Self-BLEU = higher generation diversity**; high Self-BLEU indicates repetitive mode collapse.

🔢 Step-by-Step Worked Toy Example

Example 1: Attention Step Walkthrough (Scores → Softmax → Context Vector) Suppose our Encoder has processed a 3-word input ($T_x = 3$) with 2D hidden states:

$$h_1 = \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix}, \quad h_2 = \begin{bmatrix} 0.0 \\ 2.0 \end{bmatrix}, \quad h_3 = \begin{bmatrix} 3.0 \\ 1.0 \end{bmatrix}$$

At decoder step $t = 1$, suppose the alignment scores between s_1 and h_1, h_2, h_3 are $e_1 = [\ln 2,\ \ln 1,\ \ln 1] = [0.693,\ 0.0,\ 0.0]$.

1. **Compute Softmax Attention Weights $\alpha_{1,i}$:**

$$\exp(e_1) = [e^{\ln 2},\ e^0,\ e^0] = [2,\ 1,\ 1], \qquad \sum_{j=1}^{3} \exp(e_{1,j}) = 2 + 1 + 1 = 4$$

$$\alpha_1 = \left[\frac{2}{4},\ \frac{1}{4},\ \frac{1}{4}\right] = [\mathbf{0.50},\ \mathbf{0.25},\ \mathbf{0.25}]$$

2. **Compute Context Vector $c_1 = \sum_{i=1}^{3} \alpha_{1,i} h_i$:**

$$c_1 = 0.50 \begin{bmatrix}1.0\\0.0\end{bmatrix} + 0.25 \begin{bmatrix}0.0\\2.0\end{bmatrix} + 0.25 \begin{bmatrix}3.0\\1.0\end{bmatrix} = \begin{bmatrix}0.50 + 0 + 0.75\\0 + 0.50 + 0.25\end{bmatrix} = \begin{bmatrix}1.25\\0.75\end{bmatrix}$$

Example 2: Beam Search Length Normalization Flipping the Winner (`NLP14`) Suppose Beam Search ($B = 2$) is comparing two completed candidate sentences:

- **Candidate A (short, $L_A = 2$ tokens):** $\ln P(y_1) = -1.2$, $\ln P(y_2 \mid y_1) = -1.2 \implies$ Unnormalized sum $= \mathbf{-2.4}$.
- **Candidate B (complete, $L_B = 6$ tokens):** Each of the 6 tokens has $\ln P(y_t \mid y_{<t}) = -0.8 \implies$ Unnormalized sum $= 6 \times (-0.8) = \mathbf{-4.8}$.
  - **Without Length Normalization ($\alpha = 0$):** Candidate A wins because $-2.4 > -4.8$, even though Candidate B had much higher confidence per word (-0.8 vs. -1.2)!
  - **With Length Normalization ($\alpha = 1.0$):**

$$\text{Score}_{\text{norm}}(A) = \frac{-2.4}{2^1} = -1.20 < \text{Score}_{\text{norm}}(B) = \frac{-4.8}{6^1} = \mathbf{-0.80} \quad \text{(Candidate B rightly wins!)}$$

Example 3: BLEU Clipped Precision & Brevity Penalty Calculations (`NLP16`)

- **Case A (Clipped Unigram Precision & BP):**
  - Candidate C: `"the cat the cat"` (length $c = 4$)
  - Reference R: `"the cat is on the mat"` (length $r = 6$)
  - **Step 1 (Clipped Counts):**
    - Word `"the"`: appears $\text{count}_C = 2$ times in Candidate, $\text{count}_R = 2$ times in Reference $\implies \min(2, 2) = \mathbf{2}$.
    - Word `"cat"`: appears $\text{count}_C = 2$ times in Candidate, $\text{count}_R = 1$ time in Reference $\implies \min(2, 1) = \mathbf{1}$.
  - **Step 2 (Clipped Unigram Precision p_1):** Divide by **Candidate length** $c = 4$:

$$p_1 = \frac{2 + 1}{4} = \frac{3}{4} = \mathbf{0.75}$$

  - **Step 3 (Brevity Penalty BP):** Since $c = 4 \leq r = 6$:

$$\text{BP} = \exp\left(1 - \frac{r}{c}\right) = \exp\left(1 - \frac{6}{4}\right) = \exp(1 - 1.5) = \mathbf{e^{-0.5}} \approx \mathbf{0.6065}$$

- **Case B (Half-Length Candidate BP):** What if a candidate has length $c = 6$ and the reference has length $r = 12$?

  $$\text{BP} = \exp\left(1 - \frac{12}{6}\right) = \exp(1 - 2) = \mathbf{e^{-1}} \approx \mathbf{0.3679}$$

  What if the candidate has length $c = 15$ and the reference has length $r = 12$ ($c > r$)? Then BP $= 1.0$ (never > 1!).

Example 4: ROUGE-L Longest Common Subsequence Calculation (`NLP17`)

- Reference R ($|R| = 4$): `"police killed the gunman"`
- Candidate C ($|C| = 6$): `"police ended up killing the gunman"`
- **Step 1 (Find LCS):** The words `"police"`, `"the"`, `"gunman"` appear in both R and C in that exact left-to-right order (even though `"ended up killing"` sits between `"police"` and `"the"`!). Thus $\text{LCS}(R, C) =$ **3**.
- **Step 2 (Compute Recall, Precision, and F1):**

  $$R_{\text{LCS}} = \frac{3}{|R|} = \frac{3}{4} = \mathbf{0.75}, \qquad P_{\text{LCS}} = \frac{3}{|C|} = \frac{3}{6} = \mathbf{0.50}, \qquad F_1 = \frac{2 \times 0.75 \times 0.50}{0.75 + 0.50} = \frac{0.75}{1.25} = \mathbf{0.60}$$

🎯 Exam Takeaway & Common Traps

- **BLEU vs. ROUGE Denominator Trap (`NLP16`, `NLP17`): BLEU is Precision-based** (divides matched n-grams by **Candidate length** c, plus a Brevity Penalty for recall), whereas **ROUGE-N is Recall-based** (divides matched n-grams by **Reference length** r).
- **BLEU-4 Zero-Count Geometric Mean Trap (`NLP16`):** Because standard BLEU-4 takes the geometric mean $\text{BP} \cdot (p_1 p_2 p_3 p_4)^{1/4}$, if **any single n-gram order has 0 matches** (for instance, $p_4 = 0$ on a 3-word translation or when no 4-gram matches exactly), the entire unsmoothed BLEU-4 **score equals** 0! (In practice, sentence-level BLEU uses **Chen & Cherry smoothing** to avoid zeroing out).
- **ROUGE-L Contiguity Trap (`NLP17`):** The Longest Common Subsequence (LCS) in ROUGE-L only requires matched words to appear in the **same relative order**, **NOT** adjacently/contiguously!
- **Teacher Forcing vs. Exposure Bias (`NLP12`):** Teacher Forcing is the *training technique* (feeding ground-truth y^*_{t-1}); **Exposure Bias** is the resulting *train-test discrepancy* at inference time when the model must condition on its own imperfect predictions $\hat{y}_{<t}$.

Rapid-Fire 60-Second Exam Checklist (All 20 `NLP01` – `NLP20` Targets)

↑ Mục Lục

Target ID	Concept Tested	10-Second Formula / Rule to Remember
`NLP01`	BPE vs. WordPiece Merge & Inference	BPE merges max raw $\text{count}(ab)$ & applies rules in **training order**; WordPiece merges max PMI $\frac{\text{count}(ab)}{\text{count}(a)\text{count}(b)}$ & runs greedy `MaxMatch`.

Target ID	Concept Tested	10-Second Formula / Rule to Remember
`NLP02`	Exact BPE Vocabulary Size	$\lvert V\rvert_{\text{BPE}} = \lvert\text{Base Alphabet}\rvert + N_{\text{merges}} + \lvert\text{Special Tokens}\rvert$ (256 base bytes in Byte-Level BPE $\implies$ 0 `<UNK>`).
`NLP03`	Add-k / Laplace Smoothing	$P_{\text{Add-}k}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)+k}{\text{count}(w_{t-1})+k\lvert\mathbf{V}\rvert}$ (denominator adds $k\lvert V\rvert$, never k).
`NLP04`	Perplexity (`PPL`) & Log Bases	$\text{PPL} = e^{\mathcal{L}_{\text{CE, nats}}} = 2^{\mathcal{L}_{\text{CE, bits}}}$; uniform random model gives PPL $= \lvert V\rvert$; not comparable across different $\lvert V\rvert$.
`NLP05`	TF-IDF & BM25 Hyperparameters	Stopword in all N docs $\implies$ IDF $= \log(1) = 0$. In BM25, k_1 caps TF saturation at $k_1 + 1$; $b \in [0, 1]$ controls length normalization ($b = 0$ disables it).
`NLP06`	Word2Vec Parameter Count	Maintains both $W_{\text{in}} \in \mathbb{R}^{\lvert V\rvert \times d}$ and $W_{\text{out}} \in \mathbb{R}^{\lvert V\rvert \times d}$ $\implies$ $\mathbf{2\lvert V\rvert d}$ total parameters; Skip-Gram excels on **rare words**.
`NLP07`	Softmax Approximations	Full Softmax $= O(\lvert V\rvert)$; Hierarchical Softmax (Huffman tree) $= O(\log_2 \lvert V\rvert)$; Negative Sampling (SGNS) $= O(K)$ binary sigmoids.
`NLP08`	Unit-Vector Cosine vs. L_2 Distance	When $\lVert u\rVert_2 = \lVert v\rVert_2 = 1$, $\lVert u - v\rVert_2 = \sqrt{2(1 - \cos\theta)}$ (min L_2 ranking $\equiv$ max Cosine ranking).
`NLP09`	RNN / GRU / LSTM / BiLSTM Params	Let $U = h(d + h + 1)$: **RNN** $= 1U$, **GRU** $= 3U$, **LSTM** $= 4U$, **1-Layer BiLSTM** $= 8U$ (Layer 2 of BiLSTM takes $d_2 = 2h$).
`NLP10`	LSTM Gate Equations & Highway	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (additive cell state highway prevents vanishing gradients; initialize $b_f = +1.0$).
`NLP11`	Vanishing vs. Exploding Gradients	$\sigma_{\max}(W_{hh}) > 1 \implies$ Exploding (fix: **Gradient Norm Clipping**); $\sigma_{\max} < 1 \implies$ Vanishing (fix: **LSTM/GRU highway** or **Transformer**).
`NLP12`	Teacher Forcing & Exposure Bias	Teacher Forcing feeds ground-truth y^*_{t-1} at training; causes **Exposure Bias** at test time when conditioning on self-generated $\hat{y}_{<t}$.
`NLP13`	Bahdanau vs. Luong Attention	Bahdanau (Additive): $v_a^T \tanh(W_1 s_t + W_2 h_i)$; Luong (Multiplicative): $s_t^T W_a h_i$ or $s_t^T h_i$; Vaswani scales by $1/\sqrt{d_k}$.
`NLP14`	Beam Search Length Penalty	Complexity $O(T \cdot B \cdot \lvert V\rvert)$; unnormalized sum $\sum \ln P < 0$ penalizes long sequences $\implies$ divide by L^α.
`NLP15`	Linear-Chain CRF & Viterbi	Adds $K \times K$ tag transition matrix A to prevent illegal BIO sequences (`O -> I-PER`); Forward & Viterbi run in $O(T \cdot K^2)$ time.
`NLP16`	BLEU-4 & Brevity Penalty (`BP`)	Precision-focused (divides by Candidate length c); BP $= \exp(1 - r/c)$ if $c \le r$ else 1; if any $p_n = 0$, unsmoothed BLEU-4 $= 0$.
`NLP17`	ROUGE-N & ROUGE-L	Recall-focused (divides by Reference length r); ROUGE-L uses in-order (non-contiguous) Longest Common Subsequence LCS(R, C).
`NLP18`	Levenshtein Edit Distance	$O(M \cdot N)$ Wagner-Fischer DP over Insertions, Deletions, Substitutions (`'INTENTION'` $\to$ `'EXECUTION'` $= 5$ edits for unit cost, 8 if sub $= 2$).
`NLP19`	BERT MLM 80-10-10 Rule	15% of tokens selected for loss: 80% `[MASK]`, 10% **random token**, 10% **unchanged** (mitigates pretrain-finetune discrepancy).
`NLP20`	CTC Loss Collapse Rules	**(1) Merge consecutive duplicates first, (2) drop all blanks** `'-'` (a true double letter like `'oo'` in `'too'` requires `'o-o'`).

Pillar 5: Transformers, LLMs & Efficient On-Device AI — Intuitive Study Textbook

↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **How to Read This Textbook:** If Transformers, Large Language Models (LLMs), and Efficient Edge AI aren't your daily bread and butter, dense research papers can feel like a wall of acronyms (`GQA`, `RoPE`, `SwiGLU`, `KV-Cache`, `FlashAttention`, `LoRA`, `DPO`, `W4A16`).
>
> For the **Qualcomm AI Research (QARE) Exam**, this is the flagship pillar. Every single subsection in this textbook is structured in **four clear steps**:
>
> 1. 💡 **Intuition (Plain English First):** What real-world bottleneck does this solve, and why did engineers invent it?
> 2. 📐 **The Math & Every Symbol Explained:** Every matrix shape is written using our standard $n \times m$ **(n rows, m columns)** convention, and every symbol is defined in plain English.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete numbers worked out line by line so you can trace every matrix multiplication and memory byte by hand.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and exact traps tested in HackerRank questions `LLM01`–`LLM20`.

---

1. The Transformer Architecture from First Principles

↑ Mục Lục

1.1 Why Self-Attention Replaced Recurrence

💡 Intuition (Plain English First)

Before 2017, sequence models used Recurrent Neural Networks (RNNs and LSTMs). Imagine reading a 1,000-word book where you are forced to whisper a summary of everything you've read so far into a single sticky note (h_t) and pass it word-by-word from left to right:

1. **You can't read in parallel:** You cannot compute the sticky note for word 500 until you have finished computing words $1, 2, \ldots, 499$. Modern GPUs have thousands of cores sitting idle waiting for the previous word!
2. **You forget the beginning:** By the time the sticky note reaches word 500, details from word 1 have been overwritten or squashed by 499 matrix multiplications (vanishing gradients).

Self-Attention throws away the sequential sticky note. Instead, every token in the sentence looks directly at **every other token in the sentence at the exact same time**—like a room full of people where anyone can make direct eye contact with anyone else in 1 step!

📐 The Math & Every Symbol Explained

For a sequence of N tokens (where each token is represented by an embedding vector of size D), our input matrix X has $n = N$ rows and $m = D$ columns ($X \in \mathbb{R}^{N \times D}$):

Architecture	Sequential Steps (Training Parallelism)	Max Path Length Between Token 1 and Token N	Compute per Layer (N tokens, dim D)
RNN / LSTM	$\mathcal{O}(N)$ sequential steps (cannot parallelize across time)	$\mathcal{O}(N)$ steps (gradients vanish over long distance)	$\mathcal{O}(ND^2)$
Self-Attention	$\mathcal{O}(1)$ **steps** (all N tokens processed in parallel!)	$\mathcal{O}(1)$ **step** (direct token-to-token connection)	$\mathcal{O}(N^2D + ND^2)$

- N **(or S):** Sequence length (number of tokens in the prompt/sentence, i.e., number of rows in X).
- D **(or d_{model}):** Hidden dimension of the model (number of columns in X, e.g., 4096 in LLaMA-3-8B).

1.2 Scaled Dot-Product Attention & The $\sqrt{d_k}$ Variance Proof (`LLM01`)

💡 Intuition (Plain English First)

How does Self-Attention decide which words should pay attention to which other words? It uses a **database lookup analogy** with three vectors created from every token:

- **Query (q):** "*What am I looking for?*" (e.g., the pronoun `"it"` looking for the noun it refers to).
- **Key (k):** "*What label describes me?*" (e.g., the noun `"trophy"` advertising that it is an inanimate object).
- **Value (v):** "*What actual information do I contain if you pay attention to me?*"

To measure how well token i's Query matches token j's Key, we take their dot product $q_i \cdot k_j$. **But why do we divide that dot product by $\sqrt{d_k}$?** Because if two vectors have $d_k = 64$ or 128 random numbers in them, adding up 128 products creates a sum with a huge spread (large positive and negative swings like $+25$ or -20). When you feed numbers like $+25$ into softmax(z), e^{25} completely crushes everything else: one probability becomes 0.99999999 and all other probabilities become 0.00000000. In that flat region of Softmax, the gradient is zero (**vanishing gradients**), and the model stops learning! Dividing by $\sqrt{d_k}$ shrinks the spread back down to a healthy standard deviation of 1.

📐 The Math & Every Symbol Explained

Given an input sequence matrix $X \in \mathbb{R}^{N \times D}$ (N rows of tokens, D columns of features), we multiply by three learned weight matrices:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- $X \in \mathbb{R}^{N \times D}$**:** Input token matrix (N token rows, D feature columns).
- $W_Q, W_K \in \mathbb{R}^{D \times d_k}$ **and** $W_V \in \mathbb{R}^{D \times d_v}$**:** Learned projection matrices mapping D input columns to d_k (query/key dimension) or d_v (value dimension) columns.
- $Q, K \in \mathbb{R}^{N \times d_k}$ **and** $V \in \mathbb{R}^{N \times d_v}$**:** The resulting Query, Key, and Value matrices (N rows, d_k or d_v columns).

- $QK^T \in \mathbb{R}^{N \times N}$: The pairwise similarity matrix (N query rows by N key columns), where entry (i, j) is the dot product $q_i \cdot k_j$.
- $\sqrt{d_k}$: Scaling factor (square root of the key dimension d_k).
- $M \in \mathbb{R}^{N \times N}$: Optional attention mask (0 for allowed positions, $-\infty$ for blocked future positions).

> 📌 **NOTE**
>
> **Step-by-Step Mathematical Proof of the $\sqrt{d_k}$ Scaling Factor (and the $\sigma^2 \neq 1$ Exam Twist):** Suppose the components q_i and k_i ($i = 1, \ldots, d_k$) are independent random variables with mean $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$ and variance $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = \sigma^2$:
>
> 1. **Mean of a single product $q_i k_i$:**
>
> $$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \cdot \mathbb{E}[k_i] = 0 \times 0 = 0$$
>
> 2. **Variance of a single product $q_i k_i$:**
>
> $$\mathrm{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2] \cdot \mathbb{E}[k_i^2] - 0 = \sigma^2 \cdot \sigma^2 = \sigma^4$$
>
> 3. **Variance and Standard Deviation of the sum $s = q \cdot k = \sum_{i=1}^{d_k} q_i k_i$:** By independence, the variance of the sum of d_k terms is the sum of their variances:
>
> $$\mathrm{Var}(s) = \sum_{i=1}^{d_k} \sigma^4 = d_k \sigma^4 \implies \mathrm{Std}(s) = \sqrt{d_k \sigma^4} = \sigma^2 \sqrt{d_k}$$
>
> 4. **How to scale s to restore unit variance ($\mathrm{Var} = 1$):**
>
> - **Standard Case ($\sigma^2 = 1$):** $\mathrm{Var}(s) = d_k$, so dividing by $\sqrt{d_k}$ multiplies the variance by $\left(\frac{1}{\sqrt{d_k}}\right)^2 = \frac{1}{d_k}$, giving $\mathrm{Var}\left(\frac{s}{\sqrt{d_k}}\right) = 1$.
> - **Generalized Exam Twist (`LLM01`, $\sigma^2 \neq 1$):** If $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = \sigma^2$, you must divide by $\sigma^2 \sqrt{d_k}$ so that $\mathrm{Var}\left(\frac{s}{\sigma^2 \sqrt{d_k}}\right) = \frac{d_k \sigma^4}{\sigma^4 d_k} = 1$!

🔢 Step-by-Step Worked Toy Example

Suppose a tiny head has dimension $d_k = 64$, and the elements of q and k are drawn independently with mean 0 and variance $\sigma^2 = 2$:

1. **Step 1 — Variance of one element product $q_i k_i$:**

$$\mathrm{Var}(q_i k_i) = \sigma^2 \times \sigma^2 = 2 \times 2 = 4$$

2. **Step 2 — Variance of the 64-dimensional dot product $s = \sum_{i=1}^{64} q_i k_i$:**

$$\mathrm{Var}(s) = 64 \times 4 = 256$$

3. **Step 3 — Find the exact divisor c so that $\mathrm{Var}(s/c) = 1$:**

$$\frac{\mathrm{Var}(s)}{c^2} = 1 \implies c = \sqrt{256} = 16 = \sigma^2 \sqrt{d_k} = 2\sqrt{64}$$

(If you only divided by $\sqrt{d_k} = 8$, the variance would still be $256/64 = 4$, not 1!)

🎯 Exam Takeaway & Common Traps

- **Trap in `LLM01`:** When an exam question specifies $\text{Var}(q_i) = \text{Var}(k_i) = \sigma^2 = 2$, candidates who memorized $\sqrt{d_k}$ pick $\sqrt{2d_k}$ by mistake. Remember: two independent factors of variance σ^2 multiply to σ^4, whose square root is $\sigma^2\sqrt{d_k} = 2\sqrt{d_k}$!

1.3 Multi-Head Attention (MHA): Shapes, Parameters & The $N = 2D$ Crossover (`LLM02`, `LLM03`)

💡 Intuition (Plain English First)

If a sentence says "*The animal didn't cross the street because it was too tired*", one word needs to track **multiple types of relationships simultaneously**:

- Head 1 might track **pronoun coreference** (`"it"` → `"animal"`).
- Head 2 might track **verb-object grammar** (`"cross"` → `"street"`).
- Head 3 might track **causal conjunctions** (`"because"` → `"tired"`).

Instead of running one giant attention lookup of width $D = 4096$, **Multi-Head Attention (MHA)** chops the D-dimensional vector into h smaller slices (heads), each of width $d_k = D/h$ (e.g., $h = 32$ heads of width $d_k = 128$). Each head computes its own independent attention map, and then we glue (concatenate) the h slices back together into a vector of width D!

📐 The Math & Every Symbol Explained

$$\text{head}_i = \text{Attention}\left(XW_Q^{(i)},\ XW_K^{(i)},\ XW_V^{(i)}\right), \qquad \text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

Let's trace the exact **PyTorch tensor shapes** through an MHA layer with Batch size B, Sequence length N rows, Model dimension D columns, h heads, and per-head dimension $d_k = D/h$:

1. **Input tensor X:** Shape (B, N, D) — B batches of $N \times D$ matrices.
2. **Combined Linear Projections (Q, K, V):** Multiply (B, N, D) by $W_Q, W_K, W_V \in \mathbb{R}^{D\times D}$ → shape (B, N, D).
3. **Split into h heads & transpose:** Reshape $(B, N, D) \to (B, N, h, d_k)$ and swap axes $1 \leftrightarrow 2$ → shape (B, h, N, d_k) (so each head is an $N \times d_k$ matrix).
4. **Attention Scores ($QK^T/\sqrt{d_k}$):** Multiply (B, h, N, d_k) by transposed keys (B, h, d_k, N) → shape (B, h, N, N).
5. **Weighted Values (Scores $\cdot\ V$):** Multiply (B, h, N, N) by (B, h, N, d_k) → shape (B, h, N, d_k).
6. **Concatenate heads & Output Projection W_O:** Swap axes back $(B, N, h, d_k) \to (B, N, D)$ and multiply by $W_O \in \mathbb{R}^{D\times D}$ → final shape (B, N, D).

- **Total Parameter Count of MHA (ignoring biases):**
  - All h heads of $W_Q^{(i)} \in \mathbb{R}^{D\times d_k}$ stacked side-by-side form a single $D \times D$ matrix $W_Q \in \mathbb{R}^{D\times D}$ (D^2 parameters).
  - Same for $W_K \in \mathbb{R}^{D\times D}$, $W_V \in \mathbb{R}^{D\times D}$, and $W_O \in \mathbb{R}^{D\times D}$.
  - **Total MHA Parameters: $4\mathbf{D^2}$** (or $4D^2 + 4D$ if biases are enabled).

- **Total Compute (MACs and FLOPs) of MHA for 1 Sequence of N Tokens:** Recall that multiplying an $n \times m$ matrix by an $m \times p$ matrix costs $n \cdot m \cdot p$ Multiply-Accumulates (MACs), which equals $2 \cdot n \cdot m \cdot p$ FLOPs:
  - **4 Linear Projections (Q, K, V, O):** Each multiplies $(N \times D)$ by $(D \times D) \implies 4 \times ND^2 = \mathbf{4ND^2}$ MACs ($8ND^2$ FLOPs).
  - **2 Attention Matrix Multiplications (QK^T and PV across h heads):**
    - QK^T: h heads $\times (N \times d_k) \times (d_k \times N) = h \cdot N^2 d_k = N^2 D$ MACs ($2N^2 D$ FLOPs).
    - PV: h heads $\times (N \times N) \times (N \times d_k) = h \cdot N^2 d_k = N^2 D$ MACs ($2N^2 D$ FLOPs).
    - **Total Quadratic Attention Ops: $\mathbf{2N^2D}$** MACs ($4N^2 D$ FLOPs).

🔢 Step-by-Step Worked Toy Example

Let's solve two classic exam questions with $D = 512$:

1. **Question A (`LLM02`): Suppose we double the number of heads from $h = 8$ ($d_k = 64$) to $h = 16$ ($d_k = 32$) while keeping $D = 512$ fixed. How do the parameter count and attention matrix memory change?**
   - **Parameter count at $h = 8$:** $4D^2 = 4 \times 512^2 = 1,048,576$ parameters.
   - **Parameter count at $h = 16$:** Because $h \times d_k = 16 \times 32 = 512 = D$, the combined matrices W_Q, W_K, W_V, W_O are still 512×512! The parameter count changes by 0!
   - **Attention score matrix memory (B, h, N, N):** Doubles by $2\times$ because there are now 16 attention maps of size $N \times N$ instead of 8.
2. **Question B (`LLM03`): At what exact sequence length N do the quadratic attention matrix multiplications (QK^T and PV) equal the FLOPs of the 4 linear projections (W_Q, W_K, W_V, W_O)?**
   - Set Quadratic Attention FLOPs equal to Linear Projection FLOPs:

     $$4N^2 D = 8ND^2$$

   - Divide both sides by $4ND$:

     $$N = 2D$$

   - For $D = 512$, the crossover is at $N = 2 \times 512 = 1024$ **tokens** (and for LLaMA-3-8B with $D = 4096$, the crossover is at $N = 8192$ **tokens**!).

🎯 Exam Takeaway & Common Traps

- **The h-Invariance Rule (`LLM02`):** Changing the number of heads h while keeping $D = h \cdot d_k$ constant **never changes the parameter count ($4D^2$)** or the total FLOPs—it only changes the (B, h, N, N) activation memory!
- **The $N = 2D$ Crossover (`LLM03`):** For short prompts ($N < 2D$), linear projections ($8ND^2$ FLOPs) dominate MHA compute. Only when context length exceeds $N > 2D$ does quadratic attention ($4N^2 D$ FLOPs) become the dominant bottleneck!

1.4 Causal Masking, Pre-LN vs Post-LN & Encoder vs Decoder Paradigms (`LLM04`)

💡 Intuition (Plain English First)

- **Causal Masking:** When training an autoregressive LLM (like GPT or LLaMA) to predict the next word, all N words of the training sentence are fed in at once for parallel speed. If word 3 is allowed to peek at word 4 during self-attention, the model will just copy word 4 with 100% accuracy and learn nothing! To prevent cheating, we put a blindfold (**Causal Mask**) over all future positions $j > i$.
  - *Why must the blindfold be $-\infty$ instead of 0?* Because the attention scores go into softmax$(s_j) = \frac{e^{s_j}}{\sum_k e^{s_k}}$. If you set a future score to 0, then $e^0 = 1$, which gets a positive probability! You must set blocked positions to $-\infty$ before Softmax so that $e^{-\infty} = 0$.
- **Pre-LN vs Post-LN:** In the original 2017 Transformer (**Post-LN**), LayerNorm sat *on the main highway* after the residual addition: $x_{l+1} = \text{LN}(x_l + F(x_l))$. During backpropagation through $L = 32$ layers, gradients get divided by the LayerNorm variance at every single floor, causing gradients at the bottom layers to vanish while gradients at the top layers explode. Modern LLMs use **Pre-LN**: $x_{l+1} = x_l + F(\text{LN}(x_l))$. Now the main residual highway $x_L = x_0 + \sum_{l=1}^{L} F_l(\text{LN}(x_{l-1}))$ is a clean, unobstructed identity wire from the loss all the way back to layer 0!

📐 The Math & Every Symbol Explained

1. **Causal Attention Mask Matrix $M \in \mathbb{R}^{N \times N}$ (N query rows, N key columns):**

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \quad \text{(past and present tokens allowed)} \\ -\infty & \text{if } j > i \quad \text{(future tokens blocked: } \exp(-\infty) = 0\text{)} \end{cases}$$

2. **Post-LN vs Pre-LN Residual Equations:**
   - **Post-LN (2017 Vaswani Transformer):** $x_{l+1} = \text{LN}\big(x_l + \text{Sublayer}(x_l)\big)$
   - **Pre-LN (GPT, LLaMA, Qwen, ViT):** $x_{l+1} = x_l + \text{Sublayer}\big(\text{LN}(x_l)\big)$
3. **Transformer Paradigm Taxonomy:**

Paradigm	Attention Mask	Pretraining Objective	Representative Models	Best Use Cases
Encoder-Only	Bidirectional (No mask; all $N \times N$ entries active)	**Masked Language Modeling (MLM):** mask 15% of tokens (80% `[MASK]` , 10% random, 10% unchanged)	BERT, RoBERTa, DeBERTa	Dense embeddings, text classification, NER, Cross-Encoder reranking
Decoder-Only	Causal lower-triangular ($M_{ij} = -\infty$ for $j > i$)	**Next-Token Prediction (Causal LM):** $\sum_t \log P(x_t \mid x_{<t})$	GPT-4, LLaMA-3, Qwen, Mistral	Text generation, reasoning, chat, code
Encoder-Decoder	Bidirectional Encoder + Cross-Attention + Causal Decoder	Span corruption / Seq2Seq denoising	T5, BART, Whisper	Translation, speech-to-text (Whisper)

🔢 Step-by-Step Worked Toy Example

Suppose we have a 3-token sequence (`"I"`, `"love"`, `"AI"`) and we are computing the attention probabilities for **Row** $i = 1$ (the first token `"I"`, which is only allowed to look at itself, $j = 1$, and must NOT look at $j = 2$ or $j = 3$). Suppose the raw scaled dot products $\frac{q_1 \cdot k_j}{\sqrt{d_k}}$ for row 1 are $[2.0,\ 5.0,\ 5.0]$:

- **Buggy Implementation (setting future scores to 0 before Softmax):**
  - Masked scores: $[2.0,\ 0.0,\ 0.0]$.
  - Softmax: $\left[\frac{e^2}{e^2+e^0+e^0}, \frac{e^0}{e^2+2}, \frac{e^0}{e^2+2}\right] = \left[\frac{7.39}{9.39}, \frac{1}{9.39}, \frac{1}{9.39}\right] = [0.787,\ \mathbf{0.106},\ \mathbf{0.106}]$.
  - **Disaster:** Over 21% of token 1's output is leaked from future tokens 2 and 3!
- **Correct Implementation (adding $M_{1,2} = M_{1,3} = -\infty$ before Softmax):**
  - Masked scores: $[2.0,\ -\infty,\ -\infty]$.
  - Softmax: $\left[\frac{e^2}{e^2+0+0}, \frac{0}{e^2}, \frac{0}{e^2}\right] = [\mathbf{1.0},\ \mathbf{0.0},\ \mathbf{0.0}]$. Zero future leakage!

🎯 Exam Takeaway & Common Traps

- **Causal Masking (`LLM04`):** Always add $-\infty$ (or `-1e9` in FP16) **before** Softmax, never 0!
- **Why BERT MLM uses the 80-10-10 rule:** During fine-tuning/inference, the special `[MASK]` token never appears. Keeping 10% random tokens and 10% original tokens forces BERT to maintain high-quality contextual representations for *every* token, not just `[MASK]` tokens.

---

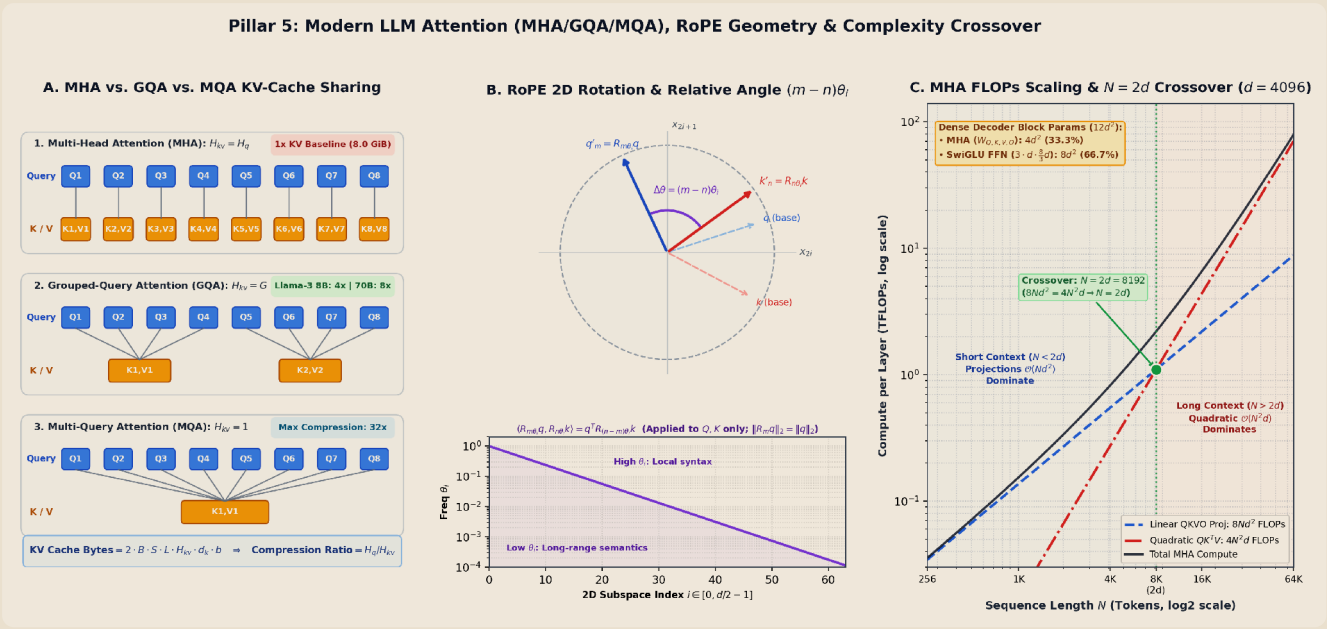
2. Modern LLM Architectural Upgrades (LLaMA / Qwen / Mistral DNA)

↑ Mục Lục

Modern frontier LLMs (LLaMA-3, Qwen-2.5, Mistral, DeepSeek) replace **five components** of the original 2017 Transformer. Know this comparison table cold:

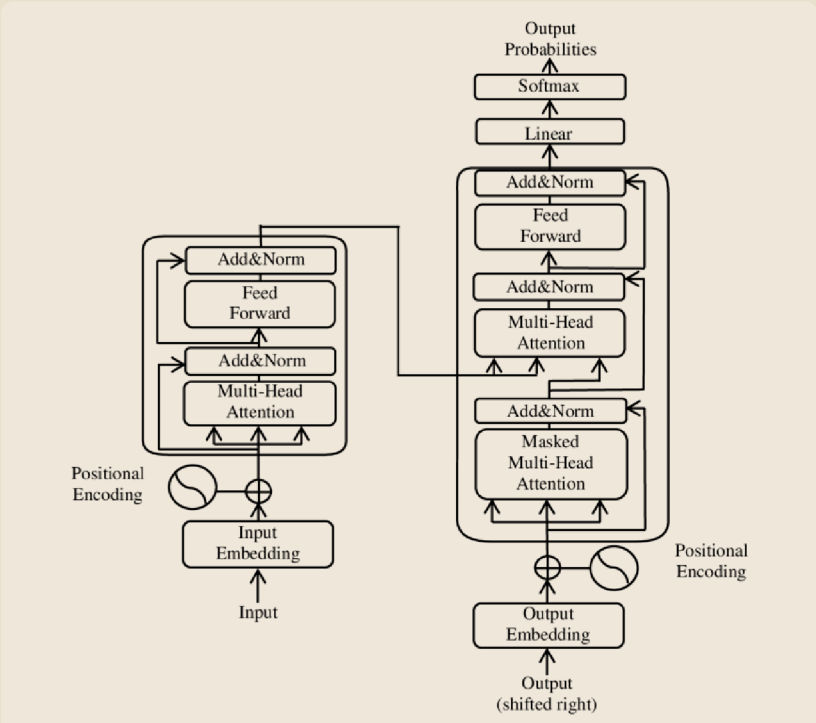
Component	Original 2017 Transformer	Modern LLM (LLaMA-3 / Qwen / Mistral)	Mathematical & Systems Advantage
Normalization	Post-LayerNorm ($2D$ params/layer: γ, β)	**Pre-RMSNorm** (D params/layer: γ only)	Drops mean-centering (μ) and bias (β); saves $\sim$ 7–15% latency with equal accuracy.
Activation / FFN	$\text{ReLU}(xW_1)W_2$ (2 matrices, $D_{\text{ff}} = 4D$)	**SwiGLU** (3 matrices: $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)	Gated linear unit $(\text{Swish}(xW_g) \odot xW_u)W_d$ yields consistently lower perplexity.
Positional Encoding	Sinusoidal / Learned Absolute added to input	**RoPE (Rotary Position Embedding)**	Applied to Q, K at every layer; inner product depends strictly on relative distance $m - n$.
Attention Heads	Multi-Head Attention (MHA: $h_{kv} = h_q$)	**Grouped-Query Attention (GQA: $h_{kv} < h_q$)**	Shares K, V heads across groups of Q heads; slashes KV-Cache memory by h_q/h_{kv} (4–8×).
Linear Biases	`bias=True` in Linear & LayerNorm	`bias=False` everywhere	Reduces parameter count and improves FP16/BF16 numerical stability at scale.

FIGURE SLIDE 1 OF 2



Modern LLM Attention (MHA/GQA/MQA), RoPE 2D Rotation & Complexity Crossover

FIGURE SLIDE 2 OF 2



Canonical 2017 Transformer Architecture (Vaswani et al.)

2.1 RMSNorm vs LayerNorm & SwiGLU FFN Parameter Count Math (`LLM05`)

💡 Intuition (Plain English First)

1. **Why RMSNorm replaced LayerNorm:** Standard LayerNorm does two passes over every token vector $x \in \mathbb{R}^D$: first it computes the mean μ to center the vector $(x - \mu)$, and second it computes the variance σ^2 to scale the vector. Zhang & Sennrich (2019) discovered that **re-centering the mean to zero does nothing for accuracy**—only scaling the vector's magnitude matters! **RMSNorm** skips the mean μ and shift β completely, dividing directly by the Root Mean Square (RMS) of the vector. That cuts memory reads/writes inside the normalization kernel and cuts norm parameters in half (D instead of $2D$).
2. **Why SwiGLU replaced ReLU FFN:** In a standard Feed-Forward Network (FFN), a single matrix W_1 projects x up to $4D$, applies ReLU, and W_2 projects back down to D. In **SwiGLU**, we use **two parallel up-projections**: one acts as a "content stream" (xW_{up}) and the other acts as a smooth, learnable "valve/gate" ($\text{Swish}(xW_{\text{gate}})$) that controls how much of each feature passes through via element-wise multiplication ($\odot$), before W_{down} projects back to D. Because SwiGLU uses **3 matrices instead of 2**, we shrink the intermediate width from $4D$ to $\frac{8}{3}D \approx 2.67D$ so total parameters stay the same ($3 \times D \times \frac{8}{3}D = 8D^2$).

📐 The Math & Every Symbol Explained

- **LayerNorm ($2D$ parameters: scale $\gamma \in \mathbb{R}^D$, shift $\beta \in \mathbb{R}^D$):**

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \qquad \text{where } \mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \quad \sigma^2 = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2$$

- **RMSNorm (D parameters: scale $\gamma \in \mathbb{R}^D$ only, no μ, no β):**

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \qquad \text{where RMS}(x) = \sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2 + \epsilon}$$

- **SwiGLU Feed-Forward Network ($3 \cdot D \cdot D_{\text{ff}}$ parameters with `bias=False`):**

$$\text{SwiGLU}(x) = \Big(\text{Swish}_1(xW_{\text{gate}}) \odot (xW_{\text{up}})\Big)W_{\text{down}}, \qquad \text{where Swish}_1(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}}$$

  - $x \in \mathbb{R}^{N \times D}$: Input token matrix ($N$ rows, D columns).
  - $W_{\text{gate}} \in \mathbb{R}^{D \times D_{\text{ff}}}$: Gate projection matrix (D rows, D_{ff} columns).
  - $W_{\text{up}} \in \mathbb{R}^{D \times D_{\text{ff}}}$: Up-projection matrix ($D$ rows, D_{ff} columns).
  - $W_{\text{down}} \in \mathbb{R}^{D_{\text{ff}} \times D}$: Down-projection matrix (D_{ff} rows, D columns).
  - $\odot$: Element-wise (Hadamard) multiplication of the two $N \times D_{\text{ff}}$ matrices.

🔢 Step-by-Step Worked Toy Example

Let's normalize a tiny 2-element vector $x = [3.0,\ 4.0]$ with $\epsilon = 0$ and $\gamma = [1.0,\ 1.0]$ using **RMSNorm**:

1. **Step 1 — Compute the mean of squares:**

$$\frac{1}{2}(3.0^2 + 4.0^2) = \frac{9 + 16}{2} = \frac{25}{2} = 12.5$$

2. **Step 2 — Take the square root to get $\text{RMS}(x)$:**

$$\text{RMS}(x) = \sqrt{12.5} \approx 3.5355$$

3. **Step 3 — Divide x by $\text{RMS}(x)$:**

$$\text{RMSNorm}(x) = \left[\frac{3.0}{3.5355}, \frac{4.0}{3.5355}\right] \approx [0.8485,\ 1.1314]$$

(Notice that the mean of the output is $\approx 0.99 \neq 0$, which is completely fine—its root-mean-square is now exactly 1.0!)

2.2 MHA vs MQA vs GQA Parameter & KV-Cache Math (`LLM05` , `LLM06`)

💡 Intuition (Plain English First)

During LLM text generation, the biggest bottleneck is **reading past Keys and Values (the KV-Cache) from memory** for every single new word generated.

- In standard **Multi-Head Attention (MHA)**, if you have $h_q = 32$ Query heads, you also store 32 separate Key heads and 32 separate Value heads in memory.
- **Multi-Query Attention (MQA)** went to the extreme: all 32 Query heads share **just 1 single Key head and 1 single Value head** ($h_{kv} = 1$). That slashes KV-Cache memory by $32\times$, but hurts model accuracy slightly.
- **Grouped-Query Attention (GQA)** is the goldilocks sweet spot used in LLaMA-3, Qwen, and Mistral: we divide the $h_q = 32$ Query heads into $g = 8$ groups (so 4 Query heads per group). Each group shares 1 Key head and 1 Value head ($h_{kv} = 8$). We get $4\times$ **smaller KV-Cache** with virtually **zero loss in accuracy**!

📐 The Math & Every Symbol Explained

For model width D, h_q Query heads of width $d_k = D/h_q$, and h_{kv} Key/Value heads of the same width d_k:

- $W_Q \in \mathbb{R}^{D \times (h_q \cdot d_k)} = \mathbb{R}^{D \times D}$: Always has D^2 parameters.
- $W_O \in \mathbb{R}^{(h_q \cdot d_k) \times D} = \mathbb{R}^{D \times D}$: Always has D^2 parameters.
- $W_K, W_V \in \mathbb{R}^{D \times (h_{kv} \cdot d_k)}$: Each has $D \cdot h_{kv} \cdot d_k = D^2 \left(\frac{h_{kv}}{h_q}\right)$ parameters!
- **Total GQA Attention Parameter Formula (`bias=False`):**

$$\text{Params}_{\text{GQA}} = \underbrace{2D^2}_{W_Q, W_O} + \underbrace{2 \cdot D \cdot h_{kv} \cdot d_k}_{W_K, W_V} = 2D^2 \left(1 + \frac{h_{kv}}{h_q}\right)$$

- **KV-Cache Reduction Factor vs MHA:**

$$\text{KV-Cache Savings Ratio} = \frac{h_q}{h_{kv}}$$

🔢 Step-by-Step Worked Toy Example (`LLM05` , `LLM06`)

Let's calculate the **exact parameter count of 1 full LLaMA-style Decoder Layer** (`LLM05`) with:

- Model dimension $D = 4096$
- Query heads $h_q = 32 \implies$ head dimension $d_k = 4096/32 = 128$
- GQA Key/Value heads $h_{kv} = 8$ (so $32/8 = 4\times$ KV-Cache reduction)
- SwiGLU intermediate dimension $D_{\text{ff}} = 11008$

- 2 RMSNorm layers per block (one before Attention, one before FFN), `bias=False` everywhere:

1. **Step 1 — GQA Self-Attention Parameters:**
   - W_Q (4096×4096) and W_O (4096×4096):

     $$2 \times (4096 \times 4096) = 2 \times 16,777,216 = 33,554,432$$

   - W_K $(4096 \times (8 \times 128) = 4096 \times 1024)$ and W_V (4096×1024):

     $$2 \times (4096 \times 1024) = 2 \times 4,194,304 = 8,388,608$$

   - **Subtotal (GQA Attention):** $33,554,432 + 8,388,608 = \mathbf{41,943,040}$
2. **Step 2 — SwiGLU FFN Parameters (3 matrices: $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$):**

   $$3 \times (4096 \times 11008) = 3 \times 45,088,768 = \mathbf{135,266,304}$$

3. **Step 3 — Two Pre-RMSNorm Layers (each has 1 vector $\gamma \in \mathbb{R}^{4096}$):**

   $$2 \times 4096 = \mathbf{8,192}$$

4. **Step 4 — Grand Total for 1 Decoder Layer:**

   $$41,943,040 + 135,266,304 + 8,192 = \mathbf{177,217,536} \text{ parameters } (\approx 177.22\text{M})$$

🎯 Exam Takeaway & Common Traps

- **3 Traps in `LLM05`:**
  1. Don't use $4D^2$ for GQA attention—W_K and W_V are shrunk by h_{kv}/h_q!
  2. Don't use $2 \cdot D \cdot D_{\text{ff}}$ for SwiGLU—SwiGLU has **3 matrices** ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)!
  3. Don't use $2D$ per RMSNorm—RMSNorm has no β shift, so each norm has only D parameters ($2D$ total for the two RMSNorms in a layer)!

2.3 RoPE (Rotary Position Embedding), ALiBi & Context Extension (`LLM07`)

💡 Intuition (Plain English First)

Self-attention is completely permutation-invariant: without position markers, `"The dog bit the man"` and `"The man bit the dog"` look identical to the attention matrix.

- **Old way (Absolute Positional Encoding):** Add a fixed vector p_m to word m at the very bottom of the network. Problem: if the model was trained on positions $1 \ldots 2048$, it has no idea what position 3000 means, and it struggles to know that position 5 and position 7 have the exact same relative gap $(+2)$ as position 1005 and position 1007.
- **Modern way (RoPE — Rotary Position Embedding):** Instead of adding a number to the vector, imagine each pair of numbers (q_{2i}, q_{2i+1}) in a Query or Key vector as the hand of a clock on a 2D dial.
  - If a word is at position m, we rotate its clock hand by $m \cdot \theta$ degrees.
  - If another word is at position n, we rotate its clock hand by $n \cdot \theta$ degrees.

- When we take the dot product between the two rotated clock hands ($q_m \cdot k_n$), the dot product of two 2D vectors depends **only on the angle between them**: $(m-n)\theta$! Thus, attention automatically knows the **exact relative distance** $m-n$ between any two tokens!

📐 The Math & Every Symbol Explained

RoPE groups the d_k coordinates of a Query vector q_m (at token position m) or Key vector k_n (at token position n) into $d_k/2$ two-dimensional pairs $(2i, 2i+1)$, and multiplies each pair by a 2×2 rotation matrix $R(m\theta_i)$:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}}_{R(m\theta_i) \in \mathbb{R}^{2\times 2}} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}, \qquad \text{where } \theta_i = \text{base}^{-2i/d_k}$$

- $m, n \in \{0, 1, \ldots, N-1\}$**:** Token position indices in the sequence.
- $\theta_i = \text{base}^{-2i/d_k}$**:** Rotation frequency for coordinate pair i (fast rotation for small i, slow rotation for large i; base $= 10,000$ in LLaMA-1/2 or $500,000$ in LLaMA-3).
- **Why the Inner Product Depends Only on Relative Distance** $(m-n)$**:** Because 2×2 rotation matrices are orthogonal ($R(\alpha)^T = R(-\alpha)$ and $R(\alpha)R(\beta) = R(\alpha+\beta)$):

  $$\langle R(m\theta)q,\ R(n\theta)k \rangle = q^T R(m\theta)^T R(n\theta)k = q^T R\big((n-m)\theta\big)k$$

- **Norm Preservation:** Because $R(m\theta)$ is an orthogonal matrix, rotating a vector never changes its length: $\|R(m\theta)q\|_2 = \|q\|_2$.

🔢 Step-by-Step Worked Toy Example

Suppose a 2D query vector $q = [1, 0]^T$ sits at position $m = 5$ and an identical key vector $k = [1, 0]^T$ sits at position $n = 7$, with rotation step $\theta = 30^\circ$:

1. **Step 1 — Rotate** q **at position** $m = 5$ **(**$5 \times 30^\circ = 150^\circ$**):**

   $$q'_5 = R(150^\circ)\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos 150^\circ \\ \sin 150^\circ \end{pmatrix}$$

2. **Step 2 — Rotate** k **at position** $n = 7$ **(**$7 \times 30^\circ = 210^\circ$**):**

   $$k'_7 = R(210^\circ)\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos 210^\circ \\ \sin 210^\circ \end{pmatrix}$$

3. **Step 3 — Compute their dot product** $q'_5 \cdot k'_7$**:** Using the trigonometric identity $\cos\alpha\cos\beta + \sin\alpha\sin\beta = \cos(\beta - \alpha)$:

   $$q'_5 \cdot k'_7 = \cos(150^\circ)\cos(210^\circ) + \sin(150^\circ)\sin(210^\circ) = \cos(210^\circ - 150^\circ) = \cos(60^\circ) = \mathbf{0.5}$$

4. **Step 4 — Check positions** $m = 1005$ **and** $n = 1007$ **(same gap** $n - m = 2$**):**

   $$q'_{1005} \cdot k'_{1007} = \cos\big((1007 - 1005) \times 30^\circ\big) = \cos(60^\circ) = \mathbf{0.5}$$

   The attention score is **100% identical** regardless of absolute position!

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Long-Context RoPE Extension (PI, NTK, YaRN) & ALiBi**

- **Position Interpolation (PI):** Linearly downscales position index $m \to m\frac{L_{\text{train}}}{L_{\text{target}}}$ so angles stay within the trained range, though it crowds high-frequency local distinctions.
- **NTK-Aware Scaling / YaRN:** Scales the RoPE base frequency ($\text{base}' = \text{base} \cdot \alpha^{d/(d-2)}$) or interpolates only low-frequency dimensions while leaving high-frequency local dimensions untouched.
- **ALiBi (Attention with Linear Biases):** Used in MPT/Falcon; adds a static, non-learned linear distance penalty directly to attention logits before softmax: $\text{softmax}\left(\frac{q_i^T k_j}{\sqrt{d_k}} - m_h \cdot |i-j|\right)$, where each head h gets a geometric slope $m_h = 2^{-8h/H}$.

🎯 Exam Takeaway & Common Traps

- **3 Guaranteed RoPE Facts (`LLM07`):**
  1. RoPE encodes **relative position** $(m-n)$ by applying an orthogonal rotation matrix to **absolute positions** m **and** n independently.
  2. RoPE is applied **only to Queries (Q) and Keys (K)** at every attention layer—**never to Values (V)**!
  3. Because rotation matrices are orthogonal, RoPE strictly preserves the L_2 norm of Q and K.

2.4 Sparse Mixture-of-Experts (MoE — Mixtral 8x7B / DeepSeek-MoE)

💡 Intuition (Plain English First)

In a dense 47B-parameter LLM, every single token—even a trivial word like `"the"`—has to multiply through all 47 billion parameters. That makes inference slow and expensive. **Sparse Mixture-of-Experts (MoE)** replaces each dense FFN layer with a team of E specialist FFN "Experts" (e.g., $E = 8$ experts in Mixtral 8x7B: one might specialize in Python code, one in math, one in French grammar). When a token arrives, a tiny traffic cop called a **Router** looks at the token and sends it to **only the top $k = 2$ experts**!

- **The Result:** You still need enough RAM/VRAM to hold all $E = 8$ experts ($\sim$ 47B **total parameters**), but for any single token, your GPU/NPU only does the math for $k = 2$ experts ($\sim$ 13B **active parameters per token**)!

📐 The Math & Every Symbol Explained

Given a token representation $x \in \mathbb{R}^D$ entering an MoE FFN layer with E experts $\{E_1, \ldots, E_E\}$ and router matrix $W_g \in \mathbb{R}^{D \times E}$:

$$G(x) = \text{Softmax}\Big(\text{TopK}(xW_g,\ k)\Big), \qquad y = \sum_{i \in \text{TopK}} G(x)_i\, E_i(x)$$

- $W_g \in \mathbb{R}^{D \times E}$**:** Lightweight linear router matrix (D rows, E expert columns) that scores how well each expert matches token x.
- $\text{TopK}(\cdot, k)$**:** Keeps the k highest router logits and sets the remaining $E - k$ logits to $-\infty$ before Softmax.
- $G(x)_i$**:** Routing weight (probability) assigned to selected expert i.

- $E_i(x)$**:** Output of the i-th SwiGLU FFN expert.
- **Router Collapse & Load-Balancing Auxiliary Loss:** During early training, whichever expert happens to get slightly better gradients gets picked more often by the router, trains even faster, and soon receives 100% of all tokens (**Router Collapse**), leaving the other $E-1$ experts dead! To prevent this, we add an **auxiliary load-balancing loss** $\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^{E} f_i P_i$ that penalizes uneven routing across a batch.

🔢 Step-by-Step Worked Toy Example

Suppose an MoE model has $E = 8$ experts per layer, where each single expert FFN has 5B parameters across all layers, and the shared Attention + Embeddings have 7B parameters. We use Top-$k = 2$ routing:

1. **Step 1 — Total Parameters (VRAM Footprint):**

   $$\text{Total Params} = \text{Shared } (7\text{B}) + 8 \times \text{Expert } (5\text{B}) = \mathbf{47}\text{B parameters}$$

   (You must load all 47B *parameters into RAM/VRAM because any expert could be called!)*

2. **Step 2 — Active Parameters per Token (Compute FLOPs):**

   $$\text{Active Params} = \text{Shared } (7\text{B}) + 2 \times \text{Expert } (5\text{B}) = \mathbf{17}\text{B parameters per token}$$

   (Each token only executes 2 *experts, giving the compute speed of a* 17B *model with the knowledge capacity of a* 47B *model!)*

🎯 Exam Takeaway & Common Traps

- **MoE Trade-off:** MoE slashes **FLOPs per token** (proportional to k active experts), NOT **VRAM footprint** (proportional to all E total experts). For batch size $B = 1$ edge inference, MoE is still memory-capacity heavy, whereas for large batches it gives massive throughput gains.

2.5 Chinchilla Compute-Optimal Scaling Laws vs. Edge Inference Over-Training

💡 Intuition (Plain English First)

Suppose your company gives you a fixed supercomputer budget of $C = 10^{23}$ FLOPs to pretrain an LLM. Should you train a **huge 70B model on a small dataset**, or a **smaller 10B model on** $7\times$ **more data**?

- In 2022, DeepMind's **Chinchilla paper** proved that to get the lowest loss for a fixed *training* budget C, you should scale model parameters N and training tokens D equally: **train for** ≈ 20 **tokens per parameter** ($D_{\text{opt}} \approx 20N$).
- **Wait—why did Meta train LLaMA-3-8B on** 15 Trillion **tokens (**$\approx 1,875$ **tokens per parameter, nearly** $100\times$ **Chinchilla)?!** Because Chinchilla only cared about **training cost** and ignored **inference cost**! If you want an LLM to run on a Qualcomm Snapdragon phone (where RAM and memory bandwidth are tight), you cannot fit a 70B model—you *must* ship a small 3B or 8B model. Spending extra supercomputer electricity once during training to "over-train" an 8B model on 15T tokens makes every single user query faster and cheaper forever!

📐 The Math & Every Symbol Explained

1. **Dense Transformer Training Compute Rule:**

$$C \approx 6 \cdot N \cdot D \quad \text{(in FLOPs)}$$

   - N **(or** Ψ**):** Number of non-embedding model parameters.
   - D**:** Number of training tokens processed.
   - **Where does the factor of** 6 **come from?**
     - Forward pass: 1 matrix multiply per weight per token $=$ 1 MAC $=$ **2** FLOPs.
     - Backward pass: computing gradients w.r.t. input activations costs 2 FLOPs, and computing gradients w.r.t. weights costs 2 FLOPs, totaling **4** FLOPs (twice the forward pass!).
     - Total per parameter per token: $2 + 4 =$ **6** FLOPs.

2. **Chinchilla Compute-Optimal Token Rule:**

$$D_{\text{opt}} \approx 20N \implies C_{\text{opt}} \approx 6 \cdot N \cdot (20N) = 120N^2 \text{ FLOPs}$$

🔢 Step-by-Step Worked Toy Example

Let's compare a Chinchilla-optimal 8B model vs. LLaMA-3-8B:

1. **Chinchilla-Optimal Training Tokens for** $N = 8\text{B}$**:**

$$D_{\text{opt}} = 20 \times 8\text{B} = \textbf{160} \text{ Billion tokens}$$

   - Training FLOPs: $C \approx 6 \times (8 \times 10^9) \times (1.6 \times 10^{11}) = 7.68 \times 10^{21}$ FLOPs.

2. **LLaMA-3-8B Over-Trained Setup (**$D = 15$ Trillion tokens**):**

$$\text{Tokens per Parameter} = \frac{15,000\text{B}}{8\text{B}} = \textbf{1,875} \text{ tokens/parameter } (\approx 94 \times \text{ Chinchilla!})$$

   - Training FLOPs: $C \approx 6 \times (8 \times 10^9) \times (1.5 \times 10^{13}) = 7.2 \times 10^{23}$ FLOPs.

🎯 Exam Takeaway & Common Traps

- **Memorize both numbers:**
  1. Training FLOPs $=$ **6ND** ($2ND$ forward $+4ND$ backward).
  2. Chinchilla compute-optimal tokens $=$ **20N** (violated intentionally by modern small/edge LLMs to minimize *inference* latency and memory footprint!).

---

3. LLM Systems, Memory Arithmetic & Inference Optimization

↑ Mục Lục

3.1 Prefill vs Decode Phases & The Hardware Roofline Model (`LLM10`)

💡 Intuition (Plain English First)

Think of a GPU or Qualcomm Hexagon NPU as a factory with two limits:

1. **How fast the workers can calculate inside the factory:** Peak Compute Π_{peak} (measured in FLOPs/sec).
2. **How fast the delivery trucks can haul weights from the warehouse (DRAM/HBM) to the factory:** Memory Bandwidth β_{BW} (measured in Bytes/sec).

On modern chips, the factory workers are $\sim 200\times$ **faster** than the delivery trucks! This means if you haul a weight matrix from DRAM and only use it once (for 1 token), the workers finish instantly and spend 99.5% of their time waiting on the delivery trucks (**Memory-Bandwidth-Bound**). This explains why LLM inference has **two completely different personalities**:

- **Phase 1: Prefill (Reading your 1,000-word prompt):** All $1,000$ prompt tokens are already available! We haul each weight matrix from DRAM **once** and multiply it against all $1,000$ tokens at the same time (**Matrix-Matrix Multiplication / GEMM**). The workers stay busy doing $1,000$ operations per byte loaded $\rightarrow$ **Compute-Bound**!
- **Phase 2: Decode (Generating the reply 1 word at a time):** Because word $t+1$ depends on word t, we can only generate 1 **token at a time**. For every single word generated, we have to haul the *entire* multi-gigabyte LLM weight matrix from DRAM just to multiply it by 1 single vector (**Matrix-Vector Multiplication / GEMV**)! We only do 1 operation per byte loaded $\rightarrow$ **Memory-Bandwidth-Bound**!

📐 The Math & Every Symbol Explained

1. **Operational Intensity (OI, also called Arithmetic Intensity):**

$$\text{OI} = \frac{\text{Total FLOPs Executed}}{\text{Total Bytes Read/Written from Memory}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right]$$

2. **The Roofline Throughput Equation & Ridge Point I_{ridge}:**

$$\text{Attainable Throughput (FLOPs/s)} = \min\Big(\underbrace{\Pi_{\text{peak}}}_{\text{Compute Ceiling}}, \underbrace{\text{OI} \times \beta_{\text{BW}}}_{\text{Memory Bandwidth Slope}}\Big), \qquad I_{\text{ridge}} = \frac{\Pi_{\text{peak}}}{\beta_{\text{BW}}}$$

   - **If** $\text{OI} < I_{\text{ridge}}$**:** Workload is **Memory-Bandwidth-Bound** (speed up by quantizing weights to INT4/INT8, using GQA, FlashAttention, or Speculative Decoding).
   - **If** $\text{OI} > I_{\text{ridge}}$**:** Workload is **Compute-Bound** (speed up by faster Tensor Cores or lower-precision MAC units).

3. **Operational Intensity of a Linear Layer (Ψ parameters stored in FP16 $= 2\Psi$ bytes) for $M = B \cdot S$ tokens:**

$$\text{OI} \approx \frac{2 \cdot M \cdot \Psi \text{ FLOPs}}{2\Psi \text{ Bytes}} = M = B \cdot S \text{ FLOPs/Byte}$$

   - **Prefill ($B = 1, S = 2048$):** $\text{OI} \approx 2048$ FLOPs/Byte $\gg I_{\text{ridge}} \implies$ **Compute-Bound**.
   - **Decode ($B = 1, S = 1$):** $\text{OI} \approx 1$ FLOP/Byte $\ll I_{\text{ridge}} \implies$ **Memory-Bandwidth-Bound**!

🔢 Step-by-Step Worked Toy Example

Suppose an edge NPU has Peak Compute $\Pi_{\text{peak}} = 100$ TFLOPs/s (10^{14} FLOPs/s) and LPDDR5X Memory Bandwidth $\beta_{\text{BW}} = 100$ GB/s (10^{11} Bytes/s), and we run a $\Psi = 7$B model in FP16 (14 GB of weights) at batch size $B = 1$:

1. **Step 1 — Find the Hardware Ridge Point I_{ridge}:**

$$I_{\text{ridge}} = \frac{100 \times 10^{12}\ \text{FLOPs/s}}{100 \times 10^{9}\ \text{Bytes/s}} = \mathbf{1000}\ \text{FLOPs/Byte}$$

2. **Step 2 — Calculate maximum Decode speed ($S = 1, \text{OI} = 1\ \text{FLOP/Byte} \ll 1000$):** Every generated token must read all 14 GB of weights over a 100 GB/s bus:

$$\text{Max Decode Speed} \approx \frac{\beta_{\text{BW}}}{\text{Model Weight Bytes}} = \frac{100\ \text{GB/s}}{14\ \text{GB/token}} \approx \mathbf{7.14}\ \text{tokens/sec}$$

3. **Step 3 — What happens if we quantize the weights from FP16 (14 GB) to INT4 (3.5 GB)?** Because Decode is memory-bandwidth-bound, shrinking the weight traffic by 4× speeds up token generation by 4× to $\frac{100}{3.5} \approx \mathbf{28.6}$ tokens/sec!

🎯 Exam Takeaway & Common Traps

- **Why do we use a KV-Cache?** During Decode step t, the query is only the newest token q_t $(1 \times d_k)$, but it must attend to all past keys $K_{1:t}$ and values $V_{1:t}$. Caching past K, V tensors avoids recomputing them at every step (reducing total generation compute of T tokens from $\mathcal{O}(T^3 D)$ to $\mathcal{O}(T^2 D)$) and converts Attention + Linear projections from GEMM into GEMV.

3.2 Exact KV-Cache Memory Calculation (`LLM08`)

💡 Intuition (Plain English First)

How much RAM does the KV-Cache eat? Just count the dimensions of what we are storing! At every single decoder layer (out of L layers), for every sequence in the batch (B), and for every token position in the sequence (S), we must save **two vectors**: one **Key (K)** vector and one **Value (V)** vector. Each K or V vector has h_{kv} heads, each head has d_{head} numbers, and each number takes Bytes_per_element bytes (e.g., 2 Bytes for FP16/BF16, or 1 Byte for INT8). Multiply those 7 numbers together and you have the exact byte count!

📐 The Math & Every Symbol Explained

$$\text{KV Cache (Bytes)} = 2 \times B \times S \times L \times h_{kv} \times d_{\text{head}} \times \text{Bytes_per_element}$$

- **2:** One tensor for Keys (K) + one tensor for Values (V).
- **B:** Batch size (number of concurrent user sequences).
- **S:** Sequence length (prompt tokens + generated tokens cached so far).
- **L:** Number of Transformer Decoder layers.
- **h_{kv}:** Number of Key/Value heads per layer ($h_{kv} = h_q$ for MHA; $h_{kv} < h_q$ for GQA; $h_{kv} = 1$ for MQA).
- **d_{head}:** Dimension of each attention head ($d_{\text{head}} = D/h_q$).
- **Bytes_per_element:** 2 for FP16/BF16, 1 for INT8/FP8, 0.5 for INT4.
- **Binary Unit Conversion Tip:** 1 KiB $= 2^{10}$ B $= 1024$ B, 1 MiB $= 2^{20}$ B $= 1,048,576$ B, and 1 GiB $= 2^{30}$ B $= 1,073,741,824$ B.

🔢 Step-by-Step Worked Toy Example (LLM08)

> 💡 **TIP**
>
> **The Powers-of-2 Mental Math Trick:** Almost every LLM hyperparameter is a power of 2! Instead of multiplying huge numbers by hand, just **add the exponents of 2** and subtract 20 for MiB (or subtract 30 for GiB)!

Problem (LLM08): Calculate the exact KV-Cache memory in MiB for an LLM with:

- $L = 32$ layers (2^5)
- Batch size $B = 2$ (2^1)
- Sequence length $S = 2048$ (2^{11})
- Query heads $h_q = 32$, **GQA Key/Value heads** $h_{kv} = 4$ (2^2)
- Head dimension $d_{\text{head}} = 128$ (2^7)
- Precision: FP16 (2 Bytes $= 2^1$)

Step-by-Step Calculation:

1. **Step 1 — Write every factor in the formula as a power of 2:**

$$\text{Bytes} = \underbrace{2}_{2^1\ (\text{K\&V})} \times \underbrace{2}_{2^1\ (\text{B})} \times \underbrace{2048}_{2^{11}\ (\text{S})} \times \underbrace{32}_{2^5\ (\text{L})} \times \underbrace{4}_{2^2\ (h_{kv})} \times \underbrace{128}_{2^7\ (d_{\text{head}})} \times \underbrace{2}_{2^1\ (\text{FP16})}$$

2. **Step 2 — Sum the exponents of 2:**

$$1 + 1 + 11 + 5 + 2 + 7 + 1 = 28 \implies \text{Total Bytes} = 2^{28}\ \text{Bytes}$$

3. **Step 3 — Divide by 2^{20} to convert Bytes to MiB:**

$$\text{KV Cache (MiB)} = \frac{2^{28}}{2^{20}} = 2^{28-20} = 2^8 = \mathbf{256}\ \text{MiB}$$

(If this model had used standard MHA with $h_{kv} = h_q = 32$ instead of GQA with $h_{kv} = 4$, the cache would be $32/4 = 8\times$ larger: $256 \times 8 = 2048$ MiB $= 2.0$ GiB!)

🎯 Exam Takeaway & Common Traps

- **Trap in LLM08 :** Exam questions always list *both* $h_q = 32$ and $h_{kv} = 4$ to tempt you into multiplying by 32. Only h_{kv} goes into the KV-Cache formula (h_q is only used if you need to find $d_{\text{head}} = D/h_q$)! Also never forget the leading factor of 2 (one for K, one for V).

3.3 Mixed-Precision Training Memory (16Ψ Rule) & Activation Checkpointing (LLM09)

💡 Intuition (Plain English First)

Why does a 7B parameter LLM need only 14 GB of VRAM for FP16 inference, but suddenly blows up an 80 GB A100 GPU when you try to run full fine-tuning with AdamW? Because during **Mixed-Precision Training with**

AdamW, for every single parameter in the model (Ψ), you don't just store the 2-byte FP16 weight—you must also store:

1. A 2-byte FP16 **gradient**.
2. A 4-byte FP32 **master copy of the weight** (if you tried to add a tiny 10^{-5} Adam update directly to a 16-bit weight, it would round to zero and stall training!).
3. A 4-byte FP32 **first moment (m_t, momentum)** for Adam.
4. A 4-byte FP32 **second moment (v_t, uncentered variance)** for Adam.

Add those up: $2 + 2 + 4 + 4 + 4 = \mathbf{16}$ Bytes per parameter (16Ψ)! And that's *before* storing any intermediate layer activations!

📐 The Math & Every Symbol Explained

Component Stored in GPU VRAM	Precision Format	Bytes per Parameter (Ψ)	Share of Static Training VRAM
1. Forward/Backward Weights (W)	FP16 or BF16	2Ψ Bytes	12.5%
2. Weight Gradients (∇W)	FP16 or BF16	2Ψ Bytes	12.5%
3. Master Weights Copy (W_{master})	FP32	4Ψ Bytes	25.0%
4. Adam First Moment (m_t)	FP32	4Ψ Bytes	25.0%
5. Adam Second Moment (v_t)	FP32	4Ψ Bytes	25.0%
Total Static Memory (16Ψ Rule)	**Mixed (FP16/BF16 + FP32)**	**16Ψ** Bytes	100% **(75% is Adam Optimizer!)**

- **Activation Checkpointing (Gradient Checkpointing):**
  - During a standard forward pass through L layers, PyTorch saves all intermediate activations inside every layer so the backward pass can compute derivatives—costing $\mathcal{O}(L)$ activation memory.
  - With **Activation Checkpointing**, we throw away the intermediate activations inside each layer during the forward pass and only save the input at the boundary of each Transformer block. During the backward pass, we **re-run a mini forward pass** through one block at a time to regenerate its activations on the fly.
  - **Memory vs Compute Trade-off:** Slashes activation memory from $\mathcal{O}(L)$ to $\mathcal{O}(\sqrt{L})$ (or $\mathcal{O}(1)$ buffer per layer) for only $\approx +33\%$ **extra training FLOPs** (since 1 normal step $= 1$ Forward $+ 2$ Backward $= 3$ units of work, and recomputing the forward pass adds $+1$ unit $\rightarrow 4/3 = 1.33\times$).

🔢 Step-by-Step Worked Toy Example (`LLM09`)

Suppose we want to perform full Mixed-Precision (BF16 + FP32 AdamW) fine-tuning of a $\Psi = 7$ Billion parameter LLM:

1. **Step 1 — BF16 Weights + BF16 Gradients:**

$$(2 + 2) \times 7\text{B} = 4 \times 7\text{B} = \mathbf{28}\ \text{GB}$$

2. **Step 2 — FP32 AdamW Optimizer States (Master Weights + m_t + v_t):**

$$(4 + 4 + 4) \times 7\text{B} = 12 \times 7\text{B} = \mathbf{84}\ \text{GB}$$

3. **Step 3 — Total Static Memory (excluding activations):**

$$28\ \text{GB} + 84\ \text{GB} = 16 \times 7\text{B} = \mathbf{112}\ \text{GB}$$

(This immediately explains why full fine-tuning a 7B model overflows a single 80 GB GPU, and why LoRA—which freezes W and eliminates the 84 GB of Adam states on W—is revolutionary!)

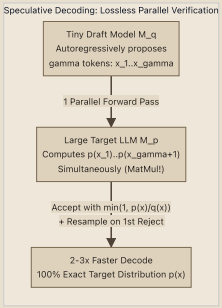
🎯 Exam Takeaway & Common Traps

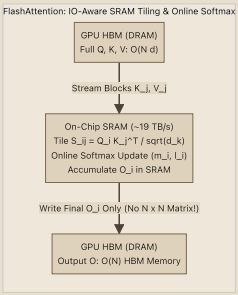
- **The 16Ψ Breakdown (`LLM09`):** Memorize $2 + 2 + (4 + 4 + 4) = 16$ Bytes/param. Notice that 12Ψ **(75%) comes purely from the FP32 AdamW optimizer states**!

3.4 FlashAttention (IO-Aware Tiling) & PagedAttention (vLLM) (`LLM10`)

💡 Intuition (Plain English First)

1. **FlashAttention:** Why is standard attention slow at $N = 8192$ tokens? Because standard PyTorch computes $S = QK^T$ (8192×8192 matrix), **writes all** 67 million **numbers out to slow GPU HBM (DRAM)**, reads them all back to compute $P = \text{softmax}(S)$, writes P back to slow HBM, and reads P back again to multiply by V! **FlashAttention** realizes that we never actually need the giant $N \times N$ matrix in slow HBM! Instead, it loads small tiles of Q, K, V into tiny, ultra-fast **on-chip SRAM** (which is $\sim$ 10–20$\times$ faster than HBM), computes the dot product, updates a running Softmax denominator using the **Online Softmax trick**, and multiplies by V right inside SRAM—writing only the final $N \times d_k$ output back to HBM!
   - *Crucial Exam Fact:* FlashAttention is **NOT an approximation**—it produces the **100% exact same mathematical output** as standard attention, while shrinking HBM memory from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$!
2. **PagedAttention (vLLM):** When serving 100 users at once, you don't know ahead of time whether User A will generate 20 tokens or 2048 tokens. Old serving engines pre-allocated a giant contiguous slab of 2048 KV-Cache slots per user, wasting 60–80% of GPU memory on empty slots (**internal fragmentation**). **PagedAttention** borrows **Virtual Memory Paging** from Operating Systems: it chops the KV-Cache into small fixed-size pages (e.g., 16 tokens per block) that can sit anywhere in physical VRAM, mapped by a lightweight Page Table!





📐 The Math & Every Symbol Explained

- **Why Online Softmax Works in FlashAttention:** Normally, $\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$ requires knowing the row maximum $m = \max_j x_j$ across all N columns before you can divide. In **Online Softmax**, as we stream tile j across SRAM, we update the running maximum $m_{\text{new}} = \max(m_{\text{old}}, \tilde{m}_j)$ and rescale the previous running sum ℓ_{old} and output O_{old} on the fly by multiplying by the exact correction factor $e^{m_{\text{old}} - m_{\text{new}}}$!

▼ 📌 Optional Deep-Dive (Skip for Timed MCQ): FlashAttention Online Softmax Recurrence Equations

$$m_{\text{new}} = \max(m_{\text{old}}, \tilde{m}), \qquad \ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} + e^{\tilde{m} - m_{\text{new}}} \tilde{\ell}, \qquad O_{\text{new}} = \frac{e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} O_{\text{old}} + e^{\tilde{m} - m_{\text{new}}} \tilde{P} V_j}{\ell_{\text{new}}}$$

Method	Compute Complexity (FLOPs)	HBM Memory Footprint	Is Output Mathematically Exact?
Standard Attention	$\mathcal{O}(N^2 d)$	$\mathcal{O}(N^2)$ (materializes $N \times N$ matrix S, P in HBM)	Yes
FlashAttention	$\mathcal{O}(N^2 d)$ (slightly more FLOPs due to rescaling)	$\mathcal{O}(N)$ (never writes $N \times N$ matrix to HBM!)	**Yes (100% Exact!)**

🔢 Step-by-Step Worked Toy Example

Suppose we have a row of 4 attention scores split into two tiles: Tile 1 is $[1.0,\ 2.0]$ and Tile 2 is $[4.0,\ 3.0]$:

1. **Step 1 — Process Tile 1** $[1.0,\ 2.0]$ **in SRAM:**
   - Local max $m_1 = 2.0$.
   - Local sum $\ell_1 = e^{1.0-2.0} + e^{2.0-2.0} = e^{-1} + 1 \approx 1.3679$.
2. **Step 2 — Stream Tile 2** $[4.0,\ 3.0]$ **into SRAM and update:**

- New global max $m_2 = \max(m_1, 4.0) = 4.0$.
- Rescale old sum ℓ_1 by $e^{m_1 - m_2} = e^{2.0-4.0} = e^{-2}$ and add Tile 2's sum:

  $$\ell_2 = e^{-2}(e^{-1} + 1) + (e^{4.0-4.0} + e^{3.0-4.0}) = e^{-3} + e^{-2} + 1 + e^{-1} \approx \mathbf{1.5530}$$

  We got the exact 4-element Softmax denominator $\sum_{j=1}^{4} e^{x_j - 4.0}$ in a single streaming pass without ever holding all 4 logits in memory at once!

🎯 Exam Takeaway & Common Traps

- **FlashAttention Trap (`LLM10`):** FlashAttention does **NOT** reduce theoretical FLOPs from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$! Its FLOPs are still $\mathcal{O}(N^2 d)$, and its output is **100% exact**—it runs 2–4× faster because it eliminates $\mathcal{O}(N^2)$ **HBM memory reads/writes** via SRAM tiling.

3.5 Decoding & Sampling Strategies (`LLM11`)

💡 Intuition (Plain English First)

At each decode step, the LLM outputs a raw score (logit z_i) for every word in its vocabulary V. How do we pick the next word?

- **Temperature T:** Controls how "adventurous" or "focused" the probability distribution is before sampling:
  - Dividing logits by a tiny temperature ($T \to 0^+$) magnifies differences between logits, turning Softmax into a winner-take-all spike on the single highest-scoring word (**Greedy Decoding**).
  - Dividing logits by a huge temperature ($T \to \infty$) squashes all logits toward 0, making every word in the dictionary equally likely (**Uniform Random Babble**).
- **Top-k Sampling:** Only keep the k most likely words (e.g., top 50) and set all other words to probability 0.
- **Top-p (Nucleus) Sampling:** Instead of a fixed count k, sort the words from most likely to least likely and add them to a bucket one by one **until their combined probability reaches or exceeds** p (e.g., $p = 0.90$), then stop and renormalize that bucket to sum to 1.0!

📐 The Math & Every Symbol Explained

1. **Temperature-Scaled Softmax:**

   $$p_i = \frac{\exp(z_i / T)}{\sum_{j=1}^{|V|} \exp(z_j / T)}$$

2. **Top-p (Nucleus) Set Definition:** Sort vocabulary tokens in descending order $p_{(1)} \geq p_{(2)} \geq \cdots \geq p_{(|V|)}$, and choose the **smallest integer** m such that:

   $$\sum_{i=1}^{m} p_{(i)} \geq p$$

   If combined with Top-k, we take at most $\min(m, k)$ tokens, zero out all remaining tokens, and **renormalize** the surviving tokens:

$$\hat{p}_{(i)} = \frac{p_{(i)}}{\sum_{j=1}^{\min(m,k)} p_{(j)}}$$

🔢 Step-by-Step Worked Toy Example (LLM11)

Problem (LLM11): At $T = 1.0$, a 4-token vocabulary has sorted probabilities $P = [0.50,\ 0.30,\ 0.15,\ 0.05]$. What is the final renormalized probability distribution after applying **both Top-k ($k = 2$) and Top-p ($p = 0.75$)**?

1. **Step 1 — Check cumulative probabilities for Top-p ($p = 0.75$):**
   - Include Token 1 (0.50): cumulative sum $= 0.50 < 0.75$ (not enough yet, keep going!).
   - Include Token 2 (0.30): cumulative sum $= 0.50 + 0.30 = \mathbf{0.80} \geq \mathbf{0.75}$ (stop! Nucleus size $m = 2$).
2. **Step 2 — Apply Top-k ($k = 2$) limit:**
   - Since $\min(m, k) = \min(2, 2) = 2$, Tokens 1 and 2 survive, and Tokens 3 and 4 are zeroed out.
3. **Step 3 — Renormalize the surviving probabilities by dividing by their sum (0.80):**

$$\hat{p}_1 = \frac{0.50}{0.80} = \mathbf{0.625}, \qquad \hat{p}_2 = \frac{0.30}{0.80} = \mathbf{0.375}, \qquad \hat{p}_3 = 0, \qquad \hat{p}_4 = 0$$

🎯 Exam Takeaway & Common Traps

- **Top-p Boundary Rule (LLM11):** Top-p keeps the **smallest prefix whose sum is $\geq p$** (so you *include* the token that pushes the cumulative sum across the p threshold—never stop strictly below p!).

---

4. Fine-Tuning (PEFT / LoRA) & Alignment (RLHF vs DPO vs GRPO)

↑ Mục Lục

4.1 LoRA & QLoRA Math & Exam Traps (LLM12 , LLM13)

💡 Intuition (Plain English First)

We just saw that full fine-tuning of a 7B model requires 112 GB of VRAM because storing FP32 Adam optimizer states (12Ψ) for billions of weights is massive. **LoRA (Low-Rank Adaptation)** asks a simple question: *When we fine-tune a giant* 4096×4096 *matrix* W_0 *on a new task, do we really change all* 4096 *directions independently, or does the weight update* ΔW *actually live in a tiny low-dimensional subspace (say, rank* $r = 8$ *or* 16*)?* LoRA **freezes W_0 completely** (so W_0 needs zero gradients and zero Adam optimizer states!) and represents the update ΔW as the product of two skinny matrices: a "down-projection" A (from width d_{in} down to tiny bottleneck r) and an "up-projection" B (from tiny bottleneck r back up to d_{out}).

- **Even better:** At inference time, since $W_0 x + \frac{\alpha}{r} BAx = \left(W_0 + \frac{\alpha}{r} BA\right) x$, we simply **add $\frac{\alpha}{r} BA$ directly into W_0 once before deployment**! That means LoRA adds **ZERO extra latency and ZERO extra FLOPs at inference time**!

📐 The Math & Every Symbol Explained

For a frozen pretrained weight matrix $W_0 \in \mathbb{R}^{n \times m}$ (where $n = d_{\text{out}}$ rows and $m = d_{\text{in}}$ columns) and an input vector $x \in \mathbb{R}^m$:

$$h = W_0 x + \frac{\alpha}{r} \Delta W x = W_0 x + \frac{\alpha}{r} B A x$$

- $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$**:** Frozen pretrained weight matrix (d_{out} rows, d_{in} columns; $d_{\text{out}} \cdot d_{\text{in}}$ frozen parameters).
- $A \in \mathbb{R}^{r \times d_{\text{in}}}$**:** Trainable low-rank down-projection matrix (r rows, d_{in} columns), initialized with random Gaussian noise $A \sim \mathcal{N}(0, \sigma^2)$.
- $B \in \mathbb{R}^{d_{\text{out}} \times r}$**:** Trainable low-rank up-projection matrix (d_{out} rows, r columns), initialized to **exact zeros** ($B = 0$**)**.
- $r \ll \min(d_{\text{out}}, d_{\text{in}})$**:** LoRA rank (typically $r \in \{8, 16, 32, 64\}$).
- α**:** Constant LoRA scaling hyperparameter (typically $\alpha = r$ or $\alpha = 2r$). Scaling by $\frac{\alpha}{r}$ ensures that if you change rank r, the magnitude of $\frac{\alpha}{r} BAx$ stays roughly constant without needing to re-tune learning rates!
- **Trainable Parameter Count of One LoRA Adapter:**

  $$\text{Params}_{\text{LoRA}} = \text{Params}(A) + \text{Params}(B) = r \cdot d_{\text{in}} + d_{\text{out}} \cdot r = \mathbf{r} \cdot (\mathbf{d}_{\text{out}} + \mathbf{d}_{\text{in}})$$

> ⚡ **IMPORTANT**
>
> **Why MUST A be random Gaussian and $B = 0$ at initialization (`LLM12`)?**
>
> 1. **Why $B = 0$?** At training step 0, $\Delta W = BA = 0 \cdot A = \mathbf{0}$, so $h = W_0 x$. The model starts fine-tuning with the **100% exact behavior of the pretrained network**, without injecting random untrained noise!
> 2. **Why can't BOTH $A = 0$ and $B = 0$?** By the chain rule, $\frac{\partial \mathcal{L}}{\partial A} = B^T \frac{\partial \mathcal{L}}{\partial h} x^T$ and $\frac{\partial \mathcal{L}}{\partial B} = \frac{\partial \mathcal{L}}{\partial h} (Ax)^T$. If both $A = 0$ and $B = 0$, both gradients are identically 0 at step 0, trapping the network at a dead saddle point forever! With $A \neq 0$ and $B = 0$, $\frac{\partial \mathcal{L}}{\partial B} \neq 0$ on step 1, updating B to become non-zero on step 2!

- **QLoRA (4-Bit Quantized LoRA — `LLM13`):** Compresses the frozen base model W_0 to **4-bit NormalFloat (`NF4`)** while keeping the tiny LoRA adapters A, B in **BF16**:
  1. **NF4 (4-Bit NormalFloat):** Because pretrained neural network weights follow a zero-mean normal (bell-curve) distribution, NF4 places its 16 quantization bins at the **exact equal-area quantiles of** $\mathcal{N}(0, 1)$ so every bin holds an equal $1/16$th of the weights (information-theoretically optimal for normal distributions).
  2. **Double Quantization (DQ):** Quantizes the 32-bit block-wise quantization scale factors themselves down to 8-bit FP8!
  3. **Paged Optimizers:** Uses NVIDIA unified memory to automatically page AdamW optimizer states between GPU VRAM and CPU RAM during long-sequence memory spikes.

🔢 Step-by-Step Worked Toy Example (`LLM12`, `LLM13`)

1. **Problem A (`LLM12` — LoRA Parameter Count & Compression Ratio):** Suppose we attach a LoRA adapter of rank $r = 16$ to a GQA Key projection matrix $W_K \in \mathbb{R}^{1024 \times 4096}$ ($n = 1024$ output rows, $m = 4096$ input columns):
   - **Original frozen parameters in W_K:** $1024 \times 4096 = 4,194,304$.
   - **Trainable parameters in A (16×4096) and B (1024×16):**

     $$\text{Params}_{\text{LoRA}} = 16 \times (1024 + 4096) = 16 \times 5120 = \mathbf{81,920} \text{ parameters}$$

- **Percentage of original matrix:** $\frac{81{,}920}{4{,}194{,}304} = \frac{16\times5120}{1024\times4096} = \frac{80}{4096} \approx \mathbf{1.95\%}$ (a $51.2\times$ reduction in trainable parameters!).

2. **Problem B (`LLM13` — QLoRA Double Quantization Exact Bit-Overhead Math):** Suppose QLoRA quantizes weights in blocks of 64 weights, storing one FP32 (32-bit) scale factor S per block:
   - **Without Double Quantization:** Each block of 64 weights pays 32 extra bits for its scale factor:

     $$\text{Scale Overhead} = \frac{32 \text{ bits}}{64 \text{ weights}} = \mathbf{0.50} \text{ bits per parameter}$$

     (So a "4-bit" model actually takes 4.50 bits/param*!)*
   - **With Double Quantization:** We quantize each block's scale factor to **8-bit FP8**, and group 256 blocks together under one second-level FP32 (32-bit) scale:

     $$\text{DQ Scale Overhead} = \frac{8 \text{ bits}}{64 \text{ weights}} + \frac{32 \text{ bits}}{64 \times 256 \text{ weights}} = 0.125 + 0.00195 \approx \mathbf{0.127} \text{ bits per parameter}$$

     (Saves $0.50 - 0.127 = \mathbf{0.373}$ bits/param*, or* ~ 3 GB *of VRAM on a 65B model!)*

🎯 Exam Takeaway & Common Traps

- **LoRA Formula (`LLM12`):** Always $r(d_{\text{out}} + d_{\text{in}})$, initialized with $A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$, merged at inference as $W_{\text{merged}} = W_0 + \frac{\alpha}{r}BA$ for **zero inference latency**.
- **QLoRA Overhead (`LLM13`):** Blocksize 64 with FP32 scales adds $32/64 = 0.5$ bits/param; Double Quantization slashes that to $8/64 + 32/(64 \times 256) \approx \mathbf{0.127}$ bits/param.

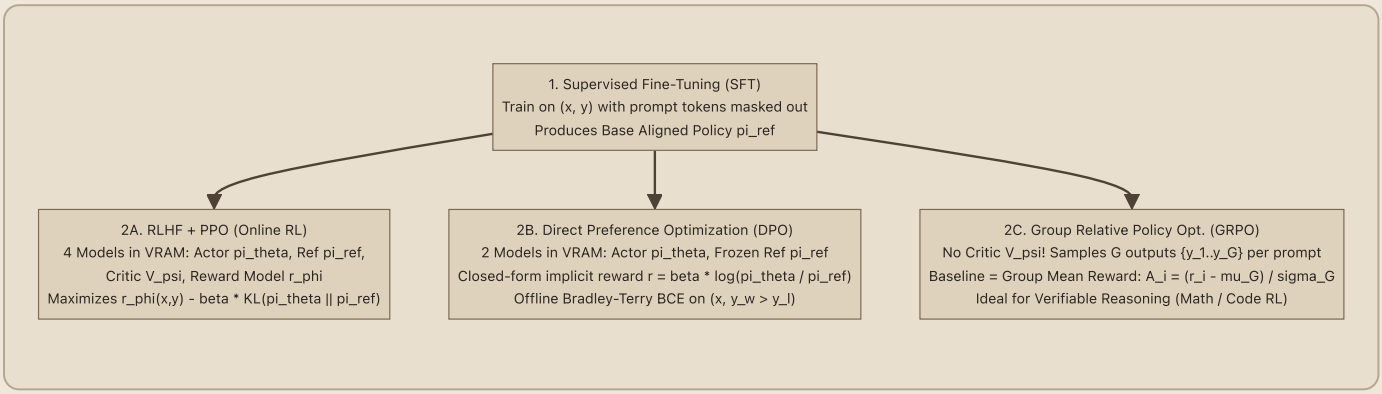
4.2 Post-Training Alignment: SFT → RLHF (PPO) vs DPO vs GRPO (`LLM14`)

💡 Intuition (Plain English First)

A pretrained base LLM only knows how to autocomplete internet text: if you ask `"How do I bake a cake?"`, it might autocomplete with `"Question 2: How do I bake a pie?"`. To turn it into a helpful, safe assistant, we run **Post-Training Alignment**:

1. **Stage 1 — Supervised Fine-Tuning (SFT):** We show the model thousands of high-quality `(User Prompt, Ideal Assistant Response)` demonstrations. *Crucial detail:* We only compute Cross-Entropy loss on the **Assistant Response tokens** (masking out the User Prompt tokens so the model isn't trained to parrot user questions!).
2. **Stage 2A — RLHF with PPO (The Classic 4-Model Pipeline):** Humans are much better at picking the better of two answers ($y_w \succ y_l$) than writing a gold answer from scratch. So we first train a **Reward Model (r_ϕ)** to score answers like a human judge. Then we use **PPO (Reinforcement Learning)** to train our LLM Policy (π_θ) to maximize the Reward Model's score.
   - *Why do we subtract a **KL-Divergence penalty** against the frozen SFT model (π_{ref})?* Without it, the LLM discovers **Reward Hacking**: it outputs bizarre gibberish like `"helpful helpful safe A+ 100%"` that tricks the Reward Model into giving a high score!
   - *Downside of PPO:* You must hold **4 full LLMs in GPU memory at once**: Actor (π_θ), Reference (π_{ref}), Critic/Value network (V_ψ), and Reward Model (r_ϕ).

3. **Stage 2B — DPO (Direct Preference Optimization — 2 Models, No RL!):** Rafailov et al. (2023) proved a stunning mathematical shortcut: you can solve the KL-constrained reward equation for the reward $r(x, y)$ directly in terms of the LLM's own probabilities $\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$! That means **the LLM itself IS the reward model**! DPO throws away the Reward Model and Critic completely, training directly on (x, y_w, y_l) pairs with a simple binary cross-entropy loss using only **2 models in VRAM** (π_θ and frozen π_{ref}).
4. **Stage 2C — GRPO (Group Relative Policy Optimization — DeepSeek-R1):** What if you *need* online RL (for verifiable math or code problems where a compiler/checker grades the answer), but PPO's Critic network V_ψ is too big for VRAM? **GRPO** deletes the Critic network V_ψ! Instead, for each prompt x, it samples a **group of G answers** $\{y_1, \ldots, y_G\}$ and uses the **group's average reward** as the baseline!



📐 The Math & Every Symbol Explained

1. **Reward Model Bradley-Terry Loss (for preference pair $y_w \succ y_l$ given prompt x):**

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big]$$

   - y_w **("winner") and** y_l **("loser"):** The human-preferred and human-dispreferred responses to prompt x.
   - $\sigma(z) = \frac{1}{1+e^{-z}}$**:** Logistic sigmoid function mapping the score difference $r_\phi(x, y_w) - r_\phi(x, y_l)$ into a probability in $(0, 1)$.

2. **RLHF (PPO) KL-Regularized Objective:**

$$\max_{\pi_\theta} \ \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x, y) - \beta\, D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big)\Big]$$

   - $\beta > 0$**:** KL penalty coefficient controlling how far the actor π_θ is allowed to drift from the frozen SFT reference policy π_{ref}.

3. **DPO (Direct Preference Optimization) Loss (`LLM14`):** Substituting the closed-form implicit reward $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ into the Bradley-Terry loss cancels the partition function $Z(x)$ and yields:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

4. **GRPO (Group Relative Policy Optimization) Advantage Formula:** For a group of G sampled outputs $\{y_1, \ldots, y_G\}$ with rewards $\{r_1, \ldots, r_G\}$:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}, \qquad \text{where } \mu_G = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad \sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_G)^2}$$

🔢 Step-by-Step Worked Toy Example

Let's trace **GRPO Advantage Calculation** on a math reasoning prompt x where we sample $G = 4$ responses from our LLM and check them with a Python math verifier ($r = 1$ if correct, $r = 0$ if wrong). Suppose only Response 1 is correct, so the rewards are $R = [1.0,\ 0.0,\ 0.0,\ 0.0]$:

1. **Step 1 — Compute the Group Mean Reward μ_G:**

$$\mu_G = \frac{1.0 + 0.0 + 0.0 + 0.0}{4} = 0.25$$

2. **Step 2 — Compute the Group Standard Deviation σ_G ($\epsilon = 0$):**

$$\sigma_G = \sqrt{\frac{(1.0 - 0.25)^2 + 3 \times (0.0 - 0.25)^2}{4}} = \sqrt{\frac{0.5625 + 0.1875}{4}} = \sqrt{0.1875} \approx 0.433$$

3. **Step 3 — Compute the Group-Normalized Advantage $\hat{A}_i$ for each response:**
   - Correct response ($r_1 = 1.0$): $\hat{A}_1 = \frac{1.0 - 0.25}{0.433} = \mathbf{+1.732}$ (reinforced!).
   - Wrong responses ($r_{2,3,4} = 0.0$): $\hat{A}_{2,3,4} = \frac{0.0 - 0.25}{0.433} = \mathbf{-0.577}$ (penalized!). *(Notice how GRPO computed exact positive and negative RL advantages without needing a 7B Critic neural network V_ψ!)*

🎯 Exam Takeaway & Common Traps

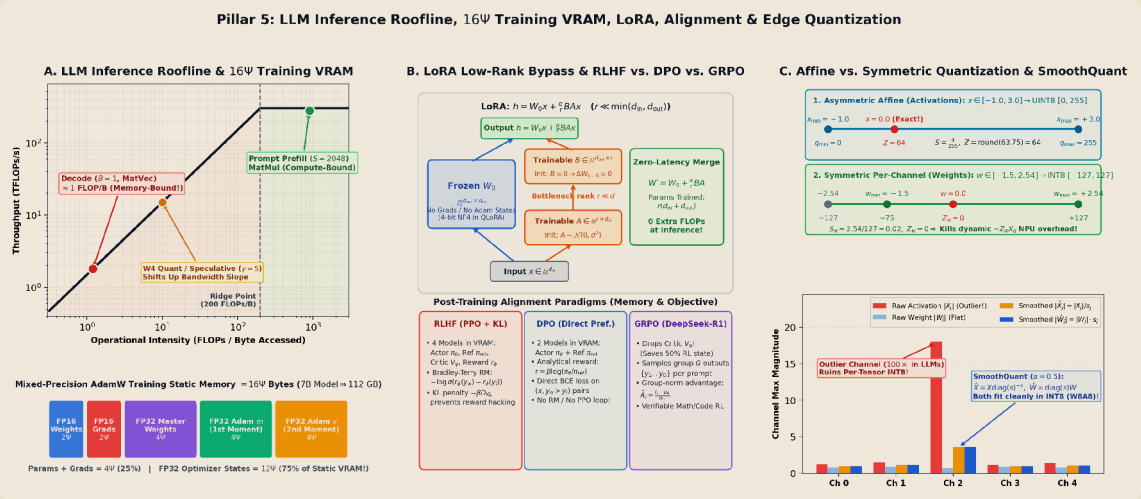
- **Alignment Comparison (`LLM14`):**
  - **RLHF (PPO):** Online RL, requires **4 models** (Actor, Ref, Critic, Reward); KL penalty prevents **reward hacking / mode collapse**.
  - **DPO:** Offline preference learning, requires **2 models** (Actor, Ref); derives implicit reward $r(x, y) = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ via Bradley-Terry substitution.
  - **GRPO:** Online RL without a Critic network; normalizes rewards across a group of G sampled outputs per prompt.

---

5. Edge AI, Quantization, Distillation & Fast Inference (Qualcomm Focus)

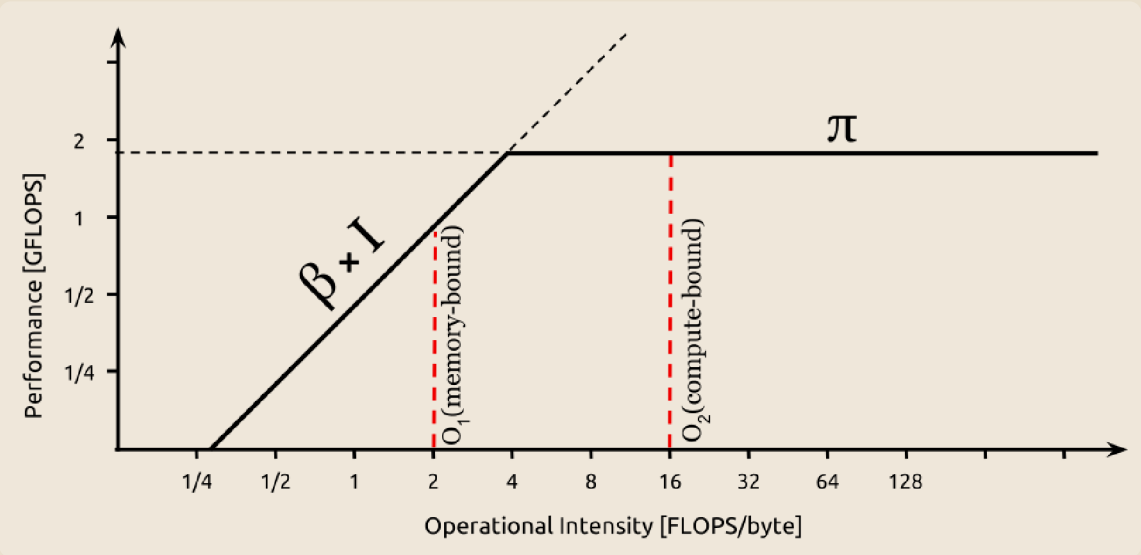
↑ Mục Lục

FIGURE SLIDE 1 OF 2



LLM Inference Roofline, 16-Psi Training VRAM, LoRA, Alignment & Edge Quantization

FIGURE SLIDE 2 OF 2



Canonical Roofline Model: Memory-Bandwidth-Bound vs Compute-Bound Regimes

5.1 Floating-Point Formats: FP32 vs FP16 vs BF16 (`LLM15`)

💡 Intuition (Plain English First)

Every floating-point number in a computer is stored in scientific notation using three parts:

$$\text{Value} = (-1)^{\text{Sign}} \times 2^{(\text{Exponent} - \text{Bias})} \times (1.\text{Mantissa})_2$$

- **The Exponent bits** control the **Dynamic Range** (how huge or how tiny a number can be before overflowing to ∞ or underflowing to 0.0).

- **The Mantissa (Fraction) bits** control the **Precision** (how many decimal digits of accuracy you get between two powers of 2).

When engineers first tried training deep networks in **IEEE FP16** (1 sign, **5 exponent**, 10 mantissa), training crashed constantly! Why? Because a **5-bit exponent** can only represent numbers up to $65,504$ and down to $\sim 6 \times 10^{-8}$, so tiny gradients underflowed to 0.0 unless you added complicated **Dynamic Loss Scaling**. Google Brain invented **BF16 (Brain Float 16)** by taking a 32-bit FP32 number (1 sign, **8 exponent**, 23 mantissa) and simply chopping off the last 16 mantissa bits (leaving 1 sign, **8 exponent**, 7 mantissa)! Because BF16 keeps all **8 exponent bits** of FP32, it has the **exact same dynamic range as FP32** ($\sim 10^{\pm 38}$) and never needs loss scaling!

📐 The Math & Every Symbol Explained

Format	Total Bits	Sign Bits	Exponent Bits (Dynamic Range)	Mantissa Bits (Precision)	Key Hardware & Deep Learning Behavior
FP32	32	1	**8 bits** ($\sim 10^{\pm 38}$)	23 bits	Standard master-weight & accumulation precision
FP16	16	1	**5 bits** (max $= 65,504$)	10 bits	Higher precision than BF16, but **narrow range** $\rightarrow$ gradients underflow; requires **Dynamic Loss Scaling**
BF16	16	1	**8 bits** ($\sim 10^{\pm 38}$)	7 bits	**Identical dynamic range to FP32**; drop-in replacement for LLM training without loss scaling!
INT8	8	1 (signed)	0 (Uniform Integer $[-128, 127]$ or $[0, 255]$)	0	4× smaller than FP32; native fast integer MACs on Qualcomm Hexagon NPU
INT4	4	1 (signed)	0 (Uniform Integer $[-8, 7]$ or $[0, 15]$)	0	8× smaller than FP32; standard for on-device LLM weights (`W4A16` / `W4A8`)

🔢 Step-by-Step Worked Toy Example

Suppose a backward pass produces a gradient value of $g = 10^{-10}$, and a giant activation spike reaches $a = 100,000$:

1. **In FP16 (5-bit exponent, range $\approx [6 \times 10^{-8},\ 65,504]$):**
   - The gradient $g = 10^{-10} < 6 \times 10^{-8}$ **underflows to** 0.0!
   - The activation $a = 100,000 > 65,504$ **overflows to** `Inf` (`NaN`)!
2. **In BF16 (8-bit exponent, range $\approx [1.2 \times 10^{-38},\ 3.4 \times 10^{38}]$):**
   - Both $g = 10^{-10}$ and $a = 100,000$ fit comfortably inside $[10^{-38}, 10^{38}]$ with **zero underflow and zero overflow**!

🎯 Exam Takeaway & Common Traps

- **Memorize the Bit Splits (`LLM15`):**
  - **FP32:** 1 sign + 8 exp + 23 mantissa.
  - **FP16:** 1 sign + 5 exp + 10 mantissa (needs loss scaling!).
  - **BF16:** 1 sign + 8 exp + 7 mantissa (same range as FP32!).

5.2 Exact Quantization Math: Asymmetric (Affine) vs Symmetric (`LLM16`, `LLM17`)

💡 Intuition (Plain English First)

How do we squeeze continuous floating-point numbers $x \in [x_{\min}, x_{\max}]$ into 256 discrete 8-bit integer buckets $x_q \in [q_{\min}, q_{\max}]$? Think of it like converting temperatures between Celsius and Fahrenheit using a **step size (Scale S)** and an **offset (Zero-Point Z)**:

1. **Asymmetric (Affine) Quantization (`LLM16`):** Maps the exact minimum float $x_{\min}$ to the bottom integer $q_{\min}$ (e.g., 0) and the exact maximum float $x_{\max}$ to the top integer $q_{\max}$ (e.g., 255). This uses 100% of all 256 buckets, even if your data is lopsided (like ReLU activations $x \in [-1.0, +3.0]$).
   - *Why is Zero-Point Z rounded to an exact integer?* Because 0.0 appears everywhere in neural networks (zero-padding around CNN images, ReLU zeros, causal masks). If 0.0 maps to an exact integer Z, our hardware can represent 0.0 with **zero quantization error**!
2. **Symmetric Quantization (`LLM17`):** Forces the Zero-Point to be $Z = 0$ (so float 0.0 always maps to integer 0) and stretches the range symmetrically to $\pm \max(|x_{\min}|, |x_{\max}|)$ across $[-127, +127]$.
   - *Why do Qualcomm Hexagon NPUs love Symmetric Quantization for Weights ($Z_w = 0$), even if it wastes a few buckets?* Because when you multiply quantized inputs $(X_q - Z_x)$ by quantized weights $(W_q - Z_w)$, expanding the product creates a cross-term $-Z_w X_q$. Since X_q changes dynamically on every single user query at runtime, if $Z_w \neq 0$, the NPU has to waste extra clock cycles summing up the rows of X_q at runtime! Setting $Z_w = 0$ **kills that runtime overhead completely**!

📐 The Math & Every Symbol Explained

1. **Asymmetric (Affine) Quantization (`LLM16`):**

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \qquad Z = \text{round}\left(q_{\min} - \frac{x_{\min}}{S}\right)$$

$$\text{Quantize: } x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z,\ q_{\min},\ q_{\max}\right), \qquad \text{Dequantize: } \hat{x} = S \cdot (x_q - Z)$$

   - $[x_{\min}, x_{\max}]$**:** Observed floating-point range of the tensor.
   - $[q_{\min}, q_{\max}]$**:** Target integer grid ($[0, 255]$ for `UINT8` where $q_{\max} - q_{\min} = 255$; $[-128, 127]$ for `INT8` where $q_{\max} - q_{\min} = 255$).
   - S **(Scale):** Floating-point step size per integer tick.
   - Z **(Zero-Point):** Exact integer value that corresponds to real floating-point 0.0.
2. **Symmetric Quantization ($Z = 0$, `LLM17`):** Using the symmetric signed grid $[-q_{\max}, +q_{\max}]$ (for `INT8`, $[-127, +127]$ so $q_{\max} = 127$; for `INT4`, $[-7, +7]$ so $q_{\max} = 7$):

$$S = \frac{\max(|x_{\min}|,\ |x_{\max}|)}{q_{\max}}, \qquad Z = 0$$

$$\text{Quantize: } x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right),\ -q_{\max},\ +q_{\max}\right), \qquad \text{Dequantize: } \hat{x} = S \cdot x_q$$

3. **The Hardware NPU Matrix Multiplication Expansion (Why $Z_w = 0$ Matters):**

$$\hat{Y} = S_x S_w (X_q - Z_x)(W_q - Z_w) = S_x S_w \Big[\underbrace{X_q W_q}_{\text{INT8 MACs}} - \underbrace{(Z_x W_q - Z_x Z_w)}_{\text{Folded into Bias Offline}} - \underbrace{Z_w X_q}_{\text{Runtime Penalty } (=0 \text{ if } Z_w=0)} \Big]$$

4. **Quantization Granularity Golden Rule:**
   - **Per-Tensor:** 1 scale S for the whole tensor (standard for **Activations**, or **Per-Token** in LLMs).
   - **Per-Channel (Per-Output-Channel):** 1 scale S_i per output row of W (standard for **CNN / Linear INT8 Weights** because different output channels have wildly different weight ranges).
   - **Per-Group (e.g., group size $g = 128$):** 1 scale per 128 contiguous weights along a row (standard for **INT4 LLM Weights** in GPTQ / AWQ).

🔢 Step-by-Step Worked Toy Example (`LLM16` , `LLM17`)

1. **Problem A (`LLM16` — Asymmetric `UINT8` Quantization by Hand):** Quantize an activation tensor with range $[x_{\min}, x_{\max}] = [-1.0,\ +3.0]$ to unsigned 8-bit integer `UINT8` ($[q_{\min}, q_{\max}] = [0, 255]$):
   - **Step 1 — Compute Scale S:**

     $$S = \frac{3.0 - (-1.0)}{255 - 0} = \frac{4.0}{255} \approx \mathbf{0.015686}$$

   - **Step 2 — Compute Zero-Point Z:**

     $$Z = \text{round}\left(0 - \frac{-1.0}{4.0/255}\right) = \text{round}\left(\frac{255}{4}\right) = \text{round}(63.75) = \mathbf{64}$$

   - **Step 3 — Verify that real $x = 0.0$ has zero quantization error:**

     $$x_q(0.0) = \text{round}\left(\frac{0.0}{S}\right) + 64 = 64 \implies \hat{x}(0.0) = S \cdot (64 - 64) = \mathbf{0.0} \quad \text{(Exact!)}$$

2. **Problem B (`LLM17` — Symmetric `INT8` Weight Quantization by Hand):** Quantize a weight tensor with range $[w_{\min}, w_{\max}] = [-1.50,\ +2.54]$ using Symmetric `INT8` on $[-127, +127]$ ($q_{\max} = 127$):
   - **Step 1 — Find the maximum absolute value:**

     $$\max(|-1.50|,\ |2.54|) = 2.54$$

   - **Step 2 — Compute Scale S_w and Zero-Point Z_w:**

     $$S_w = \frac{2.54}{127} = \mathbf{0.02}, \qquad Z_w = \mathbf{0}$$

   - **Step 3 — Quantize weight $w = -1.50$:**

     $$w_q = \text{round}\left(\frac{-1.50}{0.02}\right) = \text{round}(-75.0) = \mathbf{-75}$$

🎯 Exam Takeaway & Common Traps

- **Asymmetric vs Symmetric Summary (`LLM16` , `LLM17`):**

- **Activations** (like post-ReLU or post-GELU/Swish) are lopsided → use **Asymmetric Quantization** ($Z_x \neq 0$).
- **Weights** are roughly zero-centered and static → use **Per-Channel Symmetric Quantization** ($Z_w = 0$) so the dynamic cross-term $-Z_w X_q$ vanishes at runtime!

5.3 PTQ vs QAT & Modern LLM Quantization (GPTQ, AWQ, SmoothQuant) (`LLM18`)

💡 Intuition (Plain English First)

When we quantize a model, we have two paradigms:

1. **PTQ (Post-Training Quantization):** Take a finished model, pass a tiny calibration set (128–512 sentences) through it in a few minutes, and quantize without retraining.
   - **Why do naive PTQ methods fail on LLMs at 4-bit (`W4A16`) or 8-bit (`W8A8`)?** Because in LLMs larger than $\sim$ 6.7B parameters, a few specific **activation channels develop massive outliers (**$100\times$ **larger than normal channels)** across every token!
   - **How the Big Three LLM PTQ Algorithms solve this (`LLM18`):**
     - **GPTQ (`W4A16`):** Quantizes weights column-by-column using the **inverse Hessian** $H^{-1} = (2XX^T)^{-1}$ (second-order Taylor curvature of layer error $\|WX - \hat{W}X\|_2^2$) to slightly adjust the remaining unquantized columns so they cancel out the rounding error of the column just quantized!
     - **AWQ (Activation-Aware Weight Quantization, `W4A16`):** Notices that the $\sim 1\%$ of weight channels that multiply those huge **outlier activation channels** are $100\times$ more important ("salient")! Instead of keeping those 1% weights in FP16 (which ruins GPU/NPU SIMD memory alignment), AWQ **multiplies those salient weight channels by a scale** $s > 1$ **before INT4 quantization** (and divides the corresponding input activation channel by s) so their rounding error shrinks!
     - **SmoothQuant (`W8A8`):** If you want fast **INT8 Matrix Multiplication (`W8A8`)**, those $100\times$ activation outliers ruin per-tensor INT8 activation quantization (squashing normal channels to zero), while the weights are super flat and easy to quantize. **SmoothQuant** divides the outlier activation channels by a smoothing factor s_j and multiplies the corresponding weight channels by the same s_j! Mathematically, $(X\text{diag}(s)^{-1})(\text{diag}(s)W) = XW$ is unchanged, but now the activations are smooth enough for INT8!
2. **QAT (Quantization-Aware Training):** If PTQ still loses accuracy (e.g., pushing down to `W4A4` or `INT2`), we fine-tune the model with **Fake Quantization** nodes inserted in the forward pass so the weights learn to survive rounding noise.
   - **Wait—the rounding function** $\text{round}(u)$ **is a staircase whose derivative is** 0 **everywhere (and** ∞ **at the steps)! How can gradients flow backward through** $\text{round}(u)$**?** We use the **Straight-Through Estimator (STE)**: during the forward pass we run the real staircase $\text{round}(u)$, but during the backward pass we **pretend** $\text{round}(u)$ **is the identity line** $f(u) = u$ (derivative $= 1$ inside the clamp range $[q_{\min}, q_{\max}]$, and 0 outside)!

📐 The Math & Every Symbol Explained

Method	Target Format	Core Mathematical Mechanism	Key Problem Solved
GPTQ	`W4A16` (Weight-only INT4)	Uses inverse Hessian $H^{-1} = (2XX^T)^{-1}$ to update remaining weights as columns are quantized	Minimizes layer-wise output squared error $\|WX - \hat{W}X\|_F^2$
AWQ	`W4A16` (Weight-only INT4)	Scales up the 1% salient weight channels determined by **activation magnitude** $\|X_j\|$, not weight magnitude	Protects salient channels in uniform INT4 without slow mixed-precision hardware
SmoothQuant	`W8A8` (INT8 Weights & Activations)	Per-channel diagonal migration: $\hat{X} = X\text{diag}(s)^{-1}$, $\hat{W} = \text{diag}(s)W$ with $s_j = \frac{\max(\|X_j\|)^\alpha}{\max(\|W_j\|)^{1-\alpha}}$	Migrates 100× outlier difficulty from activations to weights so both fit in INT8
QAT + STE	Ultra-low bit (`W4A8`, `W4A4`, `W2`)	Forward: $\hat{x} = S(\text{clamp}(\text{round}(x/S) + Z))$. Backward **STE**: $\frac{\partial \text{round}(u)}{\partial u} \equiv 1$ on $[q_{\min}, q_{\max}]$	Bypasses the zero-everywhere derivative of round(·) during backpropagation

🔢 Step-by-Step Worked Toy Example (`LLM18`)

Suppose Channel j of activation X_j has a huge outlier peak $\max(|X_j|) = 100.0$, while the corresponding weight channel W_j is tiny with peak $\max(|W_j|) = 0.25$. Let's apply **SmoothQuant** with $\alpha = 0.5$:

1. **Step 1 — Compute the SmoothQuant per-channel migration factor s_j:**

$$s_j = \frac{\max(|X_j|)^{0.5}}{\max(|W_j|)^{1-0.5}} = \frac{\sqrt{100.0}}{\sqrt{0.25}} = \frac{10.0}{0.5} = \mathbf{20.0}$$

2. **Step 2 — Smooth the activation channel $\hat{X}_j = X_j/s_j$ and weight channel $\hat{W}_j = s_j W_j$:**
   - New activation peak: $\max(|\hat{X}_j|) = \frac{100.0}{20.0} = \mathbf{5.0}$ (shrunk by 20×!).
   - New weight peak: $\max(|\hat{W}_j|) = 20.0 \times 0.25 = \mathbf{5.0}$ (balanced to the exact same max of 5.0!).

3. **Step 3 — Check that the matrix product is 100% preserved:**

$$\hat{X}_j \hat{W}_j = \left(\frac{X_j}{20.0}\right)(20.0\, W_j) = X_j W_j$$

   Now both $\hat{X}$ and $\hat{W}$ have a gentle maximum of 5.0 and quantize cleanly into `INT8`!

🎯 Exam Takeaway & Common Traps

- **AWQ Trap (`LLM18`):** How does AWQ choose which 1% of weight channels are salient? By looking at the **activation magnitudes** $\|X_j\|$, NOT the weight magnitudes $\|W_j\|$!
- **STE Trap:** Outside the clamping range $[q_{\min}, q_{\max}]$, the STE gradient is 0 (clipped), and inside $[q_{\min}, q_{\max}]$ it is 1.

5.4 Knowledge Distillation, Pruning, Speculative Decoding & RAG (`LLM19`, `LLM20`)

💡 Intuition (Plain English First)

Let's master the final four edge-efficiency pillars tested in `LLM19`–`LLM20`:

1. **Knowledge Distillation (T^2 Rule):** A small Student model learns much more from a giant Teacher model's **soft probability distribution** (e.g., "*this image is* 70% *dog,* 29% *wolf,* 1% *car*") than from a hard 0/1 label ("100% *dog*"). To reveal those dark-knowledge probabilities on the non-winning classes, we soften both Teacher and Student logits with a high temperature $T > 1$ (e.g., $T = 4$).
   - *Why must we multiply the KL Distillation Loss by T^2?* Because dividing logits by T inside Softmax shrinks the backward gradient w.r.t. the student logits by a factor of $1/T$ from the student Softmax and another factor of $1/T$ from the logit difference, totaling $1/T^2$! Multiplying by T^2 restores the gradient magnitude so it doesn't get drowned out by the hard-label Cross-Entropy loss!
2. **Pruning (Unstructured vs Structured vs 2:4 Sparse Tensor Cores — `LLM19`):**
   - **Unstructured Pruning:** Zeroes out the smallest 50% of weights scattered randomly across a matrix. Sounds great on paper, right? **Trap:** On a GPU or NPU, dense matrix-multiply hardware still loads and multiplies every single 0.0, giving **ZERO wall-clock speedup** at 50% sparsity!
   - **Structured Pruning:** Deletes entire rows/columns (entire attention heads or FFN channels), physically shrinking the matrix from 4096×4096 to 2048×4096 for an immediate speedup on *any* hardware.
   - **NVIDIA / Hardware 2:4 Semi-Structured Sparsity:** Every group of 4 contiguous weights must contain **at least 2 zeros** (50% sparsity). Hardware Sparse Tensor Cores use a tiny 2-bit index mux to skip the two zeros entirely, delivering an **exact $2\times$ math speedup**!
3. **Speculative Decoding (Lossless 2–3$\times$ Speedup):** Remember from Section 3.1 that generating 1 token at a time with a 70B Target Model M_p is memory-bandwidth-bound because you haul 140 GB of weights from DRAM just to process 1 token. What if you haul those 140 GB of weights once and process 5 **tokens at the same time**? It takes almost the **exact same wall-clock time** as 1 token! **Speculative Decoding** uses a tiny 1B **Draft Model** M_q to quickly guess $\gamma = 5$ future tokens $[\tilde{x}_1, \ldots, \tilde{x}_5]$, and then runs the giant 70B **Target Model** M_p on all 5 tokens **in 1 single parallel forward pass** to verify them! Using a clever rejection-sampling rule $\min\left(1, \frac{p(x)}{q(x)}\right)$, the final generated text is **100% mathematically identical to sampling from the giant Target Model alone (Lossless!)**.
4. **RAG (Retrieval-Augmented Generation): Bi-Encoder vs Cross-Encoder (`LLM20`):** When searching 10 Million company documents for a user query:
   - **Stage 1 — Bi-Encoder (Dense Retriever):** Encodes the Query $E_Q(q)$ and each Document $E_D(d_i)$ **separately** into single vectors. Because all 10M document vectors are precomputed offline, finding the Top-100 closest vectors via dot-product search (HNSW / FAISS) takes < 5 milliseconds!
   - **Stage 2 — Cross-Encoder (Reranker):** Concatenates `[CLS] Query [SEP] Document [SEP]` and runs full token-by-token cross-attention across the Top-100 candidates. Much more accurate than a Bi-Encoder, but **impossible to precompute** (running it on all 10M docs would take hours per query!).

📐 The Math & Every Symbol Explained

1. **Knowledge Distillation Objective with T^2 Scaling:**

$$\mathcal{L}_{\text{KD}} = (1-\alpha)\,\mathcal{L}_{\text{CE}}\big(y,\ \sigma(z_s)\big) + \alpha\,\mathbf{T^2}\,D_{\text{KL}}\Big(\sigma(z_t/T)\ \|\ \sigma(z_s/T)\Big)$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Taylor Expansion Proof of 1/T² Gradient Shrinkage**

When T is large compared to the zero-mean logits z_i, Taylor expanding $e^{z_i/T} \approx 1 + \frac{z_i}{T}$ gives:

$$\frac{\partial D_{\text{KL}}}{\partial z_{s,i}} = \frac{1}{T}(p_{s,i}(T) - p_{t,i}(T)) \approx \frac{1}{T}\left(\frac{1 + z_{s,i}/T}{C} - \frac{1 + z_{t,i}/T}{C}\right) = \frac{1}{C \cdot \mathbf{T^2}}(z_{s,i} - z_{t,i})$$

Multiplying the KL loss by T^2 cancels the $1/T^2$ denominator!

2. **Speculative Decoding Acceptance & Residual Distribution Rule:** For draft token $\tilde{x} \sim q(x)$ proposed by Draft Model M_q and evaluated by Target Model M_p with probability $p(\tilde{x})$:

$$\text{Accept Probability} = \min\left(1,\ \frac{p(\tilde{x})}{q(\tilde{x})}\right)$$

If rejected at step i, we immediately resample a corrected token x_i' from the **residual distribution**:

$$p_{\text{residual}}(x) = \frac{\max\left(0,\ p(x) - q(x)\right)}{\sum_{v \in V} \max\left(0,\ p(v) - q(v)\right)}$$

🔢 Step-by-Step Worked Toy Example

Let's trace **Speculative Decoding Verification** on a 2-word vocabulary $V = \{\text{"cat"}, \text{"dog"}\}$ where at step 1:

- The **Target Model** M_p has true distribution $p = [0.80,\ 0.20]$.
- The **Draft Model** M_q has approximate distribution $q = [0.50,\ 0.50]$. Suppose the Draft Model samples $\tilde{x}_1$:

1. **Case A — Draft proposes `"cat"` (happens with draft probability $q(\text{"cat"}) = 0.50$):**
   - Acceptance probability: $\min\left(1, \frac{p(\text{"cat"})}{q(\text{"cat"})}\right) = \min\left(1, \frac{0.80}{0.50}\right) = \min(1, 1.6) = \mathbf{1.0}$ (100% accepted!).
2. **Case B — Draft proposes `"dog"` (happens with draft probability $q(\text{"dog"}) = 0.50$):**
   - Acceptance probability: $\min\left(1, \frac{p(\text{"dog"})}{q(\text{"dog"})}\right) = \min\left(1, \frac{0.20}{0.50}\right) = \mathbf{0.40}$ (40% accepted, 60% rejected!).
   - Overall probability of accepting `"dog"` $= q(\text{"dog"}) \times 0.40 = 0.50 \times 0.40 = \mathbf{0.20} = p(\text{"dog"})$!
   - And if `"dog"` is rejected (which happens with probability $0.50 \times 0.60 = 0.30$), we resample from $\max(0, p - q) = [\max(0, 0.8 - 0.5),\ \max(0, 0.2 - 0.5)] = [0.30,\ 0.0] \xrightarrow{\text{norm}} [1.0,\ 0.0]$ (always picks `"cat"`!).
   - **Total probability of outputting `"cat"`:** 0.50 (direct accept) + 0.30 (resampled after reject) $= \mathbf{0.80} = p(\text{"cat"})$! The final output probabilities $[0.80,\ 0.20]$ match the giant Target Model M_p **to the exact decimal place**!

🎯 Exam Takeaway & Common Traps

- **Pruning Trap (`LLM19`):** 50% unstructured magnitude pruning gives 0× **wall-clock speedup** on dense GEMM hardware; use **Structured Pruning** or **2:4 Semi-Structured Sparsity** for real hardware acceleration.
- **Speculative Decoding Fact:** Converts memory-bound Decode ($S = 1$ MatVec) into parallel verification ($S = \gamma + 1$ MatMul) and is **100% mathematically lossless**.
- **RAG Pipeline (`LLM20`):** Always **Bi-Encoder first** (fast offline-indexed MIPS over millions of docs) → **Cross-Encoder second** (accurate joint self-attention reranking over top-K docs).

6. Master Quick-Reference Checklist (`LLM01` – `LLM20`)

↑ Mục Lục

Question ID	Core Concept Tested	The 10-Second Formula / Golden Rule
`LLM01`	Scaled Dot-Product Attention Variance	If $\text{Var}(q_i) = \text{Var}(k_i) = \sigma^2$, then $\text{Var}(q \cdot k) = \sigma^4 d_k$; divide by $\sigma^2\sqrt{d_k}$ to restore unit variance ($\sqrt{d_k}$ when $\sigma^2 = 1$).
`LLM02`	Multi-Head Attention (MHA) Head Count h	Changing h while keeping $D = h \cdot d_k$ constant changes parameter count ($4D^2$) by 0, but scales (B, h, N, N) score memory linearly with h.
`LLM03`	MHA FLOPs & Crossover Sequence Length	Linear QKVO projections $= 8ND^2$ FLOPs; Quadratic Attention $= 4N^2D$ FLOPs; equal at $N = 2D$.
`LLM04`	Causal Attention Masking & Pre-LN	Add $-\infty$ (never 0) to upper triangle $j > i$ *before* Softmax so $e^{-\infty} = 0$. Pre-LN creates an unobstructed identity gradient highway.
`LLM05`	LLaMA Decoder Layer Parameter Count	Params $= \underbrace{2D^2 + 2Dh_{kv}d_k}_{\text{GQA}} + \underbrace{3DD_{\text{ff}}}_{\text{SwiGLU}} + \underbrace{2D}_{\text{2 RMSNorms}}$ (`bias=False`).
`LLM06`	MHA vs MQA vs GQA KV-Cache Ratio	GQA shares 1 K/V head across h_q/h_{kv} Query heads, slashing KV-Cache size by exactly h_q/h_{kv}.
`LLM07`	RoPE (Rotary Position Embedding)	Rotates 2D pairs of Q **and** K **only** (never V) by $m\theta_i$; orthogonal rotation preserves L_2 norm and makes $q_m^T k_n$ depend only on $m - n$.
`LLM08`	Exact KV-Cache Memory Formula	Bytes $= 2 \times B \times S \times L \times h_{kv} \times d_{\text{head}} \times$ Bytes_per_elem (use powers of 2: divide by 2^{20} for MiB, 2^{30} for GiB).
`LLM09`	Mixed-Precision AdamW Training Memory	16Ψ Bytes static VRAM: 2Ψ FP16 weights $+2\Psi$ FP16 grads $+4\Psi$ FP32 master weights $+4\Psi$ FP32 $m_t + 4\Psi$ FP32 v_t.
`LLM10`	Roofline Model & FlashAttention	Prefill is **Compute-Bound** (GEMM); Decode is **Memory-Bandwidth-Bound** (GEMV, OI ≈ 1). FlashAttention is **100% exact** and cuts HBM I/O to $\mathcal{O}(N)$ via SRAM tiling.
`LLM11`	Top-k & Top-p (Nucleus) Sampling	Sort probabilities descending, keep smallest prefix with cumulative sum $\geq p$ (capped at k tokens), and renormalize by dividing by their sum.
`LLM12`	LoRA Math, Initialization & Merging	$\Delta W = \frac{\alpha}{r}BA$ has $r(d_{\text{out}} + d_{\text{in}})$ params; init $A \sim \mathcal{N}(0, \sigma^2), B = 0$ so $\Delta W = 0$ at step 0; merge $W_0 + \frac{\alpha}{r}BA$ for 0 **inference latency**.
`LLM13`	QLoRA: NF4 & Double Quantization	NF4 is quantile-optimal for normal weights; Double Quantization cuts block-64 scale overhead from $32/64 = 0.50$ down to ≈ 0.127 bits/param.
`LLM14`	Alignment: RLHF (PPO) vs DPO vs GRPO	PPO needs **4 models** + KL penalty against reward hacking; DPO needs **2 models** (implicit reward $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$); GRPO drops the Critic via **group-mean baseline**.
`LLM15`	Floating-Point Bit Layouts	**FP32:** $1 + 8 + 23$; **FP16:** $1 + 5 + 10$ (narrow range, needs loss scaling); **BF16:** $1 + 8 + 7$ (**same dynamic range as FP32**).
`LLM16`	Asymmetric (Affine) Quantization	$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$, $Z = \text{round}(q_{\min} - x_{\min}/S)$; integer Z guarantees **exact** 0.0 **representation** for padding/ReLU.
`LLM17`	Symmetric Quantization & NPU Hardware	$S = \frac{\max(\lvert x_{\min}\rvert, \lvert x_{\max}\rvert)}{q_{\max}}$, $Z_w = 0$; eliminates the dynamic runtime cross-term $-Z_w X_q$ in integer GEMM!
`LLM18`	GPTQ, AWQ, SmoothQuant & QAT STE	**GPTQ:** inverse Hessian H^{-1}; **AWQ:** protects 1% salient weights chosen by **activation magnitude**; **SmoothQuant:** migrates activation outliers to weights; **STE:** $\frac{\partial \text{round}(u)}{\partial u} = 1$.

Question ID	Core Concept Tested	The 10-Second Formula / Golden Rule
`LLM19`	Knowledge Distillation T^2 & Pruning	Multiply soft-target KL loss by T^2 to compensate for $1/T^2$ gradient shrinkage; **2:4 Sparsity** (2 zeros per 4 weights) gives $2\times$ Sparse Tensor Core speedup.
`LLM20`	Speculative Decoding & Two-Stage RAG	Speculative Decoding verifies γ draft tokens in 1 parallel Target forward pass (**100% lossless**); RAG uses **Bi-Encoder** (fast MIPS retrieval) $\rightarrow$ **Cross-Encoder** (joint attention reranking).